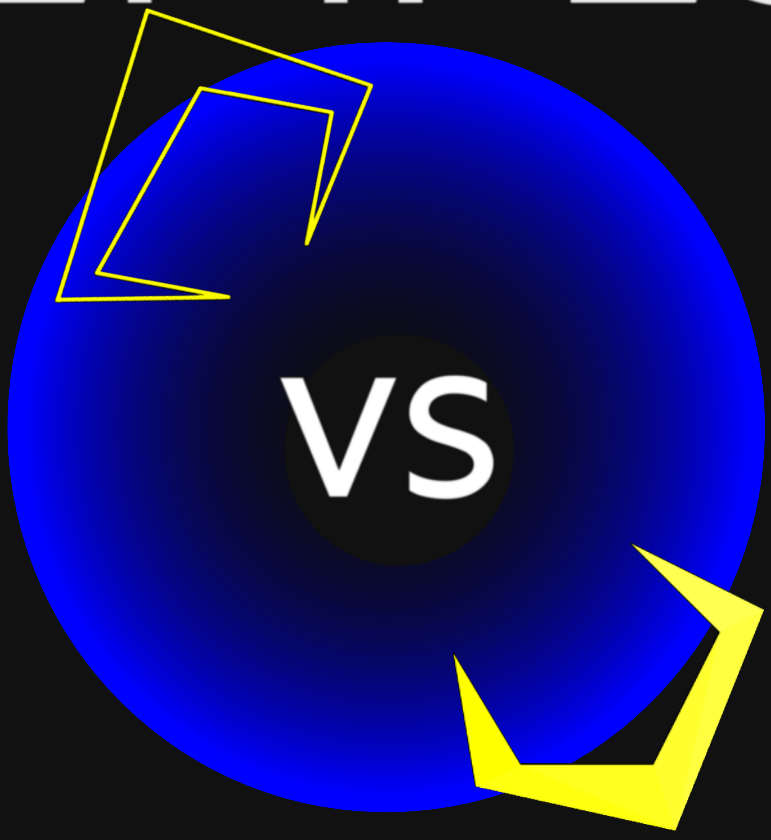
TEMPEST



TEMPEST

THE MAKING AND REMAKING OF ATARI'S ICONIC VIDEOGAME

TEMPEST
VS
TEMPEST

Notes on the Source Code of Two Video Games

For Edna.

© Rob Hogan 2026, All Rights Reserved.
Edition Date: Sunday 11th October, 2026

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 3.0 Unported" license.



Contents

how to make tempest in 1981	**13**
building tempest 2000 in 1994	**19**
mainline	**25**
mainloop	**35**
approaching logo process	**51**
cry if i want to	**61**
tracing the beam	**67**
what is blitting	**73**
copyright atari	**81**
character assassination	**91**
macro language fun	**99**

my first shader **109**

tempest program bug **115**

strings of tempest 2000 **119**

strings of tempest **125**

unused stars **127**

star planes **135**

object list **143**

wells **147**

webs **157**

cursors **167**

claws **173**

attract mode admin **177**

auto **183**

more claws **189**

flipper **195**

flipper 2000 **199**

explosions one to four **205**

meltovision **209**

splat control **219**

ouch **227**

tanker detail **233**

even more tanker detail **237**

bullet **239**

things hidden **245**

jump **251**

player charges **255**

activeobjects **261**

superzap **269**

game over **275**

spikes **281**

rotary club **291**

space game reenactment society **297**

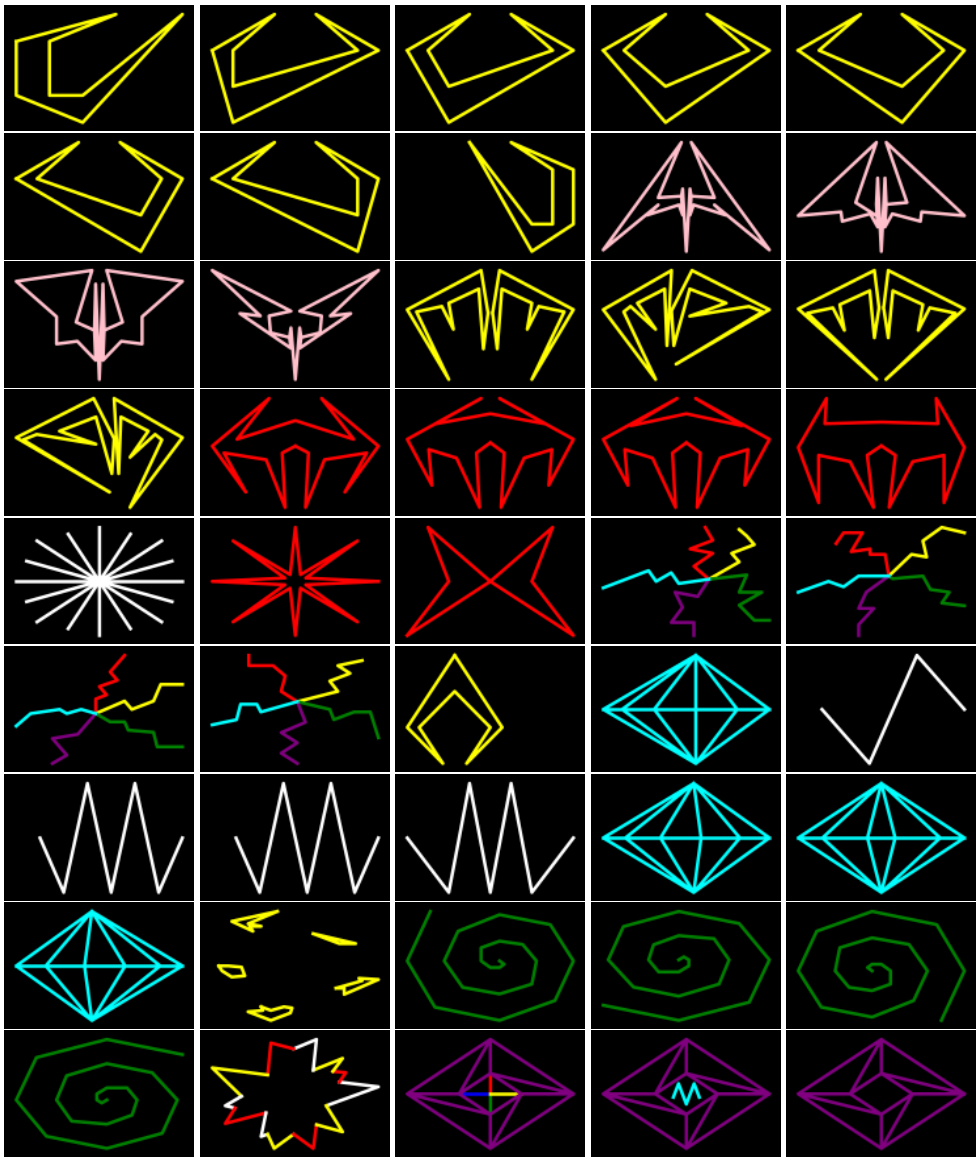
sexy yes **303**

story of a beep **311**

play us a tune pal **315**

unused explosions **323**

ntsc pal **325**



Artefacts from Tempest (1981).

about tempest & tempest 2000

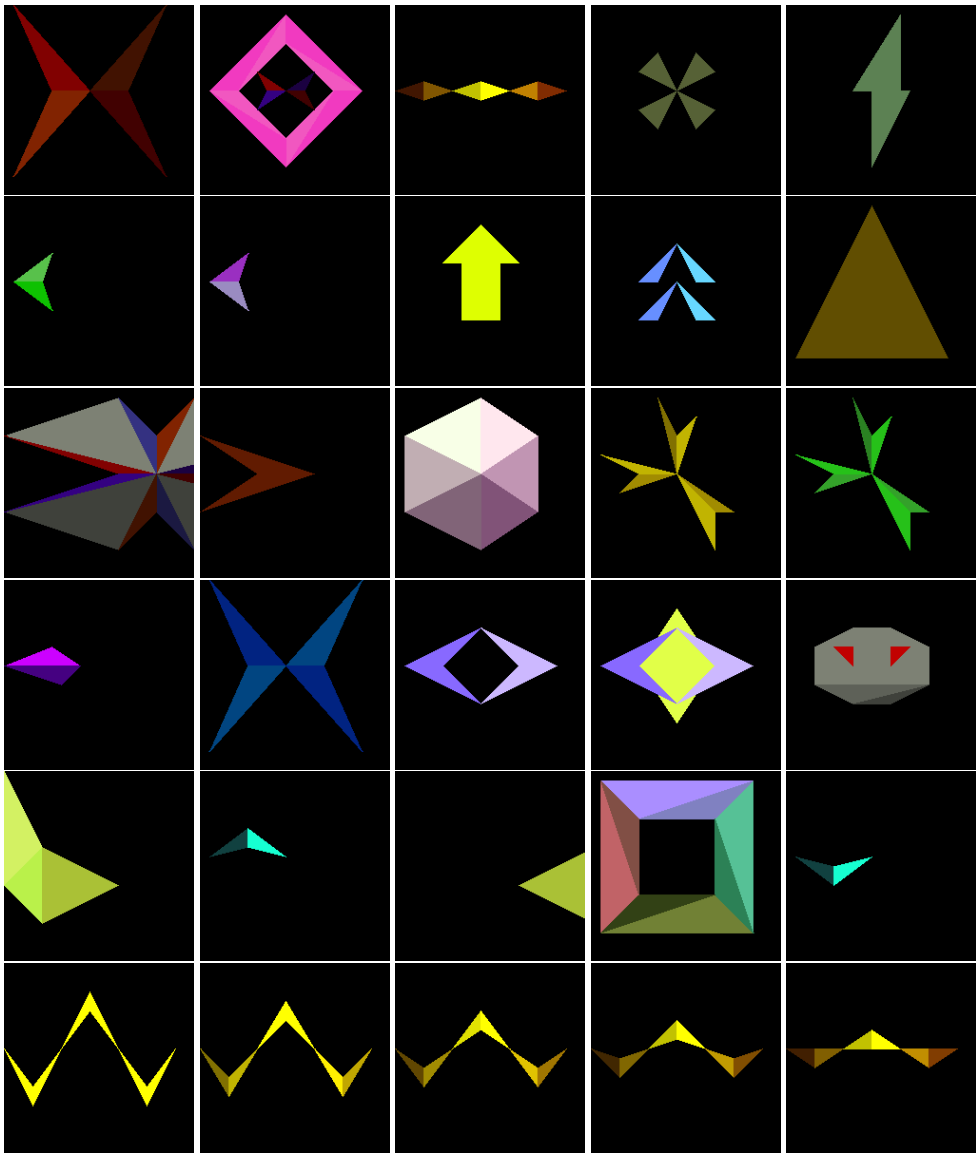
Tempest was a coin-operated arcade game released in 1981 by Atari Corporation. At the time, its graphics and gameplay were at the very edge of what was possible with contemporary hardware. Its unique appearance was in part attributable to its use of vector graphics, a technology that drew lines of color, rather than pixels, on the screen and enabled the earliest 3D graphics in the arcade games of the time.

Some 15 years later, its successor *Tempest 2000* was likewise a classic game. To its misfortune it was made for a console that crashed and burned with no survivors, and very few customers. The Jaguar was supposed to be Atari Corporation's first great leap into the 64 bit era. Instead it promptly tripped over its shoelaces and left a number of great games, Tempest 2000 among them, behind as orphans.

This book isn't about the history of *Tempest* or its importance in the history of computer games. This is a book about computer code. Each chapter aims to be an easily digested sketch of how a specific feature or effect was achieved. In the case of *Tempest* this means sifting through the 6502 assembly code David Theurer wrote by hand(!) to create the game, and wrangling with the bits and bytes of Atari's Quadrascan Vector Graphics. For *Tempest 2000* it means familiarizing ourselves with Motorola 68K assembly code and the Jaguar's powerful hardware platform.

There are lots of fascinating bits and pieces to pore over. And it is interesting to see how the years that separate the two games leave some things unchanged and many other things changed utterly. I've put together this book of squibs in the hope they will allow you, the reader, to digest small bite-sized insights into each game. And I hope, by interleaving elements from *Tempest* and *Tempest 2000*, to enable you to compare the parallel lives in the design and execution of separate incarnations of the same classic video game.

Rob Hogan (@mwenge)
2026



Artefacts from Tempest 2000 (1994).

TEMPEST
VS
TEMPEST

Notes on the Source Code of Two Video Games

how to make tempest in 1981

It is 1981 and Atari Corporation has tasked you with writing a state-of-the-art video game. To complete your assignment they have equipped you with a pencil and a bundle of sheets. You wonder where the computer is, and at what point they will give you one.

You can sit in your office and wonder in vain, because they are never going to give you one. You must devise and program the game entirely with pencil and paper. There is only one computer in the building, and it is the price of a house. They are not going to let you anywhere near that computer. You are in building 1272 and the computer is in building 1360. You are not even on the same block as the computer.

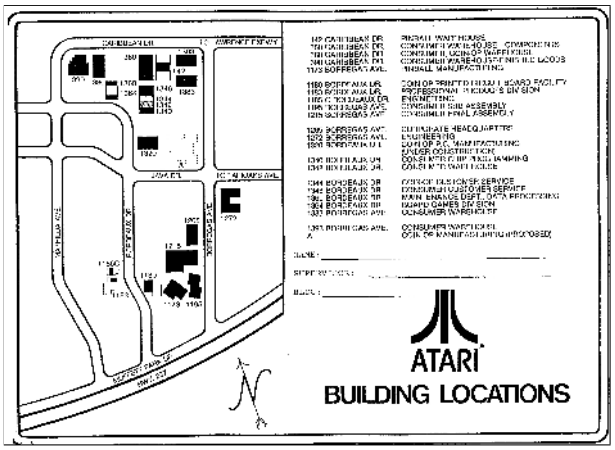
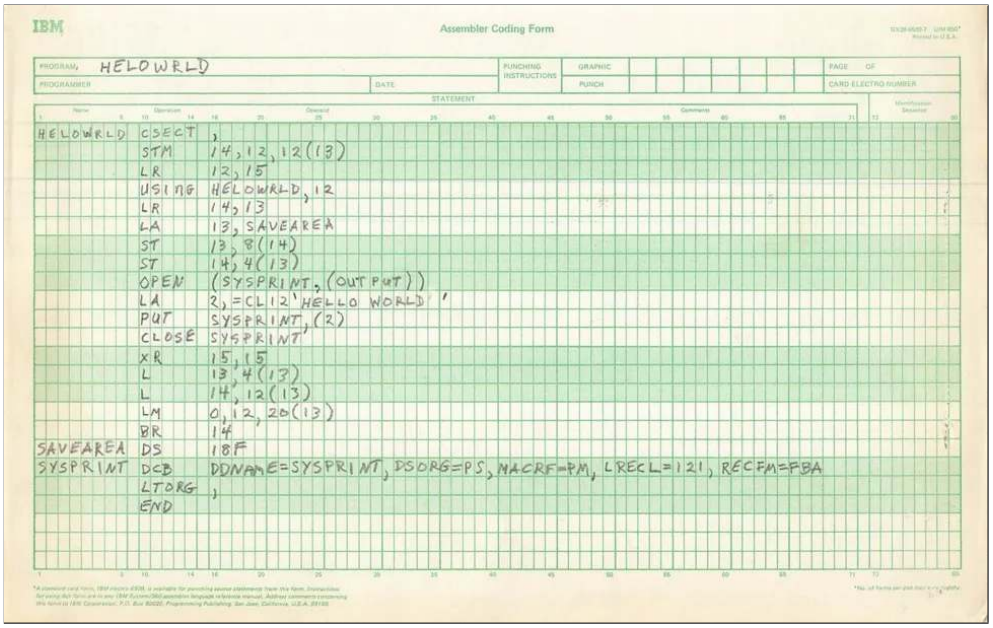


Figure .1: The computer is in another building.

Instead you are confronted with a ream of blank sheets such as this one; into which you must pencil the assembly language instructions that will make your game happen.


IBM

Assembler Coding Form

PROGRAM: HELOWRLD

```
HELOWRLD  CSECT   ,
          STM     14,12,12(13)
          LR      12,15
          USING   HELOWRLD,12
          LR      14,13
          LA      13,SAVEAREA
          ST      13,8(14)
          ST      14,4(13)
          OPEN    (SYSPRINT,(OUTPUT))
          LA      2,=CL12'HELLO WORLD'
          PUT     SYSPRINT,(2)
          CLOSE   SYSPRINT
          XR      15,15
          L       13,4(13)
          L       14,12(13)
          LM      0,12,20(13)
          BR      14
SAVEAREA  DS      18F
SYSPRINT  DCB     DDNAME=SYSPRINT,DSORG=PS,MACRF=PM,LRECL=121,RECFM=FBA
          LTORG   ,
          END
```

Figure .2: An example of a coding sheet to give you an idea of what programming was like in 1981.

If you fill out enough of these sheets with 6502 assembly you have a program that can be typed out and assembled.

dave says



I'd write the code on programming sheets and turn it in to the typists who'd type them in to a DEC computer, then give us a tape with the resulting compiled/linked program.

Once the typists had entered your program into the PDP-11 in building 1360, they would see a listing of the files such as this on a VT-100 terminal.

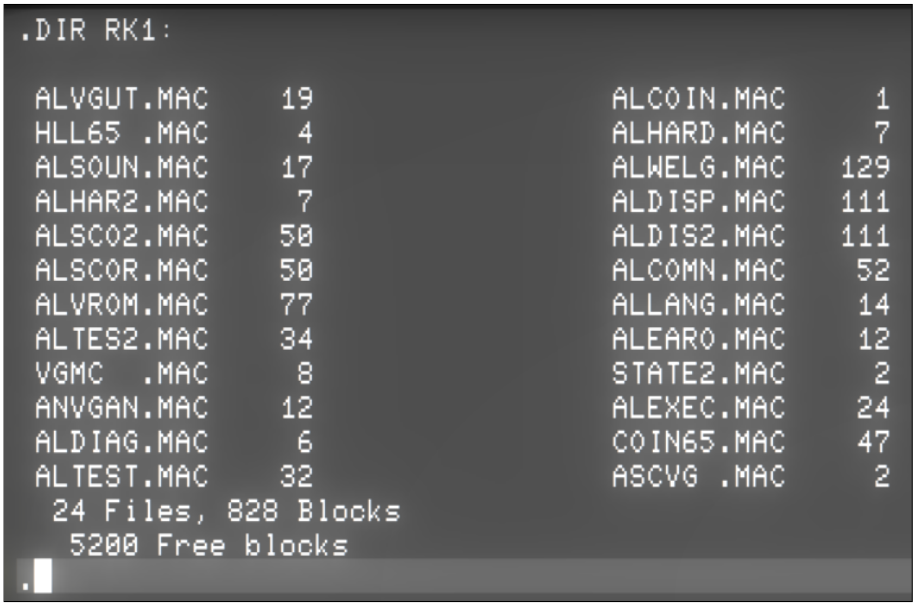


Figure .3: Listing the Tempest source files.

Their next step was to assemble each of the files using the `MAC65` command on each:

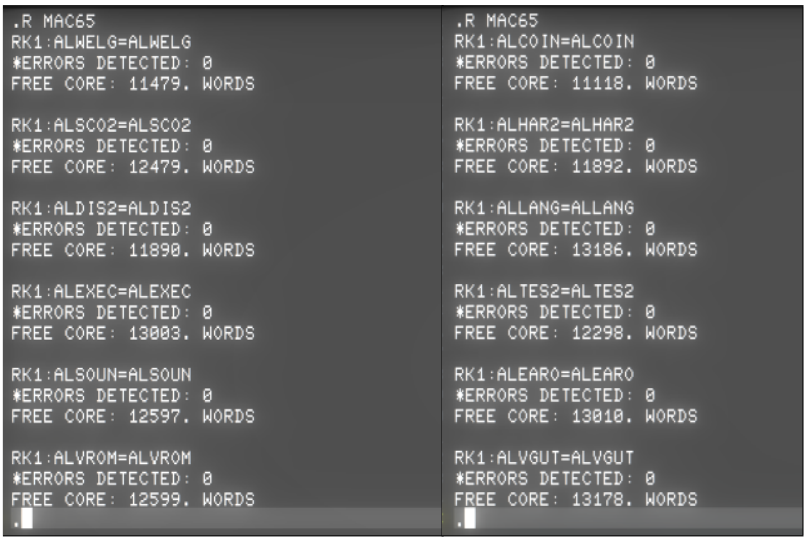


Figure .4: Assembling the Tempest source files.

Once that was done they could run the RMAC command to link the assembled files into a single object file ALEXEC.LDA.

```
.R LINKM
*RK1:ALEXEC/L,ALEXEC/A=RK1:ALWELG,ALSCO2,ALDIS2,ALEXEC,ALSOUN,ALVROM/C
*ALCOIN,ALLANG,ALHAR2,ALTES2,ALEARO,ALVGUT
MULT DEF OF VGBRIT IN MODULE:  000C
MULT DEF OF VGLIST IN MODULE:  000C
MULT DEF OF XCOMP  IN MODULE:  000C
.
```

Figure .5: Linking the Tempest object files.

dave says



We'd then take that to a blue box [which used the FORTH programming language] for debugging. We'd mark changes to the code on a listing, give it back to the typists who'd edit our files and give us a new tape. Repeat ad infinitum.

Once you have completed the ad-infinitum phase of this process you may have a game that is ready for production and distribution. For this you must talk to your project manager, Morgan, and together decide how to split the ALEXEC.LDA file into a number of different files. Each of these will be written to a specific ROM chip as detailed by the parts manifest you devise together and distribute to the guys in building 1320:

```
DATE  17-12-1981 16 23 14     USER  THEURER      JOB  TEMPEST                                          PAGE  0001

PROJECT #   28903                    PROJECT PROGRAMMER   DAVE THEURER              PROJECT LEADER   MORGAN HOFF

DOCUMENTATION DISK S   36A,36B,36C              PROJECT NAME   TEMPEST               DATE   12-17-81         VERSION  2A  ALT

VERIFICATION CONTROL FILENAME  DO2X2.DAT        VERIFICATION FILE  ALEXEC.LDA

PART #          PCB LOC.        DESCRIPTION             START ADDR      GENERIC NAME    PART SIZE       BITS USED
------          --------        -----------             ----------      ------------    ---------       ---------

136002-138      N/P3            VG EROM                 3000            2532            4KX8            7- 0
136002-237      P1              PROGRAM EROM            9000            2532            4KX8            7- 0
136002-136      L/M1            PROGRAM EROM            A000            2532            4KX8            7- 0
136002-235      J1              PROGRAM EROM            B000            2532            4KX8            7- 0
136002-134      F1              PROGRAM EROM            C000            2532            4KX8            7- 0
136002-133      D1              PROGRAM EROM            D000            2532            4KX8            7- 0
                                 IND. SELF TEST

GENERAL INFORMATION
-------------------

ASSEMBLER USED   MAC65

MAIN PROGRAM SOURCE FILES
        INCLUDES   ALCOMN, HLL65, COIN65, VGMC, ANVGAN
        FILES   ALWELG,ALSCO2,ALDIS2,ALEXEC,ALSOUN,ALVROM,ALCOIN,ALLANG,ALHAR2,ALTES2,ALEARO,ALVGUT

PICTURE FILE

LINKER USED   LINKM

LINKER COMMAND   ALEXEC/L,ALEXEC/A ALWELG,ALSCO2,ALDIS2,ALEXEC,ALSOUN,ALVROM,ALCOIN,ALLANG,ALHAR2,ALTES2,ALEARO,ALVGUT

COMMAND FILES
DOCUMENTATION FILE  TEMPST.DOC
```

Figure .6: The actual build manifest for Tempest, recreated in the style of the time.

You might expect that splitting ALEXEC.LDA is as simple as carving up its raw contents into these component files. You would be wrong. ALEXEC.LDA actually consists of a long sequence of triplets, each giving you the ROM address a set of bytes are to be written to, the number of bytes, followed by the bytes themselves. This table shows the first ten or so triplets to give you an idea of what we mean. ALEXEC.LDA contains 2000 such triplets in total!

Address	Length	Data
9000	38	02BB5A3050EE3DA84D20C592203492202B902031A8A9FA855BA9008D0601855FA90085016020
9026	38	1B9220C592208F92206F92204692209F9220AD92206EC1A9FF8D24018D4801A9008D230160A9
904C	19	108D0202A9008529852BAD2101852A1000C62B
905C	1	02
905F	38	A201A52A0A662A6629CA10F6A529186D22018D2201A52A65688568A52B65698569A55F186918
9085	17	855FA55B6900855BC9FC9000A9018D1501
9090	1	05
9096	11	A55F38E55DA55BF000E9FF
909E	1	02
90A1	18	D000A55D855FA9FF855BA90424053000A908
90B0	1	02
90B3	9	8500A63DA9009D0201
90A2	1	19
90BC	36	A9FF8D14014C4997AD2601A21CCADDFE9190FAA004AD6A012904F000AD1D07C9309000C8
90DE	1	01
90E0	5	C9509000C8
90E3	1	01
90E5	5	C9709000C8
90E8	1	01
90D7	1	12
90EA	10	A5092943C940D000A01B
90F1	1	02
90F4	8	8429E429B000A629
90F9	1	02
90FC	12	8E2701A5051000A9008D2601
9102	1	05
9108	9	A63F863DF00020B292
910D	1	03
9111	38	A904857CA9FF855BA9008D00028551857B8D0506A6051000A9148D0506A9FF8D1101A9168500
9137	13	A9088501A900859F2096C1A910
9128	1	1B

The triplets in ALEXEC.LDA for ROM address 9000 written to ROM Part 136002-237.

Notice how apparently wasteful this file format is: some of the triplets contain only one byte! You can also notice that the triplets can appear out of order. For example the bytes for address 90A2 are specified after the bytes for 90A1. So perhaps for some binaries this encoding scheme may be more efficient than just raw bytes, but not in the case of ALEXEC.LDA which uses 25,408 bytes to encode a total of 24,576 bytes that actually get written to the ROM parts on the circuit board.

Which leads nicely to our final diagram. If you look closely in the circuit board diagram on the following page you will find the final destination for these files in the printed circuit board shipped in the arcade cabinet. They are the parts indicated by stars at the top of the board.

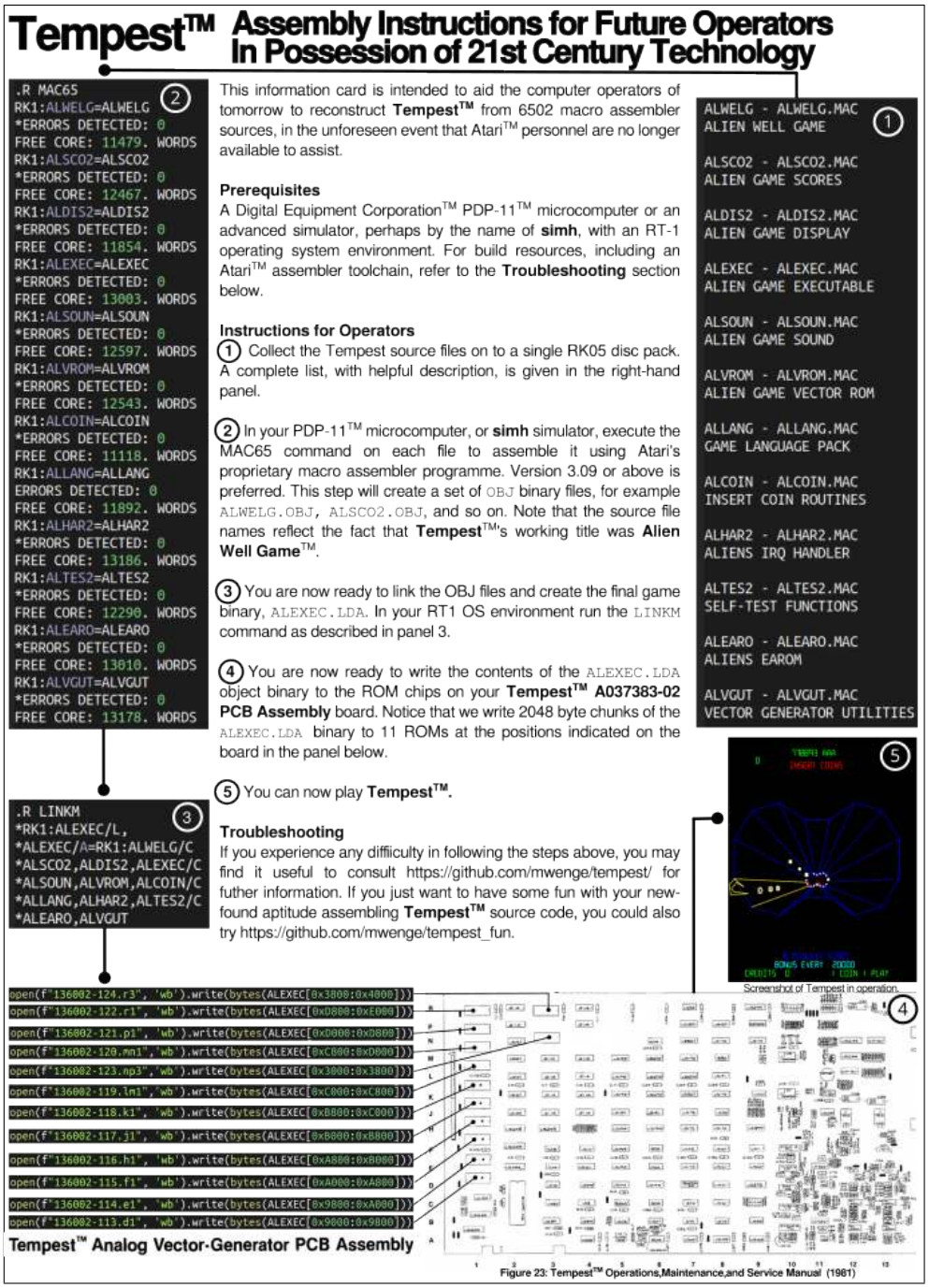


Figure .7: Information Card describing the build process on modern systems.

building tempest 2000 in 1994

It is 1994 and Atari have asked you create the flagship launch title for their new video game console, the Atari Jaguar. No need to come to Sunnyvale in California, you can stay in your valley in Wales and let us know when you're finished.



Figure .1: You are not in California, you are in Cwmcych in Wales and that's where you'll make Tempest 2000.

So instead of sitting two doors down from the computer room armed with just a pencil, you are instead expected to conjure a game that will launch a machine on which the future of Atari depends from your cottage. To get you started you will be sent a console

and a manual. You already have a computer we assume.

So because it's the way you've always done it, you get started writing `Tempest 2000` in one big file of Motoral 68K assembly code. And because you always get to name things whatever you want, you name this file `YAK.S`, because that's the the three-letter name you use on high score tables.



Figure .2: YAK has the top score in Tempest.

Assembling this code will be a simple matter of running the `madmac` assembler as follows:

```
$ mac -fb -isrc -l src/yak.s -o yak.dat
```

We're not done yet though. In addition to a cryptic name for the main megafile (`YAK.S` will eventually run to almost 20,000 lines of code), we also need to write a bunch of programs in a custom flavour of 68K Motorola assembly for the Jaguar's GPU. These programs will handle all the hard mathematics for generating polygons in three dimensions. Because no-one else will ever need to read this code we can name them whatever we want. So we'll continue the zoological theme and name them after more beasts of the field. There will be nine GPU programs in total, each doing something specific (for example, scaling and rotating objects in `antelope.gas`) and some of them doing the same things but slightly differently (variations of a Robotron-style particle explosion in `camel.gas` and `xcamel.gas`).

This is roughly what it looks like when we assemble each of our GPU programs.

```
$ mac -fr -mtom -l -isrc donky.gas -o donky.dat
$ mac -fr -mtom -l -isrc camel.gas -o camel.dat
```

```
$ mac -fr -mtom -l -isrc antelope.gas -o antelope.dat
$ mac -fr -mtom -l -isrc goat.gas -o goat.dat
$ mac -fr -mtom -l -isrc llama.gas -o llama.dat
$ mac -fr -mtom -l -isrc horse.gas -o horse.dat
$ mac -fr -mtom -l -isrc ox.gas -o ox.dat
$ mac -fr -mtom -l -isrc stoat.gas -o stoat.dat
$ mac -fr -mtom -l -isrc xcamel.gas -o xcamel.dat
```

Now is a good time to be dealing with less objects, so let's create another file called `YAKGPU.S` that collates all our GPU programs into one big GPU object. Here's how we do that in `YAKGPU.S`:

```
;
;    CONSTANT DATA (GPU PROGRAMS)
;
fastvector:
    .include "llama.dat"
xvector:
    .include "goat.dat"
demons:
    .include "antelope.dat"
parrot:
    .include "camel.dat"
xparrot:
    .include "xcamel.dat"
texter:
    .include "stoat.dat"
bovine:
    .include "ox.dat"
equine:
    .include "horse.dat"
equine2:
    .include "donky.dat"
```

And here's how we assemble `YAKGPU.S`:

```
$ mac -fb -isrc yakgpu.s -o yakgpu.dat
```

We have one more object file to pull together. An omnibus containing all our image and sound data. For this we will create a file called `IMAGES_SOUNDS.S`. At the top we will put all our image files:

```
.incbin "beasty3.cry"
.incbin "beasty4.cry"
.incbin "beasty5.cry"
.incbin "beasty6.cry"
.incbin "beasty7.cry"
.incbin "beasty8.cry"
```

We are still in a proverbial farmyard with our naming conventions. Let's take a look at what's actually in these files:



beasty3.cry



beasty5.cry



beasty5.cry



beasty6.cry



beasty7.cry



beasty8.cry

Apart from attractive image data we also have some music to play. There are seven

tunes for us to include. These are:

```
.incbin "sounds/tune13.mod"
.DC.L $0000
.incbin "sounds/tune7.mod"
.DC.L $0000
.incbin "sounds/tune1.mod"
.DC.L $0000
.incbin "sounds/tune3.mod"
.DC.L $0000
.incbin "sounds/rave4.mod"
.DC.L $0000
.incbin "sounds/tune5.mod"
.DC.L $0000
.incbin "sounds/tune12.mod"
.DC.L $0000
```

We also have a whole bunch of sound effects. But we'll get into those later. For now, let's assemble our audiovisual bag of tricks into a single object file:

```
$ mac -fb -isrc images_sounds.s -o images_sounds.dat
```

This leaves us with just one final step to perform: linking all of our products, `YAK.DAT`, `IMAGES_SOUNDS.DAT`, and `YAKGPU.DAT` together to give us a game we can actually play. For that we use Atari's purpose built linker `aln`:

```
$ aln -z -e -a 802000 4000 efa8 yak.dat vidinit.dat yakgpu.dat images_sounds
    .dat -o t2000.abs
```

mainline

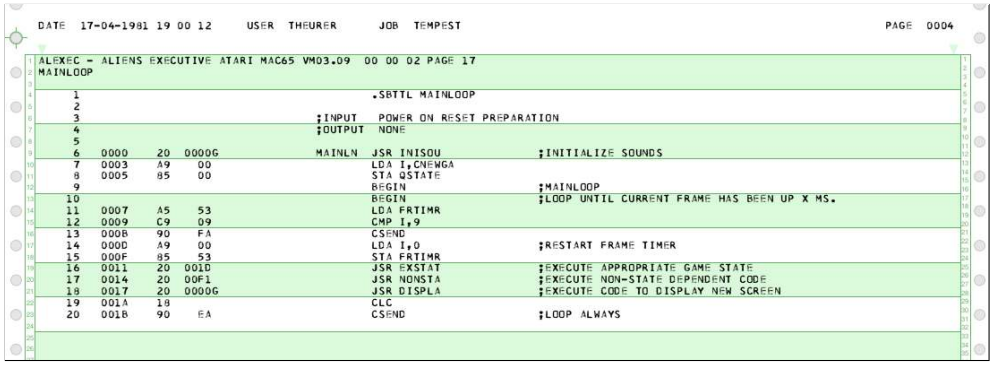
Tempest's 'mainline' is a short routine that runs the entirety of the game from a handful of lines of assembly code. Beneath lies an iceberg of strange machinery but at its core the game consists of running these 15 or so lines many hundreds of times per second:

```
;INPUT:         POWER ON RESET PREPARATION
;OUTPUT:        NONE

MAINLN: JSR INISOU              ;INITIALIZE SOUNDS
        LDA I,CNEWGA
        STA QSTATE
        BEGIN                   ;MAINLOOP
        BEGIN                   ;LOOP UNTIL CURRENT FRAME HAS BEEN UP X MS.
        LDA FRTIMR
        CMP I,9
        CSEND
        LDA I,0                 ;RESTART FRAME TIMER
        STA FRTIMR
        JSR EXSTAT              ;EXECUTE APPROPRIATE GAME STATE
        JSR NONSTA              ;EXECUTE NON-STATE DEPENDENT CODE
        JSR DISPLA              ;EXECUTE CODE TO DISPLAY NEW SCREEN
        CLC
        CSEND                   ;LOOP ALWAYS
```

The game's framerate is controlled by this tight little loop. As soon as FRTIMR reaches 9 a frame of the game is executed:

```
        LDA FRTIMR
        CMP I,9
        CSEND
```

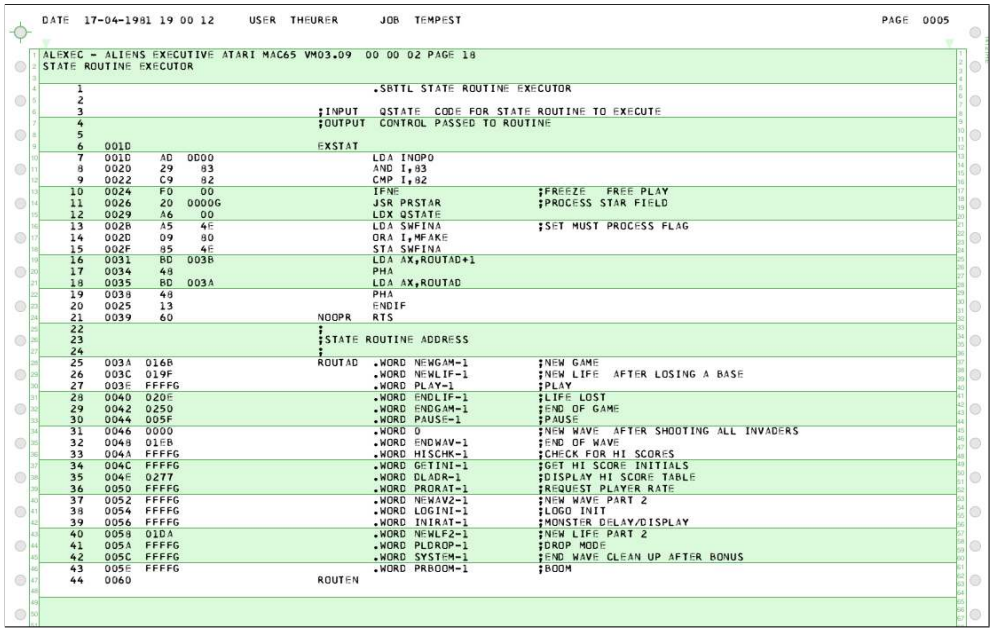


```
DATE  17-04-1981 19 00 12      USER  THEURER       JOB  TEMPEST                                  PAGE  0004

ALEXEC - ALIENS EXECUTIVE ATARI MAC65 VM03.09  00 00 02 PAGE 17
MAINLOOP

     1                                          .SBTTL MAINLOOP
     2
     3                                  ;INPUT   POWER ON RESET PREPARATION
     4                                  ;OUTPUT  NONE
     5
     6  0000    20  0000G               MAINLN  JSR INISOU              ;INITIALIZE SOUNDS
     7  0003    A9    00                        LDA I,CNEWGA
     8  0005    85    00                        STA QSTATE
     9                                          BEGIN                   ;MAINLOOP
    10                                          BEGIN                   ;LOOP UNTIL CURRENT FRAME HAS BEEN UP X MS.
    11  0007    A5    53                        LDA FRTIMR
    12  0009    C9    09                        CMP I,9
    13  000B    90    FA                        CSEND
    14  000D    A9    00                        LDA I,0                 ;RESTART FRAME TIMER
    15  000F    85    53                        STA FRTIMR
    16  0011    20  001D                        JSR EXSTAT              ;EXECUTE APPROPRIATE GAME STATE
    17  0014    20  00F1                        JSR NONSTA              ;EXECUTE NON-STATE DEPENDENT CODE
    18  0017    20  0000G                       JSR DISPLA              ;EXECUTE CODE TO DISPLAY NEW SCREEN
    19  001A    18                            CLC
    20  001B    90    EA                        CSEND                   ;LOOP ALWAYS
```

Figure .1: The `MAINLN` routine in an assembly listing.



```
DATE  17-04-1981 19 00 12      USER  THEURER       JOB  TEMPEST                                  PAGE  0005

ALEXEC - ALIENS EXECUTIVE ATARI MAC65 VM03.09  00 00 02 PAGE 18
STATE ROUTINE EXECUTOR

     1                                          .SBTTL STATE ROUTINE EXECUTOR
     2
     3                                  ;INPUT   QSTATE  CODE FOR STATE ROUTINE TO EXECUTE
     4                                  ;OUTPUT  CONTROL PASSED TO ROUTINE
     5
     6  001D                            EXSTAT
     7  001D    AD  0D00                        LDA INOPO
     8  0020    29    83                        AND I,83
     9  0022    C9    82                        CMP I,82
    10  0024    F0    00                        IFNE                    ;FREEZE   FREE PLAY
    11  0026    20  0000G                       JSR PRSTAR              ;PROCESS STAR FIELD
    12  0029    A6    00                        LDX QSTATE
    13  002B    A5    4E                        LDA SWFINA              ;SET MUST PROCESS FLAG
    14  002D    09    80                        ORA I,MFAKE
    15  002F    85    4E                        STA SWFINA
    16  0031    BD  003B                        LDA AX,ROUTAD+1
    17  0034    48                            PHA
    18  0035    BD  003A                        LDA AX,ROUTAD
    19  0038    48                            PHA
    20  0025    13                            ENDIF
    21  0039    60                      NOOPR   RTS
    22                                  ;
    23                                  ;STATE ROUTINE ADDRESS
    24                                  ;
    25  003A  016B                      ROUTAD  .WORD NEWGAM-1          ;NEW GAME
    26  003C  019F                              .WORD NEWLIF-1          ;NEW LIFE  AFTER LOSING A BASE
    27  003E  FFFFG                             .WORD PLAY-1            ;PLAY
    28  0040  020E                              .WORD ENDLIF-1          ;LIFE LOST
    29  0042  0250                              .WORD ENDGAM-1          ;END OF GAME
    30  0044  005F                              .WORD PAUSE-1           ;PAUSE
    31  0046  0000                              .WORD 0                 ;NEW WAVE  AFTER SHOOTING ALL INVADERS
    32  0048  01EB                              .WORD ENDWAV-1          ;END OF WAVE
    33  004A  FFFFG                             .WORD HISCHK-1          ;CHECK FOR HI SCORES
    34  004C  FFFFG                             .WORD GETINI-1          ;GET HI SCORE INITIALS
    35  004E  0277                              .WORD DLADR-1           ;DISPLAY HI SCORE TABLE
    36  0050  FFFFG                             .WORD PRORAT-1          ;REQUEST PLAYER RATE
    37  0052  FFFFG                             .WORD NEWAV2-1          ;NEW WAVE PART 2
    38  0054  FFFFG                             .WORD LOGINI-1          ;LOGO INIT
    39  0056  FFFFG                             .WORD INIRAT-1          ;MONSTER DELAY/DISPLAY
    40  0058  01DA                              .WORD NEWLF2-1          ;NEW LIFE PART 2
    41  005A  FFFFG                             .WORD PLDROP-1          ;DROP MODE
    42  005C  FFFFG                             .WORD SYSTEM-1          ;END WAVE CLEAN UP AFTER BONUS
    43  005E  FFFFG                             .WORD PRBOOM-1          ;BOOM
    44  0060                            ROUTEN
```

Figure .2: The `EXSTAT` and `ROUTEAD` routines in an assembly listing.

Then it is reset to zero:

```
        LDA I,0                 ;RESTART FRAME TIMER
        STA FRTIMR
```

Increasing the value from 9 to 0F (15) will slow the game down appreciably, while increasing it to 1F (31) will bring it to an unplayable crawl. Lowering it to 0 or 1 does make the game slightly faster, but not noticeably and it is still playable.

cursor says



But what increments FRTIMR you ask? The answer is a piece of code in what is known as an 'interrupt handler'. This runs separately from the main loop and is invoked by the CPU up to 60 times a second.

Once we've decided to execute a frame we are down to just three lines orchestrating the entirety of the game:

```
        JSR EXSTAT              ;EXECUTE APPROPRIATE GAME STATE
        JSR NONSTA              ;EXECUTE NON-STATE DEPENDENT CODE
        JSR DISPLA              ;EXECUTE CODE TO DISPLAY NEW SCREEN
```

The first routine we call is EXSTAT. It always updates the starfield (PRSTAR) but what it does next is dependent on the value contained in QSTATE. Take a brief look at the routine to see if you can identify how it uses QSTATE:

```
;INPUT:         QSTATE: CODE FOR STATE ROUTINE TO EXECUTE
;OUTPUT:        CONTROL PASSED TO ROUTINE

EXSTAT:
        LDA INOP0
        AND I,83
        CMP I,82
        IFNE                    ;FREEZE & FREE PLAY?
        JSR PRSTAR              ;PROCESS STAR FIELD
        LDX QSTATE
        LDA SWFINA              ;SET MUST PROCESS FLAG
        ORA I,MFAKE
        STA SWFINA
        LDA AX,ROUTAD+1
        PHA
        LDA AX,ROUTAD
        PHA
        ENDIF
NOOPR:  RTS
```

The answer is in these lines:

```
        LDX QSTATE
        ..
        LDA AX,ROUTAD+1
        PHA
        LDA AX,ROUTAD
        PHA
```

The value in QSTATE is loaded to the X register. This is then used an index to retrieve the two-byte address stored in the Xth - 1 index of the ROUTAD table:

```
;
;STATE ROUTINE ADDRESS
;
ROUTAD: .WORD NEWGAM-1          ;NEW GAME
        .WORD NEWLIF-1          ;NEW LIFE (AFTER LOSING A BASE)
        .WORD PLAY-1            ;PLAY
        .WORD ENDLIF-1          ;LIFE LOST
        .WORD ENDGAM-1          ;END OF GAME
        .WORD PAUSE-1           ;PAUSE
        .WORD 0                 ;NEW WAVE (AFTER SHOOTING ALL INVADERS)
        .WORD ENDWAV-1          ;END OF WAVE
        .WORD HISCHK-1          ;CHECK FOR HI SCORES
        .WORD GETINI-1          ;GET HI SCORE INITIALS
        .WORD DLADR-1           ;DISPLAY HI SCORE TABLE
        .WORD PRORAT-1          ;REQUEST PLAYER RATE
        .WORD NEWAV2-1          ;NEW WAVE PART 2
        .WORD LOGINI-1          ;LOGO INIT
        .WORD INIRAT-1          ;MONSTER DELAY/DISPLAY
        .WORD NEWLF2-1          ;NEW LIFE PART 2
        .WORD PLDROP-1          ;DROP MODE
        .WORD SYSTEM-1          ;END WAVE CLEAN UP AFTER BONUS
        .WORD PRBOOM-1          ;BOOM
```

Pushing each of the two bytes to the stack (using PHA) means that whatever is at the address of those two bytes will be executed next. For example, if the value in QSTATE is 0 it will retrieve the address of NEWGAM-1: which is 016B, made up of 6B which it retrieves first, and 01 which it retrieves second. When both of these bytes are pushed to the stack they are treated as an address for execution: the result is that the EXTSTAT routine will execute NEWGAM once it exits, beginning a new game.

So depending on the current state of the game, Tempest will populate QSTATE with the index for the function it needs to call next. To give a flavour of what NEWGAM looks like we've listed it below. As expected it mainly consists of accounting operations such as clearing the score, assigning the number of lives and so on.

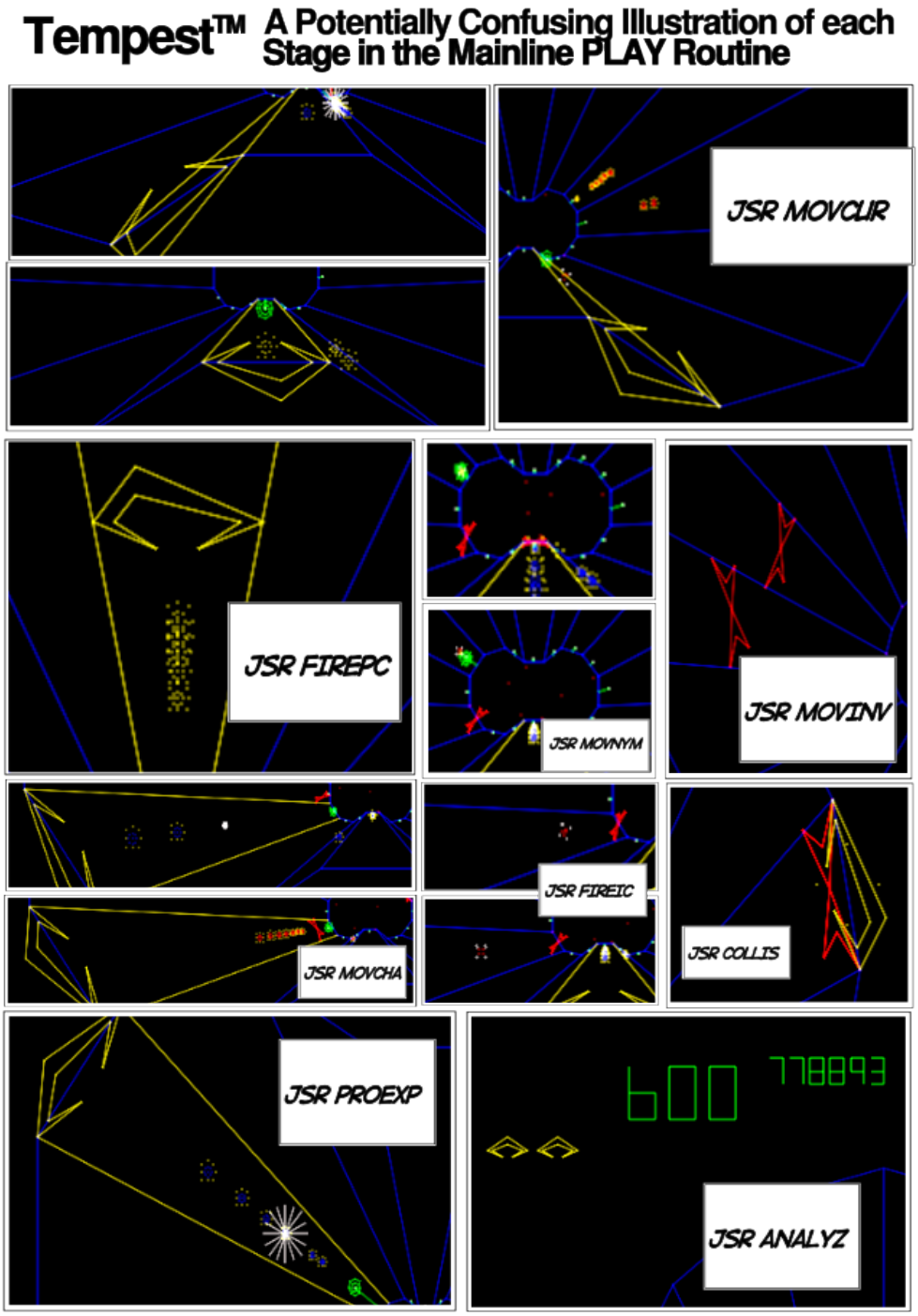
```
NEWGAM: JSR INICHK              ;INITIALIZE LANGUAGE PTRS, OPTIONS
        JSR INIDSP              ;INITIALIZE DISPLAY
        LDA QSTATUS
        IFMI                    ;ATTRACT?
```

```
        JSR CLRSCO              ;NO. CLEAR SCORES
        ENDIF
        LDA I,0
        STA LIVES2              ;ONE PLAYER GAME (DEFAULT: PLAYER 2 DEAD)
        LDX NUMPLA              ;GIVE EACH PLAYER "NEW GAME" EQUIP
        STX PLAYUP
        BEGIN                   ;LOOP FOR EACH PLAYER IN GAME (1 OR 2)
        LDX PLAYUP
        LDA LVSGAM              ;GET # LIVES
        STA AX,LIVES1           ;INITIAL # OF LIVES (GUNS)
        LDA I,-1
        STA AX,WAVEN1           ;FORCE REQUEST RATE STATE
        DEC PLAYUP
        MIEND                   ;ENDLOOP AFTER ALL PLAYERS PROCESSED
        LDA I,0
        STA NEWPLA              ;START GAME WITH 1ST PLAYER UP.
        STA PLAGRO              ;DEACTIVATE STAR FIELD
        LDA NUMPLA              ;INDUCE "PLAY PLAYER 1" MESSAGE
        STA PLAYUP              ;IF 2 PLAYER GAME.
        JMP INIRA0              ;INITIALIZE FOR PLAYER RATE REQUEST
```

You will notice at the end that we jump to a routine called `INIRA0`. This continues the accounting set-up for a new game but it also does something important, which is update `QSTATE` to direct the flow the next time it passes through `MAINLN` to a different function. In this case, it loads 16 to `QSTATE`, which will direct us to the routine `PRORAT` at the next iteration:

```
        LDA I,CREQRAT           ;GO TO REQUEST
        STA QSTATE              ;RATE STATE
```

Another way of describing what we have here is a 'state machine'. As the state of the system changes, `EXSTAT` will execute a different routine, appropriate to the current state of the game. The one executed most often is the third one listed in our `ROUTEAD` table. This is called `PLAY` and its contents, as well as its name, are almost self-explanatory.


Tempest™ A Potentially Confusing Illustration of each Stage in the Mainline PLAY Routine
JSR MOVCLIR
JSR FIREPC
JSR MOVNYM
JSR MOVINV
JSR FIREIC
JSR COLLIS
JSR MOVCHA
JSR PROEXP
600
778893
JSR ANALYZ

```
PLAY:
        JSR MOVCUR              ;MOVE CURSOR AROUND
        JSR FIREPC              ;FIRE PLAYER CHARGE
        JSR PROSUZ              ;PROCESS SUPER ZAP
        JSR MOVNYM              ;MOVE NYMPHS
        JSR MOVINV              ;MOVE INVADERS
        JSR MOVCHA              ;MOVE CHARGES
        JSR FIREIC              ;FIRE INVADER CHARGE
        JSR COLLIS              ;COLLISION DETECT
        JSR PROEXP              ;EXPLOSIONS
        JMP ANALYZ              ;ANALYZE PLAYER STATUS
```

Here every element of the game is updated and manipulated in some way. It is not surprising that somewhere in the heart of a game's runtime the logic should ultimately settle on a list of items that need to be checked, one after the other. One important thing to note here is that we're not updating how any of the game elements are displayed. We are just updating their state, which we store separately from the details of how they are displayed. We will use this updated state to determine their display in a moment, but for now we are just updating our record of their position and condition. The second thing to note is that the order in which these operations are performed is not arbitrary. First we update the player and any bullets the player has fired. Then we move the enemies and any bullets they have fired. Now we can check if the player has collided with any of the enemies. If there has been a collision between any two objects we process an explosion and finally update the player's state to determine, for example, if they are dead. The purpose of this order is to ensure that information about the player and its enemies are updated before we detect to attempt any collisions.

A lot has happened underneath the single line..

```
        JSR EXSTAT              ;EXECUTE APPROPRIATE GAME STATE
```

.. however now that we have all the properties of the game in its current frame we can move on to updating how they are displayed:

```
        JSR DISPLA              ;EXECUTE CODE TO DISPLAY NEW SCREEN
```

Again this will in most cases boil down to updating all of the elements one by one. Unlike when we were concerned with updating state, the order in which we do things is not particularly important here. The `DISPLA` routine wants to build a list of lines to paint on the screen so it will compute the lines for each type of object currently displayed in a more or less abitrary sequence determined by the programmer. Notice that in the sequence below the display of each type of object follows a fixed pattern:

```
                                ;DISPLAY CURSOR (PLAYER)
        LDA I,BCCURS
```

```
        JSR SBCLOG
        JSR DSPCUR
        LDA I,BCCURS
        JSR SBCSWI
                              ;DISPLAY CHARGES
        LDA I,BCSHOT
        JSR SBCLOG
        JSR DSPCHG
        LDA I,BCSHOT
        JSR SBCSWI
                              ;DISPLAY INVADERS
        LDA I,BCINVA
        JSR SBCLOG
        JSR DSPINV
        LDA I,BCINVA
        JSR SBCSWI
                              ;DISPLAY EXPLOSIONS
        LDA I,BCEXPL
        JSR SBCLOG
        JSR DSPEXP
        LDA I,BCEXPL
        JSR SBCSWI
                              ;DISPLAY NYMPHS
        LDA I,BCNYMP
        JSR SBCLOG
        JSR DSPNYM
        LDA I,BCNYMP
        JSR SBCSWI
                              ;DISPLAY INFORMATION (SCORES, MSGS, ETC.)
        LDA I,BCINFO
        JSR SBCLOG
        JSR INFO
        LDA I,BCINFO
        JSR SBCSWI
        ..
        JSR DSPWEL            ;DISPLAY WELL

        LDA I,BCENEL          ;DISPLAY ENEMY LINES
        JSR SBCLOG
        JSR DSPENL
        LDA I,BCENEL
        JSR SBCSWI

        LDA I,BCSTAR          ;DISPLAY STAR FIELD
        JSR SBCLOG
        JSR DSTARF
        LDA I,BCSTAR
        JSR SBCSWI
```

In each case we prime the accumulator (A) with a value specific to that object, call the routine SBCLOG, call a routine specific to the object (e.g. DSPCUR), the close by calling the routine SBCWI. These top and tail routines are concerned with setting up and preparing the header and footer material for the list of lines that displays each

object. The object-specific routine in the middle is concerned with collecting the lines themselves. We will get the opportunity to explore the detailed mechanics of how some of these objects are calculated and displayed in later sections.

mainloop

Unlike Tempest, the engine that runs Tempest 2000 is not contained within a mainline routine. Instead the donkey work is done during a vertical sync interrupt. Vertical sync is a point in time: specifically when the Atari Jaguar has finished writing pixels to the screen and has a short amount of time available before it starts writing pixels again from the top. An interrupt is a routine the system will call at a moment of the programmer's choosing. A vertical sync interrupt is when you combine the two, and in this case they are combined in a routine called `Frame`. It is this routine that manages the state of the game and prepares all the objects for display and the sound for output.

That said, Tempest 2000 does have a mainline routine but it is there to solve a problem encountered in development rather than as part of a grand design. As we shall see there are a number of ways of generating graphical data on the Atari Jaguar, and two of them have their own dedicated processors. These are the Graphics Processor (GPU), which specializes in the fast trigonometric operations required for 3D displays and the Blitter, which is suited for large copy and fill operations. It is worth being clear that neither of these processors or any of their operations actually write graphics to the screen. Instead this function is performed by a third dedicated processing unit, the Object Processor. It is the Object Processor that turns data into light. This unit takes a list of operations set up by the programmer for each frame and uses them to write pixels to the screen. These operations will usually include data prepared by the GPU and the Blitter in a long list of tasks for the Object Processor known as an Object List. It's the programmer's job to have a new Object List ready every time the Object Processor is about to paint the screen.

It was the initial intention that the `Frame` routine would be solely responsible for this task in Tempest 2000. But there was a snag: it kept crashing. We know this because

Jeff Minter left us one of his rare comments at the head of the `mainloop routine`:

```
; This loop runs the GPU/Blitter code.  I found that if you
; started up the GPU/Blitter pair from inside the FRAME
; Interrupt, the system would fall over if they got really heavily
; loaded.  MAINLOOP just waits for a sync from the FRAME routine,
; launches the GPU, then loops waiting for another sync.

mainloop:
      move #1,sync      ; Reset the sync.
      move #1,screen_ready ; Signal to 'Frame' screen ready for display.
      move pauen,_pauen ; Reset the pause indicator.

main: tst sync          ; Loop waiting for another sync..
      bne main          ; from the interrupt in 'Frame'.

      move #1,sync      ; Reset sync so that we wait for a new frame.
      ; Use dscreen as the screen that GPU will draw everything to.
      move.l dscreen,gpu_screen

      ; Do the actual mainloop work, mainloop_routine is usually
      ; a reference to the 'draw_objects' routine.
      move.l mainloop_routine,a0 ; Move mainloop_routine to a0.
      jsr (a0)          ; Call mainloop_routine.
```

So instead of doing all the necessary GPU and Blitter operations during a vertical sync interrupt, we do them here in this `mainloop`. Notice the busyloop in the second paragraph above where we wait for the value in `sync` to change to a zero. This forces the CPU to wait until the `Frame` routine resets it to zero. Once it has been reset we can go ahead and execute our `mainloop_routine`. This is actually a reference to the `draw_objects` routine - and it is this routine that does all the GPU and Blitter work required to calculate the pixels for display.

Here we see that when the game is initialized, `draw_objects` is selected as the destination for `mainloop_routine`, and then we enter the `mainloop`.

```
        ; The common entry point for single-player and head-to-head games.
go_in:  move.l #rotate_web,routine ; Make the web rotate when starting.
        move.l #0,warp_count  ; Reset the warp count.
        move.l #draw_objects,mainloop_routine ; Use draw_objects.
        move pauen,_pauen     ; Reset the 'pause' toggle.
        clr db_on             ; Enable double-buffering.
        bra mainloop          ; Enter the mainloop until we lose a life.
```

We will investigate the mechanics of using the GPU and the Blitter in later sections, but for now let's get a sense of the kind of things the `draw_objects` routine uses the GPU and Blitter for.

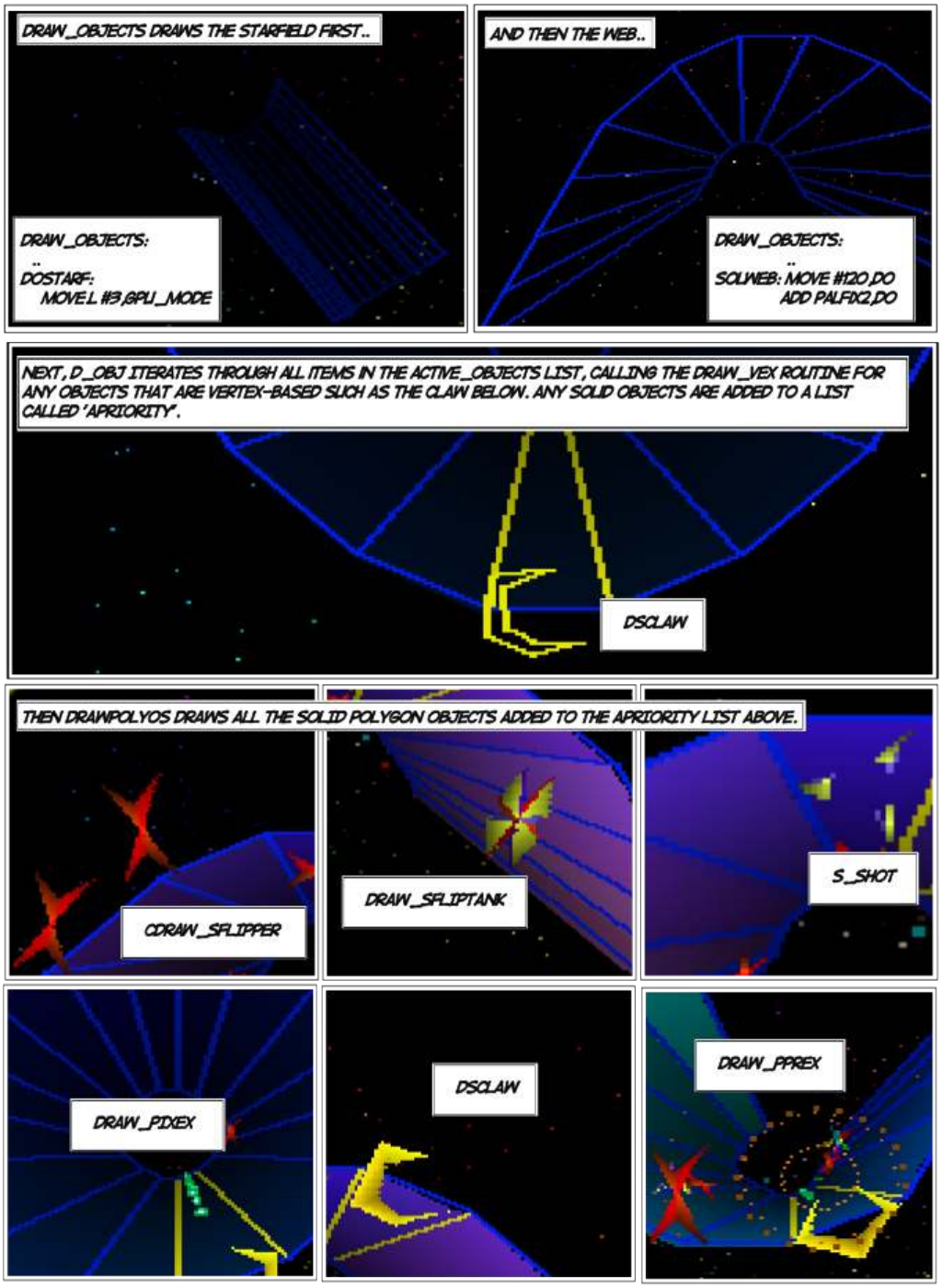
drawing objects

```
draw_objects:
        bsr clearscreen             ; Clear the screen.
        move.b sysflags,d0          ; Copy sysflags to d0.
        and.l #$ff,d0               ; Keep it between 0 and 255.
        move.l d0,_sysflags         ; pass sys flags to GPU
        tst sf_on                   ; Is the starfield active?
        bne dostarf                 ; If yes, go to 'dostarf'.
        bra gwb                     ; Otherwise do the web.

        ; Prepare the starfield!
dostarf:
        move.l #3,gpu_mode          ; mode 3 is starfield1 in llama.gas
        move.l vp_x,in_buf+4        ; Put x pos in the in_buf buffer.
        move.l vp_y,in_buf+8        ; Put y pos in the buffer.
        move.l vp_z,d0              ; Get the current z pos.
        add.l vp_sf,d0              ; Increment it.
        move.l d0,in_buf+12         ; Add it to the buffer.
        move.l #field1,d0           ; Get the starfield data structure.
        move.l d0,in_buf+16         ; And put it in the buffer.
        move.l warp_count,in_buf+20   ; Add the warp count.
        move.l warp_add,in_buf+24   ; Add the warp increment.
        lea fastvector,a0           ; Get GPU routine to use in llama.gas.
        jsr gpurun                  ; do gpu routine
        jsr gpuwait                 ; Wait until its finished.
```

A quick glance through the top of the routine shows us drawing the starfield (`dostarf`), and as you can see 'doing the starfield' involves setting up a number of registers and variables before finally invoking the GPU (`jsr gpurun`) to do its magic and waiting for it to finish (`jsr gpuwait`). We will take a closer look at the mechanics of starfield generation in a later chapter, but the above begins to give us a flavour of the formula required to get a GPU routine up and running.

The next element we find is a routine for drawing the playfield of Tempest 2000: the web. We give this on the next page and as you can see there is a lot going on here and the truth is, this isn't even all of it. And not just that, there are a number of different GPU routines used for drawing webs scattered throughout the game depending on the mode we are playing or whether there is more than one player. Just looking briefly through the code below (and you should confine yourself to that for now) gives you a sense of the overhead involved in setting up a reasonably complex piece of GPU code. Note that at the end of the listing we load a variable called `equine2` to the `A0` register. This is an address to the actual GPU code that the Graphics Processor will run, using all the data set up in the previous lines. So there is much more complexity and detail underneath, even here. We will cover this in more detail in the chapter devoted to webs.


DRAW_OBJECTS DRAWS THE STARFIELD FIRST..
DRAW_OBJECTS:
..
DOSTARF:
MOVE.L #3,GPU_MODE
AND THEN THE WEB..
DRAW_OBJECTS:
..
SOLWEB: MOVE #120,D0
ADD PALFIX2,D0
NEXT, D_OBJ ITERATES THROUGH ALL ITEMS IN THE ACTIVE_OBJECTS LIST, CALLING THE DRAW_VEX ROUTINE FOR ANY OBJECTS THAT ARE VERTEX-BASED SUCH AS THE CLAW BELOW. ANY SOLID OBJECTS ARE ADDED TO A LIST CALLED 'APRIORITY'.
DSCLAW
THEN DRAWPOLYOS DRAWS ALL THE SOLID POLYGON OBJECTS ADDED TO THE APRIORITY LIST ABOVE.
CDRAW_SFLIPPER
DRAW_SFLIPTANK
S_SHOT
DRAW_PIXEX
DSCLAW
DRAW_PPREX

```
gwb:    move.l vp_x,d3          ; Stash player's X viewpoint in d3.
        move.l vp_y,d4          ; Stash player's Y viewpoint in d4.
        move.l vp_z,d5          ; Stash player's Z viewpoint in d5.

solweb: move #120,d0            ; Set register d0 to 120.
        add palfix2,d0          ; Adjust for PAL screens if required.
        and.l #$ff,d0           ; Keep it between 0 and 255
        move.l d0,ycent         ; Store result as the Y centre.
        tst l_solidweb          ; Are we drawing a solid web?
        beq vweb                ; If not, skip to vweb.
        cmp #1,webcol           ; Drawing a transparent web?
        beq vweb                ; If so, skip to vweb.
        lea _web,a6             ; Load the web data structure to a6.
        tst 34(a6)              ; Is the web active?
        beq n_wb                ; If not, skip
        lea in_buf,a0           ; Point our GPU RAM input buffer at a0.
        move.l 46(a6),d0        ; Get ppinter to web data structure.
        move.l d0,(a0)          ; Store it in the in_buf.
        move.l 4(a6),d0         ; Get the source X position.
        sub.l d3,d0             ; Subtract vp_x, player's X viewpoint.
        move.l d0,4(a0)         ; Store it in the in_buf.
        move.l 8(a6),d0         ; Get the object's Y position.
        sub.l d4,d0             ; Subtract vp_y, player's Y viewpoint.
        move.l d0,8(a0)         ; Store it in the in_buf.
        move.l 12(a6),d0        ; Get the object's Z position.
        sub.l d5,d0             ; Subtract vp_z, player's Z viewpoint.
        bmi n_wb                ; If web not visible to player, skip.
        move.l d0,12(a0)        ; Store it in the in_buf.
        move l_solidweb,d0      ; Get X centre of Web.
        and.l #$ff,d0           ; Keep it between 0 and 255
        move.l d0,16(a0)        ; Store it in the in_buf.
        move 28(a6),d0          ; Get Y centre of web.
        and.l #$ff,d0           ; Keep it between 0 and 255
        move.l d0,24(a0)        ; Store it in the in_buf.
        move frames,d0          ; Stash the frame count in d0.
        and.l #$ff,d0           ; Keep it between 0 and 255
        move.l d0,28(a0)        ; Store it as the rotation angle in in_buf.
        move.l #w16col,32(a0)   ; Store the rotation table in in_buf.
        move.l #0,gpu_mode      ; Select the web3d shader in donky.gas.
        lea equine2,a0          ; Load the GPU module in donky.gas.
        jsr gpurun              ; Run the selected GPU module.
        jsr gpuwait             ; Wait for the GPU to finish.
        jsr WaitBlit            ; Wait for the blitter to finish.
```

You may note at the end that we call a routine called `WaitBlit`. This is because in addition to computing the polygons that make up the 3D web, the GPU also 'blits' or writes its results to a buffer that will ultimately be used by the Object Processor to write the pixels to the screen. It is this dual operation that caused Minter a headache when attempting it from within the vertical sync interrupt. (My own suspicion is that there just wasn't enough time in the interrupt to do everything he wanted on the GPU and Blitter - so moving it here to the mainloop ensured that it would only be done opportunistically

and at the risk of missing a frame every now and then.)

But there is more to Tempest 2000 than starfields and webs so there must be plenty of other things being drawn in here too. At first it is not easy to see where though. The next section of the `draw_objects` routine is cryptic at best but on close inspection does contain some clues. As elsewhere, I've added comments to assist understanding:

```
        ; Finished drawing the web, now for the activeobjects.
n_wb:   move.l activeobjects,a6    ; Store activeobjects pointer in a6.
        bsr d_obj                  ; Run the d_obj routine below.
        bra odend                  ; And go to end when done.

        ; A loop for processing everything in 'activeobjects'.
d_obj:  cmpa.l #-1,a6              ; Have we reached the end of activeobjects?
        beq oooend                 ; If yes, skip to end.
        move 50(a6),d0             ; Is the object marked for deletion?
        beq no_unlink              ; If not, skip to no_unlink and draw it.

        ; This verbose section up until no_unlink is concerned entirely
        ; with deleting the dead object from the activeobjects list.
        move.l 56(a6),d1           ; Get address of previous object.
        bmi tlink                  ; If none, skip to tlink.
        move.l d1,a5               ; Move previous object to a5.
        move.l 60(a6),60(a5)       ; Make it invisible to the vsync interrupt.

tlink:  move #-1,50(a6)            ; Mark the object for deletion.
        move.l 60(a6),-(a7)        ; Stash address of next object in the stack.
        move d0,-(a7)              ; Stash d0 in the stack.
        move.l a6,a0               ; Store the object (a6) in a0.
        move 32(a6),-(a7)          ; Stash the player ownership tag.
        move (a7)+,d1              ; Store the player ownership tag in d1.
        move (a7)+,d0              ; Store the object type in d0.
        lea uls,a1                 ; Point a1 at uls.
        asl #2,d0                  ; Multiply it by 4.
        move.l -4(a1,d0.w),a1      ; Use d0 as an index in to 'uls'.
        jmp (a1)                   ; Jump to one of afinc, ashinc, pshinc.

        ; Object specific unlinking/deletion routines
uls:    dc.l afinc,ashinc,pshinc

        ; For unlinking a player shot object.
pshinc: tst d1                     ; Is it owned by a player?
        beq ulsh1                  ; If not, skip to ulsh1.
        add #1,shots+2             ; Make another shot available to player 1.
ulo:    move #1,locked             ; Lock activeobjects while we delete from it.
        bsr unlinkobject           ; Remove the object from activeobjects.
        clr locked                 ; Unlock the activeobjects list again.
        bra nxt_o                  ; Finished deleting, go the the next object.
ulsh1:  add #1,shots               ; Make another shot available to player 2.
        bra ulo
```

```
        ; For unlinking an enemy shot object.
ashinc: add #1,ashots           ; Make another shot available to enemies.

        ; For unlinking a default object.
afinc:  add #1,afree            ; Make slot available for a new activeobject.
        bra ulo

        ; Actually draw the object.
        ; No need to remove the object, just draw it.
no_unlink:
        lea draw_vex,a0         ; Get our table of draw routines.
        move 34(a6),d0          ; Is this object smaller than a pixel?
        bpl notpxl              ; If not, go to notpxl.
        move.l #draw_pel,a0     ; Use draw_pex for pixel-size objects.
        bra apal                ; Jump to the draw call.
notpxl: asl #2,d0               ; Multiply the val in d0 by 4.
        move.l 0(a0,d0.w),a0    ; Use it as an index into draw_vex.
apal:   move.l 60(a6),-(a7)     ; Store the index of next object in a7.
        jsr (a0)                ; But first call the routine in draw_vex.
        jsr gpuwait             ; Wait for the GPU to finish.
nxt_o:  clr locked              ; Clear 'locked' just in case.
nxt_ob: move.l (a7)+,a6         ; Put the index of next object back in a6.
        bra d_obj               ; Go to the next object.
```

This part of the `draw_objects` routine is concerned with processing a linked list called `activeobjects` using a loop that runs from `d_obj` to `bra d_obj` on the last line. This `activeobjects` list contains the detail for all objects that are active on the screen and require drawing by the GPU/Blitter. Some of the original (but sparse) code comments added by Minter suggest that this was initially conceived as a list for managing just the player and enemy bullets but it expanded over time.

Each object in the `activeobjects` list has the following structure:

Bytes	Description
0-4	Vector Object or Solid Object
4-8	X
8-12	Y
12-16	Z
16-20	Position on web.
20-24	Velocity
24-28	Acceleration & XY orientation
28-30	XZ Orientation & XZ orientation
30-32	Y Rotation
32-34	Z Rotation
34-36	Index into draw routine in `draw_vex`.
36-38	Start address of pixel data. & Delta Z
38-40	Y & Colour change value
40-42	Colour
42-44	Scale factor

The structure of objects in `activeobjects`.

Bytes	Description
44-46	Mode to climb, descend or cross rail
46-48	Size of Pixel Data & Duration.
48-50	Fire Timer
50-52	Marked for deletion
52-54	Whether an enemy or not.
54-56	Object Type
56-60	Address of Previous Object
60-64	Address of Next Object

The structure of objects in `activeobjects` continued.

So in the 64 bytes of each object we cover quite a bit of ground. There is colour, co-ordinate, motion, and orientation information. There's also detail that defines the behaviour of the object and most immediately relevant to what we are looking at here, an index into the draw routine to use for the object at bytes 34-36.

This index is a value that references the routine given at the appropriate position in the `draw_vex` array:

```
draw_vex:
   dc.l rrts,draw,draw_z,draw_vxc,draw_spike,draw_pixex,draw_mpixex,
    draw_oneup,draw_pel,changex
   dc.l draw_pring,draw_prex,dxshot,drawsphere,draw_fw,dmpix,dsclaw,dsclaw2
```

The draw routine can vary depending on the type of object. There's a distinct treatment

of vector object and objects that are solid polygons. Vector objects require the least work and if the header in bytes 0-4 indicates that it requires vector drawing only (for example the player's claw) then a vector draw in the GPU will suffice. The `draw` routine in the second item in `draw_vex` is the base routine for deciding whether a vector draw is sufficient or not, and most of the other routines make use of it in addition to the object-specific detail they manage:

```
; ********************************************************************
; draw
; The 'base' draw routine when processing 'activeobjects'. If a 'vector'
; draw is good enough, we just do that. Otherwise we add the object to the
; 'apriority' list for processing by 'drawpolyos'. The object to draw is
; passed in the a6 register.
; ********************************************************************
draw:
        move.l a6,oopss     ; Stash the header.
        ; If the header contains a negative value it's an index into the
        ; 'solids' list, otherwise it's the address of the vector data
        ; object we can draw directly with the gpu.
        move.l (a6),d0      ; Is the header value greater than zero?
        bpl vector          ; If yes, then a vector draw will suffice.

        ; Otherwise we need to do add this object to the 'apriority'
        ; list so that it can be drawn as a solid polygon.
        ; The 'apriority' list stores objects in the descending order
        ; of their Z co-ordinate. This ensures that nearer objects are
        ; painted in front of objects that are further away or 'behind' them
    .
        move.l fpriority,a0 ; Get a free priority object
        move.l a6,(a0)      ; Store the object we're drawing in a0.
        move.l 12(a6),d0    ; Put the object's Z value in d0.
        move.l d0,12(a0)    ; Put z in prior object
        move.l apriority,a1 ; Store pointer to apriority in a1.
        move.l a1,a2        ; And also in a2.

        ; Loop through the apriority list looking for the right place
        ; to store the current object.
chklp:  cmp.l #-1,a1        ; Is the list empty?
        bne prio1           ; If not, skip to prio1 otherwise..
        bra insertprior     ; Insert the object in the apriority list.

prio1:  cmp.l 12(a1),d0     ; Check Z value of object in list against ours.
        bge insertprior     ; If it's less, then insert our object here.
        move.l a1,a2        ; Otherwise stash that object.
        move.l 8(a1),a1     ; And store the next object in a1.
        bra chklp           ; loop until list end or next object in front of
     us
        rts                 ; return with object at right place in the list
```

Reading through the above we can see that if a `vector` draw won't do the job then the object gets added to a list called `apriority` and nothing else is done with it for now.

So what we are doing in our `d_obj` loop is a first pass through the `activeobjects` list, with any items on the list that require attention to the order in which they are painted, passed off to the `apriority` list. Notice that we don't mutate or update the objects in any way, we simply pass them over so the structure of the objects we described above remains unchanged. What we're doing with this list is ensuring that the Z co-ordinate position of each object is respected, so that objects nearer the viewer are painted after the ones that are further away. This ensures that nearer objects are painted over further objects.

Once we have finished this first run through the `activeobjects` list our next order of business is to process the `apriority` list we've populated. This is done in `drawpolyos` which we call right after we've finished with our first pass of `activeobjects`.

```
        ; We've finished our first pass of activeobjects.
odend:
        bsr showscore     ; Show the score.
        ; In Tempest Classic mode we don't need solid polygons.
        tst blanka        ; Are we doing solid polygons?
        beq odvec         ; If not, skip.
        bsr drawpolyos    ; if we are, draw them.
```

Drawing our polygons happens below. We iterate through the 'apriority' list, deleting items in it as we go, and calling the object-specific draw routine for each to render them to the screen.

```
; Routine for drawing all solid polygons.
; Process each object in the 'apriority' list. We remove each item after
; processing.The draw routine for each item is given by its index into
; the 'solids' array.
drawpolyos:
        move.l #192,xcent   ; Set 192 as X centre.
        move.l #120,d6      ; Set 120 as Y centre.
        add palfix2,d6      ; Adjust for PAL if necessary.
        move.l d6,ycent     ; Store it as Y centre.
        move.l apriority,a0 ; Get our 'apriority' list.

dpoloop:cmp.l #-1,a0        ; Have we reached the end of the list?
        beq rrts            ; If yes, bail out now.
        move.l (a0),a6      ; Get the index to 'solids'
        move.l (a6),d0      ; Store it in d0.
        move.l a0,-(a7)     ; Stash our current position in the list.
        bsr podraw          ; Go do object type draw
        jsr gpuwait         ; wait for gpu
        move.l (a7)+,a0     ; Get our current position in the list
        move.l 8(a0),-(a7)  ; Get the next position in the list
        bsr unlinkprior     ; Delete the current object.
        move.l (a7)+,a0     ; Move to the next position in the list.
        bra dpoloop         ; Loop until all objects drawn and unlinked
```

```
podraw: move.l #9,d4          ; Set X centre as 9.
        move.l #9,d5          ; Set Y centre as 9.
        neg d0                ; Negate d0.
        lea solids,a4         ; Get the 'solids' list.
        lsl #2,d0             ; Multiply our index by 2.
        move.l 0(a4,d0.w),a0  ; Get the draw routine address from 'solids'.
        move.l 4(a6),d2       ; Get the X position from our object.
        sub.l vp_x,d2         ; Subtract our X viewpoint.
        move.l 8(a6),d3       ; Get the Y position from our object.
        sub.l vp_y,d3         ; Subtract our Y viewpoint.
        move.l 12(a6),d1      ; Get the Z position from our object.
        sub.l vp_z,d1         ; Subtract our X viewpoint.
        bmi rrts              ; Skip if not visible.
        move 28(a6),d0        ; Get orientation of object.
        and.l #$ff,d0         ; Use only the least significant bytes.
        ; Because we jmp to the draw routine, when it returns, execution
        ; will go back to the caller of 'podraw' in dpoloop above.
        jmp (a0)              ; Call the object's draw routine.
```

You'll notice that we're choosing an object draw routine from something called a `solids` list. This is a list of draw routines and each object contains an index into this list for its specific routine. For example, a flipper object will have an index of 1 pointing ot the `cdraw_sflipper` routine in `solids`:

```
solids:
        dc.l rrts,cdraw_sflipper,draw_sfliptank,s_shot,draw_sfuseball
        dc.l draw_spulsar,draw_sfusetank,ringbull,draw_spulstank
        dc.l draw_pixex,draw_pup1,draw_gate,draw_h2hclaw,draw_mirr
        dc.l draw_h2hshot,draw_h2hgen,dxshot,draw_pprex,draw_h2hball
        dc.l draw_blueflip,ringbull,supf1,supf2,draw_beast,dr_beast3
        dc.l dr_beast2,draw_adroid
```

creating and updating objects

Drawing objects is all very well, but first we must create them. And once we've created them, we need to keep updating them. Enemies, bullets and even players have a habit of moving around.

As we mentioned previously, initially Minter tried doing both the drawing and the updating of objects in the same place, during the vertical sync interrupt that occurs when the Jaguar has finished painting a frame. Since it turned out there wasn't enough time to do both in the short interval that the vertical sync interrupt allows, he had to move the drawing work to `mainloop`. So when we first look at the code that remains in the vertical sync interrupt routine (which is called `Frame`), we find that is concerned

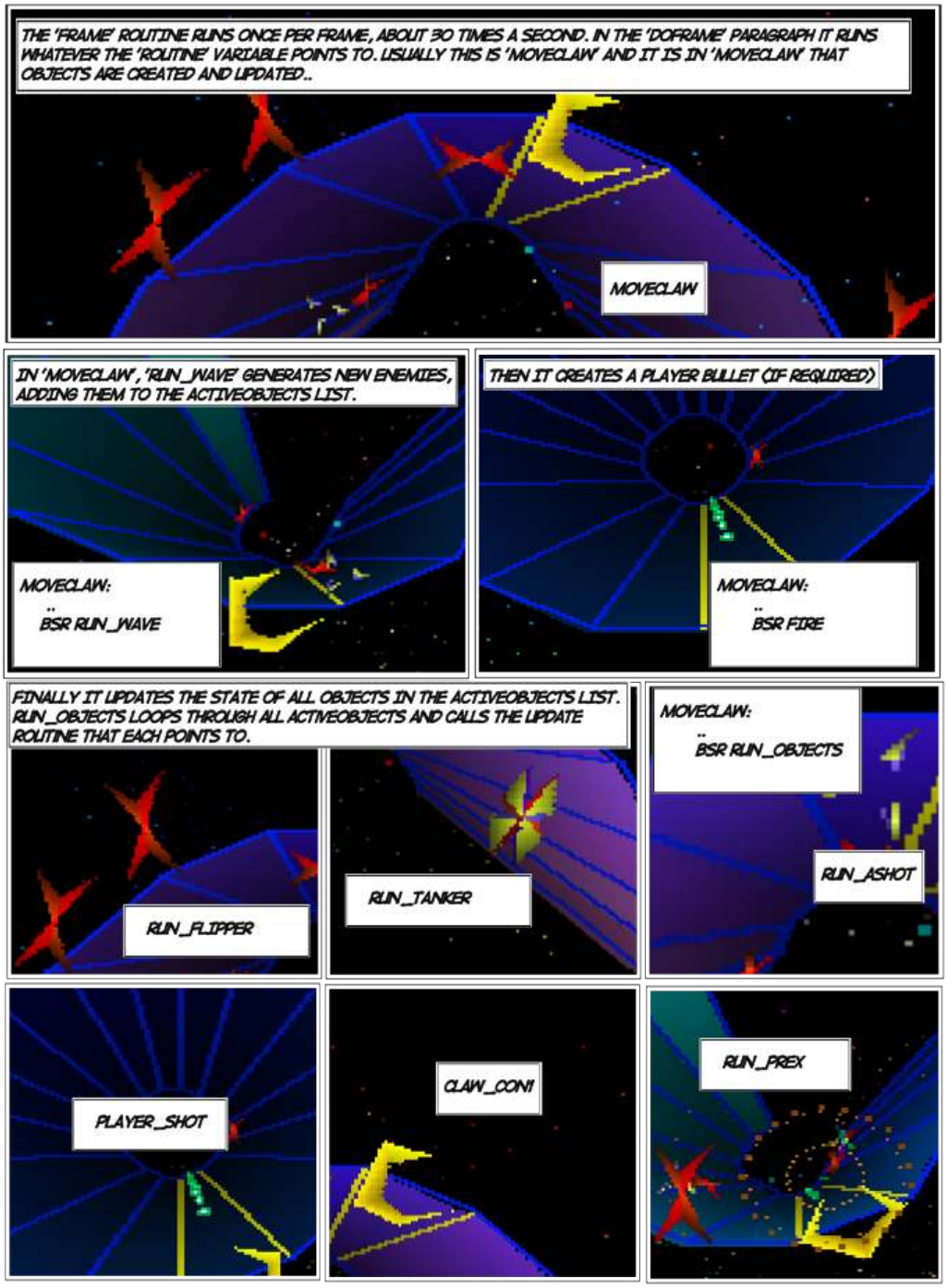
almost everything except keeping the `activeobjects` list fresh up-to-date. There is a lot of time spent on managing double-buffering, playing sounds, checking for player input and even some redundant logic for checking the rotary controller.

The updating of objects in the `activeobjects` list, and the creation of new ones, is instead buried in this little paragaph:

```
doframe:
        move.l routine,a0  ; Stash the current 'routine' in a0.
        jsr (a0)           ; Run it.
```

The routine that `routine` points to is updated regularly to reflect the current state of the game. There are a number of different routines it can point to, some of them more self-explanatory than others.

```
;  rrts, pauson, pausing, budb, pausoff, paws, zoom1, zoomto,
;  waitfor, zprev, znext, zshow, oselector, seldb, tunrun, text_o_run,
;  m7run, vicrun, failcount, viccount, txen, tgoto, rotate_web,
;  zoom3, zap_player, take_me_to_your_leader, snatch_it_away,
;  moveclaw, zoom2
```


THE 'FRAME' ROUTINE RUNS ONCE PER FRAME, ABOUT 30 TIMES A SECOND. IN THE 'DOFRAME' PARAGRAPH IT RUNS WHATEVER THE 'ROUTINE' VARIABLE POINTS TO. USUALLY THIS IS 'MOVECLAW' AND IT IS IN 'MOVECLAW' THAT OBJECTS ARE CREATED AND UPDATED..
MOVECLAW
IN 'MOVECLAW', 'RUN_WAVE' GENERATES NEW ENEMIES, ADDING THEM TO THE ACTIVEOBJECTS LIST.
MOVECLAW:
..
BSR RUN_WAVE
THEN IT CREATES A PLAYER BULLET (IF REQUIRED)
MOVECLAW:
..
BSR FIRE
FINALLY IT UPDATES THE STATE OF ALL OBJECTS IN THE ACTIVEOBJECTS LIST. RUN_OBJECTS LOOPS THROUGH ALL ACTIVEOBJECTS AND CALLS THE UPDATE ROUTINE THAT EACH POINTS TO.
RUN_FLIPPER
RUN_TANKER
MOVECLAW:
..
BSR RUN_OBJECTS
RUN_ASHOT
PLAYER_SHOT
CLAW_CON1
RUN_PREX

Every now and then, in fact more often than not, `routine` will point to a routine buried in the list above called `moveclaw`. Despite its name, it is `moveclaw` that looks after the bits we're interested in.

```
; ********************************************************************
; Despite its name this routine updates the state of most active game
; objects in addition to just the claw.
; Move the claw, generate waves, update some objects.
; run_wave: Manages all enemies for this frame, including generating
;           new ones.
; ********************************************************************
moveclaw:
        bsr vp_xform                ; do dynamic vp
        tst h2h                     ; Are we playing a head-to-head game?
        bne h2hrun                  ; If yes, go to h2hrun.
        bsr run_wave                ; Otherwise run the wave generater!
        tst chenable                ; Is cheating enabled?
        beq nofire                  ; If not, skip to nofire.

        ; Check if the Option button cheat was selected.
        cmp #-2,wave_tim            ; Is the wave finished already?
        beq nofire                  ; If so, skip to nofire.
        btst.b #1,pad_now+2         ; Is the option button pressed?
        beq noptcheat               ; If not skip to noptcheat.
        ; Otherwise cheat, get more points and go to next level.
        jsr douttah                 ; Add points and go to next level.
        bra nofire                  ; Update objects and clean up.

        ; Another cheat, pressing 6 lets us warp to next level.
noptcheat:
        move.l pad_now,d0           ; Store pressed buttons in d0.
        and.l #six,d0               ; Is the 6 button pressed?
        beq nofire                  ; If not skip to nofire.
        ; Otherwise the cheat has been selected.
        jsr swarp                   ; Warp to next level!

        ; Fire bullets, update all activeobjects, and update
        ; the player's viewpoint.
nofire: bsr fire                    ; Fire a bullet if necessary.
        ; The main routine for updating the state of all the active
        ; objects in the game and the overall game state.
        bsr run_objects             ; Update active objects
        bsr vp_set                  ; Update camera viewpoint.
        rts
```

The key elements in here are `run_wave` near the top, and `run_objects` near the bottom. The first creates new enemies if required, while the second does the important job of updating all the active objects in the game such as the flippers, bullets, and the player's claw itself. The illustration opposite hopefully gives you a general idea.

Both of these routines are quite substantial, so I'm not going to list them in full. Instead

we're going to concentrate on the section in `run_objects` that deals with updating the state of objects.

```
        ; Iterate through the activobjects list in a6 and run
        ; the appropriate routine in 'run_vex' for each object.
r_obj:  cmpa.l #-1,a6         ; Are we at the end of the activeobjects list?
        beq r_end             ; If so, clean up and exit.
        tst 50(a6)            ; Is the object marked for deletion?
        bmi r_end1            ; If so, exit and clean up.
        lea run_vex,a0        ; Store run_vex in a0
        ...
        move 54(a6),d0        ; Put the object type in d0.
        ..
r_o1:   asl #2,d0             ; Multiply it by 4.
        ; Put address of next object in stack in case this one is unlinked.
        move.l 60(a6),-(a7)   ; Push address of next object to stack.
        move.l 0(a0,d0.w),a0  ; Use d0 as index into run_vex.
        tst 52(a6)            ; Is the object an 'Enemy'?
        beq zokk              ; (don't count player bulls)
        bmi zokk              ; (or claw VULNERABLE flags)
        cmp 12(a6),d6         ; Check against previous nearest
        blt gokk              ; If less, don't swap.
        swap d6               ; Swap position of first 2 bytes with last 2.
        move 16(a6),d6        ; get nearest one's Lane #
        swap d6               ; Swap position of first 2 bytes with last 2.
        move 12(a6),d6        ; make this z nearest
gokk:   cmp 12(a6),d7         ; Is the z value the nearest so far?
        bgt zokk              ; If not, skip to zokk, otherwise..
        move 12(a6),d7        ; Store it as the new nearest.
zokk:   movem.l d6-d7,-(a7)   ; Stash values in the stack.
        move #3,diag          ; Update our diagnostic bytes to step 3..
        move.l a0,diag+4      ; .. used for debugging where we are..
        move.l a6,diag+8      ; .. in this routine.
        tst locked            ; Are we alreadyadding stuff to activeobjects?
        bne hack              ; If so, don't run the object update routine.
        jsr (a0)              ; Run the update routine selected from run_vex.
hack:   movem.l (a7)+,d6-d7   ; retrieve furthest-z
        move.l (a7)+,a6       ; Put the next object in a6.
        move #4,diag          ; Update our diagnostic bytes to step 4..
        move.l a6,diag+12     ; Store the object pointer in diagnostic bytes.
        bra r_obj             ; Move to the next object in activeobjects.
```

There is some accounting in here to ensure that the z position of the objects are updated depending on their current position relative to each other, but the main business is towards the very end where we call the appropriate update routine for the object given by its index into the `run_vex` list. This is stored at at byte 54 of the object and will tell us which of the routines in `run_vex` to use. For example a flipper object will have an index of 2 pointing to `run_flipper`. A player bullet will have an index of 1, pointing to `player_shot`.

```
run_vex:
    dc.l rrts,player_shot,run_flipper,run_zap,kill_shot,run_tanker,run_spike
    dc.l run_spiker,run_ashot,run_fuseball,blowaway,run_pulsar,oblow
    dc.l go_downc,go_downf,claw_con2,claw_con1,rez_claw,czoom1,czoom2
    dc.l pzap,rundroid,dc.l run_pixex,run_pspark,run_prex,run_pup,xshot
    dc.l run_gate,xr_pixex,run_fw,run_h2hclaw,run_mirr,run_h2hshot
    dc.l run_h2hgen,run_h2hball,oblow2,rumirr,refsht,refsht2,run_adroid
```

In fact `player_shot` is a good example of how straightforward these object update routines can be. As we can see below, it consists simply of rotating the object, updating its color to make if flash, incrementing its Z position to move it along the web, and finally deciding whether it has reached the end of its life and should be deleted.

```
; ********************************************************************
; player_shot: move a standard Tempest player's shot
; ********************************************************************
player_shot:
        add #8,28(a6)         ; Increment the XZ orientation to spin it.
        add.b #$30,41(a6)     ; Increment the color value to make it flash.
        move.l shotspeed,d0   ; Put shot speed in d0.
        ; Increment the z position by the shot speed, so that
        ; it moves along the web.
        add.l d0,12(a6)       ; Increment the z position.
        cmp #webz+80,12(a6)   ; Check if it has reached the end of the web.
        blt rrts              ; If not, we're done.
        move.l 24(a6),a1      ; Otherwise, point a1 at the XY orientation..
        clr.l (a1)            ; .. and clear it.
        move.l a6,a0          ; Point a0 at the object data.
        move #3,50(a6)        ; Mark the object for deletion.
        rts
```

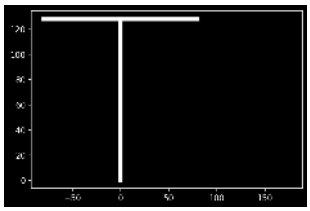
approaching logo process



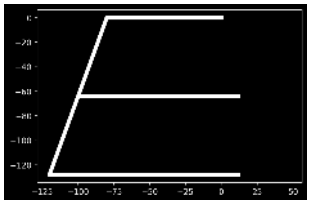
Figure .1: The Tempest Title Rainbow.

The *TEMPEST* title sequence is a thing of simple beauty. It cascades towards us from afar in a rainbow of shifting colours. It is a glimpse into the future of video games, where the illusion of three dimensions will become commonplace. Despite its rawness, and even its crudity, it beguiles us into accepting Tempest's world of near infinite depth. A world in which things that are so far away they are almost invisible quickly come up on us from the deep.

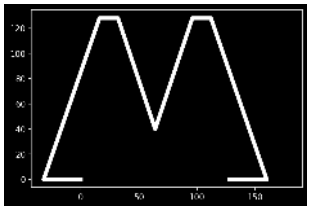
To understand how it was achieved we must start with the bare building blocks. The routines that get the letters onto the screen. There are five individual letters in Tempest, and we have a routine for each.



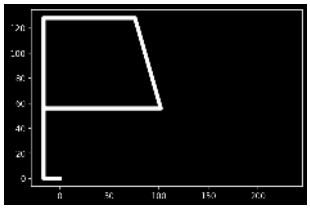
```
T:      VCTR 0,80,CB            ;T
        VCTR -50,0,0
        VCTR 0A0,0,CB
```



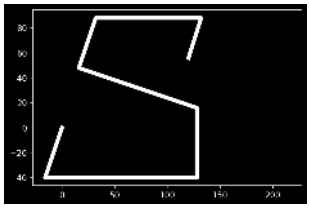
```
E:      VCTR -50,0,CB           ;E
        VCTR -14,-40,CB
        VCTR 70,0,CB
        VCTR -70,0,0
        VCTR -14,-40,CB
        VCTR 84,0,CB
```



```
M:      VCTR -20,0,CB           ;M
        VCTR 30,80,CB
        VCTR 10,0,CB
        VCTR 20,-58,CB
        VCTR 20,58,CB
        VCTR 10,0,CB
        VCTR 30,-80,CB
        VCTR -20,0,CB
        RTSL
```

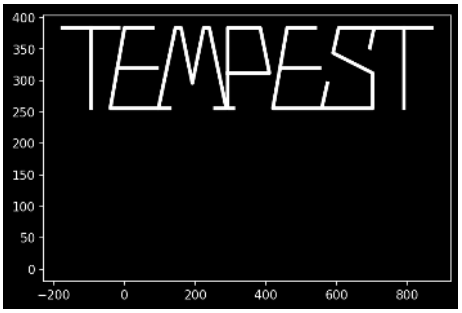


```
P:      VCTR -10,0,CB           ;P
        VCTR 0,80,CB
        VCTR 5C,0,CB
        VCTR 1A,-48,CB
        VCTR -76,0,CB
        RTSL
```



```
S:      VCTR -10,-28,CB         ;S
        VCTR 90,0,CB
        VCTR 0,38,CB
        VCTR -70,20,CB
        VCTR 10,28,CB
        VCTR 64,0,CB
        VCTR -0C,-20,CB
        RTSL
```

If you look closely you can see each letter is a routine in its own right. Each consists of the vector commands we encountered in 'cursors' and 'wells'. Each concludes with an RTSL statement that returns from the routine. This mean that in order to build the *TEMPEST* title image we can create a routine called VORLIT that writes it out in full.




```
VORLIT:
        CNTR
        VCTR -1B0,100,0
        JSRL T
        VCTR 60,0,0
        JSRL E
        VCTR 24,0,0
        JSRL M
        VCTR 34,0,0
        JSRL P
        VCTR 0F8,48,0
        JSRL E
        VCTR 16,28,0
        JSRL S
        VCTR 60,-60,0
        JMPL T
```

Equipped with VORLIT as a shorthand for drawing the TEMPEST title logo onto the screen we can use two other Vector Drawing instructions to alter the scale of the logo and colour of the logo when we draw it. If we adjust the scale and the colour as we go along we can create the effect of a cascading series of differently coloured titles receding into the distance.

Fortunately there are vector commands that will allow us to do all of this. Each command is a two-byte Op Code. There's one for setting the color to be used when drawing, one for setting the scale to use when drawing (allowing us to make the title big or small), and one for calling our VORLIT routine to actually draw the title logo in our selected colour and scale.

OpCode Description	OpCode	How We Use It.
New color/intensity	0x6000	000 will give the color and intensity.
New scale	0x7000	000 will give the scale to draw at.
Jump to subroutine	0xA000	000 can be populated with an address, e.g. VORLIT.

So we can imagine issuing a series of three vector commands such as the ones given below to draw our logo.

Op Code	Parsed Data	Written to Screen
7220	Set Scale: 220	
6800	New Color: White Intensity: 00	
AFA7	Jump to Routine: VORLIT	

If we repeat these commands, changing the color and reducing the scale as we go, we should expect to achieve the effect we're looking for. And this is exactly what we find in SCARNG. This routine builds the title rainbow by adding the three commands above to the vector list and it does this 19 times, adjusting the color and reducing the scale as it goes.

```
        .SBTTL  LOGO RAINBOW BUILDER
SCARNG:
    ; DRAW THE TEMPEST TITLE RAINBOW. THIS CONSISTS OF 19 ITERATIONS
    ; OF THE TITLE IMAGE IN EVER INCREASING DEPTH AND VARYING COLORS.
    ; A - CONTAINS THE LOW BYTE OF THE ADDRESS FOR VORLIT.
    ; X - CONTAINS THE HI  BYTE OF THE ADDRESS FOR VORLIT.
    STA PYL             ; STORE LOW BYTE OF VORLIT IN PYL
    STX PXL             ; STORE HI  BYTE OF VORLIT IN PYL
    LDA NEARY           ; PUT THE NEAREST ALLOWED POSITION IN A
    STA INDEX1          ; STORE IT IN INDEX1

    ; THIS IS THE LOOP THAT DRAWS THE TITLE RAINBOW, ONE TITLE
    ; AT A TIME, UNTIL WE REACH THE FURTHEST ALLOWED POSITION.
    ; WITH THE NEAREST POSITION STORED IN INDEX1 WE INCREMENT
    ; IT AT EACH ITERATION TO DRAW THE NEXT TITLE A LITTLE
    ; FURTHER AWAY UNTIL IT EQUALS FARY.
    BEGIN                           ; START OF THE LOOP

    ; FIRST WE SET THE SCALE OF THE TITLE IMAGE TO DRAW IN THIS
    ; ITERATION. THIS WILL DETERMINE HOW FAR AWAY IT SEEMS. TO DO
    ; THIS WE BASE IT ON THE DISTANCE VALUE IN INDEX1 AND CALL
    ; VGSCAL TO ADD A VECTOR COMMAND THAT SETS THE SCALE WE WILL
    ; DRAW THE TITLE AT. WE SET TWO SCALE TYPES, LINEAR AND BINARY.
    ; LINEAR SCALE FACTOR (0=FULL SIZE, FF=1/256 SIZE, FE=2/256, ETC.)
    ; BINARY SCALE FACTOR (0=FULL SIZE,1=1/2 SIZE, ETC.)
    LDA INDEX1          ; GET THE CURRENT DISTANCE VALUE.
    ASL                 ; SHIFT LEFT ONE BIT (I.E. MULTIPLY BY 2).
    ASL                 ; MULTIPLY BY 2 AGAIN.
    AND I,7F            ; AND WITH 7F TO GET OUR SCALE VALUE.
    TAY                 ; SET RESULT AS OUR LINEAR SCALE VALUE.
    LDA INDEX1          ; GET THE CURRENT DISTANCE VALUE.
    LSR                 ; DIVIDE BY 32 BY SHIFTING RIGHT FIVE TIMES.
    LSR                 ; TWO TIMES.
    LSR                 ; THREE TIMES.
```

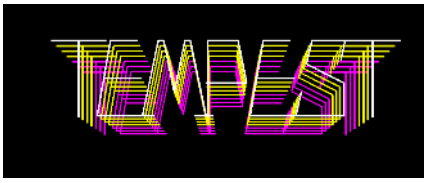
```
    LSR                 ; FOUR TIMES.
    LSR                 ; FIVE TIMES.
    JSR VGSCAL          ; ADD THE COMMAND SETTING BINARY AND LINEAR SCALE.

    ; NEXT WE SET THE COLOR WE WILL DRAW THE TITLE WITH. WE USE THE
    ; CURRENT DISTANCE TO CHOOSE THIS COLOR AND 'SET' IT BY CALLING
    ; VGSTAT. WE TURN THE DISTANCE INTO A COLOR BY SHIFTING IT RIGHT
    ; THREE BITS - THIS GIVES US A COLOUR VALUE THAT VGSTAT CAN USE.
    LDA INDEX1          ; GET THE CURRENT DISTANCE VALUE.
    CMP NEARY           ; IS IT EQUAL TO THE NEAREST POINT (I.E. THE TOP)?
    IFEQ                ; IF SO..
    LDA I,WHITE         ; .. PAINT IT WHITE.
    ELSE                ; OTHERWISE..
    LSR                 ; DIVIDE BY 8 BY SHIFTING LEFT THREE TIMES.
    LSR                 ; TWO TIMES.
    LSR                 ; THREE TIMES.
    NOP                 ; A NO-OP.
    AND I,7             ; MAKE THE RESULT BETWEEN 0 AND 7
    CMP I,7             ; IS IT EQUAL TO 7 (I.E. BLACK)?
    IFEQ                ; IF SO..
    LDA I,RED           ; USE RED FOR BLACK
    ENDIF
    ENDIF
    TAY                 ; STORE THE COLOR IN Y
    LDA I,68            ; SET THE COMMAND BITS FOR VGSTAT
    JSR VGSTAT          ; SET COLOR BY CALLING VGSTAT

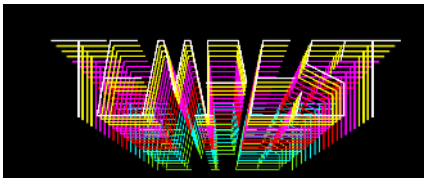
    ; NOW WE'RE READY TO PAINT THE TITLE. REMEMBER WE STORED THE ADDRESS
    ; TO VORLIT IN PYL AND PXL. SO VGJSRL WILL USE THAT TO DRAW THE VECTORS
    ; CONTAINING THE TEMPEST TITLE IN THE VORLIT ROUTINE.
    LDA PYL             ; LOAD THE LOW BYTE OF VORLIT TO A.
    LDX PXL             ; LOAD THE HI  BYTE OF VORLIT TO A.
    JSR VGJSRL          ; DRAW THE TEMPEST TITLE.

    ; NOW GET READY TO DRAW THE NEXT ITERATION. WE INCREMENT INDEX1
    ; BY 2 AND DRAW ANOTHER TITLE FURTHER AWAY, UNTIL WE REACH THE FARTHEST
    ; ALLOWED DISTANCE (FARY).
    LDA INDEX1          ; GET THE CURRENT DISTANCE VALUE.
    CLC                 ; CLEAR THE CARRY SO ADDING WILL HAPPEN OK.
    ADC I,2             ; ADD 2 TO THE CURRENT DISTANCE.
    STA INDEX1          ; AND STORE IT IN INDEX1.
    CMP FARY            ; IF IT IS EQUAL TO FARY, EXIT..
    CSEND               ; OTHERWISE LOOP AND DRAW ANOTHER TITLE.
```

We can see how the commands mutate and slowly build up the rainbow piece by piece for all 19 iterations in the table that follows.

Vector Commands	Command Meaning	Result on Screen
7228 6801 AFA7	Set Scale: 228 New Color: Yellow Intensity: 00 Jump to Routine: VORLIT	
7230 6801 AFA7	Set Scale: 230 New Color: Yellow Intensity: 00 Jump to Routine: VORLIT	
7238 6801 AFA7	Set Scale: 238 New Color: Yellow Intensity: 00 Jump to Routine: VORLIT	
7240 6802 AFA7	Set Scale: 240 New Color: Magenta Intensity: 00 Jump to Routine: VORLIT	
7248 6802 AFA7	Set Scale: 248 New Color: Magenta Intensity: 00 Jump to Routine: VORLIT	
7250 6802 AFA7	Set Scale: 250 New Color: Magenta Intensity: 00 Jump to Routine: VORLIT	

Vector Commands	Command Meaning	Result on Screen
7258	Set Scale: 258	
6802	New Color: Magenta Intensity: 00	
AFA7	Jump to Routine: VORLIT	
7260	Set Scale: 260	
6803	New Color: Red Intensity: 00	
AFA7	Jump to Routine: VORLIT	
7268	Set Scale: 268	
6803	New Color: Red Intensity: 00	
AFA7	Jump to Routine: VORLIT	
7270	Set Scale: 270	
6803	New Color: Red Intensity: 00	
AFA7	Jump to Routine: VORLIT	
7278	Set Scale: 278	
6803	New Color: Red Intensity: 00	
AFA7	Jump to Routine: VORLIT	
7300	Set Scale: 300	
6804	New Color: Cyan Intensity: 00	
AFA7	Jump to Routine: VORLIT	

Vector Commands	Command Meaning	Result on Screen
7308 6804 AFA7	Set Scale: 308 New Color: Cyan Intensity: 00 Jump to Routine: VORLIT	
7310 6804 AFA7	Set Scale: 310 New Color: Cyan Intensity: 00 Jump to Routine: VORLIT	
7318 6804 AFA7	Set Scale: 318 New Color: Cyan Intensity: 00 Jump to Routine: VORLIT	
7320 6805 AFA7	Set Scale: 320 New Color: Green Intensity: 00 Jump to Routine: VORLIT	
7328 6805 AFA7	Set Scale: 328 New Color: Green Intensity: 00 Jump to Routine: VORLIT	
7330 6805 AFA7	Set Scale: 330 New Color: Green Intensity: 00 Jump to Routine: VORLIT	

We now have a sense of how to draw the rainbow, but in a way this is just one iteration of the effect. As we know from experience, the logo gets gradually larger approaching us from the black depths of the screen. The way this is achieved is by running the process we've described above over and over, while adjusting the values that define the nearest and farther point on the screen. These two values (NEARY and FARY) control the overall maximum and minimum scale that we draw the logo with in SCARNG. So by incrementing them in the routine LOGPRO below, then calling SCARNG to draw the logo with these values as their limits, we get it to draw the rainbow again, but this time slightly larger. If we do this every frame or two, the logo will appear to get nearer and nearer to us. Since the color we draw with is a function of distance we are drawing at this has the added bonus of also gradually changing the colours each instance of the title is drawn with: creating the cascading effect, as opposed to a rainbow with static colors.

```
    .SBTTL  LOGO - LOGO RAINBOW ; APPROACHING LOGO PROCESS
LOGPRO:
    ; THE LOGO RAINBOW CONSISTS OF 19 ITERATIONS OF THE TEMPEST
    ; LOGO IN VARYING COLORS. EACH CALL TO THIS ROUTINE MOVES
    ; THE LOGO RAINBOW GRADUALLY CLOSER TO THE VIEWER.

    ; HERE WE SET 'TEMPEST' AS ADDRESS OF PICTURE TO DRAW IN
    ; IN A AND X REGISTERS, AND THEN DRAW THE FULL RAINBOW WITH IT.
    LDAH VORLIT+1       ; LOW BYTE OF VORLIT ADDRESS GOES IN A
    LXL VORLIT          ; HI BYTE OF VORLIT ADDRESS GOES IN X
    JSR SCARNG          ; DRAW RAINBOW OF LOGO

    ; CALCULATE POSITION OF RAINBOW FOR NEXT TIME.
    ; NEARY IS THE NEAREST POINT DEPTH-WISE TO DRAW THE LOGO.
    ; FARY IS THE FARTHEST POINT DEPTH-WISE TO DRAW THE LOGO.
    LDA NEARY           ; PUT NEARY IN A
    CMP I,30            ; HAS IT REACHED 48 (THE NEAREST POINT) YET?
    IFCS                ; IF NOT..
    SBC I,1             ; NO. BRING IT CLOSER BY 1 PIXEL.
    STA NEARY           ; STORE NEW VALUE IN NEARY
    ENDIF
    ; CHECK IF WE'VE REACHED THE FINAL PHASE WHERE THE
    ; RAINBOW REDUCES IN DEPTH.
    CMP I,80            ; HAS THE FAR POINT REACHED ITS LIMIT OF 128?
    IFCC                ; IF SO BRING FAR PT. CLOSER
    LDA FARY            ; BY LOADING FARY TO A
    SEC                 ; SET THE CARRY
    SBC I,1             ; REDUCE IT BY 1
    CMP NEARY           ; IS IT THE SAME AS THE NEAR POINT YET?
    IFCC                ; IF SO SET NEARY AND FARY TO BE THE SAME BY..
    LDA NEARY           ; LOADING NEARY TO A
    ENDIF
    STA FARY            ; STORE A AS THE NEW VALUE FOR FARY
    ENDIF
    RTS                 ; WE'RE DONE FOR THIS ROUND, RETURN.
```

cry if i want to

Before the dawn of the three horsemen *png*, *gif* and *jpg*, computer people were free to invent their own schemes for storing images. In 1991 technology was still rudimentary enough that console and computer developers were largely pre-occupied with waging something resembling a *Color War*. So when Atari sat down to brew its own method of storing and representing image data one of its overriding concerns was *how many colours can we fit into this thing*.

The **CRY** file format is the result. 'Format' in this case is a slightly grandiose term for: here is a list of bytes in a file, each pair of bytes in the list represents a color. Since a single byte has 256 possible values, two bytes has 65,536 possible values. So welcome to our system that has a state-of-the-art 65,536 possible colours. Well, not quite. Let's see why.

claw says



Good luck figuring out what CRY is an acryonym of. Maybe it refers to the three axes of the 'colour space': cyan, red, and yellow. Maybe it refers to the three components contained in each byte pair: color, radiance, and luminosity?

In your innocence dear reader, you may expect that this color scheme is as simple as assigning a color to each of the possible values and considering ourselves done for the day. But that is nowhere near complex enough to generate true job sastifaction. Instead, we must declare something called a 'colour space' and declare co-ordinates along an X and Y axis. Even better, since this is not the 1980s anymore, we ought to thinking in three dimensions and throw in a Z axis while we're at it.

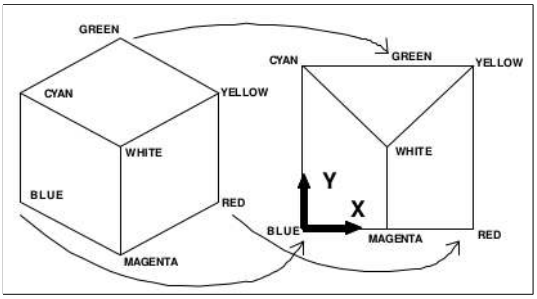


Figure .1: This kind of thing. This is what we want.

With just two bytes (or 16 bits) to work with we have to come up with a way that is suitably complicated for representing all our colours along so many co-ordinate axes. So let's imagine an 8x8x8 colour cube with something resembling white at its top corner and all the colours of the spectrum radiating out of it.

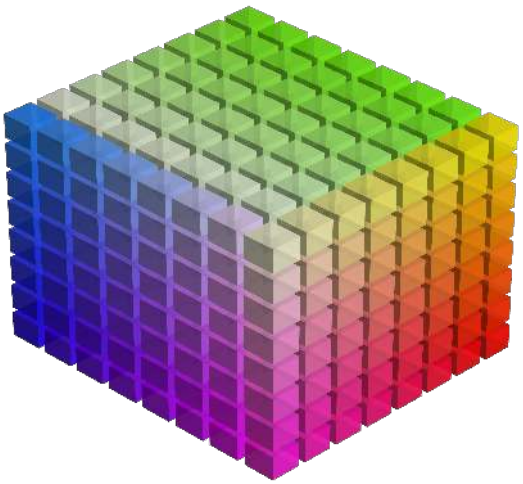


Figure .2: This is not quite right, but you get the idea.

Now if we project this three dimensional object onto a flat surface, with our white top corner at the centre, we will have a 16x16 square with 256 different colours:

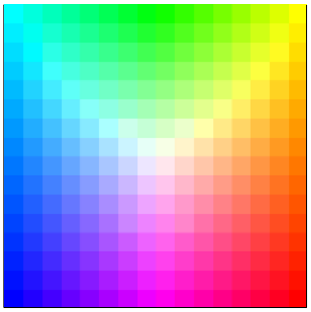


Figure .3: 3d, but in 2d: it's the 1990s.

Since 256 is exactly the number of permutations in a single byte we have lit upon a use-case for the first of our two bytes.



Figure .4: The first 8 bits now have a purpose. The first four are our X axis on the square, the second our Y axis.

This is all very nice, we have 256 colors going on and a whole other 8 bits to do something with. After thinking about it a bit, we don't have too many options. In fact we have one option: use our extra 8 bits to vary each of our 256 colors in 256 different ways. We can make this twiddle factor sound intelligent by calling it 'luminosity'. So depending on how much light we think is being cast on a surface with the color in question we can use our final 8 bits to dial its brightness up or down.

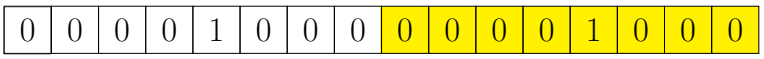


Figure .5: The 8 bits representing the luminosity/intensity of the color chosen by the first 8 bits.

To get an idea of the effect this has in practice we can visualize our little color square repeating across a much bigger square with ascending values of luminosity starting at 0 and ending at 255.

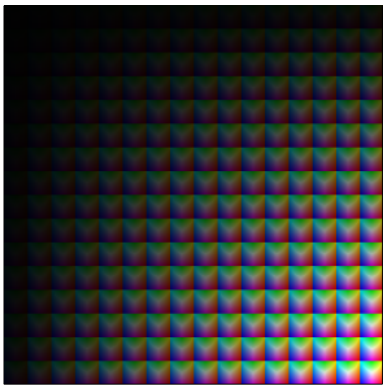


Figure .6: A big square with all our little squares in ascending luminosity.

That's a lot of dark. We better not wear shaded spectactles while playing games on this thing. Is there really 65,536 colors in there, it seems like we have an awful lot of black going on. Let's try something and see what we get if we ignore all color values that appear more than once.

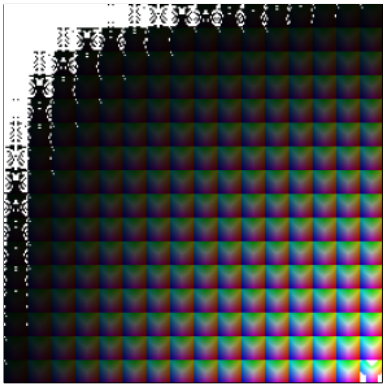


Figure .7: Of course the 65,536 colors is a lie. I've counted them so you don't have to: there are 63,762 unique colours.

There are some strange little fractal shapes in the black stuff there. In the bottom right hand corner we can see that we eventually run out of luminosity, our last little sqaure can only muster up 120 or so new colours.

the tempest files

Now that we know what the bytes in a CRY image mean let's take a peek at the 6 image files in the Tempest 2000 sources to see what some of these bytes look like in practice. (Don't worry we'll be looking at all of these files in detail under one pretext or another during the course of the rest of this book.) The seven image files are given slightly random, but very Minter-ish names: `beasty3.cry`, `beasty4.cry`, `beasty5.cry`, `beasty6.cry`, `beasty7.cry` and `beasty8.cry`.

claw says



Oh and let's not forget `beasty8-xtra.cry`. This is a slightly larger version of `beasty8.cry` included with the sources but for some reason is not used. There is also a story to tell about `beasty3.cry` but we will come to that later.

Remember there is nothing else in these files but the series of byte-pairs. There's no information about how big the image is or what its dimensions are. It really is just a list. To cope with this fact the game hard-codes its own expectation that the images are 320 pixels wide (i.e. the pixel width of the Jaguar's screen) and fixes the address that each image at will be loaded at. It does this early on in the declaration of constants at the top of the main file containing all the game's code (`yak.s`):

```
pic             EQU $820000             ; beasty3.cry
pic2            EQU pic+$1f400          ; beasty4.cry
pic3            EQU pic2+$1f400         ; beasty5.cry
pic4            EQU pic3+$25800         ; beasty6.cry
pic5            EQU pic4+(640*128)      ; beasty7.cry
pic6            EQU pic5+(640*200)      ; beasty8.cry
```

Each image has been loaded to a specific position in memory and the game is instructed exactly where each one is. While `beasty3.cry` is loaded to `$820000` (and assigned the variable name `pic`), `beasty4.cry` is loaded to the address 128,000 (`$1f400`) bytes after that and assigned the name `pix2` because 128,000 bytes is how large `beasty3.cry` is. You get the idea hopefully.

As you can see in the above listing extract some of the images are larger than others so need more memory, but in all cases the assumption of treating every image as 320 pixels wide (640 bytes, since we have 2 bytes per pixel) holds.

Before passing on let's take a look at the contents of `beasty7.cry`.



Figure .8: `beasty7.cry`, packed with goodness.

Looks great, you have to agree. We're in for some fun.

tracing the beam

Tempest does its drawing to the screen using a piece of hardware called the *Analog Vector Generator* (AVG). We can make this piece of kit draw stuff by supplying it with operation codes (opcodes) that it treats as instructions for drawing lines to the screen and updating their colour, position, and size. Most opcodes are a pair of bytes, since this is usually enough to convey the necessary information.

Description	OpCode Hex	Bitmap
Draw relative vector.	`0x0___ ____`	`000YYYYY YYYYYYYY IIIXXXXX XXXXXXXX`
	`0x1___ ____`	`000YYYYY YYYYYYYY IIIXXXXX XXXXXXXX`
Halt	`0x2___`	`00100000 00000000`
Draw short relative vector	`0x4___`	`010YYYYY IIIXXXXX`
	`0x5___`	`010YYYYY IIIXXXXX`
New color/intensity	`0x6___`	`0110URGB IIIIIIII`
New scale	`0x7___`	`0111USSS SSSSSSSS`
Center	`0x8___`	`10000000 00000000`
Jump to subroutine	`0xA___`	`101AAAAA AAAAAAAA`
	`0xB___`	`101AAAAA AAAAAAAA`
Return from subroutine	`0xC___`	`11000000 00000000`
Jump to new address	`0xE___`	`111AAAAA AAAAAAAA`
	`0xF___`	`111AAAAA AAAAAAAA`

The table above lists all the known available opcodes for the AVG at the time that *Tempest* was released. As you can see, various values like the X and Y position, the color intensity (`I`), scale (`S`), and memory address (`A`) are encoded at the bit level in order to pack them into the available byte pair. This means that there is a limit, for example, on the values we can provide for things like X and Y positions: 4 bits in the 'Draw Short Relative Vector' instruction allows values between 0 and 16 with an additional fifth bit

to indicate a positive or negative value.

So equipped with this table as our cheat sheet, let's imagine we want to start drawing something on the screen. First we will move the beam to the centre of the screen:

Description	Op Code	Bitmap
Center	`0x8000`	`10000000 00000000`

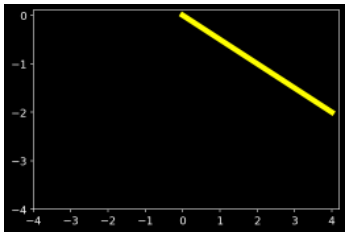
So our opcode instruction is a two byte `0x8000`. This thing we want to draw, we're going to make it yellow. So let's issue a colour command:

Description	Op Code	Bitmap
	`0x6___`	`0110URGB IIIIIIII`
Set Color to Yellow	`0x68C1`	`01101000 11000001`

In practice we find that our colour opcode always starts with `0x68C`. In this instance we add a 1 to set the color to yellow. To set it to red we would have used 3 (i.e. `0x68C3`). To set it to blue we would have used 7 (i.e. `0x68C7`). The values are abitrary, but you get the idea.

Now we're ready to draw. Let's project a line 4 units to the right and 2 units down:

Description	Op Code	Bitmap
	`0x5___`	`010YYYYY IIIXXXXX`
Draw vector to (4,-2)	`0x5EC4`	`01011110 11000100`
		`5 E C 4`



Now you might notice something that gives you pause. Where we have added the 4 in our vector opcode is relatively obvious (at the end of `5EC4`). But why has the -2 become an E? Shouldn't it be `52C4` instead?

The answer is *two's complement*. This is the way we represent negative numbers. The first bit under the first Y above tells us that what we have is a negative number (it's why the opcode begins with a 4 instead of a 5). But for the remaining 4 bits of our Y value the following mapping applies:

Decimal	Hexadecimal
-1	F
-2	E
-3	D
-4	C
-5	B
-6	A
-7	9
-8	8
-9	7

Negative Values

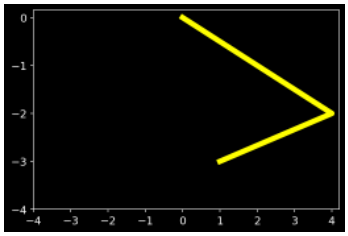
Decimal	Hexadecimal
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9

Positive Values

As you can see, to represent negative integers with *two's complement*, we 'start again' at F and count downwards. So to represent a value of -2 we use E.

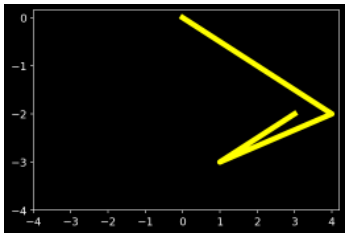
With this in mind we can start drawing again. This time we want to move left 3 units and down 1 unit. So we issue the opcode command `0x5FDD` to the vector generator:

Description	Op Code	Bitmap
	`0x5___`	`010YYYYY IIIXXXXX`
Draw vector to (-3,-1)	`0x5FDD`	`01011111 11011101`
		`5 F D D`



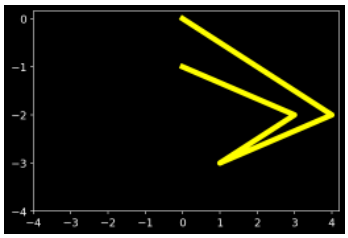
Next we move right 2 units and up 1 unit:

Description	Op Code	Bitmap
	`0x4___`	`0100YYYY IIIXXXXX`
Draw vector to (2,1)	`0x41C2`	`01000001 11000010`
		`4 1 C 2`



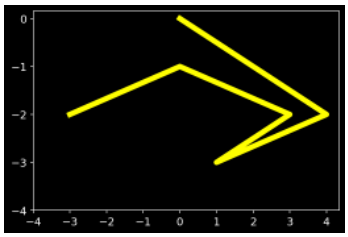
Then left 3 units and up 1 unit:

Description	Op Code	Bitmap
Draw vector to (-3,1)	0x4___ 0x41DD	0100YYYY IIIXXXXX 01000001 11011101 4 1 D D



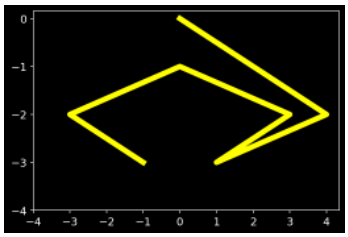
Things are starting to take shape. Can you guess what we're drawing yet? Our next line will go left 3 units and down 1 unit:

Description	Op Code	Bitmap
Draw vector to (-3,-1)	0x5___ 0x5FDD	010YYYYY IIIXXXXX 01011111 11011101 5 F D D

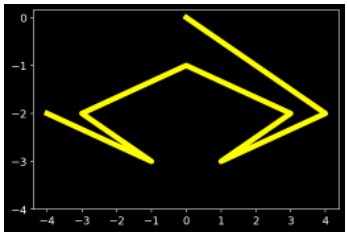


OK I think you can see how we're building up our picture here, so let's get it over with:

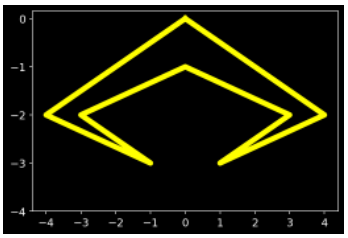
Description	Op Code	Bitmap
Draw vector to (2,-1)	0x5___ 0x5FC2	010YYYYY IIIXXXXX 01011111 11000010 5 F C 2



Description	Op Code	Bitmap
Draw vector to (-3,1)	0x4___ 0x41DD	0100YYYY IIIXXXXX 01000001 11011101 4 1 D D



Description	Op Code	Bitmap
	0x4___	0100YYYY IIIXXXXX
Draw vector to (4,2)	0x42C4	01000010 11000100
		4 2 C 4



And there it is. We've drawn the 'lives left' symbol used in *Tempest* to indicate the number of player lives remaining. Not bad for a handful of bytes.

Before signing off, let's look at the dense little assembler macro used to construct these opcode values. This function, called VCTR, takes a pair of X and Y values, e.g. (4,2), as well as a color intensity value, and converts them into the packed two-byte opcode. It is an object lesson in old-school bit manipulation written by Ed Logg in 1979 to support the original version of the Analogue Vector Generator. As we read more about *Tempest* we'll find that Dave Theurer's source code makes heavy use of it when drawing player and enemy to the screen.

```
;
;       VCTR - DRAW VECTOR INSTRUCTION
;       THIS INSTRUCTION DRAWS A VECTOR ON THE DISPLAY AREA
;       RELATIVE TO THE PREVIOUS BEAM POSITION BEFORE THE
;       INSTRUCTION IS EXECUTED.  DX IS THE CHANGE IN BEAM X
;       POSITION; DY IS THE CHANGE IN BEAM Y POSITION; ZZ
;       SPECIFIES THE BEAM INTENSITY (0 THROUGH 7., 0 IS
;       NO INTENSITY, 7. IS BRIGHTEST INTENSITY).
;
        .SBTTL  VCTR
        .MACRO  VCTR DX,DY,ZZ
        ...1=DX                     ; STORE X ARG IN ...1
        ...2=DY                     ; STORE Y ARG IN ...2
        .IF      LT,...1            ; IF NEGATIVE X
        ...1=-DX                    ; CONVERT TO TWO'S COMPLEMENT.
        .ENDC                       ; END IF
        .IF      LT,...2            ; IF NEGATIVE Y
        ...2=-DY                    ; CONVERT TO TWO'S COMPLEMENT.
        .ENDC                       ; END IF
        ...5=...1!...2              ; OR X AND Y TOGETHER IN ...5

        .IF     NE,...1+...2        ; IF X AND Y ARE 0 - SKIP SHORT VECTOR
        .IF      EQ,...5&^H0FFE1    ; IF WE HAVE AN X OR Y VALUE..
        ; DRAW IT WITH A SHORT VECTOR OPCODE.
        ; THE BITMAP FOR THIS IS: 010YYYYY IIIXXXXX
        ; BRIEF EXPLANATION OF THE BIT MANIPULATIONS:
        ; ^H4000 - HEX 4000 IS OUR BASE FOR SHORT VECTOR OPCODE.
        ; <ZZ*^H20> - MOVE THE INTENSITY VALUE TO THE III SLOT ABOVE.
        ; <DX/2&^H1F> - MOVE THE X VALUE TO THE END OF THE 2ND BYTE.
```

```
        ; <DY*^H80&^H1F00> - MOVE THE Y VALUE TO THE END OF THE 1ST BYTE.
        .WORD   ^H4000+<ZZ*^H20>+<DX/2&^H1F>+<DY*^H80&^H1F00>
        .MEXIT                      ; EXIT MACRO.
        .ENDC                       ; END IF
        .ENDC                       ; OTHERWISE..
        ; DRAW AS A RELATIVE VECTOR, WHICH ALLOWS FOR LARGER VALUES.
        ; THE BITMAP FOR THIS IS 4 BYTES LONG:
        ;   000YYYYY YYYYYYYY IIIXXXXX XXXXXXXX
        ; DY&^H1FFF - PUT THE Y VALUE IN 2ND BYTE OF 1ST BYTE PAIR.
        ; <ZZ*^H2000> INTENSITY VALUE AT START OF 1ST BYTE IN 2ND PAIR.
        ; <DX&^H1FFF> X VALUE IN 2ND BYTE OF 2ND BYTE PAIR.
        .WORD   DY&^H1FFF,<ZZ*^H2000>+<DX&^H1FFF>
        .ENDM                       ; END MACRO
```

what is blitting

Blitting is the process of writing pixels to the screen. A bltter is a *bit block processor*. When we *blit* an image to the screen we are copying a specific block of image data from storage to the screen where the user can see it.

This short extract from the routine that looks after the 'High Score' table in Tempest 200 gives you an idea of how easy using the blitter can be. All we have to do is point at the position in RAM containing our pixel data (in this case the pixels from `beasty7.cry` that we loaded to `pic5`), tell the blitter the x and y position we want to copy from, the width and height we want to copy, and the x and y position on the screen we want to copy to.

```
; Paint the 'Top Guns' graphic
move.l #pic5,a0    ; file images/beasty7.cry, our source data.
move.l #screen3,a1 ; the destination we're copying to.
move #1,d0         ; x position in pic5
move #1,d1         ; y position in pic5
move #223,d2       ; width of block to copy
move #79,d3        ; height of block to copy
move #70,d4        ; x pos of destination
move #10+8,d5      ; y pos of destination
jsr CopyBlock
```

So in the above we specify a block in the image data from *beasty7.cry* opposite from the top-left corner (x:1, y:1) with a width of 223 pixels and a height of 79 pixels. We then tell it that we want to write this to x position 70 and y position 18 on the screen, in other words somewhere just offset from the top left of the screen. This is the piece of the image we are selecting for copying:



Figure .1: The bit of `beasty7.cry` we've selected for copying.

If we look more closely at the snippet I extracted above we can wee that We seem to be doing this 'specifying' by moving our values into things called `a0`, `a1`, `d0`, `d1`, and so on. These are 'registers'. An old computer such as the Motorola 68K CPU in the Atari Jaguar has only so many fingers and thumbs it can use for counting, and these are them. `a0`, `a1` and so on are 'address registers'. So if we have the address of a piece of code or data we want the CPU to use an 'address register' is where we store them. If we have some 'values' on the other hand, i.e. some numbers, then the 'data registers' `d0`,`d1` are where we put them.

As you read on you'll find it's a common pattern in Tempest 2000's source code to load up a bunch of registers with bits and pieces and then call a function (or 'routine' in assembler parlance) that expects those registers to contain the values it should work with. If this sounds familiar, and that we are essentially using these registers as parameters to a function, then you are correct: that's exactly what we're doing.

The simple operation we're performing here is to copy a portion of the `beasty7.cry` image file to the screen. The `CopyBlock` routine opposite just needs to know the position and dimensions of the portion we're copying and the position on the screen we want to place it. To achieve this the snippet of game code on the previous page sets up the registers as follows:

Register	Type	Value	Description
`a0`	Address	`pic5`	Source Screen
`a1`	Address	`screen3`	Destination Screen
`d0`	Data	`1`	X Position in Source Screen
`d1`	Data	`1`	Y Position in Source Screen
`d2`	Data	`223`	Width to Copy
`d3`	Data	`79`	Height to Copy
`d4`	Data	`70`	X Position in Destination Screen to Copy to
`d5`	Data	`18`	Y Position in Destination Screen to Copy to

Setting up the registers for `CopyBlock`.

```
CopyBlock:
;
; Copy from screen at a0 to screen at a1
; d0/d1=origin of sourceblock
; d2/d3=width and height of block to copy
; copy from blitter a1 to a2.
; d4/d5=destination XY
;
; This simple routine will assume both screens are the same width
; Using this blitter is a piece of piss.

  move.l #PITCH1|PIXEL16|WID320|XADDINC,d7
  move.l d7,A1_FLAGS  ;a1 (Source) Gubbins

  move.l #PITCH1|PIXEL16|WID384|XADDPIX|YADD1,d7
  move.l d7,A2_FLAGS  ;a2 (Dest) Gubbins

  move d3,d7
  swap d7
  move d2,d7
  move.l d7,B_COUNT   ;set inner and outer loop counts

  move d1,d7
  swap d7
  move d0,d7
  move.l d7,A1_PIXEL  ;origin of source

  move d5,d7
  swap d7
  move d4,d7
  move.l d7,A2_PIXEL  ;origin of destination

  move.l #0,A1_FPIXEL


  move.l #$0001,A1_INC
  move.l #$0,A1_FINC

  move #1,d7
  swap d7
  move d2,d7
  neg d7
  move.l d7,A1_STEP
  move.l d7,A2_STEP    ;set loop steps

  move.l a0,d7
  move.l d7,A1_BASE
  move.l a1,d7
  move.l d7,A2_BASE    ;set screen window bases

  move.l #SRCEN|UPDA1|UPDA2|DSTA2|LFU_A|LFU_AN,d7
  move.l d7,B_CMD
  bra WaitBlit
```

claw says



The source and destination are described as 'screens' in the Tempest 2000 code. This is a useful way of thinking of our pixel data, which although it is just a list of bytes, always represents a screen of a certain height and width. While in practice we are copying a section from an image to the screen here, we can think of it as copying from one screen to another - with the destination screen the one being shown to the player.

The mechanics of how the `CopyBlock` routine uses this information to actually copy a segment of our image to the screen are not obvious to the unintiated. This is because it involves a lot of precise specification of many different instructions to the Blitter (Bit Block Processor) hardware. The amount of set up involved makes our preparation of the `a0-a1` and `d0-d5` registers look modest in comparison. There is a quite involved amount of bit-twiddling required to get the Blitter set up the way we want it. Using this routine may be a 'piece of piss', but writing it was fairly painstaking. For example:

```
    move.l #PITCH1|PIXEL16|WID320|XADDINC,d7
    move.l d7,A1_FLAGS  ;a1 (Source) Gubbins
```

Here we are formulating some of the parameters we want the Blitter to use when it copies the pixels from our source. We're combining the parameters into a single 64-bit value, storing it in the `d7` data register and then writing that to the Blitter's `A1_FLAGS` register. This is the register it will read to figure out some of the things we want to do with the copy operation.

Parameter	Bits				Hex	Description
PITCH1	00000000	00000000	00000000	00000000	00000000	Pixel data has no gaps
PIXEL16	00000000	00000000	00000000	00110000	00000020	Pixel Size 16 bits
WID320	00000000	00000000	01000010	00000000	00004200	Screen Width of 320
XADDINC	00000000	00000011	00000000	00000000	00030000	Add 1 to X at each pass
A1_FLAGS	00000000	00000011	01000010	00110000	00034220	Blitter's A1 Flags Register

Combining our parameters into the Blitter's A1 Flags Register.

There is a lot more of this kind of thing in there and it would definitely be too tedious to itemize each operation here in detail. But there is at least one other pattern worth pointing out in the routine, exemplified by the following snippet that copies the X (`d0`) and Y (`d1`) positions in *beasty7.cry* that we are telling the blitter to use as its origin for copying:

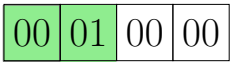
```
    move d1,d7
    swap d7
```

```
    move d0,d7
    move.l d7,A1_PIXEL  ;origin of source
```

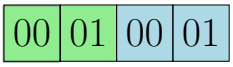
Here the A1_PIXEL register we're writing to (like all our address and data registers) is 32-bits (or 4 bytes) long. This means it can contain both our X and Y values which will be at most 2 bytes long each. So to get our d0 and d1 values in there we first write the d1 value to d7 giving:



Next, we swap its position:



Now we write d0 to d7:



The result is our X and Y values are both successfully stowed in d7, which we can now write in its totality to A1_PIXELS. This little exercise illustrates the granularity at which our code has to operate.

After setting up all our registers on the Blitter we finally get to tell it to set to work:

```
    move.l #SRCEN|UPDA1|UPDA2|DSTA2|LFU_A|LFU_AN,d7
    move.l d7,BLIT_CMD
```

As with everything else Blitter-related this is achieved above by writing a value to the Blitter command register. The register affords up to 30 different commands and here we just use a few of them by populating the register with six different values.

Parameter	Bits				Hex	Description
SRCEN	00000000	00000000	00000000	00000001	00000001	Enable Source Read in Loop
UPDA1	00000000	00000000	00000001	00000000	00000100	Add the A1 value in the outer loop
UPDA2	00000000	00000000	00000010	00000000	00002000	Add the A2 value in the outer loop
DSTA2	00000000	00000000	00000100	00000000	00004000	Reverse role of A1 and A2 registers
LFU_A	00000001	00000000	00000000	00000000	01000000	AND the bits
LFU_AN	00000000	10000000	00000000	00000000	00800000	AND NOT the bits
BLIT_CMD	00000000	00000011	01000010	00110000	01804210	Blitter's Command Register

Combining our parameters into the Blitter's Command Register.

In a nutshell this command is instructing the blitter to read through our source data using an inner loop for the values to write along the X co-ordinates and an outer loop for the Y co-ordinates. The implementation details are obviously complex and we won't get into them here. Instead we can treat this as a recipe for the blitter when we have a chunk from a source image that we want to position on the screen as we do here.

One last feature is worth pointing out explicitly. The final step in the `CopyBlock` routine is to call the `WaitBlit` sub-routine. As the name suggests this piece of code busy-loops until the `BLIT_CMD` register indicates that the blitter has finished:

```
WaitBlit:
  move.l BLIT_CMD,d7 ; Get Blitter status regs
  btst #0,d7         ; Check if d7 is still zero
  beq WaitBlit       ; Loop if d7 is zero
  rts
```

Before rounding up this piece let's take a look at another instance of `CopyBlock` in use. This time we're in the `versionscreen` routine which is responsible for painting the title screen that greets us when we launch Tempest 2000.



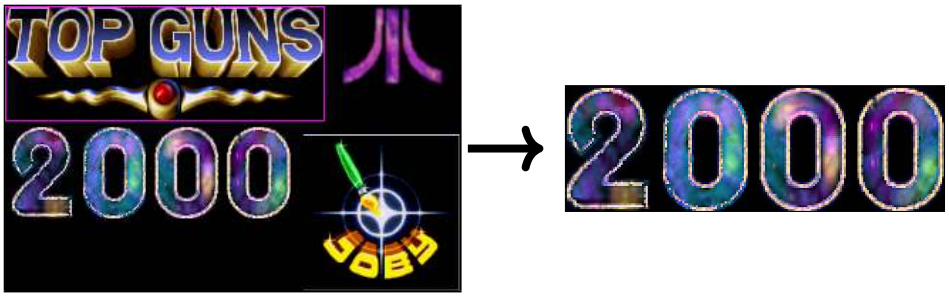
Figure .2: The title screen of Tempest 2000.

Here we find a slightly different order of business in setting up the blitting copy:

```
WaitBlit:
  move #4,d0         ; X position in pic5.
  move #84,d1        ; Y position in pic5.
  move #197,d2       ; Width to copy.
  move #65,d3        ; Height to copy.
  move #92,d4        ; X pos of write to.
  move #120-15,d5    ; Y pos to write to.
```

```
   tst pal             ; Are we PAL or NTSC?
   beq mypal           ; If PAL, skip to mypal.
   add #10,d5          ; We're NTSC so offset Y pos by 10.
mypal:
   move.l #pic5,a0a    ; file 'images/beasty7.cry' again
   move.l #screen3,a1  ; destination we're copying to
   jsr CopyBlock       ; Copy the pixels.
```

Can you guess the piece of `beasty7.cry` we're copying here? It's this:



There is a little extra twiddling in our code to figure out whether the display we're on is PAL or NTSC. These were slightly conflicting standards for analogue television display through the 1960s to the 1990s. NTSC was the standard in the US and PAL the standard in Europe. Most video games of the time needed to account for the slightly different dimensions each standard offered. In the case of our title screen that meant adding an extra offset of 10 pixels to the Y position of the '2000' when painting it to the screen on an NTSC set in order to ensure that it appeared correctly positioned.

Now that we've got the general idea of how to get at least some pictures on to the screen, let's take a look at how we get some text up there.

copyright atari

Making arcade games was an expensive business. The outlay in time and money expended on *Tempest* was premised on selling enough units to arcades in the United States and Europe to justify the investment. And once the game became successful, Atari could fully expect unscrupulous entrepreneurs to cash in on *Tempest's* success by doing something much more profitable than developing a game of their own from scratch. This shortcut to riches involved patching the ROM boards to rename the game, apply some minimal changes to the game's graphical assets, and rebadge it as their own product.

Atari laid a number of traps in the game's code to catch these guys out. The primary focus of nearly all of them was in ensuring that the copyright message displayed throughout the game had not been tampered with. Each of these checks consists of calculating a checksum value on some segment of data in the game and ensuring that it contains the expected value.

QT1: copyright atari

An obvious place to check for tampering is in the copyright literal itself. This is defined in ALLANG.MAC:

```
ZATLIS: ASCVH <^ MCMLXXX ATARI>
ZATLIE:
ZATLIC==ZATLIE-ZATLIS
```

Note that that the © symbol is given as a circumflex. Also note that we've used the labels `ZATLIS` and `ZATLIE` to identify the beginning and end of the copyright literal - this allows us to calculate the length of the message at assembly time and store it in `ZATLIC`. We need this value because our check will be to initialize the accumulator (`A`) with a value of 85, iterate through the copyright string in reverse order adding the value of each character to the accumulator, and store the result (which should be `00`) in `QT1`. The idea is that the cumulative additions will keep wrapping around each time we reach 255 in the accumulator and when they are all complete our final result will be zero, as long as the copyright message has not been tampered with.

```
ZATLIV: CLC           ; CLEAR THE CARRY BIT.
        LDY I,ZATLIC  ; GET POS OF LAST CHAR IN STRING.
        LDA I,085     ; INITIALIZE QT1.
        BEGIN         ; LOOP THROUGH COPYRIGHT STRING.
        ADC Y,ZATLIS  ; ADD VALUE OF CURRENT CHARACTER.
        DEY           ; GO TO NEXT CHAR IN STRING.
        MIEND         ; KEEP LOOPING UNTIL END.
        STA QT1       ; VERIFY ATARI LITERAL
        RTS
```

Here is a rough visualization of the process. At each step we take in the value of the current byte and add it to our accumulator giving us an updated value. Note that the start position of our check is not the last character of the copyright string, but the byte after it (`A0`). This may have been intentional but it does look like a lurking off-by-one error, since we are including a value in our checksum that does not strictly belong to the copyright literal.

String So Far	Byte Value	Updated Accumulator.
I	A0	25
I	A6	CB
RI	38	04
ARI	16	1A
TARI	3C	56
ATARI	16	6C
ATARI	00	6C
X ATARI	44	B0
XX ATARI	44	F4
XXX ATARI	44	39
LXXX ATARI	2C	65
MLXXX ATARI	2E	93

String So Far	Byte Value	Updated Accumulator.
CMLXXX ATARI	1A	AD
MCMLXXX ATARI	2E	DB
MCMLXXX ATARI	00	DB
@ MCMLXXX ATARI	50	2C
@ MCMLXXX ATARI	D3	00

The question now is what happens if we get a non-zero value, i.e. the checksum fails. The answer is that we lie in wait and at some point make the game unplayable. Every time the display state of the game's enemies is updated we check the value of QT1. If it's not zero and the player has advanced past the tenth level we set an impossibly high framerate. This will result in enemies updating too quickly and will render the game unplayable.

```
ZQATLI: LDA QT1         ; GET QT1.
        IFNE            ; IF IT'S NON ZERO..
        LDA WAVEN1      ; CHECK THE CURRENT LEVEL..
        CMP I,10.       ; IF IT'S GREATER THAN 10..
        IFCS            ; THEN..
        LDA I,7A        ; SET AN IMPOSSIBLE TO PLAY
        STA FRTIMR      ; FRAMERATE.
        ENDIF
```

```
        ENDIF
```

QT2: copyright atari II

We've checked the literal value in QT1, but it's possible an enterprising pirate will find another way of altering the copyright message displayed to the player. For example, they could create an entirely new message instead of altering the existing one. Then all they would have to do is updated the routine that displays the message to point at it. This code lives in the routine ZATC4S. All we would have to do is change MATARI to reference our own pirated version, for example MPIRAT.

```
ZATC4S: LDX I,MATARI  ; GET THE ATARI (C) MESSAGE.
        JSR MSGS      ; DISPLAY IT.
        LDX I,MCREDI  ; GET THE 'CREDITS' MESSAGE.
        JSR MSGS    ; DISPLAY IT.
ZATC4E:
ZATC4C==ZATC4E-ZATC4S
```

So what we need to do to prevent this is to come up with a way of checking that this routine itself has not been modified. As before, notice how we identify the end of the code block by adding a label ZATC4E and then used that and the label at the start of the block (ZATC4S) to calculate the length of the code block and store it as ZATC4C. This comes in handy when we want to iterate through the codeblock at runtime and verify that it hasn't been altered:

```
ZATC4V: LDY I,ZATC4C  ; GET THE LENGTH OF THE CODEBLOCK.
        LDA I,0A7       ; INITIALIZE OUR CHECKSUM.
        BEGIN           ; LOOP THROUGH EACH BYTE IN THE CODEBLOCK.
        EOR Y,ZATC4S    ; XOR VAL OF CURRENT BYTE WITH CHECKSUM.
        DEY             ; GO TO NEXT BYTE
        MIEND           ; LOOP THROUGH FULL CODE BLOCK.
        STA QT2         ; STORE CHECKSUM IN Q2.
```

To give us an idea of what 'iterating through the codeblock' actually means, we have to assemble the instructions at ZATC4S to see the byte values that they correspond to. We list them below, side by side with the routine itself. As you can see, the LDX instruction maps to A2, while the address for MATARI corresponds to a pair of bytes 08 and 92, and so on.

```
ZATC4S::
        LDX I,MATARI
        JSR MSGS
        LDX I,MCREDI
        JSR MSGS
ZATC4E:
```

```
ZATC4S::
        A2  2C
        20  AB14
        A2  2E
        20  AB14
ZATC4E:
```

With this in hand we can reconstruct the activity of ZATC4V as it performs an exclusive OR on each byte in the code block, in reverse order, computing a checksum for QT2. As with the computation of QT1 checksum, the 'off by one' nature of the loop means that we actually pick up an extra byte (A5), after the position ZATC4E at the very start of our loop.

Bytes So Far	Byte Value	Updated Checksum.
A5	A5	02
14, A5	14	16
AB, 14, A5	AB	BD
20, AB, 14, A5	20	9D
2E, 20, AB, 14, A5	2E	B3
A2, 2E, 20, AB, 14, A5	A2	11
14, A2, 2E, 20, AB, 14, A5	14	05
AB, 14, A2, 2E, 20, AB, 14, A5	AB	AE
20, AB, 14, A2, 2E, 20, AB, 14, A5	20	8E
2C, 20, AB, 14, A2, 2E, 20, AB, 14, A5	2C	A2
A2, 2C, 20, AB, 14, A2, 2E, 20, AB, 14, A5	A2	00

As we can see, the expected end result for our checksum, which we store in QT2, is 00. What delights are in store if the checksum is not zero? A brutal end: we use the SED instruction to switch the 6502 CPU to Binary Coded Decimal (BCD) mode. This has a fundamental impact on how data is stored and calculations are performed. Any future computation will yield highly unexpected values and result in all kinds of chaos, probably a very rapid and complete crash.

```
ZQAT4C: LDA QT2    ; LOAD THE QT2 CHECKSUM.
        IFNE       ; IF IT'S NOT ZERO..
        LDA I,19.  ; LOAD 19 TO THE ACCUMULATOR.
        CMP CURWAV ; IF THE CURRENT LEVEL..
        IFCC       ; IS GREATER THAN 19..
        SED        ; SWITCH THE CPU TO BINARY DECIMAL MODE.
        ENDIF
        ENDIF
```

QT3: copyright atari III

This one checks the contents of the vector display list used to render the Atari copyright message (as well as everything else the game writes to the screen). Since the copyright message is displayed on a few different screens, with different colors, the contents of the display list can vary, even when the Atari copyright banner is correctly displayed, so our checksum has to tolerate a bit of variety.

Every time a message is written, the MSGS routine checks if the copyright message has been passed to it: if so, it stores the current location in the display list in SECUVG for checking later. Note that this is an address, so is two bytes long. We make sure to save both bytes of the address in SECUVG so we can use it later.

```
ZSECL0:
  CPX I,MATARI  ; IS THIS THE COPYRIGHT MSG?
  IFEQ          ; IF IT IS THEN..
  LDA VGLIST    ; GET CURRENT ADDRESS IN VGLIST
  STA SECUVG    ; SAVE THE LOW BYTE IN SECUVG.
  LDA VGLIST+1  ; GET THE HIGH BYTE OF VGLIST ADDRESS.
  STA SECUVG+1  ; SAVE IT IN SECUVG+1
  ENDIF
```

When the DISPLAY routine runs, we are processing the contents of the display list contained in VGLIST. It's time for us to check the portion that contains our hallowed copyright message.

```
ZATVG2::
  LDA SECUVY      ; ARE WE DISPLAYING ONE OF THE TITLE SCREENS?
  IFNE            ; IF WE ARE THEN:
  LDY I,27        ; FOR ALL 27 BYTES IN THE ROW.
  LDA I,0E        ; INITIALIZE OUR CHECKSUM VALUE
  SEC             ; CLEAR THE CARRY BIT SO IT DOESN'T INTERFERE
  ; LOOP THROUGH EACH CHARACTER IN THE LINE POINTED TO BY SECUVG
  ; THIS SHOULD CONTAIN '(C) MCMLXXX ATARI', OTHERWISE WE WILL GET
  ; AN INVALID CHECKSUM.
  BEGIN           ; LOOP FROM 27 TO 0
  SBC NY,SECUVG   ; SUBTRACT VALUE IN THE CURRENT CHAR FROM 'A' REGISTER.
  DEY             ; DECREMENT Y TO GO TO THE PREVIOUS CHARACTER.
  MIEND           ; LOOP UNTIL Y IS 0.
  TAY             ; STORE THE RESULT IN A IN Y.
  IFNE            ; IF THE RESULT IS ZERO, THE CHECKSUM PASSES OTHERWISE:
  EOR I,0E5       ; CHECK IT AGAINST THE CHECKSUM FOR ANOTHER SCREEN
  ENDIF
  IFNE            ; IF THAT PASSES WE'RE DONE OTHERWISE:
  EOR I,02A       ; CHECK IT AGAINST THE CHECKSUM FOR ANOTHER SCREEN
  ENDIF
  STA QT3         ; STORE THE RESULT OF OUR CHECK IN QT3.
  ENDIF
```

```
   ENDIF
```

The consequences of failing the QT3 are even more devious than the ones we have encountered so far. Rather than corrupting gameplay nearly instantaneously, Dave Theurer devised somthing much more playful and unpredictable. If the player has a score exceeding 170,000, the program will take the lower digits of their score and use these as an index into an area in memory which contains a wide variety of parameters that control the game's behaviour. Commonly referred to as the 'zero page', this is the region of 256 bytes at the very start of the program's addressable memory. Changing values at random in this area will lead to increasingly strange unbehaviour, rendering the game unplayable in a way that will vary from player to player and guaranteed to puzzle the owner of the pirated arcade cabinet. Unfortunately, if there is a mistake in the calculation of the checksum then the same result will obtain for perfectly legitimate owners of Atari cabinets. And this is exactly what happened to owners of the first edition of Tempest in the final months of 1980. A devious bug that 'we cover in detail in a later chapter.'

```
ZQVAVG::
  LDA QT3     ; CHECK THAT BOTH QT3 AND
  ORA QT6     ; QT6 ARE ZERO.
  IFNE        ; IF THEY ARE NOT THEN
  LDA I,17    ; CHECK IF THE PLAYER' SCORE IS GREATER THAN 170,000
  CMP LSCORH  ; LSCORH CONTAINS THE FIRST 2 DIGITS OF THE PLAYER SCORE
  IFCC        ; IF IT IS GREATER THAN OR EQUAL TO 17..
  LDX LSCORL  ; LOAD WHATEVER IS IN THE LSCORL BYTE
  INC X,0     ; AND USE THAT TO INCREMENT ONE OF OUR 'ZERO-PAGE' BYTES.
  ENDIF       ; IN THE HOPE OF CAUSING SOME HAVOC.
  ENDIF       ; HAVOC SECURED.
```

QT4: sound check

This tests the sound chips (known as 'pokeys') for expected behaviour. There are two sound chips on the board and each has a hardware-based random number generator. Retrieving a random number from each, one after the other, should return matching values. If they don't, something is wrong. The check is looking out for signs of the pokeys temporarily stopping when queried for a random number. Don't know why this is bad.

```
ZPOKST: LDX I,4     ; QUERY THE POKEYS 4 TIMES.
        LDA RANDOM  ; GET A RANDOM NUMBER FROM POKEY 1.
        LDY RANDO2  ; GET A RANDOM NUMBER FROM POKEY 2.
        BEGIN       ; LOOP AND COMPARE SUBSEQUENT VALUES 4 TIMES.
        CMP RANDOM  ; GET A NEW VALUE FROM POKEY 1 AND COMPARE TO ORIGINAL.
```

```
        IFEQ        ; IF IT MATCHES..
        CPY RANDO2  ; GET A NEW VALUE FROM POKEY 2 AND COMPARE TO ORIGINAL.
        ENDIF
        IFNE        ; IF NEITHER MATCHED, THEN STORE THE FAILURE..
        STA QT4     ; IN OUR QT4 CHECKSUM VALUE.
        LDX I,0     ; SKIP TO END BY SETTING X TO 0.
        ENDIF
        DEX         ; DECREMENT X
        MIEND       ; LOOP UNTIL X IS -1
```

If this check fails, then we are going to ruin the player's day by corrupting the computer's stack. We do this in the DSPEXP routine, so as long as you never hit anything on your bootlegged version of *Tempest*, you should be OK.

```
ZQPOKS: LDA QT4     ; CHECK THE SOUND CHIP CHECKSUM.
        IFNE        ; IF IT FAILED THEN SET UP A BOMB..
        LDA CURWAV  ; GET THE CURRENT LEVEL..
        CMP I,13.   ; COMPARE IT TO 13.
        IFCS        ; IF IT IS GREATER THAN 13..
        STA 1FF     ; KILL TOP OF STACK
        ENDIF
        ENDIF
        RTS
```

QT5: sound check II

A similar, but more elaborate check, of the game's sound board is performed when displaying the high score tables. Again, we query the *pokey*'s random number generators in quick succession. We bitshift the results around a bit (in a way that is quite hard to follow), and after a bix of X-OR'ing expect a zero byte to store in QT5. What all of this is doing is ensuring that successive requests to each random number generator return the same result - an expected behaviour of the Atari sound ROM board which presumably was not known to its pirate competitors.

```
        .SBTTL  DISPLAY HIGH SCORE LADDER
LDRDSP:
  JSR INFO      ; DISPLAY SCORE & LIVES INFO
ZPONTS:
  SEI           ; DISABLE INTERRUPTS.
  LDA RANDOM    ; PUT A RANDOM NUMBER FROM POKEY 1 IN A.
  LDY RANDOM    ; PUT A RANDOM NUMBER FROM POKEY 1 IN Y.
  STY TEMP0     ; STORE Y IN TEMP0.
  LSR           ; SHIFT ACCUMULATOR RIGHT 1 BIT.
  LSR           ; SHIFT ACCUMULATOR RIGHT 1 BIT.
  LSR           ; SHIFT ACCUMULATOR RIGHT 1 BIT.
```

```
  LSR          ; SHIFT ACCUMULATOR RIGHT 1 BIT.
  EOR TEMP0    ; X-OR ACCUMULATOR WITH TEMP0.
  STA TEMP0    ; STORE RESULT IN TEMP0.
  LDA RANDO2   ; PUT A RANDOM NUMBER FROM POKEY 1 IN A.
  LDY RANDO2   ; PUT A RANDOM NUMBER FROM POKEY 1 IN Y.
  CLI          ; ALLOW INTERRUPTS AGAIN.
  EOR TEMP0    ; X-OR ACCUMULATOR WITH TEMP0.
  AND I,0F0    ; AND TOP 4 BITS OF ACCUMULATOR.
  EOR TEMP0    ; X-OR ACCUMULATOR WITH TEMP0.
  STA TEMP0    ; STORE RESULT IN TEMP0.
  TYA          ; TRANSFER VALUE IN Y TO ACCUMULATOR.
  ASL          ; SHIFT ACCUMULATOR LEFT 1 BIT.
  ASL          ; SHIFT ACCUMULATOR LEFT 1 BIT.
  ASL          ; SHIFT ACCUMULATOR LEFT 1 BIT.
  ASL          ; SHIFT ACCUMULATOR LEFT 1 BIT.
  EOR TEMP0    ; X-OR ACCUMULATOR WITH TEMP0.
  STA QT5      ; STORE THE RESULT IN QT5.
```

The consequences for failing this checksum test are tagged on to the end of the display starfield routine (DSTARF). If the score is greater than 150,000 then we use the last few digits of the score to randomly increment a byte in memory at position 200 or above. Like the damage wrought in QT3 this will be done every time the starfield display is updated during a level change so will inevitably, and usually promptly, bring the game to a crashing halt.

```
ZQPONS:
  LDA QT5     ; LOAD QT5 IN THE ACCUMULATOR.
  IFNE        ; IF IT'S NON-ZERO THEN..
  LDX LSCORH  ; GET THE FIRST 2 DIGITS OF THE SCORE..
  CPX I,15    ; COMPARE WITH 15..
  IFCS        ; IF SCORE GREATER THAN 150,000 THEN..
  LDX LSCORL  ; GET THE LAST 2 DIGITS OF THE SCORE..
  INC X,200   ; AND USE THAT TO INCREMENT ONE OF THE BYTES AT ADDRESS $200.
  ENDIF       ; IN THE HOPE OF CAUSING SOME HAVOC.
  ENDIF       ; HAVOC SECURED.
  RTS
```

QT6: copyright atari, one last time

This is the same check as QT3 but is run in attract mode. The display list is inspected and a checksum calculated on the line where the Atari copyright message is expected to be.

```
ZATVG1:
  LDA QSTATUS      ; ARE WE IN ATTRACT MODE?
```

```
   IFPL             ; IF YES THEN.. ATARI BETTER BE ON SCREEN
   LDA I,0F2        ; INITIALIZE THE CHECKSUM VALUE TO F2.
   CLC              ; CLEAR THE CARRY BIT.
   LDY I,39.        ; ITERATE OVER ALL 39 BYTES ON THE LINE
   BEGIN            ; LOOP THROUGH ALL 39 BYTES..
   ADC NY,SECUVG    ; ADDING THE VALUE AT EACH BYTE TO OUR CHECKSUM
   DEY              ; KEEP LOOPING FOR ALL 39.
   MIEND            ; END LOOP
   STA QT6          ; SAVE THE RESULT (WHICH SHOULD BE 0)
   ENDIF
```

Like QT3, an invalid checksum in QT3, will cause rapid corruption of bytes in the zero-page area of the machine's memory.

```
ZQVAVG::
   LDA QT3      ; CHECK THAT BOTH QT3 AND
   ORA QT6      ; QT6 ARE ZERO.
   IFNE         ; IF THEY ARE NOT THEN
   LDA I,17     ; CHECK IF THE PLAYER' SCORE IS GREATER THAN 170,000
   CMP LSCORH   ; LSCORH CONTAINS THE FIRST 2 DIGITS OF THE PLAYER SCORE
   IFCC         ; IF IT IS GREATER THAN OR EQUAL TO 17..
   LDX LSCORL   ; LOAD WHATEVER IS IN THE LSCORL BYTE
   INC X,0      ; AND USE THAT TO INCREMENT ONE OF OUR 'ZERO-PAGE' BYTES.
   ENDIF        ; IN THE HOPE OF CAUSING SOME HAVOC.
   ENDIF        ; HAVOC SECURED.
```

character assassination

Writing some text to the screen, such as the copyright information on the title screen, is surprisingly convoluted in its detail, it will even introduce us to the machinations of the Jaguar's GPU. But although there is a surprising amount of code required from the developer to get a string of characters onto the display the general principle is one familiar from other platforms: you define a font, you specify the text you want to display and where you want to display it, and finally you call a routine that will take care of all the detail.

In our case the routine that takes care of all the detail will be our first taste of the implementation and use of what are known as 'shaders', free-standing programs that are written in a customized version of Motorola 68000 assembly lanage for the Atari's Graphics Procesing Unit(GPU), capriciously dubbed 'Tom' by the hardware designers of Atari at the time.

Before we get to that we can coast through the relatively easy part of defining our font and specifying our string of text, as well as plugging in the values to the GPU's buffer that will be required by the shader when it takes care of actually putting pixels on screen.

When it comes to defining a font for the copyright information on Tempest 2000's title screen you may have already guessed where information is contained: in one of our `beasty` cry files, in this case `beasty4.cry`:



Figure .1: `beasty4.cry` with the copyright font at the bottom.

And as you may remember the data in `beasty4.cry` is pointed by the variable `pic2`:

```
pic         EQU $820000          ; beasty3.cry
pic2        EQU pic+$1f400       ; beasty4.cry
pic3        EQU pic2+$1f400      ; beasty5.cry
pic4        EQU pic3+$25800      ; beasty6.cry
pic5        EQU pic4+(640*128)   ; beasty7.cry
pic6        EQU pic5+(640*200)   ; beasty8.cry
```

So all we have to do to is devise a scheme that maps from a given letter or number to its location (and dimensions) in `beasty.cry`. For example, if we want to display the letter 'A' we need to come up with a way of extracting this:



A.

It helps us that all of our font characters in the `cry` file are the same dimensions so we can make the height and width a consistent assumption for every glyph. Armed with the location of the pixel data and this dimension information all that remains is to create a look-up table that tells us the X and Y co-ordinates in `beasty4.cry` for each character.

The easiest form of look-up table we can make is an 'indexed' one. In other words, a procedure that converts each letter and digit in our character set into a number that we then use as an index into a list of the X and Y locations. For example, if 'A' is converted to the number 41 then we fetch its coordinates in `beasty4.cry` from the

42nd member of the list. If 'B' is coverted to number 42 we fetch its co-ordinates from the 43rd member of the list, and so on. Fortunately we don't have to invent a method of our own for relating letters and digits to a number. The ASCII committee has done this for us already. So as long as we store the co-ordinates in our list in the same order as given by the ASCII committe we will simply be able to use the corresponding ASCII value for each letter and digit as our index.

With the main planks of our cunning plan settled in place we can iron out a few details. The best way of storing our co-ordinate information in this list is to cram the X and Y values into a single place. More specifically, we can assume our X and Y values will always be between 0 and 65,536 so each will fit in one half of a single 4 byte (64 bit value). Another way of describing a value of this length is as a 'long' and in 68K assembler the notation to denote such a value is `dc.l`.

So let's assume we have 'A' at Y co-ordinate 165 and X co-ordinate 2 in our `beasty4.cry` 'sprite sheet'. Since we're dealing with zero-based indices we subtract 1 from each of these values to give us 164 and 1 respectively. In hex 164 is `$A4` and 1 is `$01`. We can fit this into our 4 byte entry as follows:

00	$A4$	00	01

Likewise with our fixed set of dimensions for each of the characters or glyphs. We get out our ruler, apply it to the screen, and determine they each have a height of 19 and a width of 16. We subtract 1 from each to give us 18 and 15. Convert to hex: `$12` and `$0F`. Then plug them into our 4 byte value:

00	12	00	$0F$

With this information we can now isolate our 'A' glyph and extract it.



Figure .2: The 'A' glyph located in a red box by our dimension and co-ordinate information.

The mechanics of our look-up table are now fully worked out so we are ready to create it. Since it has over a hundred entries we store it in a separate file called `afont.s` and give it the very descriptive name `afont`. I think this is intended as an abbreviation of 'Atari Font', since it is the font used to write the Atari copyright information. An incomplete excerpt of the file follows below with ellipses (...) to indicate lines that have been omitted.

```
*
*
* Page 2 font, by Joby

afont:
  dc.l pic2      ; pixel data
  dc.l $0012000f ; height ($0012) and width ($000f)

  dc.l $b6011e   ;Space
  dc.l $b600d2   ;!
  dc.l $a40001   ;"
  dc.l $a40001   ;#
  dc.l $a40001   ;$
  dc.l $a40001   ;%
  dc.l $a40001   ;&
  dc.l $a40001   ;'
  dc.l $9100a5   ;(
  dc.l $9100ba   ;)
  dc.l $a40001   ;*
  dc.l $a40001   ;+
  dc.l $b600f8   ;,
  dc.l $b6010b   ;-
  dc.l $b600c2   ;.
  dc.l $a40001   ;/
  ...
  dc.l $a40001  ;A <-- This is the one! Y: $00a4, X: $0001
  dc.l $a40014  ;B
  dc.l $a40027  ;C
  ...
```

Listing .1: The opening block of the `afont` data structure in source file `afont.s`.

Notice that the order in which we have defined the location of each character corresponds to the order (or value) of each character according to the ASCII specification. This means that when we define the text we want to display all we have to do is convert each character in the text to its corresponding ASCII value 'n' and get that nth value from our `afont` list.

So let's define the text we want to display:

```
ataricop1: dc.b "COPYRIGHT 1981,",0
ataricop2: dc.b "1994 ATARI CORP.",0
```

This is straightforward enough. Note that we have a zero at the end of each string. This makes them 'null-terminated' - a feature that will come in handy later when we need to identify that the end of the string when writing it to the screen.

Now that we have our text defined we can start the process of writing it to the screen. These two concise little paragraphs do this job for us. Given the amount of info encoded in our `afont` data structure we only need to set up one additional piece of info for the `centext` routine that will do our heavy lifting. This is the Y position that we want to write the text to on the screen (adjusted for whether we're on a PAL or NTSC screen).

```
    lea afont,a1      ; Load the font data structure
    lea ataricop1,a0  ; Load the first line of the copyright info
    move #190-8,d0    ; Set the Y position for the text
    add palfix2,d0    ; Adjust for PAL screens
    jsr centext       ; Write the text

    lea afont,a1      ; Load the font data structure
    lea ataricop2,a0  ; Load the second line of the copyright info
    move #207-5,d0    ; Set the Y position for the text
    add palfix2,d0    ; Adjust for PAL screens
    jsr centext       ; Write the text
```

The `centext` routine (short for 'center the text') we call into at the end of each paragraph above is where things start to get detailed. It is still not the tangled core of this particular beast but it is certainly a busier proposition that anything we've looked at so far. But we can settle our nerves for the moment because it is in fact doing something quite straightforward: it is setting up a buffer of data for the GPU to use. It stores this buffer at an address in the GPU RAM given by the `inbuf` variable. This buffer is a 36 byte array containing the 4-byte addresses of all the different bits of data the GPU will need to render the text. Here are all the bits and piece in the order that they appear in `inbuf` once `centext` is finished:

GPU RAM Address	Name	Description
$F0003F60	ataricop1	Address in RAM of the text to write
$F0003F64	afont	Address in RAM of the font data structure
$F0003F68	$00000000	Drop Shadow Vector X axis
$F0003F6C	$00000000	Drop Shadow Vector Y axis
$F0003F70	$00010000	Text Scale X axis
$F0003F74	$00010000	Text Scale Y axis
$F0003F78	$00000000	Text Shear X axis
$F0003F7C	$00000000	Text Shear Y axis
$F0003F84	$00B600A7	Y & X Co-ordinates to Write Text To
$F0003F80	$00000000	Drop Shadow Mode Enabled

The 10 4-byte addresses stored beginning in `inbuf` by `centext`.

In addition to the items of data we've collected so far we can see that we plug in a few extras in the `centext` routine. These are graphical effects such as the scaling to apply to each character, the dimensions of a drop shadow to apply on each, and finally the 'shear' to apply. 'Shear' distorts an object along the depth or 'Z' axis to make it appear like we are looking at it at an angle in 3 dimensions. The only one of these parameters we using is the scaling, for the rest we just plug in zero. But here's what the copyright text would look like with some shear applied:



Figure .3: What the characters look like when we apply a shear value of 800 on both the X and Y axes.

And here's what they looks like if we bump up the scale on the X and Y axis:



Figure .4: What the characters look like when we apply a scale value of 2000 instead of 1000. The pixellated, exploded appearance gives some hint that 'scale' here is not simply concerned with size but a specific 'explosion type' effect.

We'll encounter effective use of these features elsewhere in Tempest 2000 so won't go any further into them for now.

```
centext:
  move.l a0,d2          ; Store 'text' in d2 so we don't overwrite it.
  lea in_buf,a0         ; Use 'in_buf' as the buffer to write to.
  move.l d2,(a0)        ; Store 'text' in the first position.
  move.l a1,4(a0)       ; Store 'afont' in the 2nd position.
  move.l #0,8(a0)       ; Store 0 in the 3rd position.
  move.l #0,12(a0)      ; Store 0 in the 4th possition (Dropshadow vector).
  move.l #$10000,16(a0) ; Store 10000 in the 5th position (X Text Scale).
  move.l #$10000,20(a0) ; Store 10000 in the 6th position (Y Text Scale).
  move.l #0,24(a0)      ; Store 0 in the 7th position (X Text Shear).
  move.l #0,28(a0)      ; Store 0 in the 8th position (Y Text Shear).
  move.l #0,36(a0)      ; Store 0 in the 10th position.

  ; This paragraph is about figuring out the X pos we want to place
  ; the texta nd then storing that along with the Y pos in the 10th
  ; position in the buffer
  bsr g_textlength ; Get the length of the text in d2 and store it in d7.
```

```
; This uses the text length to center it along the X axis and
; store the result in d0;
lsr #1,d7
neg d7
add #192,d7
; d0 contains the Y pos, so move it to the left hand side
; of the 4 byte word.
swap d0
move d7,d0       ; Move the X pos we've calculated into the right.
move.l d0,32(a0) ; Move the X/Y pos in the 9th position.

; We're now ready to run the GPU.
lea texter,a0
jsr gpurun
jmp gpuwait
```

Listing .2: The `centext` routine responsible for setting up the values in our GPU buffer.

`centext`, given above, stuffs all these items into a buffer of GPU RAM called `in_buf` and when that is done loads and runs a separate little GPU module called a 'shader'. In this case the name of the shader program is `texter`, giving us a little clue to its domain of expertise. This shader is a free-standing piece of code written in a slightly different flavour of 68000 assembly language. Despite it's name `texter` is contained in a source file called `stoat.gas`. (Nearly all of the shaders, like the main program source file, are named after beasts of one sort or another. There is no logic or convention that can be applied to the names. They're just a bit of fun.)

```
texter:
 .incbin  "bin/stoat.o"
```

`texter` is slightly more than just a 'writing strings to the screen' routine. In addition to deforming the text as we saw in the 'shear' example above, it can also 'explodes' the text using the 'scale' parameters. A comment near the top of `stoat.gas` (the source file for the `texter` routine) tells the possible inspiration for this effect:

```
;*****
;*
;* REX: Robotron explosion generator. Takes an image from the source screen
    and expands
;* it in X and Y, then uses a1_n to draw the resultant matrix of single
    pixels.
;*
;*  Provide: dest screen in gpu_screen, in_buf: 0=source image address
;*  4=source image start pixel address, 8=x and y size of source, 12=X scale
     (16:16),
;*  16=Y scale (16:16), 20=X shear (16:16), 24=Y shear,
;*  28=Mode (0=Top edge, 1=Centered), 32=Dest X and Y
;*
;*****
```

The 'robotron explosion' is the effect visible below. As you gradually dial up the X and Y scale parameters and call this shader at each iteration it can appear as though the text is 'exploding' outwards. There is no real mystery to how the shader writes our characters to the screen. It uses the same blitting operation we covered previously but with the difference that the preparation of the blitting command and all the special calculations we might want to do if adding shear, scaling and drop shadows in advance of calling the blitter are done from within the special purpose processor of the GPU rather than in the CPU. The reason for doing it this way is because the GPU is designed to be fast at a small number of arithmetic operations such as adding and multiplying so if we're going to do a lot of them and don't want to slow the rest of the game down, the GPU is the man for our job. We'll next take a look at the internals of the `texter` shader, including the relatively trivial task of blitting our characters to the screen, in the next chapter.

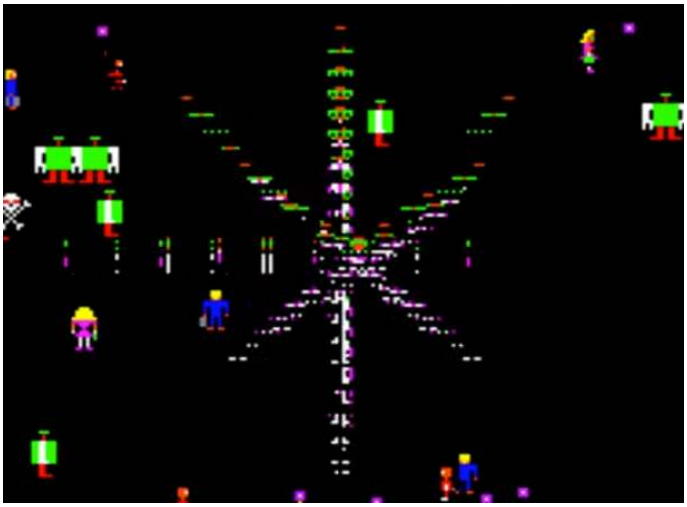


Figure .5: The explosion effect in 'Robotron 2084'.

macro language fun

The extensive language localization present in *Tempest*, the cabinet's ability to display English, French, German or Spanish versions of itself, is a salutary education in how computing enterprises even as early as the 1980s embedded the need for catering to international markets. These noble considerations aside, it also happens to present an object lesson in the very sophisticated use of macros in assembly language programming.

Macros, in case you're wondering, were an early form of 'meta-programming'. By this we mean the ability to write code that generates more code. We occasionally encounter the use of macros elsewhere in this book, for example as a sort of shorthand to make a complex set of instructions into a simple one-line command, but nowhere are they used to the same degree of depth and complexity as when turned to the humble purpose of generating messages to the player in multiple languages.

To get a flavour of how it is 'macros all the way down' in Tempest's localization implementation, let's start all the way near the bottom with a macro that achieves the simple task of converting a string of letters into text that will get displayed on the screen. This is the macro called `ASCVG`. [1] It was a common part of the Atari toolkit, developed inhouse by Rich Moore.

[1] Short for ASCii Vector Generator.

```
;MACRO .ASCVG CONVERTS STRING OF ALPHANUMERICS TO OFFSET VALUES INTO
;VGMSGA TABLE (TABLE OF VG CHARACTER JSRL'S)
;
    .MACRO .ASCVG .STRING      ; START OF MACRO

    .NCHR ..N,<.STRING>        ; PUT LENGTH OF STRING IN 'N'.

    ; LOOP THROUGH THE STRING, CONVERTING EACH CHAR INTO
    ; AN OFFSET IN THE VGMSGA TABLE.
    .IRPC ..C,<.STRING>        ; PUT NEXT CHAR IN STRING IN 'C'.
    .IF EQ,''..C-^H20          ; IF CHAR IS A SPACE..
    ..X=0                      ; THEN OFFSET IS ZERO.
    .IFF                       ; ELSE IF..
    .IF GT,''..C-^H40          ; CHAR IS BETWEEN A AND Z..
    ..X=''..C-^H36             ; THEN OFFSET IS BETWEEN 16 AND 48
    .IFF                       ; ELSE
    ..X=''..C-^H2F             ; OFFSET IS BETWEEN 0 AND 16
    .ENDC                      ; END IF
    .ENDC                      ; END IF
    ..X=..X*2                  ; MULTIPLY BY 2 TO GET OFFSET
    ..N=..N-1                  ; DECREMENT CURRENT LENGTH
    .IIF EQ,..N,..X=..X!^H80   ; IF REACHED ZERO, MARK LAST CHAR.
    .BYTE ..X                  ; OUTPUT THE OFFSET.
    .ENDR                      ; REPEAT FROM .IRPC UNTIL END OF STRING.

    .ENDM                      ; END OF MACRO
```

As we said, the purpose of this macro is to take an arbitrary string, for example `ATARI`, and turn it into a series of bytes. This is what those bytes look like, five in total, one for each letter:

```
;      A   T   A   R   I
.BYTE 16, 3C, 16, 38, A6
```

These values are not arbitrary. Each is an offset into a table called `VGMSGA`. [2] We've reproduced this table below. If you look up the value 16 in the rightmost column you'll see that it points to an entry in the table that reads `'JSRL CHAR.A'`, for example:

[2] Short for Vector Generator MeSsGage Alphabet

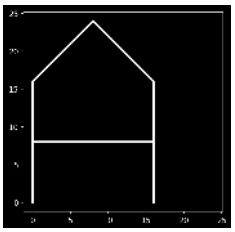
Code			Content In Bytes	Offset
VGMSGA:	JSRL	CHAR.	A8B4	00
	JSRL	CHAR.0	A865	02
	JSRL	CHAR.1	A8B6	04
	JSRL	CHAR.2	A8BA	06
	JSRL	CHAR.3	A8C2	08
	JSRL	CHAR.4	A8C9	0A
	JSRL	CHAR.5	A8D0	0C
	JSRL	CHAR.6	A8D7	0E
	JSRL	CHAR.7	A8DE	10
	JSRL	CHAR.8	A8E3	12
	JSRL	CHAR.9	A8EB	14
	JSRL	CHAR.A	A800	16
	JSRL	CHAR.B	A808	18
	JSRL	CHAR.C	A815	1A
	JSRL	CHAR.D	A81B	1C
	JSRL	CHAR.E	A823	1E
	JSRL	CHAR.F	A82B	20
	JSRL	CHAR.G	A832	22
	JSRL	CHAR.H	A83B	24
	JSRL	CHAR.I	A842	26
	JSRL	CHAR.J	A849	28
	JSRL	CHAR.K	A84F	2A
	JSRL	CHAR.L	A855	2C
	JSRL	CHAR.M	A85A	2E
	JSRL	CHAR.N	A860	30
	JSRL	CHAR.O	A865	32
	JSRL	CHAR.P	A86B	34
	JSRL	CHAR.Q	A872	36
	JSRL	CHAR.R	A87B	38
	JSRL	CHAR.S	A883	3A
	JSRL	CHAR.T	A88A	3C
	JSRL	CHAR.U	A890	3E
	JSRL	CHAR.V	A896	40
	JSRL	CHAR.W	A89B	42
	JSRL	CHAR.X	A8A2	44
	JSRL	CHAR.Y	A8A7	46
	JSRL	CHAR.Z	A8AE	48

Figure .1: The VGMSGA table, along with the contents in bytes, and the offset value for each entry.

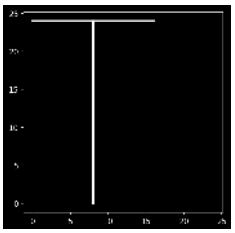
You will notice something funny about the last letter. Instead of 26, which would point to 'JRSL CHAR.I', as we might expect, the last byte is A6 instead. This is because the last letter in a string is always AND'd with 80 to signal that it is indeed the last letter: so 80 + 26 = A6.

So now that we know each of the bytes is an index into our VGMSGA table, and each

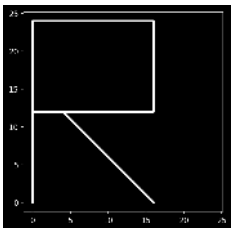
entry in the table calls a mysterious-looking routines such as `CHAR.A`, `CHAR.T` and so on., we might wonder what each of these routines actually does. It is not surprising to learn that each one is responsible for drawing the invididual letter it is named after onto the screen, like so:



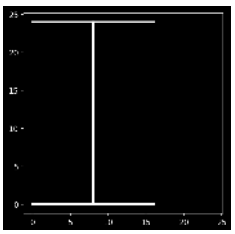
```
CHAR.A: VCTR 0,16,.BRITE
        VCTR 8,8,.BRITE
        VCTR 8,-8,.BRITE
        VCTR 0,-16,.BRITE
        VCTR -16,8,0
        VCTR 16,0,.BRITE
        VCTR 8,-8,0
        RTSL
```



```
CHAR.T: VCTR 8,0,0
        VCTR 0,24,.BRITE
        VCTR -8,0,0
        VCTR 16,0,.BRITE
        VCTR 8,-24,0
        RTSL
```



```
CHAR.R: VCTR 0,24,.BRITE
        VCTR 16,0,.BRITE
        VCTR 0,-12,.BRITE
        VCTR -16,0,.BRITE
        VCTR 4,0,0
        VCTR 12,-12,.BRITE
        VCTR 8,0,0
        RTSL
```



```
CHAR.I: VCTR 16,0,.BRITE
        VCTR -8,0,0
        VCTR 0,24,.BRITE
        VCTR 8,0,0
        VCTR -16,0,.BRITE
        VCTR 24,-24,0
        RTSL
```

Nor is surprising that if we call them one after the other, in a particular order, that we might get the anticipated result of drawing `ATARI` to the screen:




```
JSRL    CHAR.A ; 36
JSRL    CHAR.T ; 1C
JSRL    CHAR.A ; 36
JSRL    CHAR.R ; 38
JSRL    CHAR.I ; A6 (16)
```

So if we look for the piece of code responsible for using ASCVG to write *ATARI* to the screen (as part of the copyright statement) we find this:

```
EATARI:
FATARI:
GATARI:
SATARI: ASCVH <^ MCMLXXX ATARI>
```

There appears to be an immediate catch, it is using something called ASCVH [3] rather than the ASCVG that we expected. But all is not lost, it turns out that ASCVH is a wrapper around ASCVG. In addition to calling ASCVG it first figures out the horizontal position of the string. In fact, figuring out (and writing out) this horizontal position is its sole function, apart from calling ASCVG to write the string itself.

```
   .MACRO ASCVH ...AR1,...AR2 ; MACRO ASCVH WITH 2 POSSIBLE ARGS.
   .NARG ...NUM               ; PUT NO. OF ARGS PASSED IN 'NUM'.
   .IF EQ ...NUM-1            ; IF ONLY THE STRING WAS PASSED..
   .NCHR ...HLO,<...AR1>      ; STORE THE NO. OF CHARS IN STRING IN 'HLO'.
   .BYTE -<...HLO*3>          ; HORIZONTAL POSITION IS HLO * 3.
   .ASCVG <...AR1>            ; CALL ASCVG TO WRITE OUT THE STRING.
   .IFF                       ; OTHERWISE POSITION AND STRING WERE PASSED..
   .BYTE ...AR1               ; SO OUTPUT ARG 1 AS THE HORIZONTAL POSITION
   .ASCVG <...AR2>            ; AND WRITE OUT THE STRING IN ARG 2.
   .ENDC                      ; END IF
   .ENDM                      ; END MACRO
```

ASCVH can take two arguments: one for the horizontal position to draw the string, and one for the string itself. If only one argument is passed, it assumes that its the string and that it should display it in the horizontal center. This is the case with our current example:

```
EATARI:
FATARI:
GATARI:
SATARI: ASCVH <^ MCMLXXX ATARI>
```

So we have managed to come quite far here: we have figure out how to draw text to the screen and how to position it (horizontally) at least. But this still leaves a number of missing elements. What about the vertical position? The color of the text? And most of

[3] Short for ASCii Vector Horizontal.

all, what about writing text in languages other than English? `MCMLXXX ATARI` doesn't need translation, but messages like *'GAME OVER'* certainly do.

So what we want is something to layer on top of `ASCVH` that will give us all of the above and produce a result that contains the vector instructions for writing all of our messages in all of the languages that we're interested in.

To do this we have to think a little harder about the end result we want: lists of messages in all four desired languages. This means we have to create four separate tables, one for each language. And that each entry in each table should point to the message in that language. Since the position on screen, color, and size is going to be common for each message, we can store that in a single, separate table.

So we could imagine five tables in total, something like this:

Message	ENGMSG	FREMSG	SPAMSG	GERMSG	MSGLABS
GAME OVER	EGAMOV (012C)	FGAMOV (0136)	SGAMOV (0144)	GGAMOV (014E)	1C 9E
PLAYER	EPLAYR (015E)	FPLAYR (0166)	SPLAYR(0166)	GPLAYR (016E)	80
©MCMLXXX ATARI	EATARI(0544)	FATARI(0544)	SATARI(0544)	GATARI(0544)	80

The message tables. The pointer mnemonic, e.g. `EGAMOV`, is a nickname for the hex address of the message, e.g. `(012C)`.

The key here is that for each message we have a separate table for each language, and each entry in the table points to the instructions for drawing the string in that language. For words that require no translation, the pointer will be to the same address. So in the case of *©MCMLXXX ATARI*, the entry for each language points to the same address `0544` in memory where the following bytes are stored:

```
EATARI:
FATARI:
GATARI:
SATARI:
;  (C)   M  C  M  L  X  X  X     A  T  A  R  I
D3 50 00 2E 1A 2E 2C 44 44 44 00 16 3C 16 38 A6
```

You will hopefully recognize the bytes for 'ATARI' from earlier. As we discovered previously, these are offsets into the alphabet table `VGMSGA` that will draw each character.

An example where we actually have different messages for each language is *GAME OVER*. For this we have a different hex address in the entry for each language table, because we are pointing at a different message for each.

```
EGAMOV: ASCVH <GAME OVER>
FGAMOV: ASCVH <FIN DE PARTIE>
GGAMOV: ASCVH <SPIELENDE>
```

```
SGAMOV: ASCVH <JUEGO TERMINADO>
```

So what we need is a way of adding these entries to the tables programatically, for example an instruction that we can write as follows. One that will automatically add the *GAME OVER* message to all four language tables plus the table that defines the color, scale, and vertical position.

```
MESS    GAMOV,GREEN,1,56   ; 'GAME OVER', COLOR, SCALE, VERTICAL POSITION.
```

Wouldn't it be great if this instruction created the first entry in each of our tables, at a stroke, like so:

Message	ENGMSG	FREMSG	SPAMSG	GERMSG	MSGLABS
GAME OVER	EGAMOV (012C)	FGAMOV (0136)	SGAMOV (0144)	GGAMOV (014E)	06 56

Well it turns out there is a macro we can write to do exactly that. It's called MESS. [4]

```
   .SBTTL  PREPARE TO MAKE MSG TABLES
   ; ARGS ARE: STRING, COLOUR, SCALE, Y-POSITION
   .MACRO MESS ...LIT,...COL,...SCA,...YPO ; START OF MACRO

   ; THE COMMENTS ASSUME WE HAVE PASSED GAMOV IN 'LIT'
   ; ADD ENTRY TO ENGMSG (ENGLISH TABLE)
   .=...ENG              ; POINT TO CURRENT POS IN ENGMSG
   .WORD E...LIT         ; ADD EGAMOV ('GAME OVER') TO ENGMSG.
   ...ENG=...ENG+2       ; MOVE CURRENT POS IN ENGMSG 2 BYTES FOR NEXT TIME.

   ; ADD ENTRY TO FRAMSG
   .=...FRE              ; POINT TO CURRENT POS IN FRAMSG
   .WORD F...LIT         ; ADD FGAMOV ('FIN DE PARTIE') TO FRAMSG.
   ...FRE=...FRE+2       ; MOVE CURRENT POS IN FRAMSG 2 BYTES FOR NEXT TIME.

   ; ADD ENTRY TO GERMSG
   .=...GER              ; POINT TO CURRENT POS IN GERMSG
   .WORD G...LIT         ; ADD GGAMOV ('SPIELENDE') TO GERMSG.
   ...GER=...GER+2       ; MOVE CURRENT POS IN GERMSG 2 BYTES FOR NEXT TIME.

   ; ADD ENTRY TO SPAMSG
   .=...SPA              ; POINT TO CURRENT POS IN SPAMSG
   .WORD S...LIT         ; ADD SGAMOV ('JUEGO TERMINADO') TO SPAMSG.
   ...SPA=...SPA+2       ; MOVE CURRENT POS IN SPAMSG 2 BYTES FOR NEXT TIME.

   ; ADD GLOBAL VARIABLE
M'...LIT=.MSNUM
   .GLOBL M...LIT        ; ADD GLOBAL VAR MGAMOV
   .MSNUM=.MSNUM+2
```

[4] Short for **MESS**age.

```
   ; ADD COLOR, SCALE, YPOS TO MSGLABS
   .=...CSY
   .BYTE <...COL*10>!...SCA  ; COLOR & SCALE ARE AND'D TOGETHER
   .BYTE ...YPO              ; AND Y POS IN A BYTE BY ITSELF.
   ...CSY=...CSY+2           ; MOVE CURRENT POS IN MSGLABS 2 BYTES FOR NEXT TIME.

   .ENDM                     ; END OF MACRO
```

Notice some of the special behaviour available to the macro language here. We can take the literal `GAMOV` passed in the argument `LIST` and manipulate it directly to reference each of `EGAMOV`, `FGAMOV`, `SGAMOV`, and `GGAMOV` individually. This allows us to output entries for each language table during assembly as though we had typed this:

```
ENGMSG:
  .WORD EGAMOV
```

```
FRAMSG:
  .WORD FGAMOV
```

```
SPAMSG:
  .WORD SGAMOV
```

```
GERMSG:
  .WORD GGAMOV
```

Because the macros are interpreted at assembly time and converted into machine language instructions, this will make building our complete language tables a simple matter of issuing this instruction for each message that we have defined:

```
        .SBTTL  MESSAGES: COLOR, SCALE, Y POSITION, LANGUAGE PTRS
MESS    GAMOV,GREEN,1,56        ;GAME OVER
MESS    PLAYR,WHITE,0,1A        ;PLAYER (BIG)
MESS    PLYR2,WHITE,1,20        ;PLAYER (NORMAL)
MESS    PRESS,RED,1,56          ;PRESS START
MESS    PLAY,WHITE,1,38         ;PLAY
MESS    ENTER,RED,1,0B0         ;ENTER
MESS    PRMOV,TURQOI,1,0        ;SPIN
MESS    PRFIR,YELLOW,1,-10.     ;PRESS FIRE
MESS    HIGHS,RED,0,38          ;HIGH SCORE
MESS    RANK,RED,1,-50.         ;RANK
MESS    RATE,GREEN,1,10.        ;RATE YOURSELF
MESS    NOVIC,RED,1,-30.        ;NOVICE
MESS    EXPER,RED,1,-30.        ;EXPERT
MESS    BONUS,GREEN,1,-70.      ;BONUS
MESS    TIME,GREEN,1,98         ;TIME
MESS    LEVEL,GREEN,1,-40.      ;LEVEL
MESS    HOLE,GREEN,1,-55.       ;HOLE
MESS    INSER,RED,1,56          ;INSERT COINS
MESS    CMODE,GREEN,1,80        ;FREE PLAY
MESS    CMOD1,GREEN,1,80        ;1 COIN 2 PLAYS
MESS    CMOD2,GREEN,1,80        ;1 COIN 1 PLAY
MESS    CMOD3,GREEN,1,80        ;2 COINS 1 PLAY
MESS    ATARI,BLULET,1,92       ;MCMLXXX ATARI
MESS    CREDI,GREEN,1,80        ;CREDITS
MESS    BONPT,RED,1,0B0         ;BONUS PTS
MESS    2GAME,GREEN,1,89        ;2 GAME MINIMUM
MESS    BOLIF,TURQOI,1,89       ;BONUS EVERY
MESS    SPIKE,WHITE,0,0         ;AVOID SPIKES
MESS    APROA,BLULET,1,5A       ;APPROACH
```

```
MESS    SUPZA,BLULET,1,0A0      ;NEW SUPER
```

All of which will give us our four complete language tables.

```
ENGMSG:
  .WORD EGAMOV
  .WORD EPLAYR
  .WORD EPLYR2
  .WORD EPRESS
  .WORD EPLAY
  .WORD EENTER
  .WORD EPRMOV
  .WORD EPRFIR
  .WORD EHIGHS
  .WORD ERANK
  .WORD ERATE
  .WORD ENOVIC
  .WORD EEXPER
  .WORD EBONUS
  .WORD ETIME
  .WORD ELEVEL
  .WORD EHOLE
  .WORD EINSER
  .WORD ECMODE
  .WORD ECMOD1
  .WORD ECMOD2
  .WORD ECMOD3
  .WORD EATARI
  .WORD ECREDI
  .WORD EBONPT
  .WORD E2GAME
  .WORD EBOLIF
  .WORD ESPIKE
  .WORD EAPROA
  .WORD ESUPZA
```

```
FRAMSG:
  .WORD FGAMOV
  .WORD FPLAYR
  .WORD FPLYR2
  .WORD FPRESS
  .WORD FPLAY
  .WORD FENTER
  .WORD FPRMOV
  .WORD FPRFIR
  .WORD FHIGHS
  .WORD FRANK
  .WORD FRATE
  .WORD FNOVIC
  .WORD FEXPER
  .WORD FBONUS
  .WORD FTIME
  .WORD FLEVEL
  .WORD FHOLE
  .WORD FINSER
  .WORD FCMODE
  .WORD FCMOD1
  .WORD FCMOD2
  .WORD FCMOD3
  .WORD FATARI
  .WORD FCREDI
  .WORD FBONPT
  .WORD F2GAME
  .WORD FBOLIF
  .WORD FSPIKE
  .WORD FAPROA
  .WORD FSUPZA
```

```
SPAMSG:
  .WORD SGAMOV
  .WORD SPLAYR
  .WORD SPLYR2
  .WORD SPRESS
  .WORD SPLAY
  .WORD SENTER
  .WORD SPRMOV
  .WORD SPRFIR
  .WORD SHIGHS
  .WORD SRANK
  .WORD SRATE
  .WORD SNOVIC
  .WORD SEXPER
  .WORD SBONUS
  .WORD STIME
  .WORD SLEVEL
  .WORD SHOLE
  .WORD SINSER
  .WORD SCMODE
  .WORD SCMOD1
  .WORD SCMOD2
  .WORD SCMOD3
  .WORD SATARI
  .WORD SCREDI
  .WORD SBONPT
  .WORD S2GAME
  .WORD SBOLIF
  .WORD SSPIKE
  .WORD SAPROA
  .WORD SSUPZA
```

```
GERMSG:
  .WORD GGAMOV
  .WORD GPLAYR
  .WORD GPLYR2
  .WORD GPRESS
  .WORD GPLAY
  .WORD GENTER
  .WORD GPRMOV
  .WORD GPRFIR
  .WORD GHIGHS
  .WORD GRANK
  .WORD GRATE
  .WORD GNOVIC
  .WORD GEXPER
  .WORD GBONUS
  .WORD GTIME
  .WORD GLEVEL
  .WORD GHOLE
  .WORD GINSER
  .WORD GCMODE
  .WORD GCMOD1
  .WORD GCMOD2
  .WORD GCMOD3
  .WORD GATARI
  .WORD GCREDI
  .WORD GBONPT
  .WORD G2GAME
  .WORD GBOLIF
  .WORD GSPIKE
  .WORD GAPROA
  .WORD GSUPZA
```

my first shader

The busy heart of the `texter` shader is its `textloop`. This is a simple, but not especially short, piece of code that will write an individual character to the screen, dilating, distorting and deforming it as required. The `textloop` will keep doing this until the string in `textptr` runs out of characters, i.e. hits a `$00` byte such as at the end of `ataricop1`:

```
ataricop1: dc.b "COPYRIGHT 1981,",0
```

This 'have we run out of characters to paint' logic is at the very start of the loop:

```
textloop:
  loadb (textptr),r0 ; Put our text in r0.
  movei #ntxt,r1     ; Put our cleanup-and-exit routine in r1.
  addq #1,textptr    ; Move to the next character in textptr
  cmpq #0,r0         ; Check if it's $00
  jump eq,(r1)       ; If it is, jump to our cleanup-and-exit routine.
```

In this instance `textptr` is pointing to `ataricop1`. As soon as we hit the `0` at the very end we will `jump` to `ntxt`, which still stop the shader and bail out - unless 'drop shadow' mode was selected: in which case it will paint the string a second time (slightly offset) to achieve the dropshadow effect:

```
ntxt:
  cmpq #1,mode       ; In drop-shadow mode?
  jr ne,StopGPU      ; No, bail.
  nop
  moveq #0,mode      ; Turn off drop-shadow flag so we bail next time.
```

```
   movei #textloop,r0 ; Put our textloop routine in r0.
   movefa r6,textptr  ; Rest textptr to the start of the string.
   movefa r7,dstxy    ; Reset our destination position to its original.
   jump (r0)          ; Jump to r0 (textloop)
```

Listing .1: In fact there is no offset for drop-shadow mode implemented so the feature is unused. I suspect this is because the code is copy-pasted from elsewhere as we shall encounter a cousin of this routine later on.

Actually stopping the GPU involves clearing its command set to zero and loading that to the command register:

```
StopGPU:
   movei #G_CTRL,r1   ; Get our GPU commands.
   load (r1),r0       ; Load them to r0.
   bclr #0,r0         ; Clear the GPU flags in r0 to make it stop.
   store r0,(r1)      ; Actually stop the GPU by loading the cleared flags.
stoploop:
   jr stoploop        ; Spin until the GPU actually stops.
```

Now that we see how our loop through the text starts and exits, we will obviously be curious to know how the 'writing to the screen' part is effected. This is nearly identical to the method we stepped through for the 'Blitter'. The moment that pixels are written to the screen is in these statements at the end of `textloop`:

```
   movei #(SRCEN|CLIP_A1|UPDA1F|UPDA1|UPDA2|LFU_A|LFU_AN|DCOMPEN),r0
   ...
   store r0,(blit)    ;draw the sprite
```

Here `blit` is the equivalent of the `B_CMD` register we encountered in blitting. This is defined earlier on in the routine:

```
   movei #B_CMD,blit   ; Use blit as an alias for B_CMD
```

By writing the contents of the `r0` register to `blit` we are initiating a paint to the screen using all of the data we have setup earlier in the loop. This leads naturally to the question: what data? Well, not unlike 'Blitting' we set up a bunch of parameters and data points defining our 'source' (`A1`) and our 'destination'(`A2`). For both, these parameters and data points are concerned with things like positon, dimensions, screen width, and so on, not to mention where we are going to get the pixels for drawing from (in our case we already know that this is going to be from `pic`, the variable point to the contents of `beasty4.cry`).

We'll see how this data is calculated and derived shortly but first we'll take a look at the parameters and data points that are fed to the GPU and how. Below we have the GPU blitter registers that control the way our pixel data is written to the screen, in the order in which our `texter` routine writes them:

Common Name	T2K Name	Address in GPU RAM	Sample Value	Description
A1_PIXEL	a1_n+_pixel	$F0220C	00000001	Current X Position in the Pixel Data
A1_FPIXEL	a1_n+_nfpixel	$F02218	00000001	Current Y Position in the Pixel Data
A1_STEP	a1_n+_step	$F02210	00000001	Current Y Position in the Pixel Data
A1_FSTEP	a1_n+_fstep	$F02214	00000001	Current Y Position in the Pixel Data
A1_INC	a1_n+_inc	$F0221C	00000001	Current Y Position in the Pixel Data
A1_FINC	a1_n+_finc	$F02220	00000001	Current Y Position in the Pixel Data
A1_FLAGS	a1_n+_flags	$F02204	00000001	Current Y Position in the Pixel Data
A1_BASE	a1_n	$F02200	00000001	Current Y Position in the Pixel Data
A1_CLIP	a1_n+_clip	$F02208	00000001	Current Y Position in the Pixel Data

The GPU blitter registers in the order in which `texter` populates them in the listing below.

And here is the relevant code in the `texter` populating the registers:

```
   blit     REGEQU r13
   a1_n     REGEQU r14
   a2_n     REGEQU r15
   ...
rex:
   ...
   movei #A1_BASE,a1_n                        ; Make a1_n A1_BASE
   ...
   store xx,(a1_n+_pixel)                     ; Write to A1_PIXEL
   store yy,(a1_n+_fpixel)                    ; Write to A1_FPIXEL
   store r0,(a1_n+_step)                      ; Write to A1_STEP
   store r1,(a1_n+_fstep)                     ; Write to A1_FSTEP
   store xinc,(a1_n+_inc)                     ; Write to A1_INC
   store yinc,(a1_n+_finc)                    ; Write to A1_FINC

   movei #gpu_screen,r0         ; Prep gpu_screen as our destination
   movei #(PITCH1|PIXEL16|WID384|XADDINC),r1  ; Prep r1 for A1_FLAGS
   load (r0),r31                              ; Store gpu_screen in r31

   store r1,(a1_n+_flags)                     ; Write flags to A1_FLAGS
   movei #$1180180,r1                         ; Prep clip value.
   store r31,(a1_n)                           ; Write gpu_screen to A1_BASE
   store r1,(a1_n+_clip)                      ; Write clip value to A1_CLIP
```

This takes care of how we want the pixels to be written. On the other side of the equation we have to define for the GPU how want the source data in `beasty7.cry` to be read. Here is the order in which `texter` populates this other side of the scale:

Common Name	T2K Name	Address in GPU RAM	Value	Description
A2_BASE	a2_n	$F02224	_bass	Address of the source data
A2_PIXEL	a2_n+_pixel	$F02230	spixel	Address of the pixel data in pic5/beasty7.cry
A2_STEP	a2_n+_step	$F02234	Derived from ssize	The X and Y values for stepping
A2_FLAGS	a2_n+_flags	$F02228	Calculated	Current Y Position in the Pixel Data

The GPU blitter registers in the order in which `texter` populates them in the listing below.

And here we are doing the populating:

```
   store _bass,(a2_n)                  ; Write source address to A2_BASE
   store spixel,(a2_n+_pixel)          ; Write pixel data address to A2_PIXEL
   move ssize,r0                       ; Calculate the step size
   and lomask,r0
   neg r0
   and lomask,r0
   bset #16,r0                         ; Set the width of the character
   movei #(PITCH1|PIXEL16|WID320|XADDPIX),r1 ; Prep flags
   store r0,(a2_n+_step)               ; Write step size to A1_STEP
   store r1,(a2_n+_flags)              ; Write flags to A1_FLAGS
```

So far we've worked backwards from writing to the screen and got to the point where we completed the necessary admin of stuffing our source (A2) and destination (A1) registers with the necessary odds and ends the GPU needs to write a character of our string to the screen. We've even received some clue as to what some of this data may actually mean.

The value in `_bass` that we write to A2_BASE is the value at the very start of our `afont` data structure, in other words `pic2`, the contents of `beasty7.cry` where our font pixels live. This is the blob of data the GPU will index into for its pixels to actually paint:

```
afont:
  dc.l pic2      ; pixel data
```

We loaded this from the `in_buf` passed to `texter` by first storing it in `fontbase` and then passing it into `_bass`:

```
   movei #in_buf,r0    ;load the parameters into registers
   load (r0),textptr    ;point at start of textstring
   addq #4,r0
   load (r0),fontbase    ;point to start of font datastructure
   ...
   load (fontbase),_bass
```

The value `spixel` we write to A2_PIXEL is the X and Y position of the character we want to paint in `beasty7.cry/pic2`. We get this value by indexing the ASCII value of our current character into our `afont` data structure.

```
textloop:
  loadb (textptr),r0 ; Put our text in r0.
  movei #ntxt,r1     ; Put our cleanup-and-exit routine in r1.
  addq #1,textptr    ; Move to the next character in textptr
  cmpq #0,r0         ; Check if it's $00
  jump eq,(r1)       ; If it is, jump to our cleanup-and-exit routine.
  nop                ; It's no zero so we can get the x/y value for it from
```

```
    afont.
  subq #32,r0         ;ASCII 0-31 do NOT print
  shlq #2,r0
  add fontbase,r0     ; Add the fontbase address to our offset so we get the
                      ; actual address afont
  load (r0),spixel    ; Write the actual address to spixel.
```

In the above, if our character is `A`, for example, this will result in us pulling out the following entry in `afont`

```
afont:
  ...
 dc.l $a40001 ;A
```

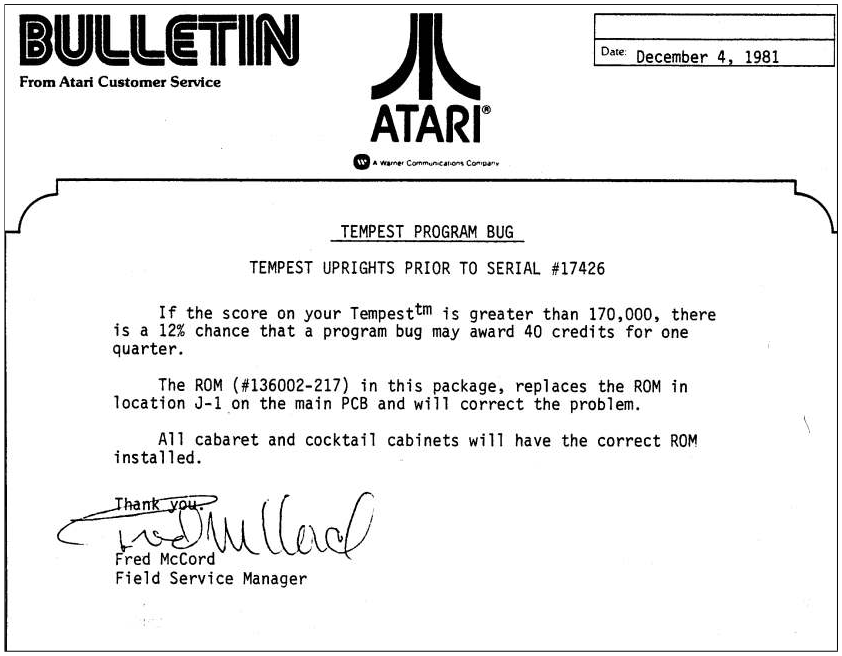
As we saw earlier this contains the Y (`a4`) and X (`01`) position of our `A` character in the `beasty7.cry` image:



Figure .1: The 'A' glyph located in a red box by our dimension and co-ordinate information.

tempest program bug

There is more than one version of Tempest because some time after the game's release in October 1981 players discovered a juicy little bug that hit arcade owners where it hurts: in the pocketbook.



BULLETIN

From Atari Customer Service

ATARI®

A Warner Communications Company

Date: December 4, 1981

TEMPEST PROGRAM BUG

TEMPEST UPRIGHTS PRIOR TO SERIAL #17426

If the score on your Tempest[tm] is greater than 170,000, there is a 12% chance that a program bug may award 40 credits for one quarter.

The ROM (#136002-217) in this package, replaces the ROM in location J-1 on the main PCB and will correct the problem.

All cabaret and cocktail cabinets will have the correct ROM installed.

Thank you.

Fred McCord
Field Service Manager

If you managed to reach a score of 170,000 or more there was a roughly one-in-eight chance that the next coin you popped in the cabinet would give you a whopping forty credits to play with. The run-up to Christmas was thoroughly ruined for Dave Theurer as this little heisenbug evolved from impossible and easily-dismissed reports from arcade owners into something that the Atari hardware technicians could reproduce in the lab. Theurer finally had to admit that something was wrong somewhere, and it was probably his fault.

Whatever the culprit was, the end result was that the number of credits available to the player was magically inflated to 40 when they reached game over. So the only possible place to start is by first looking at all the places that increment the variable responsible for storing credits. This is called `$$CRDT`, and like a lot of other variables, it lives in a region of memory known as the 'base page'. Another common name for this region is the 'zero page'. This latter name gives us a hint about the region's location, which starts at address zero. The first nine variables in Tempest's 'zero page' are given below. As you can see, `$$CRDT` is the seventh in the list, which means it has an address of `$06`.

```
;
;CONTROL & TIMING VARIABLES
;
QSTATE:  .BLKB 1    ;CONTAINS CODE FOR STATE ROUTINE (INDEX INTO ROUTAD)
QDSTATE: .BLKB 1    ;DISPLAY STATE
QNXTSTA: .BLKB 1    ;NEXT STATE CODE TO EXECUTE AFTER PAUSE
QFRAME:  .BLKB 1    ;FRAME COUNTER (WRAPS AT FF)
QTMPAUS: .BLKB 1    ;PAUSE TIMER (IN SECOND UNITS)
QSTATUS: .BLKB 1    ;STATUS FLAGS
;
;OTHER OVERHEAD
$$CRDT:  .BLKB 1    ;# OF CREDITS
$INTCT:  .BLKB 1    ;INTERRUPT COUNT
$COINA:  .BLKB 1    ;COIN MECHS
```

We've gone to some pains to point this out because when we look at all the places where we increment `$$CRDT` in the Tempest codebase there really isn't anything that jumps out as unusual. The routines in `COIN65.MAC` are responsible for counting the coins inserted by the player. There is a very remote chance our problem is in here, but since it is common code used by nearly all Atari titles we should only look there as a very last resort. We do find an interesting nugget in `ALSCO2.MAC`, one which explains the magic number of 40 reported by increasingly irate arcade managers:

```
ZATC4C==ZATC4E-ZATC4S
        LDA $$CRDT    ; GET CURRENT CREDITS
        CMP I,28      ; IS IT MORE THAN HEX $28 (i.e. 40)?
        IFCS          ; IF SO THEN..
        LDA I,28      ; PUT 40 IN THE 'A' REGISTER
        STA $$CRDT    ; AND MAXIMIZE # CREDITS TO 40.
```

```
        ENDIF
```

The value in $$CRDT is deliberately capped at 40, so the reason for this strangely specific number is because we are clamping it to keep the number of credits at a reasonable limit. This means that more often than not the real number in $$CRDT is much higher and whatever is responsible for this bug is incrememenenting it many times over. Since the maximum value we can store in a single byte is 255, and incrementing it beyond that would result in it wrapping around to zero again, this starts to suggest to us that perhaps $$CRDT is not being incremented haphazardly, every now and then, but repeatedly and rapidly many times over, perhaps many times a second. This points us in the direction of the code we run every time we paint a frame of the game.

I think at this point in his investigation Dave Theurer might have started to get an inkling of where the problem was. The devious mechanics we described in our copy protection chapter would have started to come back to him. Wasn't there something about writing crazy values to the region of zero page memory when his code detected when a potential pirate had altered the appearance of any of the game's screens? What was it we did again?

```
ZQVAVG::
  LDA QT3     ; CHECK THAT BOTH QT3 AND
  ORA QT6     ; QT6 ARE ZERO.
  IFNE        ; IF THEY ARE NOT THEN
  LDA I,17    ; CHECK IF THE PLAYER' SCORE IS GREATER THAN 170,000
  CMP LSCORH  ; LSCORH CONTAINS THE FIRST 2 DIGITS OF THE PLAYER SCORE
  IFCC        ; IF IT IS GREATER THAN OR EQUAL TO 17..
  LDX LSCORL  ; LOAD WHATEVER IS IN THE LSCORL BYTE
  INC X,0     ; AND USE THAT TO INCREMENT ONE OF OUR 'ZERO-PAGE' BYTES.
  ENDIF       ; IN THE HOPE OF CAUSING SOME HAVOC.
  ENDIF       ; HAVOC SECURED.
```

A realization is beginning to dawn in the Tempest lab. We have discovered another magic number from the frantic bug reports. If the score is at or above 170,000 (i.e. if the first two digits of the score are 17), we unleash a bit of havoc. Of course, we only do this if the checksums in QT3 or QT6 are non-zero. And this only happens if the screen content has been altered by a third party. For example, if they have altered the copyright line to something else on the title screen.

This would appear to be our smoking crater. If our copy-protection checks fail, and the player's score is 170,000 or above, we take the last two digits of their score from the byte LSCORL and use that as index into the 'zero page' region of memory to select a random variable to increment. So for example if the player's score is 172,004 we will increment the fifth byte in zero-page, which is QTMPAUS. If their score is 172,002 we will increment QDSTATE. And if it is 172,006, or any value that ends with a 06, we will increment our old friend $$CRDT. We won't just increment it once and done - we'll

increment it once every frame, dozens of times a second. So it would appear that as long as you can score relatively highly and get your score ending with a 06 we will happily clock up your credits like billy-oh.

But of course for any of this to happen, the copy-protection has to fail. There's no good reason that should happen. Unless we forgot to update the expected checksum values the last time we altered some screen content before shipping the game. We wouldn't have done that, would we?

We take a look at the routine responsible for performing the copyright protection check.

```
ZATVG2::
  LDA SECUVY      ; ARE WE DISPLAYING ONE OF THE TITLE SCREENS?
  IFNE            ; IF WE ARE THEN:
  LDY I,27        ; FOR ALL 39 BYTES IN THE THE ATARI COPYRIGHT STRING..
  LDA I,0E        ; STARTING FROM OUR HARD-CODE LITERAL OF '0E'
  SEC             ; CLEAR THE CARRY BIT SO IT DOESN'T INTERFERE
  ; LOOP THROUGH EACH CHARACTER IN THE LINE THAT CONTAINS '(C) MCMLXXX
  ; ATARI' TO CALCULATE A FINAL CHECKSUM VALUE.
  BEGIN           ; LOOP FROM 27 TO 0
  SBC NY,SECUVG   ; SUBTRACT VALUE IN THE CURRENT CHAR FROM 'A' REGISTER.
  DEY             ; DECREMENT Y TO GO TO THE PREVIOUS CHARACTER.
  MIEND           ; LOOP UNTIL Y IS 0.
  TAY             ; STORE THE RESULT IN A IN Y.
  IFNE            ; IF THE RESULT IS ZERO, THE CHECKSUM PASSES OTHERWISE:
  EOR I,0E5       ; CHECK IT AGAINST THE CHECKSUM FOR ANOTHER SCREEN
  ENDIF
  IFNE            ; IF THAT PASSES WE'RE DONE OTHERWISE:
  EOR I,02A       ; CHECK IT AGAINST THE CHECKSUM FOR ANOTHER SCREEN
  ENDIF
  STA QT3         ; STORE THE RESULT OF OUR CHECK IN QT3.
  ENDIF
  ENDIF
```

It comes back to us now. We iterate through all the characters on a particular line in the screen and use them to calculate a primitive checksum. This is as simple as cumulatively subtracting the value of each byte on the line from an initial value and checking that the result is what we expect. After a bit of pencil and paper testing we find that the first test passes: the checksum on the high score screen results in a value of E5 in hex. We run the same test on the title screen and discover our problem: the result is not 2A, it's 29. Oops.

Time to get on to Fred McCord and let him know he has a Christmas Field Service Bulletin to write and a bunch of replacement ROMS to burn and package out to customers.

strings of tempest 2000

Three different fonts are used in *Tempest 2000*. Each one is set out in a sprite sheet, and comes with an accompanying array that maps ASCII characters a set of co-ordinates in the sheet.

afont - the 'Atari' font

For example in the `afont`, also known as the *Atari* font, the letter `A` is found at X position 1 and Y position 164 (`$A4`), while `B` is at X position 20 (`$14`) and Y position 164 (`$A4`), in the sprite sheet `beasty4.cry`.




```
afont:
  dc.l pic2       ; beasty4.cry
  dc.l $0012000f  ;
  dc.l $b6011e    ; Space
  dc.l $a40001    ; A
  dc.l $a40014    ; B
  dc.l $a40027    ; C
  dc.l $a4003a    ; D
  dc.l $a4004d    ; E
  dc.l $a40060    ; F
  dc.l $a40073    ; G
  dc.l $a40086    ; H
  dc.l $a40099    ; I
  dc.l $a400ac    ; J
  dc.l $a400bf    ; K
  dc.l $a400d2    ; L
  dc.l $a400e5    ; M
  dc.l $a400f8    ; N
```

What this looks like in practice is demonstrated below. The copyright message from the title screen is displayed by loading the `afont`, the text string, and then using the `centext` routine to render the string to the display.




```
; Write "Copyright 1981" to screen.
lea afont,a1     ; Select afont image map for our font.
lea ataricop1,a0 ; Load "Copyright 1981" string to a0
move #190-8,d0   ; Set the Y Pos.
add palfix2,d0   ; Adjust for PAL.
jsr centext      ; Write the text in a0 to the screen.

; Write "1994 Atari Corp" to screen.
lea afont,a1     ; Select afont image map for our font.
lea ataricop2,a0 ; Load "1994 Atari Corp" string to a0.
move #207-5,d0   ; Set the Y pos of the text.
add palfix2,d0   ; Adjust for PAL.
jsr centext      ; Write the text in a0 to the screen.
```

The routine `centext` is useful for display static, horizontally centred messages. But what about cases where we want to display a message for a brief period of time? For this we have a pair of routines called `setmsg` and `drawmsg` that are used to display contextual messages to the player in the 'Atari' font. For example, when they are doing especially well and get a 'power up':




```
lea drtxt,a0                ; Point a0 to  "yes! yes! yes!" string.
clr.l d0                    ; Set X position.
move.l #$8000,d1            ; Set Y position.
bsr setmsg                  ; Display the message.
```

Once we've defined the message and its horizontal and vertical position above, we call `setmsg` to put it on a display timer. We also set its initial X and Y position as well as the final position of the message as it 'decays'.

Every time we draw objects on the screen we will call the `drawmsg` routine to render any active messages in `afont`. We don't give that routine in full here as it is quite long!

```
; ******************************************************************
; Update the message displayed to the player with whatever is in a0.
; The display of the message is handled by 'drawmsg'.
; ******************************************************************
setmsg:
        move #100,msgtim1          ; set default Messager parameters
        move #50,msgtim2
        move.l d0,msgxv            ; X position for second phase of message.
        move.l d1,msgyv            ; Y position for second phase of message.
        move.l #$10000,msgxs       ; Initial X position.
        move.l #$10000,msgys       ; Initial Y position.
        move.l a0,msg              ; Set the message object.
        rts
```

bfont - the 'Paused' font

This metallic space-opera of a font is used primarily for the 'Demo' and 'Paused' messages but is also used when entering a high score.



```
bfont:
  dc.l pic6     ; beasty8.cry
  dc.l $001f001b
  dc.l $470119  ;Space
  dc.l $010001  ;A
  dc.l $010024  ;B
  dc.l $010047  ;C
  dc.l $01006a  ;D
  dc.l $01008d  ;E
  dc.l $0100b0  ;F
  dc.l $0100d3  ;G
  dc.l $0100f6  ;H
  dc.l $010119  ;I
  dc.l $240001  ;J
  dc.l $240024  ;K
  dc.l $240047  ;L
  dc.l $24006a  ;M
  dc.l $24008d  ;N
```

Here's how it looks during the 'Pause' screen.



```
jsr text2_setup
; Display the 'paused' text.
lea in_buf,a0           ; Point our GPU RAM input buffer at a0.
```

```
move.l #$490074,32(a0)  ; was 35
move.l #pautext,(a0)    ; Point a0 at "paused" string.
lea texter,a0           ; Load the GPU module in stoat.gas.
jsr gpurun              ; Run the selected GPU module.
jsr gpuwait             ; Wait for the GPU to finish.
```

The 'Pause' screen has a commented-out feature that was likely in use when the game was going through testing and review. This is a date-stamped version string, referencing the date of the current build. What we find in the source code is the date of the last build that this feature was used: 12th November 1993, or 11th December 1993. My guess is the latter - since the intended audience for the test builds were American.




```
jsr text2_setup ; Select 'bfont'.
move.l #$400040,32(a0) ; Set position.
move.l #vertext,(a0) ; Select version string.
lea texter,a0   ; Load GPU module in stoat.gas.
jsr gpurun      ; Run selected GPU module.
jsr gpuwait     ; Wait for GPU to finish.
```

cfont - the 'Arcade' font

This one has the unusual distinction of containing two fonts in one: or rather the same font in different colours. Upper case strings are mapped to a green/orange version of the font, while lower case strings are mapped to a blue/white version.




```
cfont:
  dc.l pic      ; beasty3.cry
  dc.l $00080008
  dc.l $b50137  ;Space
  dc.l $b5001b  ;A
  dc.l $b50026  ;B
  dc.l $b50031  ;C
  dc.l $b5003c  ;D
  dc.l $b50047  ;E
  dc.l $b50052  ;F
  dc.l $b5005d  ;G
  dc.l $bf001b  ;a
  dc.l $bf0026  ;b
  dc.l $bf0031  ;c
  dc.l $bf003c  ;d
  dc.l $bf0047  ;e
  dc.l $bf0052  ;f
  dc.l $bf005d  ;g
```

We can see an example of this in the string *"press FIRE to play"*. *"FIRE"* is rendered in

the green/orange font, while the rest of the text is rendered in the blue/white font used for lower case text.




```
; Draw 'press FIRE to play'
lea cfont,a1    ; Load the small font
lea autom2,a0   ; "press FIRE to play"
move #180,d0    ; Set y position
jsr centext     ; Draw the text in the centre.
```




```
lea cfont,a1          ; Load the small regular font to a1.
move.l enl3,a0        ; "enter something EGOTISTICAL!"
move #80,d0           ; Set Y position of text.
jsr centext           ; Display horizontally centred text.
```

```
; ***************************************************************
; text_setup
; Set up text in the 'Atari' font.
; ***************************************************************
text_setup:
        lea in_buf,a0               ; Point our GPU RAM input buffer at a0.
        move.l #afont,4(a0)         ; 'Atari' font data structure
tsu:
        move.l #0,8(a0)             ; Start of font data structure.
        move.l #0,12(a0)            ; Drop-shadow in X direction.
        move.l #$10000,d0           ; Store 10000 in d0.
        move.l d0,16(a0)            ; Drop-shadow in Y direction.
        move.l d0,20(a0)            ; Scale factor in X direction.
        move.l #0,24(a0)            ; Scale factor in Y direction.
        move.l #0,28(a0)            ; Text shear in X direction.
        move.l #0,36(a0)            ; Mode 0: Straight Draw.
        rts
```

strings of tempest

We covered the mechanics of generating strings in *Tempest* in the earlier chapter 'macro language fun'. So here we'll take a small selection of the message strings used in *Tempest* and recreate how they appear on screen.




```
MESS ATARI,BLULET,1,92
SATARI: ASCVH <^ MCMLXXX ATARI>
```




```
MESS RATE,GREEN,1,10.
ERATE: ASCVH <RATE YOURSELF>
```




```
MESS HIGHS,RED,0,38
EHIGHS: ASCVH 0BC,<HIGH SCORES>
```




```
MESS BOLIF,TURQOI,1,89
EBOLIF: ASCVH 0C8,<BONUS EVERY >
```




```
MESS SUPZA,BLULET,1,0A0 ;NEW
     SUPER
ESUPZA: ASCVH <SUPERZAPPER
     RECHARGE>
```




```
MESS SPIKE,WHITE,0,0  ;AVOID
     SPIKES
ESPIKE: ASCVH -72.,<AVOID SPIKES>
```




```
MESS 2GAME,GREEN,1,89 ;2 GAME
     MINIMUM
E2GAME: ASCVH <2 CREDIT MINIMUM>
```




```
MESS PRFIR,YELLOW,1,-10. ;PRESS
     FIRE
EPRFIR: ASCVH <PRESS FIRE TO
     SELECT>
```



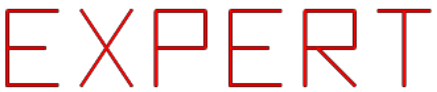

```
MESS CMODE,GREEN,1,80 ;FREE PLAY
ECMODE: ASCVH 0,<FREE PLAY>
```




```
MESS HOLE,GREEN,1,-55. ;HOLE
EHOLE: ASCVH 8B,<HOLE>
```




```
MESS NOVIC,RED,1,-30. ;NOVICE
ENOVIC: ASCVH 0AA,<NOVICE>
```




```
MESS EXPER,RED,1,-30. ;EXPERT
EEXPER: ASCVH 4A,<EXPERT>
```




```
MESS GAMOV,GREEN,1,56  ;GAME OVER
EGAMOV: ASCVH <GAME OVER>
```

unused stars

Tempest 2000 is full of starfields, from the title screen to the game itself - there's always a field of stars up to something in the background.



Figure .1: Example of starfield effects. The middle one is not used by the game.

We'll start in totally the wrong place by looking at piece of code that actually goes unused in the game. A routine called `initstarfield` that populates a random space of stars. This was probably a first iteration at drawing a star-spangled background and dropped later when fancier alternatives were developed. Before we look at the fancy, and necessarily more complex alternatives, let's take a look at what populating a random-ish starfield looks like.

What we have to do first is populate a data structure for our starfield. This is a list (or array) of elements with each element containing an X, Y, and Z co-ordinate plus a color value for the star. There are 127 such elements in total, and we store the number of elements as a 'header' or first entry in the array. We'll store this array in an address called `field1` and do the work of populating it in a routine called `initstarfield`.

The values we come up with for X and Y are totally random. All we do is come up with a number between 0 and 127 for the X and Y co-ordinates and a number between 0 and 512 for our Z co-ordinate. We then use the X and Y co-ordinates to seed a color value.

Here is the routine. As we said before, it ended up unused but has the merit of being relatively simple to read and understand:

```
initstarfield:
;
; initialise a starfield data structure for the GPU to display

  lea field1,a0     ; field1 is where the data structure is stored
  move #127,d7      ; The number of times to loop through 'isf' below.
  move.l #128,(a0)+ ; Store the number of stars at the start of field1.
  ; Create 128 stars and store them as an array in field1.
  ; Each star is: X,Y,Z,cry_index
isf:
  bsr rannum      ; Get a random number between 0 and 255 and store in d0
  move d0,d2      ; Stow d0 in d2 for use later
  sub #$80,d0     ; Get rid of the high bit so the num is between 0 and 128
  swap d0         ; Turn e.g. 00000032 into 00320000
  move.l d0,(a0)+ ; Store our rand num as our X co-ordinate.

  bsr rannum      ; Get a random number between 0 and 255  and store in d0
  move d0,d3      ; Stow d0 in d3 for use later
  sub #$80,d0     ; Get rid of the high bit so the num is between 0 and 128
  swap d0         ; Turn e.g. 00000032 into 00320000
  move.l d0,(a0)+ ; Store our rand num as our Y co-ordinate.

  bsr rannum      ; Get a random number between 0 and 255  and store in d0
  asl #1,d0       ; Multiply the number by 2 so that its between 0 and 512
  swap d0         ; Turn e.g. 00000032 into 00320000
  move.l d0,(a0)+ ; Store our rand num as our Z co-ordinate.

  ; Use our X and Y co-ordinates to come up with a random color for the star
    .
  ; So if X(d2) is 00000032 and Y(d3) is 00000088:
  and #$f0,d2  ; 00000032 -> 00000030
  lsr #4,d3    ; 00000088 -> 80000008
  and #$0f,d3  ; 80000008 -> 00000008
  or d2,d3     ; 00000030 or 00000008 -> 00000038
  lsl #8,d3    ; 00000038 -> 00003800
  move d3,(a0) ; Store our color value.

  lea 20(a0),a0  ; Move a0 20 bytes ahead ready for the next element.
  dbra d7,isf    ; Loop until d7 is 0.
  rts
```

Listing .1: Populating an unused starfield data structure. This is a fuzzier version of the ring starfield used in the credits screen.

This is our data structure (or at least the first 2 elements in it) in a table:

Value	Decimal	Description
0000000E	127	Number of Elements
00064000	127	Element 1: X co-ordinate
00064000	127	Element 1: Y co-ordinate
00064000	127	Element 1: Z co-ordinate
00064000	127	Element 1: Color Value
00064000	127	Element 2: X co-ordinate
00064000	127	Element 2: Y co-ordinate
00064000	127	Element 2: Z co-ordinate
00064000	127	Element 2: Color Value

First 2 elements of the data structure created by `initstarfield`.

And this is what our data structure looks like when its animated by our shader `fastvector` (more of which later).

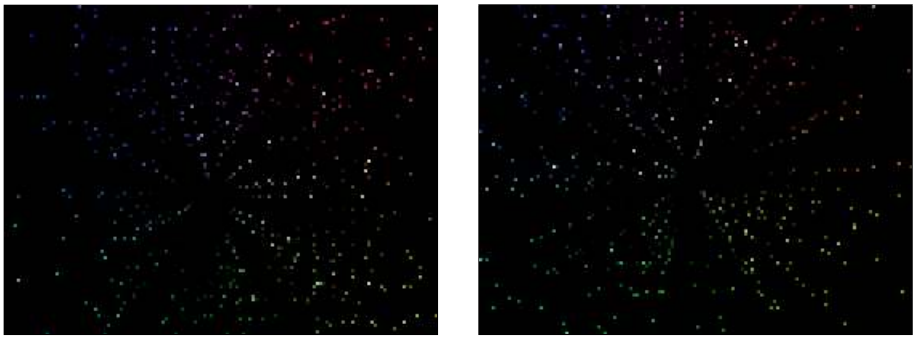


Figure .2: Frames of the unused starfield during animation.

The actual starfield used in the credits screen makes use of a sine table to accurately calculate the co-ordinates of pixels in an expanding ring structure.

```
sines:
dc.w $0003,$0609,$0C0F,$1215,$181B,$1E21,$2427,$2A2D
dc.w $3033,$3639,$3B3E,$4144,$4649,$4B4E,$5053,$5557
dc.w $595C,$5E60,$6264,$6667,$696B,$6D6E,$7071,$7274
dc.w $7576,$7778,$797A,$7B7B,$7C7D,$7D7E,$7E7E,$7E7E
dc.w $7E7E,$7E7E,$7E7E,$7D7D,$7C7B,$7B7A,$7978,$7776
dc.w $7573,$7271,$6F6E,$6C6B,$6967,$6563,$615F,$5D5B
dc.w $5957,$5552,$504D,$4B48,$4643,$403E,$3B38,$3533
dc.w $302D,$2A27,$2421,$1E1B,$1815,$120F,$0C08,$0502
```

```
dc.w $00FD,$FAF7,$F4F0,$EDEA,$E7E4,$E1DE,$DBD8,$D5D2
dc.w $CFCD,$CAC7,$C4C1,$BFBC,$B9B7,$B4B2,$B0AD,$ABA9
dc.w $A6A4,$A2A0,$9E9C,$9A98,$9795,$9392,$908F,$8D8C
dc.w $8B8A,$8988,$8786,$8584,$8483,$8382,$8282,$8282
dc.w $8282,$8282,$8283,$8383,$8485,$8586,$8788,$898A
dc.w $8B8D,$8E8F,$9192,$9496,$9799,$9B9D,$9FA1,$A3A5
dc.w $A7AA,$ACAE,$B1B3,$B6B8,$BBBD,$C0C3,$C5C8,$CBCE
dc.w $D1D4,$D7D9,$DCDF,$E2E6,$E9EC,$EFF2,$F5F8,$FBFE
dc.w $0000,$0192,$0323,$04B5,$0645,$07D5,$0963,$0AF0
dc.w $0C7C,$0E05,$0F8C,$1111,$1293,$1413,$158F,$1708
dc.w $187D,$19EF,$1B5C,$1CC5,$1E2A,$1F8B,$20E6,$223C
dc.w $238D,$24D9,$261F,$275F,$2899,$29CC,$2AFA,$2C20
dc.w $2D40,$2E59,$2F6B,$3075,$3178,$3273,$3366,$3452
dc.w $3535,$3611,$36E4,$37AE,$3870,$3929,$39DA,$3A81
dc.w $3B1F,$3BB5,$3C41,$3CC4,$3D3D,$3DAD,$3E14,$3E70
dc.w $3EC4,$3F0D,$3F4D,$3F83,$3FB0,$3FD2,$3FEB,$3FFA
dc.w $3FFF,$3FFA,$3FEB,$3FD2,$3FB0,$3F83,$3F4D,$3F0D
dc.w $3EC4,$3E70,$3E14,$3DAD,$3D3D,$3CC4,$3C41,$3BB5
dc.w $3B1F,$3A81,$39DA,$3929,$3870,$37AE,$36E4,$3611
dc.w $3535,$3452,$3366,$3273,$3178,$3075,$2F6B,$2E59
dc.w $2D40,$2C20,$2AFA,$29CC,$2899,$275F,$261F,$24D9
dc.w $238D,$223C,$20E6,$1F8B,$1E2A,$1CC5,$1B5C,$19EF
dc.w $187D,$1708,$158F,$1413,$1293,$1111,$0F8C,$0E05
dc.w $0C7C,$0AF0,$0963,$07D5,$0645,$04B5,$0323,$0192 ; <-- sine ($0C7C)
dc.w $0000,$FF6E,$FDDD,$FC4B,$FABB,$F92B,$F79D,$F610
dc.w $F484,$F2FB,$F174,$EFEF,$EE6D,$ECED,$EB71,$E9F8
dc.w $E883,$E711,$E5A4,$E43B,$E2D6,$E175,$E01A,$DEC4
dc.w $DD73,$DC27,$DAE1,$D9A1,$D867,$D734,$D606,$D4E0
dc.w $D3C0,$D2A7,$D195,$D08B,$CF88,$CE8D,$CD9A,$CCAE
dc.w $CBCB,$CAEF,$CA1C,$C952,$C890,$C7D7,$C726,$C67F
dc.w $C5E1,$C54B,$C4BF,$C43C,$C3C3,$C353,$C2EC,$C290
dc.w $C23C,$C1F3,$C1B3,$C17D,$C150,$C12E,$C115,$C106 ; <-- cosine ($C23C)
dc.w $C101,$C106,$C115,$C12E,$C150,$C17D,$C1B3,$C1F3
dc.w $C23C,$C290,$C2EC,$C353,$C3C3,$C43C,$C4BF,$C54B
dc.w $C5E1,$C67F,$C726,$C7D7,$C890,$C952,$CA1C,$CAEF
dc.w $CBCB,$CCAE,$CD9A,$CE8D,$CF88,$D08B,$D195,$D2A7
dc.w $D3C0,$D4E0,$D606,$D734,$D867,$D9A1,$DAE1,$DC27
dc.w $DD73,$DEC4,$E01A,$E175,$E2D6,$E43B,$E5A4,$E711
dc.w $E883,$E9F8,$EB71,$ECED,$EE6D,$EFEF,$F174,$F2FB
dc.w $F484,$F610,$F79D,$F92B,$FABB,$FC4B,$FDDD,$FF6E
```

A table like this allows us to 'hard code' the points of an arbitrarily sized circle by providing the sine and cosine values for each point. To get the X and Y co-ordinates of a specific point in the circle you pull out the corresponding sine and cosine values from the table, multiply each by your desired radius, and that gives you your X and Y value to draw your point on the screen. You do that for each point (say 32 in total) and you have a ring of dots that you can connect to form a circle.

$$x = \cos(\theta) * radius \tag{.1}$$

$$y = \sin(\theta) * radius \tag{.2}$$

The `sines` table seems inscrutable at first glance but the specific use of it made by the `ringstars` routine is straightforward. The routine plots a circle of 32 points and for each point it uses the current point number (e.g. 31) to pull out a sine and cosine value from the first entry in the corresponding row on the table. If we imagine we're plotting the 31st point this means for the sine value, we will take the first value from row 31, and for the cosine value it will take the first value 8 rows after that. As you can see in the above table this corresponds to `$0C7C` for the sine and `$C23C` for the cosine.

With this in hand, and a radius of 200 pixels, we can calculate our X and Y co-ordinates. The steps are the same for both the X and Y values, except for the use of `sine` and `cosine`:

Step No.	Description	Cosine	Sine
1	Value taken from `sine` table	`$0C7C`	`$C23C`
2	Use last byte only	`$007C`	`$003C`
3	Multiply by radius (200)	`$60E0`	`$2EE0`
4	Shift Left by 7 bits	`$00307000`	`$00177000`
5	Final value treated as 16:16 fraction	X:`48.28672`	Y:`23.28672`

Steps to calculate the X and Y value for Point 31.

The whole process is admittedly convoluted but the last step may seem especially mysterious. In order to treat the X and Y values as fractions rather than whole numbers, the blitter will split our final value in half and treat the left-hand side as a whole number and the right-hand side as a fraction. So `$00307000`, for example, becomes `$0030` and `$7000`, which are `48` and `28672` in decimal respectively, giving us a decimal fractional value of `48.28672`.

There is another complication we have glossed over in Step 2 above. Since sine and cosine values can be positive or negative we have to know how +/- values are indicated here. The answer is that any value of `$80` or above is treated as negative: so `$FE` is -1, `$FD` is -2, all the way down to `$81` which is -127 and `$8000` which is -128. In our example both `$7C` and `$3C` are less than `$80` so both are treated as positive.

Here then is the first half of the `ringstars` routine where the process we've outlined is implemented:

```
ringstars:
;
; 'initialise a starfield of 8 rings of 64 stars each': it says this,
```

```
; but its actually 8 rings of 32 stars each.

  move #200,d5       ; the radius we will use for the ring
rst:
  lea field1,a0      ; field1 is where we'll store it
  lea sines,a1       ; our sine table (see later) in a1
  lea p_sines,a2     ; our positive-only sine table in a2
  move #7,d7         ; d7 will track our 8 rings
  move.l #256,(a0)+  ; header of our structure: no. of stars (256)

  move #$0000,d4     ; d4 will contain the star color
ring1:
  move #32,d6        ; d6 will track no. of stars per ring (32)
ring2:
  move d6,d0         ; Put no. of current star in d0
  asl #3,d0          ; Multiply by 8 to get the row in our sine table
  move.b 0(a1,d0.w),d1  ; Get the sine from position d0 in our sine table
  add.b #$40,d0      ; Add offset to index the cos in our sine table
  move.b 0(a1,d0.w),d2  ; Get the cos from position d0 in our sine table.
  ext d1             ; Chop off everything but the last byte for sine.
  ext d2             ; Chop off everything but the last byte for cos.
  muls d5,d1         ; Get Y by multiplying sin * radius
  muls d5,d2         ; Get X by multiplying cos * radius
  asl.l #7,d1        ; Shift left 7 bits to create an X value
  asl.l #7,d2        ; Shift left 7 bits to create a Y value
  move.l d1,(a0)+    ; Move X into our star data structure.
  move.l d2,(a0)+    ; Move Y into our star data structure.
```

Listing .2: Populating the data structure for the starfield used in the Tempest 2000 credits screen.

And here is what plotting these points ourselves looks like. Iterating above for a full 32 points, pulling values from our `sines` table as we go, gives us a simple circle as expected:

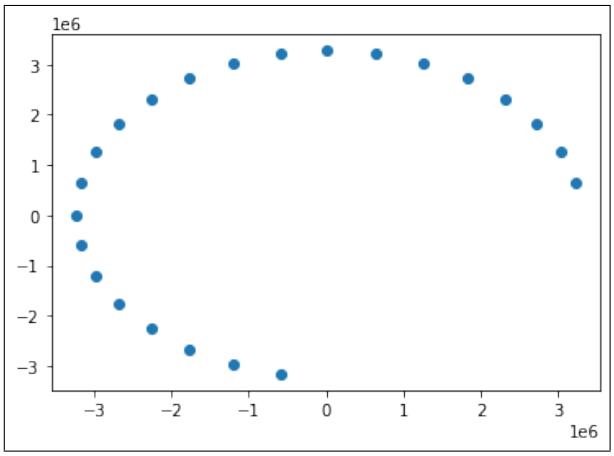


Figure .3: Each of the 32 points as plotted using the X/Y values calculated using the procedure above.

The remainder of the routine calculates a Z value for our point in 3D space and this one is even more intricate.

```
   ; Calculate the Z value.
   lsl #2,d0
   and #$ff,d0
   move.b 0(a1,d0.w),d1
   move.b 0(a2,d0.w),d2
   and #$f0,d2
   lsl #4,d2
   ext d1
   bpl sposss
   neg d1
 sposss:
   swap d1
   clr d1
   asr.l #1,d1
   clr.l d0
   move d7,d0
   swap d0
   lsl.l #6,d0    ;Z position according to ring no.
   add.l d1,d0
   move.l d0,(a0)+ ; Move Z into our star data structure

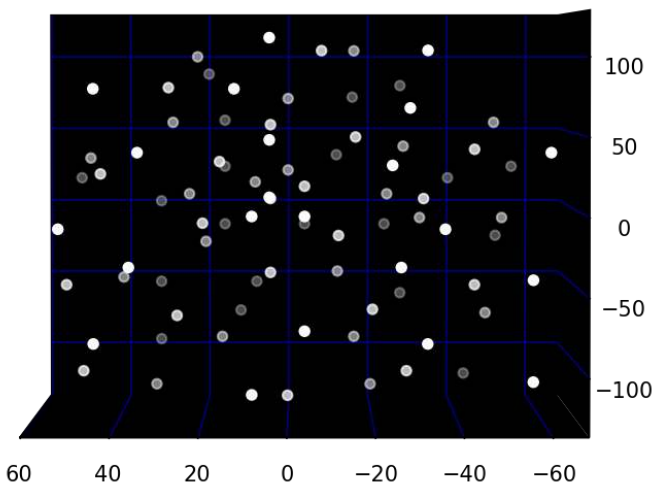
   move d4,d0
   add d2,d0
   move d0,(a0)    ;colour
   lea 20(a0),a0
   dbra d6,ring2
   add #$2000,d4
   dbra d7,ring1
```

```
    rts
```

Listing .3: Calculating the Z and color value for the starfield data structure used in the Tempest 2000 credits screen.

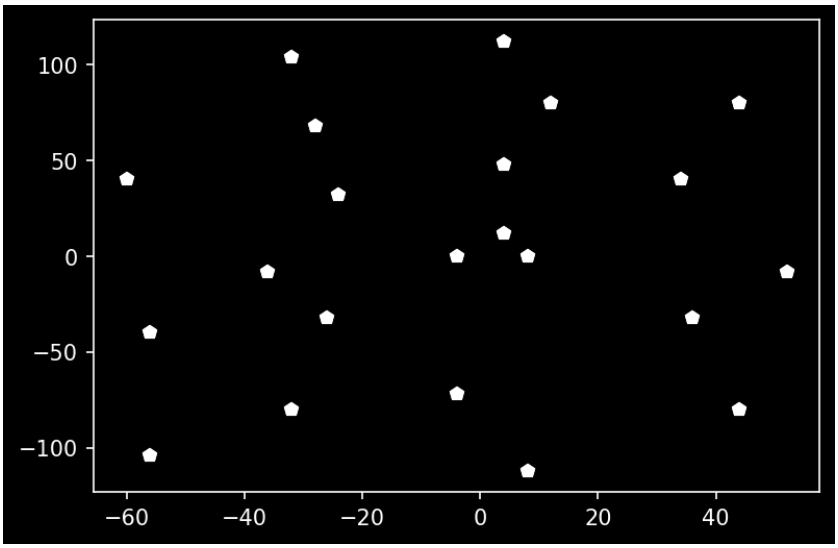
star planes

Between levels in *Tempest*, the player flies through a starfield as they travel to the next level. The game successfully creates the impression of stars passing above and below the player as they dive towards a well in the distance. We're going to look at how this effect is constructed from relatively simple elements. We can start by a head on view of a starfield in the distance.

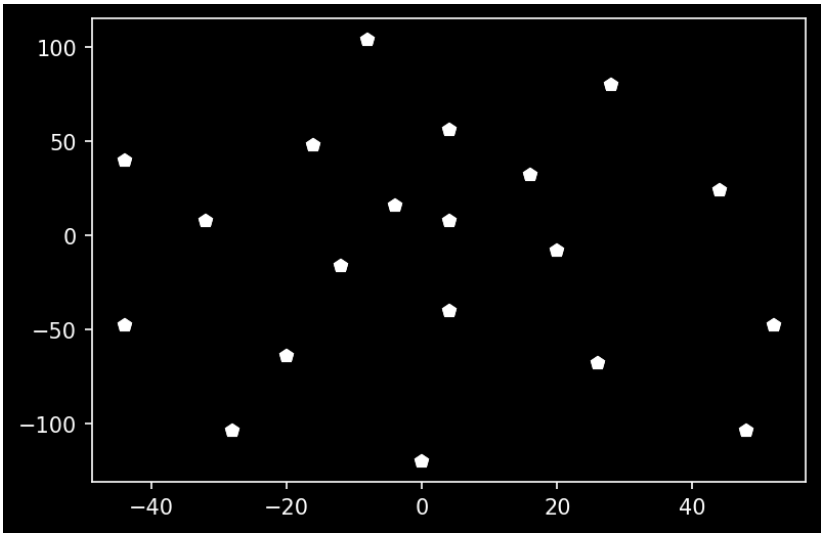


Starfield viewed head on.

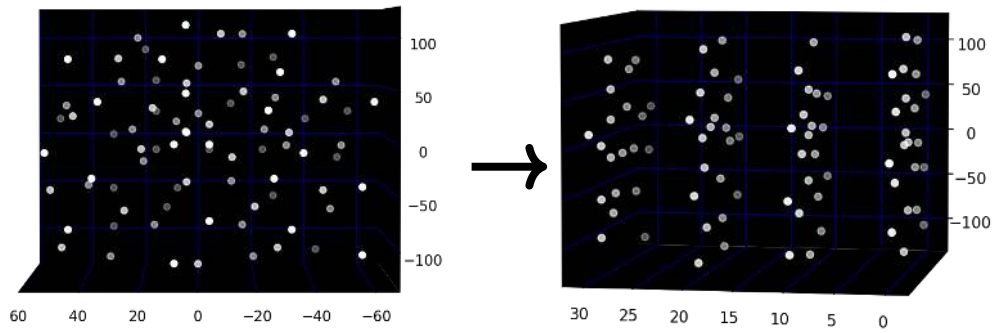
```
    .MACRO MSTAR1
    ICVEC
    SCDOT -8,0
    SCDOT 8,0C
    SCDOT 10,0
    SCDOT 8,30
    SCDOT -30,20
    SCDOT -34,-20
    SCDOT -8,-48
    SCDOT 48,-20
    SCDOT 44,28
    SCDOT 18,50
    SCDOT -38,44
    SCDOT -48,-8
    SCDOT -40,-50
    SCDOT 10,-70
    SCDOT 58,-50
    SCDOT 68,-8
    SCDOT 58,50
    SCDOT 8,70
    SCDOT -40,68
    SCDOT -78,28
    SCDOT -70,-28
    SCDOT -70,-68
    SCVEC 0,0,0
    RTSL
    .ENDM
```




```
    .MACRO MSTAR2
    ICVEC
    SCDOT 8,8
    SCDOT -8,10
    SCDOT -18,-10
    SCDOT 8,-28
    SCDOT 28,-8
    SCDOT 20,20
    SCDOT 8,38
    SCDOT -20,30
    SCDOT -40,8
    SCDOT -28,-40
    SCDOT 34,-44
    SCDOT 58,18
    SCDOT 38,50
    SCDOT -10,68
    SCDOT -58,28
    SCDOT -58,-30
    SCDOT -38,-68
    SCDOT 0,-78
    SCDOT 60,-68
    SCDOT 68,-30
    RTSL
    .ENDM
```



Hopefully this diagram gives you an appropriate idea of stars in a deep field receding into the distance. In this toy rendering we are actually faithful to the method used in *Tempest*. We can begin to get a sense of what the method is if we rotate our view of the field to the right so that we're looking at it side on.



As you can see the star field in this instance consists of four layers of stars, one behind the other. Tempest defines each of these layers separately and we've rendered each in full 2-dimensional mode in the facing and following pages.

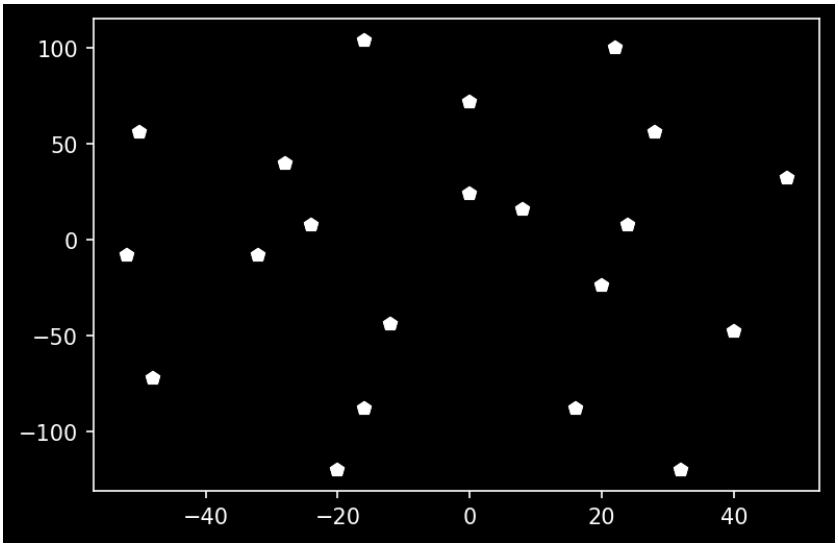
```
STAR1:  MSTAR1
STAR2:  MSTAR2
STAR3:  MSTAR3
STAR4:  MSTAR4
```

This gives us the general idea of how the starfield is constructed but it has been simplified quite a bit. As it turns out, there are not four layers to `Tempest`'s starfield, but eight. We find this defined here as `NPLANE`, which is used to create an array of eight bytes that will hold the current Z position (or depth in field) of each starfield. This array is confusingly named `PLANEY`.

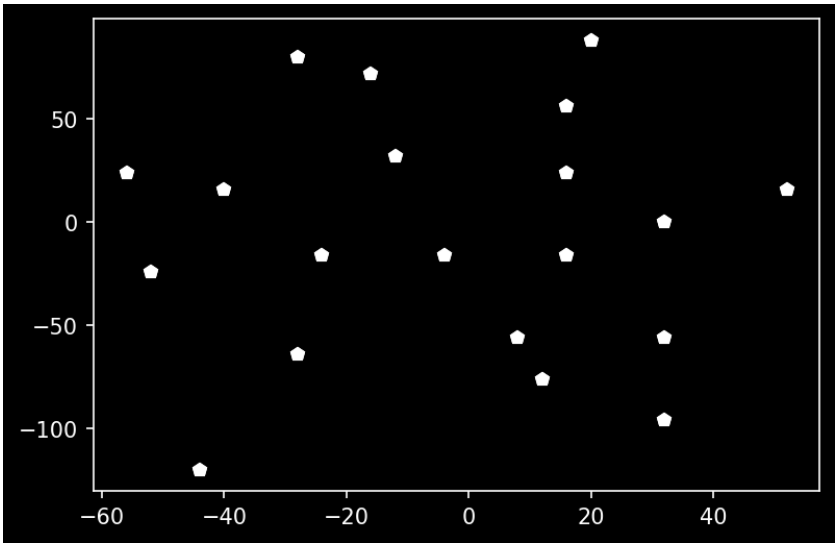
```
 .SBTTL CONSTANTS-COUNTS
NPLANE= 8
PLANEY: .BLKB NPLANE ;STAR FIELD PLANES
```

I say confusingly, because we might reasonably have expected it to be called `PLANEZ`, as the Z axis is the one typically used to represent depth. But the co-ordinate system used in Tempest thought of the Y axis as representing depth and treated X and Z as representing the horizontal and vertical axis respectively. For the sake of simplicity, I've not adopted this quirk in our descriptions here. So for us, `PLANEY` contains the Z positions of all 8 layers in our starfield.

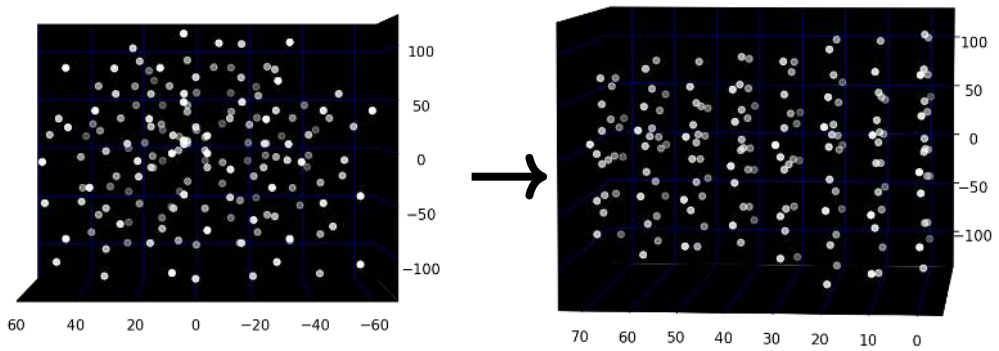
```
    .MACRO MSTAR3
    ICVEC
    SCDOT 10,10
    SCDOT 0,18
    SCDOT -30,8
    SCDOT -18,-2C
    SCDOT 28,-18
    SCDOT 30,8
    SCDOT 38,38
    SCDOT 0,48
    SCDOT -38,28
    SCDOT -40,-8
    SCDOT -20,-58
    SCDOT 20,-58
    SCDOT 50,-30
    SCDOT 60,20
    SCDOT 2C,64
    SCDOT -20,68
    SCDOT -64,38
    SCDOT -68,-8
    SCDOT -60,-48
    SCDOT -28,-78
    SCDOT 40,-78
    RTSL
    .ENDM
```




```
    .MACRO MSTAR4
    ICVEC
    SCDOT -8,-10
    SCDOT 20,-10
    SCDOT 20,18
    SCDOT -18,20
    SCDOT -30,-10
    SCDOT 10,-38
    SCDOT 40,0
    SCDOT 20,38
    SCDOT -20,48
    SCDOT -50,10
    SCDOT -38,-40
    SCDOT 18,-4C
    SCDOT 40,-38
    SCDOT 68,10
    SCDOT 28,58
    SCDOT -38,50
    SCDOT -70,18
    SCDOT -68,-18
    SCDOT -58,-78
    SCDOT 40,-60
    RTSL
    .ENDM
```



As it happens the 8 layers build on our simple example by repeating the 4 layers of MSTAR1, MSTAR2, MSTAR3, and MSTAR4 twice over. So with all eight in play our full starfield will look something like this.



Drawing these starfields is a simple loop through the PLANEY array. We do this in DSTARF.

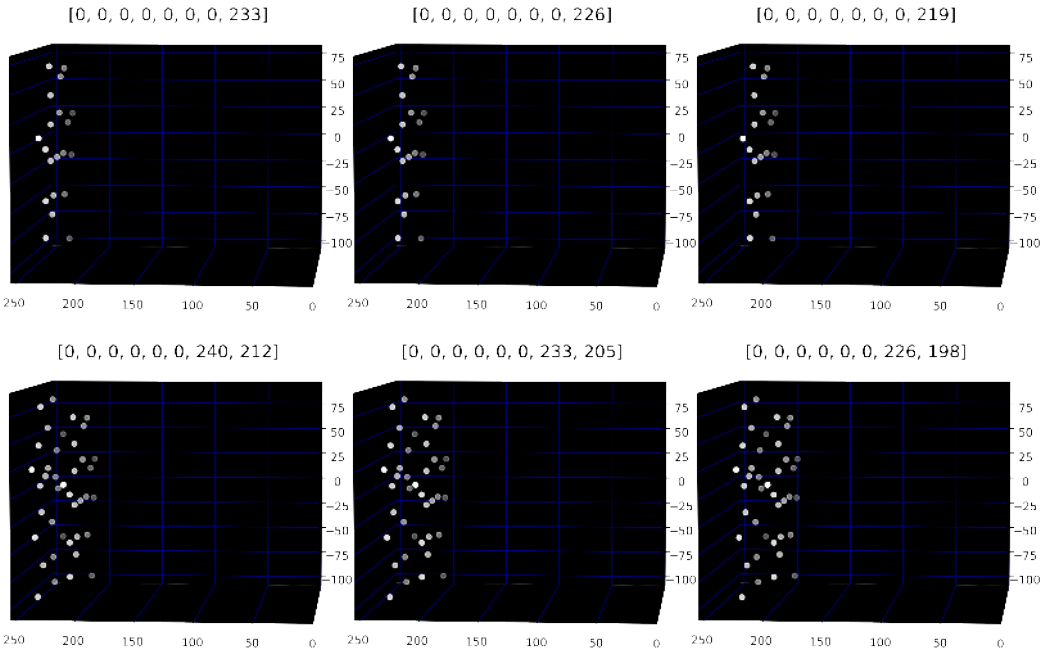
```
 .SBTTL  DISPLAY STAR FIELD
DSTARF: LDA PLAGRO      ; ARE WE STILL DRAWING THE STARFIELD?
        IFNE            ; IF NOT, SKIP EVERYTHING BELOW.
        LDX I,NPLANE-1  ; PUT NO. OF PLANES (8) IN X.
        STX INDEX1      ; STORE X AS THE LOOP INDEX
        BEGIN           ; BEGIN THE LOOP FOR EACH PLANE OF STARS.
          LDX INDEX1    ; LOAD INDEX TO X.
          LDA X,PLANEY  ; GET THE Z-POS FOR THIS PLANE.
          IFNE          ; IF NON-ZERO IT IS ACTIVE..
          STA PYL       ; STORE IT AS THE 'Z' POSITION.
          LDA I,80      ; USE 128 AS THE X POSITION (CENTER).
          STA PXL       ; STORE IT AS THE 'X' POSITION.
          LDA I,80      ; USE 128 AS THE Y POSITION (CENTER).
          STA PZL       ; STORE IT AS THE 'Y' POSITION.
          LDA I,BLUE    ; STORE BLUE COLOR IN A.
          STA COLOR     ; AND USE IT AS OUR COLOR.
          LDA I,MZCOLO  ; CLEAR THE VECTOR COLOR.
          JSR VGSTAT    ; RESET THE VECTOR DRAWER.
          LDA INDEX1    ; GET THE CURRENT INDEX.
          AND I,3       ; CLAMP IT TO A VALUE BETWEEN 0 AND 3.
          ASL           ; SHIFT LEFT..
          ADC I,PTSTR1  ; AND ADD THE TABLE OFFSET..
          STA OBJIND    ; TO GIVE US A POINTER TO MSTAR1/2/3/4.
          JSR SCAPI2    ; DRAW THE SELECTED STARFIELD PICTURE.
          ENDIF         ; END OF NON-ZERO CHECK.
          DEC INDEX1    ; GO TO THE NEXT ENTRY IN PLANEY.
        MIEND           ; KEEP LOOPING UNTIL WE DO THEM ALL.
        ENDIF           ; END OF SKIP EVERYTHING CHECK.
        RTS
```

This means that in order to control the display of the starfield we just have to manage the contents of the PLANEY array. Each byte in this array will give the Z co-ordinate position of each plane in the starfield. Where a byte is zero, the plane is not displayed, otherwise its value is treated as the position of the plane in the depth of field.

PRSTAR, given below, is the slightly chunky routine that takes care of this task. But it's bulk is largely a function of the verbosity inherent in assembly language. For example, a full 10 lines are occupied solely with getting an index to the element in the array previous to the current one in the routine's main loop.

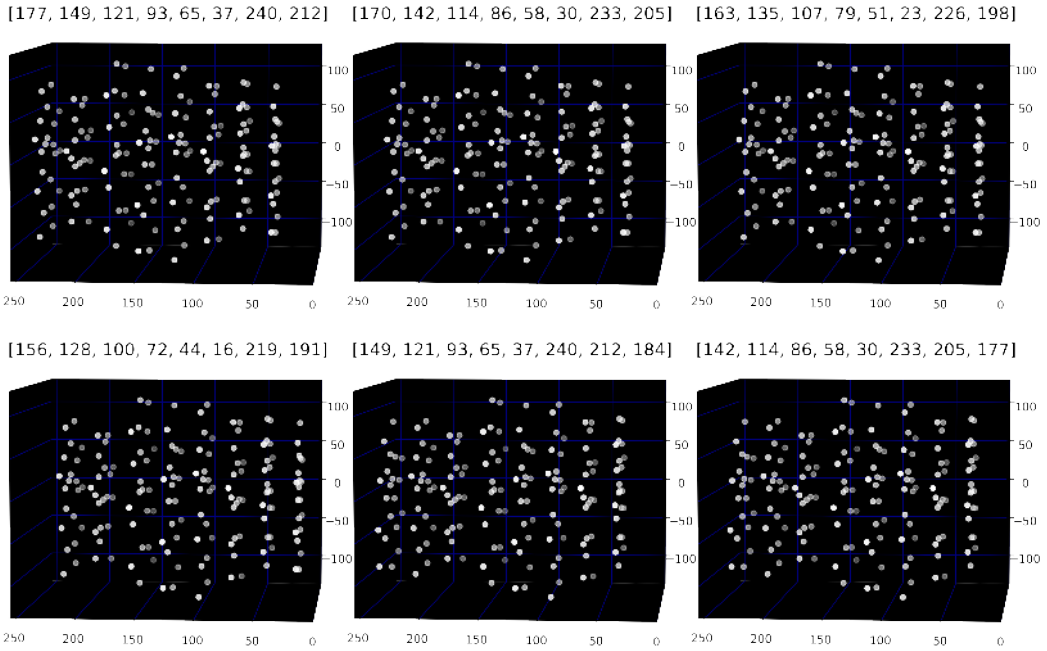
The overall approach for managing the starfield planes is to add one to the field at Z-position 240 and move it forward in steps of 7. The next one gets added when the one before it has reached Z-position 213. All 8 planes are added in this way and moved forward until they reach position 16. At this point they either start again at the beginning or, if the starfield has been turned off by resetting PLAGR0 to 0, deactivated.

This is what the first few steps of this process looks like, with the value of each byte in PLANEY given in square brackets.



```
PRSTAR: LDA PLAGRO           ; ARE WE STILL DRAWING THE STARFIELD?
        IFNE                 ; IF NOT, SKIP EVERYTHING BELOW.
          LDA I,0            ; YES. PROCESS PLANES
          STA TEMP0          ; CLEAR COUNT OF ACTIVE PLANES
          LDX I,NPLANE-1     ; PUT NO. OF PLANES (8) IN X.
          STX INDEX1         ; STORE X AS THE LOOP INDEX
          BEGIN              ; BEGIN THE LOOP FOR EACH PLANE OF STARS
            LDX INDEX1       ; LOAD INDEX TO X.
            LDA X,PLANEY     ; GET THE Z-POS FOR THIS PLANE.
            IFNE             ; IF NON-ZERO IT IS ACTIVE..
              SEC            ; RESET THE CARRY BIT.
              SBC I,07       ; DECREMENT THE PLANE'S Z-POS BY 7.
              IFCS           ; IF STILL A POSTIVE VALUE..
              CMP I,10       ; HAS IT REACHED 16 YET?
              ENDIF
              IFCC           ; IF IT HAS THEN..
              LDY PLAGRO     ; IS THE STARFIELD STILL GROWING?
              IFMI           ; IF IT IS THEN..
              LDA I,0F0      ; START IT AGAIN AT THE BACK..
              ELSE           ; OTHERWISE WE ARE NO LONGER GROWING SO..
              LDA I,0        ; DEACTIVATE IT.
              ENDIF
              ENDIF
            ELSE             ; OTHERWISE IT'S NOT ACTIVE SO CHECK
              LDY PLAGRO     ; IS THE STARFIELD STILL GROWING?
              IFMI           ; IF IT IS THEN..
                ; GET THE INDEX FOR THE PREVIOUS PLANE BY..
                TXA          ; PUTTING CURRENT INDEX IN A.
                CLC          ; PREPARE FOR AN ADDITION.
                ADC I,1      ; ADD 1 TO INDEX TO GET PREVIOUS PLANE.
                CMP I,NPLANE ; IS THE PREVIOUS ONE THE ZERO INDEX?
                IFCS         ; IF IT IS..
                LDA I,0      ; THEN SET PREVIOUS PLANE INDEX TO 0.
                ENDIF
                TAY          ; PUT PREVIOUS PLANE INDEX IN Y.
                LDA Y,PLANEY ; GET Z-POS OF PREVIOUS PLANE.
                IFNE         ; IF IT'S ACTIVE THEN..
                CMP I,0D5    ; HAS IT REACHED Z-POS 213?
                IFCC         ; IF SO THEN..
                LDA I,0F0    ; ACTIVATE THIS PLANE AT Z-POS 240.
                ELSE         ; OTHERWISE
                LDA I,0      ; IT REMAINS INACTIVE.
                ENDIF
                ENDIF
              ENDIF
            ENDIF
            STA X,PLANEY     ; STORE THE UPDATED Z-POS FOR THE PLANE
            ORA TEMP0        ; IF NOT ALREADY IN BITMAP OF ACTIVE PLANES..
            STA TEMP0        ; THEN ADD IT.
            DEC INDEX1       ; DECREMENT OUR INDEX...
          MIEND              ; AND LOOP UNTIL INDEX REACHES 0.
        ENDIF
        RTS
```

And if we flip forward a few dozen interations we get a sense of what it looks like in full flow.



This process never really dies: we have to kill it by resetting PLAGRO to a value that no longer looks like a negative value. Anything that does not set the first bit of PLAGRO will do, for example a 1 or a 0. This is what we find in the 'new wave' routine. Once the well has been moved into position, we can turn off the tap and the starfield will no longer be displayed.

```
        LDA I,1                 ;TURN OFF STAR FIELD
        STA PLAGRO
```

object list

About 60 times a second the Jaguar will want to write something to the screen. A module in the Jaguar system called the 'Object Processor' will take whatever the current list of things to draw happens to be and draw them. It's Tempest 2000's job to keep this beast well fed with new stuff. The way to do this is to keep shovelling fresh imagery into a structure called an 'Object List' for the Object Processor to chew on. As the name suggests this is a list of objects, but since everything is an object these days we might need to be more specific about what these objects are and what they contain.

There are a few different types of objects, but the only one of real interest is the one that contains something that can be displayed: an image. This image takes the form of a section of data similar to the cry pixel data we looked at in 'cry if i want to'. There can be a number of different flavours of this data that make the Object Processor's job easier in different circumstances but in Tempest 2000 we only ever detain ourselves with the rich cry data, the good stuff.

Below we set out the raw contents of an actual Object List used for a single frame in the 'Demo' sequence. The interesting objects in the list are the 'Bit Mapped Objects'. You can tell they're interesting because we have to specify so much about them that we need two 'phrases' (i.e. two sets of 8 bytes each) to encapsulate it all.

Data	Object Type
13,48,00,1D,6E,45,C1,60	Bit Mapped Object (First Phrase)
00,00,80,06,01,80,CF,F8	Bit Mapped Object (Second Phrase)
00,00,00,00,00,00,00,04	Stop Object
00,00,00,00,00,00,00,04	Stop Object
05,00,00,1D,72,0C,01,E0	Bit Mapped Object (First Phrase)
00,00,80,06,01,80,CF,F8	Bit Mapped Object (Second Phrase)

Figure .1: Object List

It is kind of amazing how much you can pack into 16 bytes if you try. Below we parse out the contents of the first 'Bit Mapped Object' in our Object List and show the screen of pixel data it references in the first 8-byte phrase.

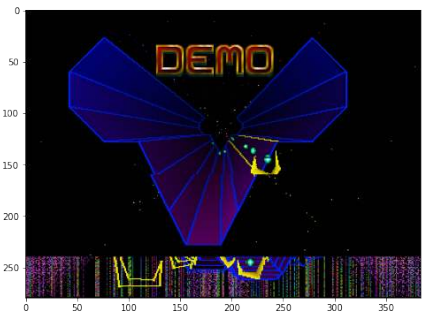
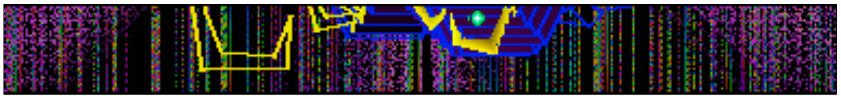
Raw Data	Parsed Data	Image at DATA: 0x134800
13,48,00,1D,6E,45,C1,60	TYPE: 0 LINK: 0xeb70 YPOS: 44 DATA: 0x134800 HEIGHT: 279	
00,00,80,06,01,80,CF,F8	XPOS: -7 REFLECT: 0 DEPTH: 4 RMW: 0 PITCH: 1 TRANS: 1 DWIDTH: 96 RELEASE: 0 IWIDTH: 96 FIRSTPIX: 0 INDEX: 0	

Figure .2: First Bit Mapped Object in the Object List

Here is what a selection of the parameters we supply above are a way of saying:

```
DATA: 0x134800 ⟶ Paint the data found at address 0x134800 in RAM.
DEPTH: 4       ⟶ Treat the data as using 16 bits for each pixel.
YPOS: 44       ⟶ Start painting this image at line 44.
XPOS: -7       ⟶ Start painting this image at X pos -7.
DWIDTH: 96     ⟶ Treat the data as having a line width of 380 (96*4).
IWIDTH: 96     ⟶ Treat the image as having a width of 380 (96*4).
HEIGHT: 279    ⟶ Treat the data as having a height of 280 lines.
```

You'll notice we have what looks like a lovely bit of glitch at the bottom of our image data.



This handsome scramble is the left-over data from previously drawn frames in the RAM. It is not visible because as we noted above we are painting this image at Y position 49, so although we are painting the full 280 lines of data only the first 231 are visible, and our glorious glitch sits off screen.

These vacant 49 lines at the top of the screen are occupied by our second 'Bit Mapped Object'. This is the player's score and their remaining lives:

Raw Data	Parsed Data	Image in Data `0x134800`

`05,00,00,1D,72,0C,01,E0`	`TYPE: 0 LINK: 0xeb90`
	`YPOS: 60 DATA: 0x50000`
	`HEIGHT: 48`	

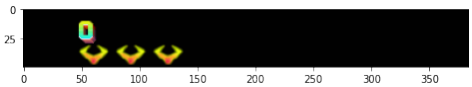
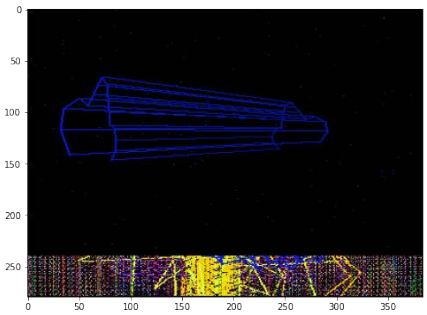
`00,00,80,06,01,80,CF,F8`	`XPOS: -7 REFLECT: 0`
	`DEPTH: 4 RMW: 0`
	`PITCH: 1 TRANS: 1`
	`DWIDTH: 96 RELEASE: 0`
	`IWIDTH: 96 FIRSTPIX: 0`
	`INDEX: 0`
		

Figure .3: Second Bit Mapped Object in the Object List

We can be flexible about how many Bit Mapped Objects we have and even what we put in them. Here are the two objects we use for the high score screen. The first is the rotating web in the background:

Raw Data	Parsed Data	Image at `DATA: 0x100000`

`10,00,00,1D,6E,45,C1,60`	`TYPE: 0 LINK: 0xeb70`
	`YPOS: 44 DATA: 0x100000`
	`HEIGHT: 279`	

`00,00,80,06,01,80,CF,F8`	`XPOS: -7 REFLECT: 0`
	`DEPTH: 4 RMW: 0`
	`PITCH: 1 TRANS: 1`
	`DWIDTH: 96 RELEASE: 0`
	`IWIDTH: 96 FIRSTPIX: 0`
	`INDEX: 0`
		

The second is the high-score screen itself:

Raw Data	Parsed Data	Image at DATA: 0x5000
05,00,00,1D,72,45,C1,60	TYPE: 0 LINK: 0xeb90 YPOS: 44 DATA: 0x5000 HEIGHT: 279	
00,00,80,06,01,80,CF,F8	XPOS: -7 REFLECT: 0 DEPTH: 4 RMW: 0 PITCH: 1 TRANS: 1 DWIDTH: 96 RELEASE: 0 IWIDTH: 96 FIRSTPIX: 0 INDEX: 0	

Notice that unlike our previous case we are superimposing these two images (or compositing them) so that the high score appears above the rotating web.

wells

Wells are fun and should be easy to draw. To start with, we can describe a 2d shape of our choice using a bunch of x/y co-ordinates as points.

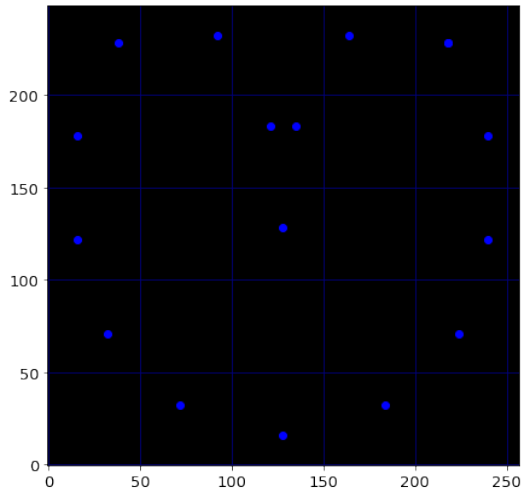


Figure .1: Make our dots.

Next we can join these together to give us a 2d shape.

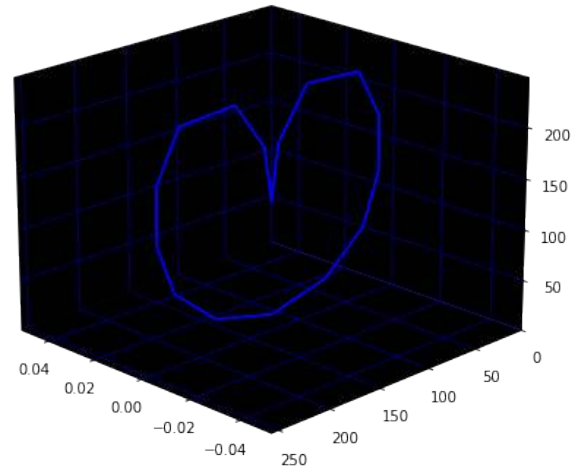


Figure .2: Start with a 2d shape..

To make this thing three dimensional all we have to do is project our shape onto some chosen distant position along the Z plane.

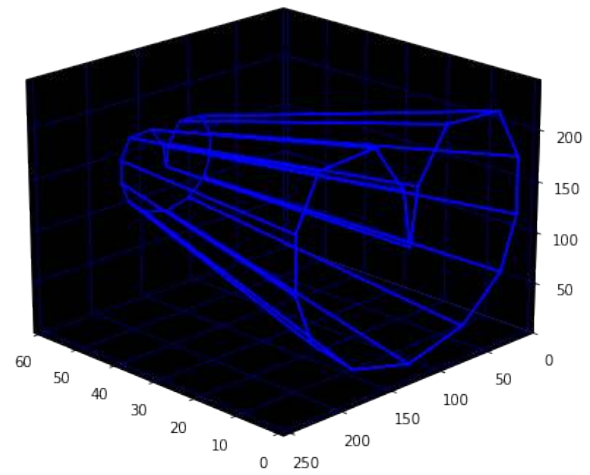


Figure .3: ..then make it three dimensional.

Seems simple enough. With this as our objective the data structure below gives us everything we need to create the web. In addition to listing the 16 vertices that make up the web's shape in two dimensions.

```
; Data structure for the 'HEART' well.
; X Co-Ordinates
    .BYTE 0DA,0A4, 87, 80, 79, 5C, 26, 10    ; HEART
    .BYTE  10, 20, 48, 80,0B8,0E0,0F0,0F0
; Y Co-Ordinates
    .BYTE 0E4,0E8,0B7, 80,0B7,0E8,0E4,0B2    ; HEART
    .BYTE  7A, 47, 20, 10, 20, 47, 7A,0B2
```

To help you relate this table to the co-ordinates in our graphs let's list it in decimal instead:

```
; Data structure for the 'HEART' well.
; X Co-Ordinates
    .BYTE 218,164,135,128,121, 92, 38, 16    ; HEART
    .BYTE  16, 32, 72,128,184,224,240,240
; Y Co-Ordinates
    .BYTE 228,232,183,128,183,232,228,178    ; HEART
    .BYTE 122, 71, 32, 16, 32, 71,122,178
```

To understand how we turn this list of numbers into a well, we're goint to look at the routine that is used to draw a miniature version on the level selection screen. The procedure consists of reading each X and Y co-ordinate pair in the order that they're given, starting with 218 for X and 228 for Y, and drawing a line to the position given by the next pair. As we saw in our 'cursors' chapter, we do this by constructing 'vector' commands and adding them to the vector list. Each command will consist of the X and Y direction in which to move the beam. In order to construct this list of vector commands we must loop through our co-ordinates above. In the routine given below, these co-ordinates are stored in `NEWLIX` (for the X co-ordinates) and `NEWLIZ` (for the Y co-ordinates).

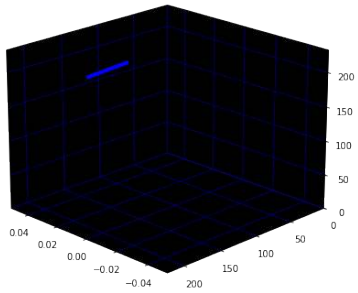
```
        .SBTTL  UTILITY-DRAW WELL SHAPE
DSPHOL:
        ..
        BEGIN             ; LOOP FOR EACH PT ON EDGE
        LDY SAVEY         ; GET THE CURRENT INDEX
        LDA Y,NEWLIX      ; STORE CURRENT X CO-ORD IN A
        TAX               ; MOVE IT TO X
        SEC               ; SET THE CARRY BIT.
        SBC PXL           ; SUBTRACT THE LAST X POSITION.
        PHA               ; STORE THE X MOVEMENT ON THE STACK.
        STX PXL           ; STORE THE CURRENT VALUE AS THE LAST X POSITION.
        LDA Y,NEWLIZ      ; STORE CURRENT Y CO-ORD IN A.
        TAY               ; MOVE IT TO Y
        SEC               ; SET THE CARRY BIT.
        SBC PYL           ; SUBTRACT THE LAST Y POSITION.
```

```
        TAX             ; STORE THE Y MOVEMENT IN X.
        STY PYL         ; STORE THE CURRENT VALUE AS THE LAST Y POSITION.
        PLA             ; PUT OUR X MOVEMENT IN A.
        ; WE NOW HAVE THE X MOVEMENT IN A AND THE Y MOVEMENT IN X.
        ; WE CAN DRAW OUR VECTOR LINE.
        JSR VGVTR1      ; DRAW OUR VECTOR.
        DEC SAVEY       ; DECREMENT THE INDEX TO MOVE TO NEXT CO-ORDINATE.
        MIEND           ; LOOP UNTIL WE'RE DONE.
```

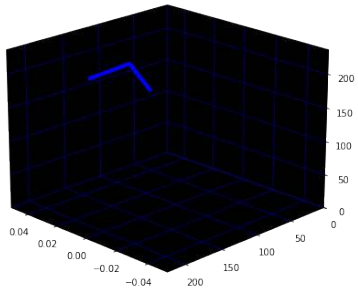
The procedure consists of calculating an X and Y movement for each vector line by subtracting the last X and Y position in the co-ordinate list from the current one. This difference or 'delta' tells the vector ROM the number of units in the X and Y direction it should move the line. The end result is one single face of our web in two dimensions.

We can visualise this by following the construction of the well face, vector by vector, as follows. In each case we list the actual vector instruction used, encoded into two bytes by the `VGVTR1` macro.

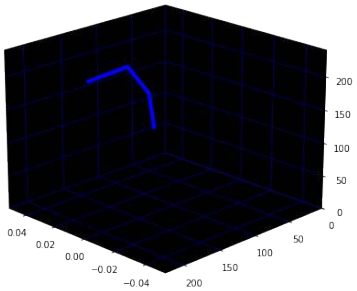
Description	Op Code	Bitmap
Draw vector to (164,232)	0x5___ 0x48C4	010YYYYY IIIXXXXX 01001000 11000100 4 8 C 4



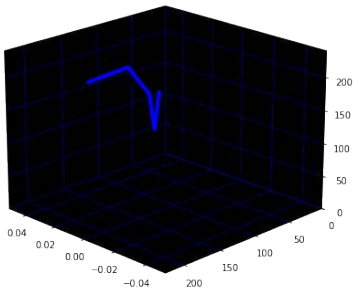
Description	Op Code	Bitmap
Draw vector to (-83,-45)	0x5___ 0x53CD	010YYYYY IIIXXXXX 01010011 11001101 5 3 C D



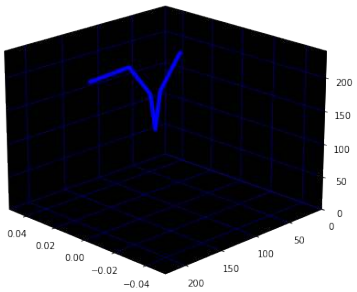
Description	Op Code	Bitmap
Draw vector to (-36,-104)	0x5___ 0x58DC	010YYYYY IIIXXXXX 01011000 11011100 5 8 D C



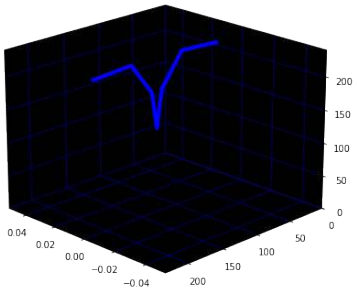
Description	Op Code	Bitmap
Draw vector to (-14,0)	0x5___ 0x40D2	010YYYYY IIIXXXXX 01000000 11010010 4 0 D 2



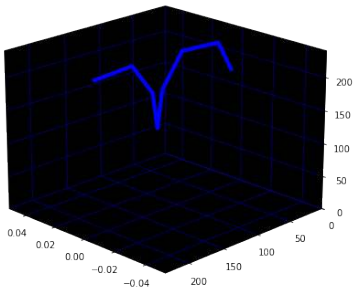
Description	Op Code	Bitmap
Draw vector to (-36,104)	0x5___ 0x48DC	010YYYYY IIIXXXXX 01001000 11011100 4 8 D C



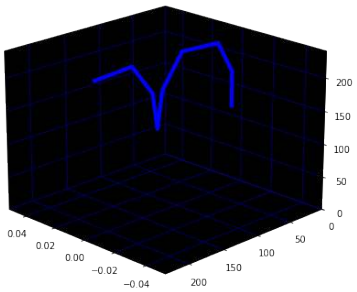
Description	Op Code	Bitmap
Draw vector to (-83,45)	0x5___ 0x4DCD	010YYYYY IIIXXXXX 01001101 11001101 4 D C D



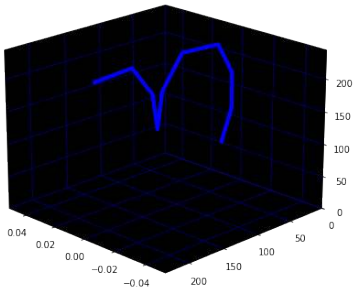
Description	Op Code	Bitmap
Draw vector to (-76,-54)	0x5___ 0x4AD4	010YYYYY IIIXXXXX 01001010 11010100 4 A D 4



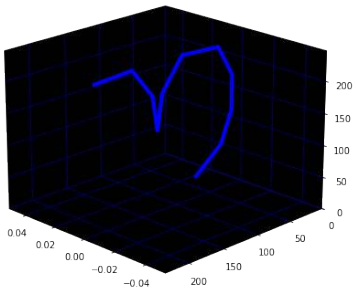
Description	Op Code	Bitmap
Draw vector to (-22,-106)	0x5___ 0x56CA	010YYYYY IIIXXXXX 01010110 11001010 5 6 C A



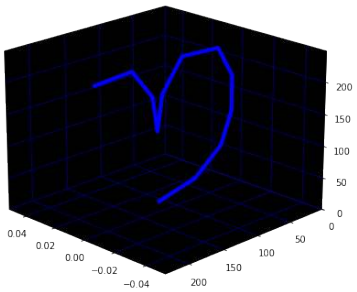
Description	Op Code	Bitmap
Draw vector to (16,-107)	0x5___ 0x55D0	010YYYYY IIIXXXXX 01010101 11010000 5 5 D 0



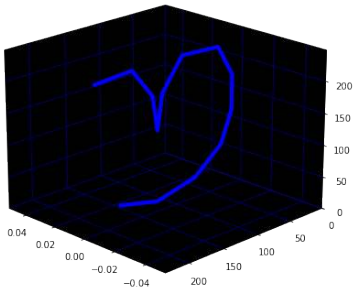
Description	Op Code	Bitmap
Draw vector to (56,-90)	0x5___ 0x46D8	010YYYYY IIIXXXXX 01000110 11011000 4 6 D 8



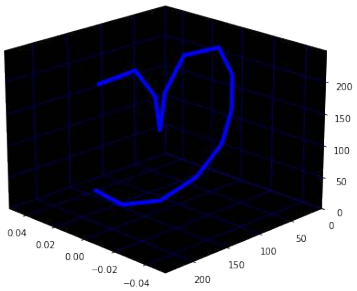
Description	Op Code	Bitmap
Draw vector to (96,-55)	0x5___ 0x49C0	010YYYYY IIIXXXXX 01001001 11000000 4 9 C 0



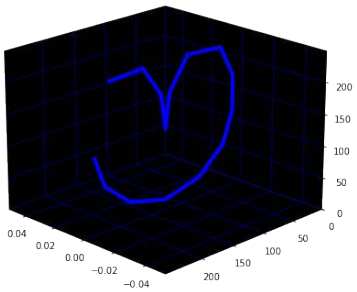
Description	Op Code	Bitmap
Draw vector to (112,0)	0x5___ 0x40D0	010YYYYY IIIXXXXX 01000000 11010000 4 0 D 0



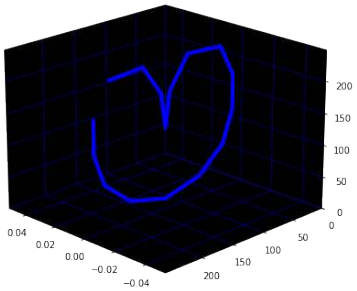
Description	Op Code	Bitmap
Draw vector to (96,55)	0x5___ 0x57C0	010YYYYY IIIXXXXX 01010111 11000000 5 7 C 0



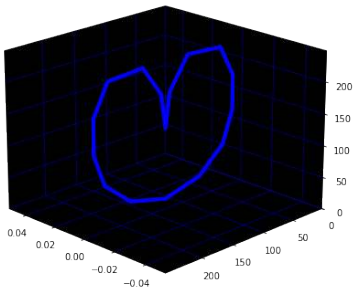
Description	Op Code	Bitmap
Draw vector to (56,90)	0x5___ 0x5AD8	010YYYYY IIIXXXXX 01011010 11011000 5 A D 8



Description	Op Code	Bitmap
Draw vector to (16,107)	0x5___ 0x4BD0	010YYYYY IIIXXXXX 01001011 11010000 4 B D 0



Description	Op Code	Bitmap
Draw vector to (-22,106)	0x5___ 0x4ACA	010YYYYY IIIXXXXX 01001010 11001010 4 A C A



webs

Webs are fun and should be easy to draw. To start with, we can describe a 2d shape of our choice using a bunch of x/y co-ordinates as points.

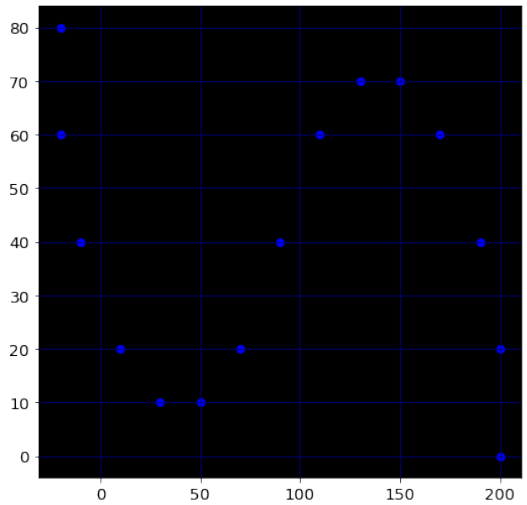


Figure .1: Make our dots.

Next we can join these together to give us a 2d shape.

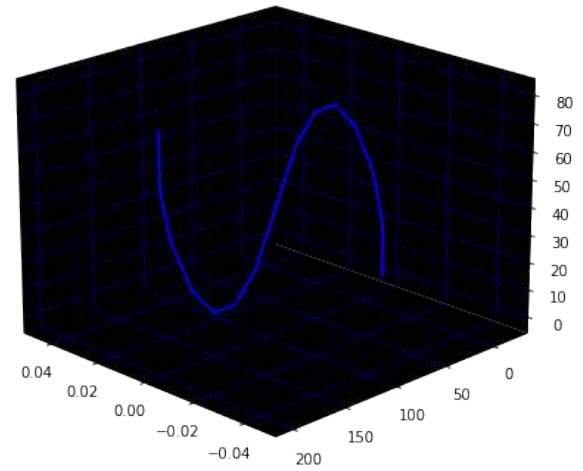


Figure .2: Start with a 2d shape..

To make this thing three dimensional all we have to do is project our shape onto some chosen distant position along the Z plane.

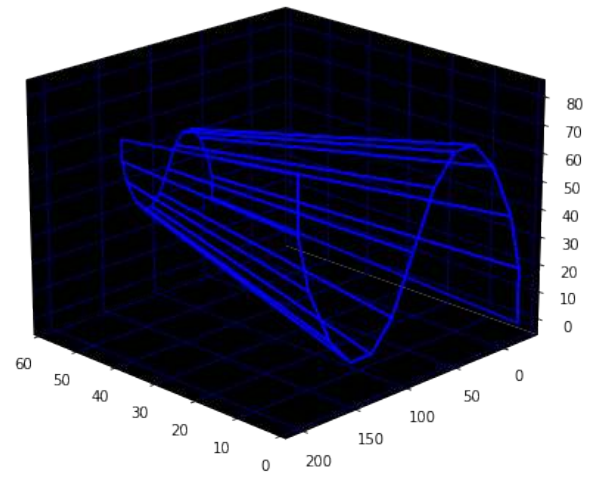


Figure .3: ..then make it three dimensional.

Seems simple enough. With this as our objective the data structure below gives us everything we need to create the web. In addition to listing the 16 vertices that make up the web's shape in two dimensions, we also include some information on whether the web should be open (like the one pictured above) or closed (like a circle or square). Since a web of 16 vertices will give us 14 lanes which the player and enemies can occupy we also add a 'Rotation Table' to tell us how players and enemies should be oriented when on that lane.

```
; Data structure for the 'Sine Wave' web.
web13:
  dc.w 14    ; Number of lanes in the web.
  dc.w  6    ; The lane the player starts on.

  ; The x/y pairs of all vertices in the web. So for example
  ; -2/14 indicates the X and Y co-ordinates of the first vertex.
  ; There are always 16 in total.
  dc.w -2,14,-2,12,-1,10, 1, 8
  dc.w  3, 7, 5, 7, 7, 8, 9,10
  dc.w 11,12,13,13,15,13,17,12
  dc.w 19,10,20, 8,20, 6,-2,14

  dc.w 0     ; 0 = Open Web, -1 = Closed Web

  ; Rotation Table (angle of an object within a particular lane)
  ; The length of this list is specified by the '14' above.
  dc.w -64,-48,-32,-16, 0,16,32,32,16, 0,-16,-32,-48,-64
```

The values given in the Rotation Table are angles of roll to apply to the claw (and to the claw's enemies) when placing them in the lane. Negative values rotate the object to the right, while positive values rotate it to the left.

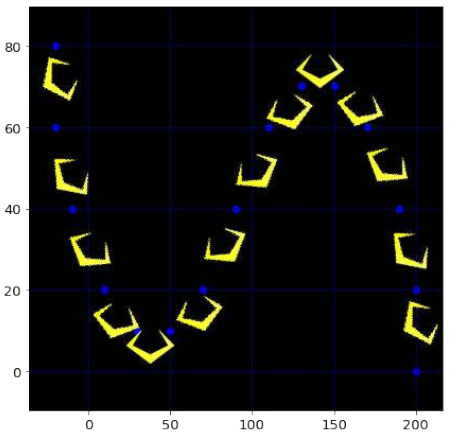


Figure .4: The rotation table applied to the claw in each lane of the web.

This is the important part of the routine that converts the web data structure into a list of vertices and lines between vertices that will make up the web itself:

```
   ; Read in the x/y pairs from our data structure. We create an interleaved
   ; list of near and far points in a2 and a 'connlist' in a3 that describes
   ; how each point is connected to the others.
xweb:
   ; Get the current x and y pair
   move (a1)+,d0    ; Store the Y value in d0.
   move (a1)+,d1    ; Store the X value in d1.
   ext.l d0         ; Make d0 4 bytes long.
   ext.l d1         ; Make d1 4 bytes long.

   ; Check if this is the largest X value so far
   cmp d5,d1        ; Compare X with the largest so far, stored in d5.
   blt xweb2        ; If it's less, skip to xweb2 below.
   move d1,d5       ; It's bigger, so save it in d5.

xweb2:
   ; Store the x,y,z value for the near point in the web
   move.l d0,(a2)+ ; Y value
   move.l d1,(a2)+ ; X value
   clr.l (a2)+     ; Z value for near point (always 0)

   ; Store the x,y,z value for the far point in the web
   move.l d0,(a2)+ ; Y value
   move.l d1,(a2)+ ; X value
   move.l d7,(a2)+ ; z value for far point (calculated by initvo).

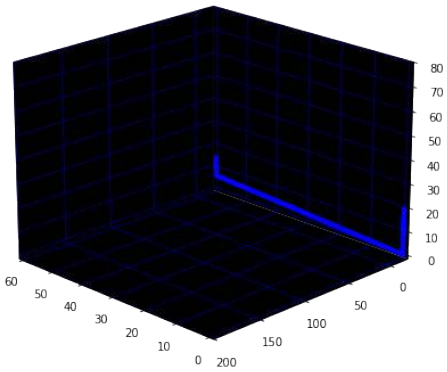
   ; a3 is the 'connlist', the list describing how each point is connected
   ; to the others.

   ; Create entries in the connlist for this point. First we create an entry
   ; for the 'near' point.  This will be connected to the far point and the
   ; next point in the web data structure. Each entry starts with an index
   ; to the point in question, followed by indices to the points its
     connected to.
   move d3,(a3)+   ; Make this a connlist entry for the 'near' point.
   move d3,d4      ; Store the 'near' point index in d4.
   addq #1,d4      ; Add 1 to it, so it's now an index to the 'far' point.
   move d4,(a3)+   ; Store this as first connected point in the entry.
   addq #1,d4      ; Add 1 to d4 again so it's an index to the next point.
   move d4,(a3)+   ; Store this as the second connected point in the entry.
   move #0,(a3)+   ; Mark the end of the entry with a zero.

   ; Create an entry in the connlist for the 'far' point.
   subq #1,d4      ; Subtract 1 from d4 so it's an index to the far point.
   move d4,(a3)+   ; Make this an entry for the 'far' point.
   addq #2,d4      ; Add 2 to d4 so it's an index for the next 'far' point.
   move d4,(a3)+   ; Store this as the connected point in the entry.
   move #0,(a3)+   ; Mark the end of the entry with a zero.
   move.l a3,(a5)+ ; Add our entries to the overall connlist list of lists.
   add #2,d3       ; Point d3 to the next X/Y pair.
```

```
    ; Keep looping until we finish reading the X/Y pairs in the data structure
      .
    dbra d6,xweb    ; Get the next pair
```

Its job is to read in X and Y pairs in our `web13` data structure. To make sense of them as describing a web it uses the pairs to create a connection list or `connlist` that describes how each of the points in the web are connected to each other. Let's take a look at how the loop in `xweb` builds up this `connlist` as it iterates over the X and Y pairs in `web13`.

X/Y Pairs and Connections	Current Draw State
`web13:  (-2,14),  (-2,12)` `connlist:  (0,1,2,0), (1,3,0)`	

In this first iteration through `xweb` we've managed to construct the first three lines of our web. On the near and far side we've connected the first two points in the web, and we've connected the near and far side of the first point.

In our `connlist` the entry `(1,3,0)` connects the far point (1) with the next far point (3). The `0` at the end is a terminator sentinel telling us it's the end of that particular entry.

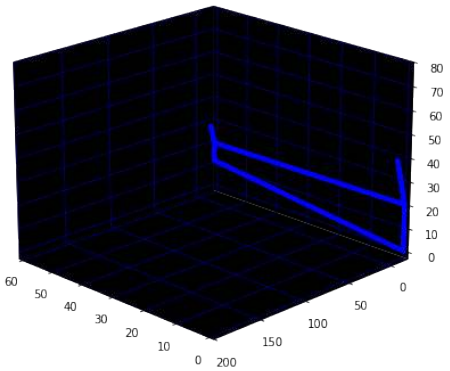
The first entry in the `connlist` tells us that the first near point (`0`) is connected to its far point counterpoint (1) and the the next near point (2). Again the zero at the end is a terminator for the entry.

Moving to the next X/Y pair in our next diagram we repeat this process, layering on the same shape and the same set of connections between the near and far points for the next point and its successor.

X/Y Pairs and Connections	Current Draw State

```
web13:      (-2,14),   (-2,12)
            (-1,10)

connlist:   (0,1,2,0), (1,3,0)
            (1,2,3,0), (2,4,0)
```

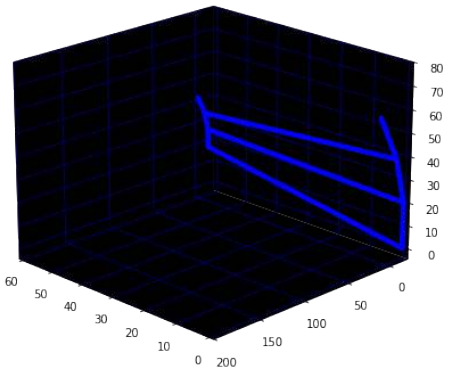


This process now continues for each pair, building up the `connlist` for each pair of 'near' and 'far' point counterparts.

X/Y Pairs and Connections	Current Draw State

```
web13:      (-2,14),   (-2,12)
            (-1,10),   (1,8)

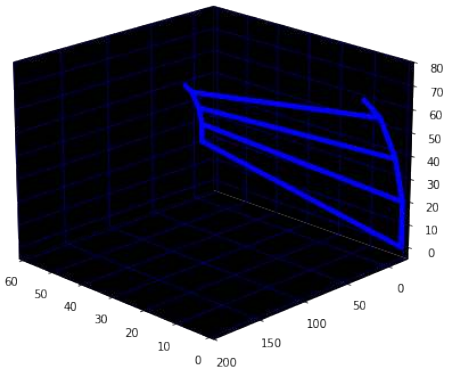
connlist:   (0,1,2,0), (1,3,0)
            (1,2,3,0), (2,4,0)
            (2,3,4,0), (3,5,0)
```



X/Y Pairs and Connections	Current Draw State

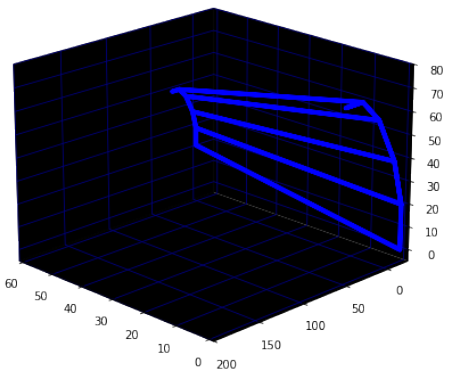
```
web13:      (-2,14),   (-2,12)
            (-1,10),   (1,8)
            (3,7)

connlist:   (0,1,2,0), (1,3,0)
            (1,2,3,0), (2,4,0)
            (2,3,4,0), (3,5,0)
            (3,4,5,0), (4,6,0)
```



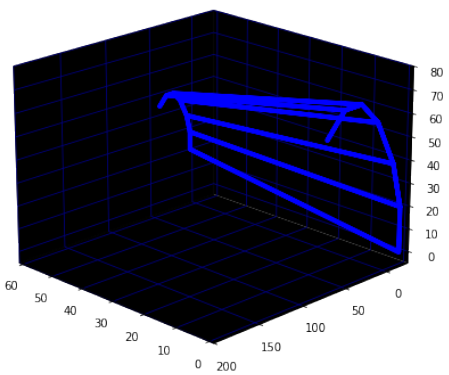

```
web13:      (-2,14),   (-2,12)
            (-1,10),   (1,8)
            (3,7),    (5,7)

connlist:   (0,1,2,0), (1,3,0)
            (1,2,3,0), (2,4,0)
            (2,3,4,0), (3,5,0)
            (3,4,5,0), (4,6,0)
            (4,5,6,0), (5,7,0)
```




```
web13:      (-2,14),   (-2,12)
            (-1,10),   (1,8)
            (3,7),    (5,7)
            (7,8)

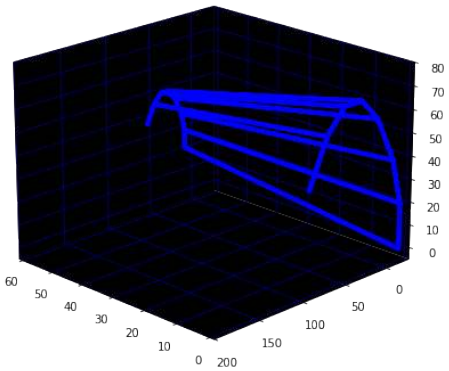
connlist:   (0,1,2,0), (1,3,0)
            (1,2,3,0), (2,4,0)
            (2,3,4,0), (3,5,0)
            (3,4,5,0), (4,6,0)
            (4,5,6,0), (5,7,0)
            (5,6,7,0), (6,8,0)
```



X/Y Pairs and Connections	Current Draw State

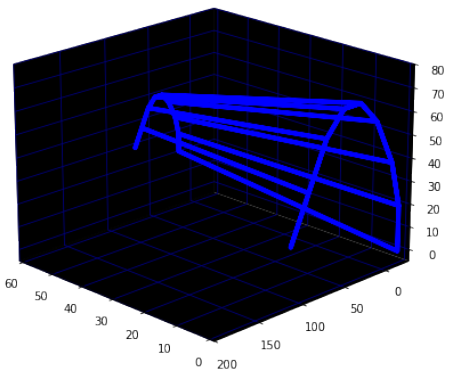
```
web13:      (-2,14),   (-2,12)
            (-1,10),   (1,8)
            (3,7),     (5,7)
            (7,8),     (9,10)

connlist:   (0,1,2,0), (1,3,0)
            (1,2,3,0), (2,4,0)
            (2,3,4,0), (3,5,0)
            (3,4,5,0), (4,6,0)
            (4,5,6,0), (5,7,0)
            (5,6,7,0), (6,8,0)
            (6,7,8,0), (7,9,0)
```

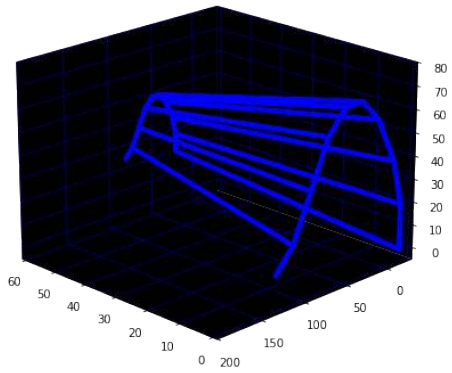
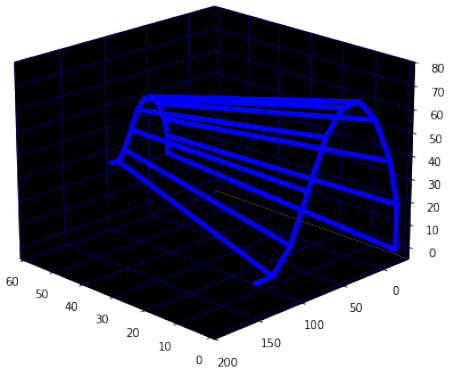
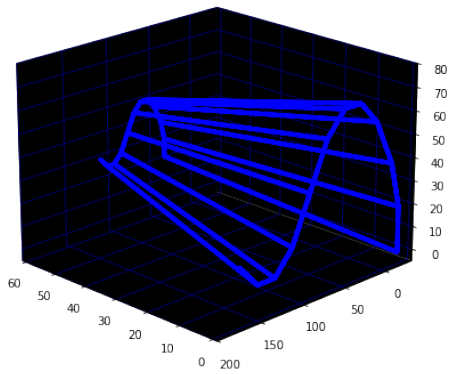
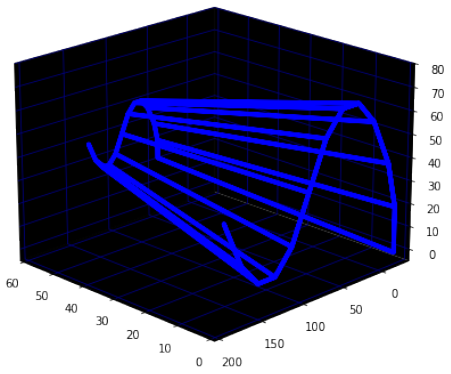
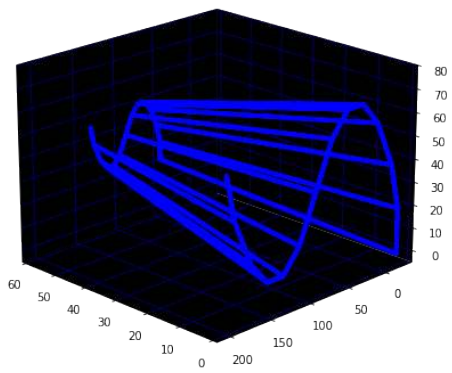
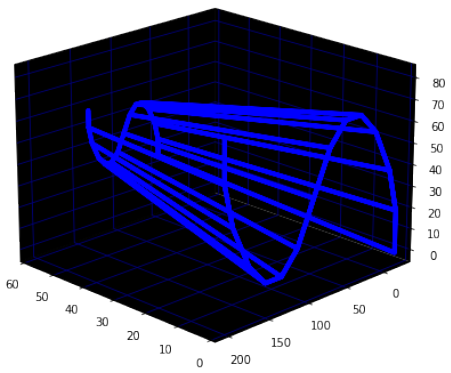



```
web13:      (-2,14),   (-2,12)
            (-1,10),   (1,8)
            (3,7),     (5,7)
            (7,8),     (9,10)
            (11,12)

connlist:   (0,1,2,0), (1,3,0)
            (1,2,3,0), (2,4,0)
            (2,3,4,0), (3,5,0)
            (3,4,5,0), (4,6,0)
            (4,5,6,0), (5,7,0)
            (5,6,7,0), (6,8,0)
            (6,7,8,0), (7,9,0)
            (7,8,9,0), (8,10,0)
```



OK I think we're getting the idea. There's a clear pattern in the way the `connlist` is constructed for each of the point pairs. Let's finish off the construction.

Draw State	Draw State
	
	
	

cursors

The Tempest source code refers to the player's ship as a CURSOR. It does this because it is the 1980s and that is what you call something on the screen that you can control.

The graphics in Tempest are generated using Atari's 'Quadrascan' vector technology. The images are all defined using a series of vectors. A vector in this case is a (X,Y) value pair that moves the beam in an X,Y direction to a new point on the screen. A series of vectors moving from point to point around the screen will eventually form a complete image.

When we take a look at the data structure defining a relatively upright version of the player's ship we find this. I'm going to speculate that NCSR4S is short for 'New Cursor 4 Start' and NCSR4E for 'New Cursor 4 End':

```
NCRS4S:
        VEC 3,-2
        VEC 5,2
        VEC -3,1
        VEC 2,-1
        VEC -4,-1
        VEC -2,1
        VEC 2,1
        VEC -3,-1
NCRS4E:
```

The number pairs in the above listing are the vectors we're talking about, and we're going to see how we can use them to build up an image of the ship assuming an origin point of zero. Our first step, therefore, is to draw a line from (0,0) to (3,-2) as follows:

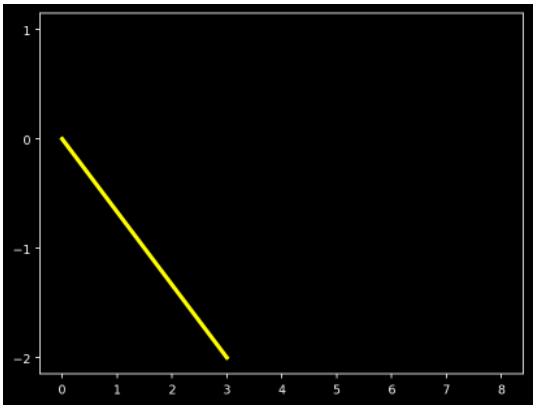


Figure .1: Draw a line from `(0,0)` to `(3,-2)`.

Our next step reveals a bit more about the actual nature of the operation we are performing. To draw a line from our new position using a vector of `5,2` we add it to our current position of `(3,-2)` to draw a line to `8,0`:

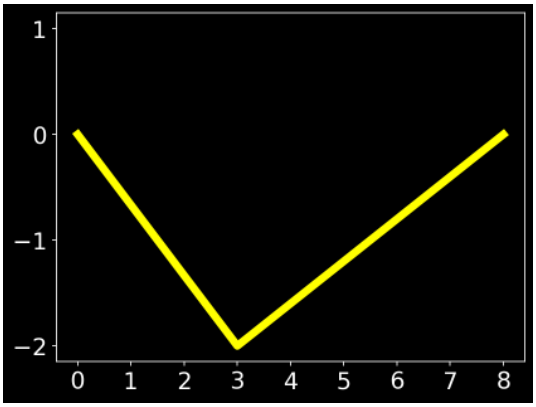
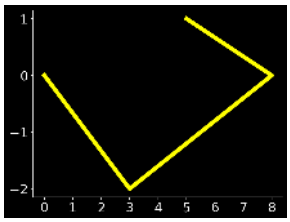


Figure .2: `(3,-2)` + `(5,2)`. ⟶ `(8,0)`

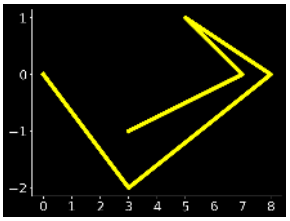
With this as our method, we can start building up our complete image, adding each vector in our array to the previous result to define a new line to draw.



(a) `(8,0) + (-3,1).` ⟶ `(5,1)`



(b) `(5,1) + (2,-1).` ⟶ `(7,0)`



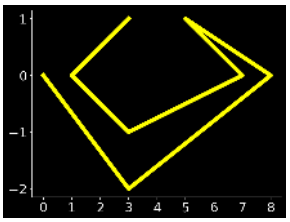
(c) `(7,0) + (-4,-1).` ⟶ `(3,-1)`

Figure .3: Starting to take shape.

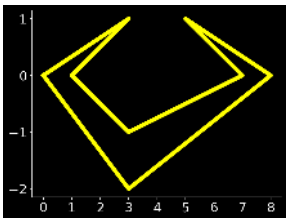
Adding our last three vectors completes the picture, and we have a claw.



(a) `(3,-1) + (-2,1).` ⟶ `(1,0)`



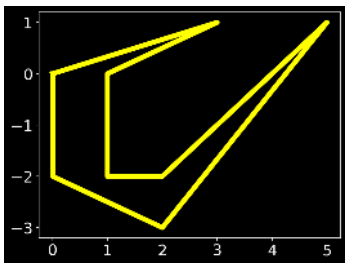
(b) `(1,0) + (2,1).` ⟶ `(3,1)`



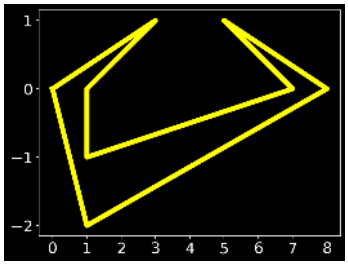
(c) `(3,1) + (-3,-1).` ⟶ `(0,0)`

Figure .4: Putting the final pieces in place.

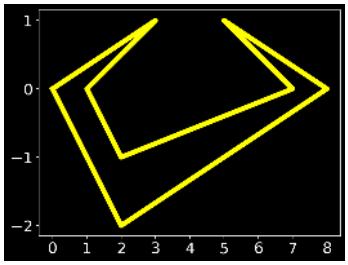
Finally, here are all of the `CURSOR` elements along with their data structures.



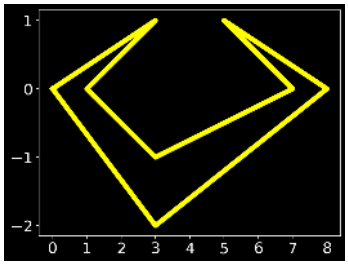
```
NCRS1S:
 VEC 0,-2
 VEC 2,-1
 VEC 3,4
 VEC -3,-3
 VEC -1,0
 VEC 0,2
 VEC 2,1
 VEC -3,-1
NCRS1E:
```



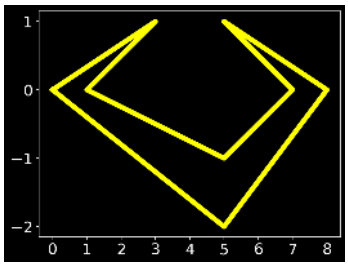

```
NCRS2S:
 VEC 1,-2
 VEC 7,2
 VEC -3,1
 VEC 2,-1
 VEC -6,-1
 VEC 0,1
 VEC 2,1
 VEC -3,-1
NCRS2E:
```



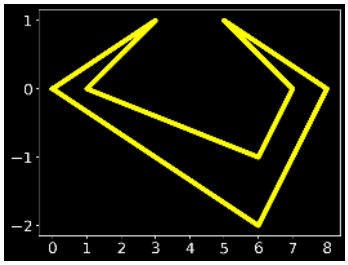

```
NCRS3S:
 VEC 2,-2
 VEC 6,2
 VEC -3,1
 VEC 2,-1
 VEC -5,-1
 VEC -1,1
 VEC 2,1
 VEC -3,-1
NCRS3E:
```



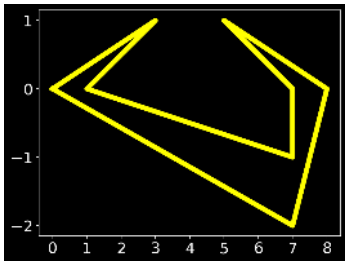

```
NCRS4S:
 VEC 3,-2
 VEC 5,2
 VEC -3,1
 VEC 2,-1
 VEC -4,-1
 VEC -2,1
 VEC 2,1
 VEC -3,-1
NCRS4E:
```



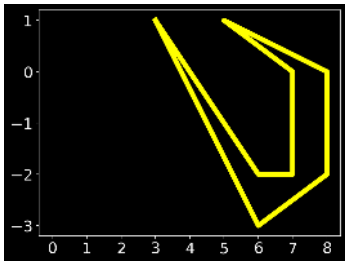

```
NCRS5S:
 VEC 5,-2
 VEC 3,2
 VEC -3,1
 VEC 2,-1
 VEC -2,-1
 VEC -4,1
 VEC 2,1
 VEC -3,-1
NCRS5E:
```




```
NCRS6S:
 VEC 6,-2
 VEC 2,2
 VEC -3,1
 VEC 2,-1
 VEC -1,-1
 VEC -5,1
 VEC 2,1
 VEC -3,-1
NCRS6E:
```




```
NCRS7S:
 VEC 7,-2
 VEC 1,2
 VEC -3,1
 VEC 2,-1
 VEC 0,-1
 VEC -6,1
 VEC 2,1
 VEC -3,-1
NCRS7E:
```




```
NCRS8S:
 VEC 3,1,0
 VEC 3,-4
 VEC 2,1
 VEC 0,2
 VEC -3,1
 VEC 2,-1
 VEC 0,-2
 VEC -1,0
 VEC -3,3
NCRS8E:
```

claws

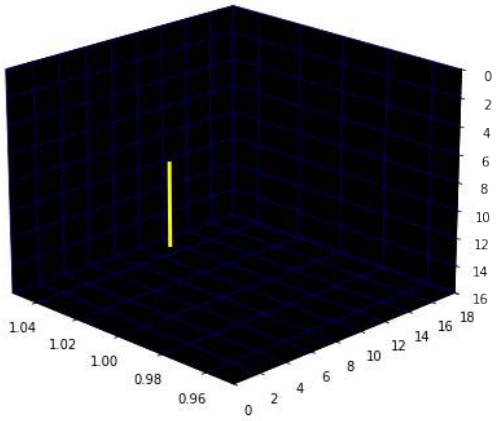
In Tempest 2000, for no good reason, the 'classic' tempest claws use trios of bytes to give co-ordinates in 3D space. I say 'for no good reason' because the claw is still flat, so the Z co-ordinate is always a 1. We define our claws using a list of X,Y,Z co-ordinates, and each co-ordinate is paired with a list of points that it connects to. It is this list of connecting points that allows us to draw lines:

```
claw3:  dc.b 4,5,1              ; Co-ordinate 1
        dc.b 2,8,0              ; Draw line to co-ords 2 and 8.
        dc.b 4,11,1             ; Co-ordinate 2
        dc.b 3,0                ; Draw line to co-ord 3.
        dc.b 11,12,1            ; Co-ordinate 3
        dc.b 4,0                ; Draw line to co-ord 4.
        dc.b 15,8,1             ; Co-ordinate 4
        dc.b 5,0                ; Draw line to co-ord 5.
        dc.b 12,4,1             ; Co-ordinate 5
        dc.b 6,0                ; Draw line to co-ord 6.
        dc.b 17,8,1             ; Co-ordinate 6
        dc.b 7,0                ; Draw line to co-ord 7.
        dc.b 11,15,1            ; Co-ordinate 7
        dc.b 8,0                ; Draw line to co-ord 8.
        dc.b 2,12,1             ; Co-ordinate 8
        dc.b 0,0                ; Don't draw a line.
```

This is what it looks like when we consume the first co-ordinate and its paired points:

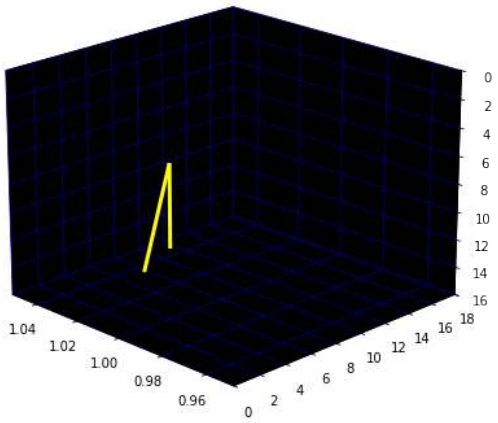
```
claw3:  dc.b 4,5,1              ; Co-ordinate 1
        dc.b 2,8,0              ; Draw line to co-ords 2 and 8.
```

This is telling us to connect our first trio 4,5,1 to co-ordinates '2' (4,11,1)..



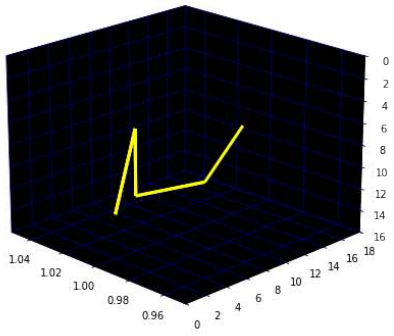
Draw a line from `(4,5,1)` to `(4,11,1)`.

.. and '8' (`2,12,1`).

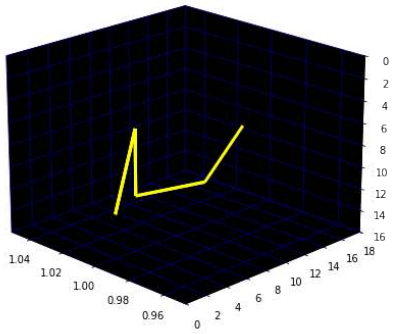


Draw a line from `(4,5,1)` to `(2,12,1)`.

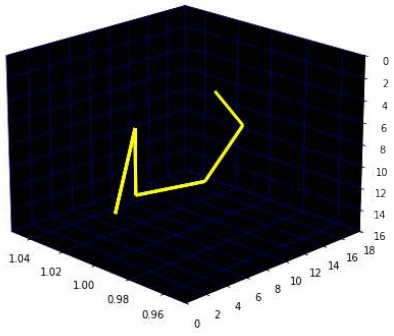
With this as our method, we can start building up our complete image, adding each point in our array to the previous result to define a new line to draw.



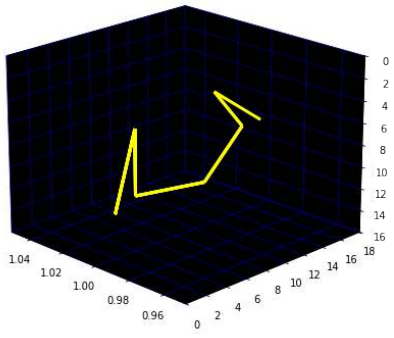

```
dc.b 4,11,1   ; Co-ordinate 2
dc.b 3,0      ; Draw line to co-ord 3.
dc.b 11,12,1  ; Co-ordinate 3
```



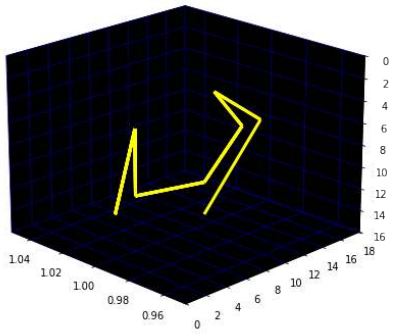

```
dc.b 11,12,1 ; Co-ordinate 3
dc.b 4,0     ; Draw line to co-ord 4.
dc.b 15,8,1  ; Co-ordinate 4
```



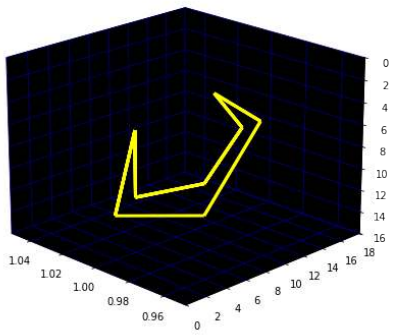

```
dc.b 15,8,1  ; Co-ordinate 4
dc.b 5,0     ; Draw line to co-ord 5.
dc.b 12,4,1  ; Co-ordinate 5
```



```
dc.b 12,4,1  ; Co-ordinate 5
dc.b 6,0     ; Draw line to co-ord 6.
dc.b 17,8,1  ; Co-ordinate 6
```



```
dc.b 17,8,1   ; Co-ordinate 6
dc.b 7,0      ; Draw line to co-ord 7.
dc.b 11,15,1  ; Co-ordinate 7
```



```
dc.b 11,15,1 ; Co-ordinate 7
dc.b 8,0     ; Draw line to co-ord 8.
dc.b 2,12,1  ; Co-ordinate 8
```

attract mode admin

There is naturally a bit of administrative overhead associated with operating a game in 'attract mode'. In case you may be wondering, 'attract mode' is when a game cabinet sits unattended and is attempting to attract customers. The manufacturer can display a colourful title screen and high score tables, maybe play a tune. But often the most effective way of getting people to play your game is to show the game being played. This allows the uninitiated to get a sense of what they're missing out on and also gives them an idea of how to proceed when they enter their first coin. Providing this kind of demonstration gameplay means devising some rudimentary intelligence in the game logic that will mimic the activity of a fairly competent paying customer. Fortunately, in *Tempest* there are only two things that a player actually does: move and shoot. This means that our task of emulating a player negotiating a level in *Tempest* is simpler than it might at first appear. We can split it out into two separate concerns, and if we solve each one we will have an artificial player that can help bring in some business.

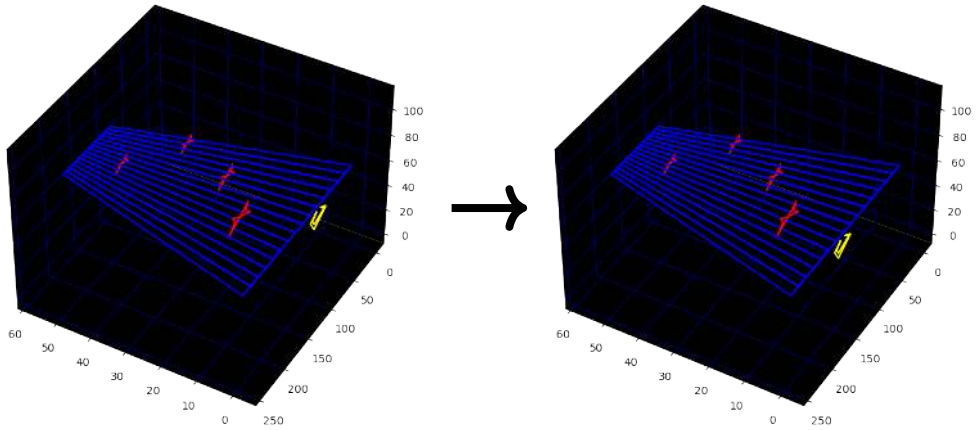
When we get our 'attract mode' up and running we have a small bit of setup to perform first. The demonstration player is only given one life, and the level they will play is chosen at random from the first eight.

```
        BIT QSTATUS   ; CHECK FOR ATTRACT MODE..
        IFPL          ; IF IT IS ENABLED THEN..
        LDY I,1       ; LOAD 1 INTO Y REGISTER.
        STY LIVES1    ; AND STORE THAT AS AVAILABLE LIVES.
        LDA RANDOM    ; STORE A RANDOM NUMBER IN ACCUMULATOR.
        AND I,7       ; CLAMP IT TO BETWEEN 0 AND 7.
        ENDIF
        STA X,WAVEN1  ; USE THE RANDOM NUM TO SELECT LEVEL.
        STA CURWAV    ; STORE IT AS THE CURRENT LEVEL TOO.
```

With this environment for our player established we can first figure out how to make him move.

movement admin

This one is surprisingly easy. At any given moment we just have to decide whether the cursor (representing the player) is going to move left or right. The only thing that should ever affect this decision is whether the movement gets us nearer the same line as the enemy that is nearest the top of the web. The diagram below shows this decision-making in action. We are much nearer the line of the invader that is second farthest away from us, but rather than move right towards that enemy, we move left: towards the enemy that is closest to reaching the top of the web.



This approach, while it may be suboptimal in some ways, has the merit of being very simple and we can implement it in a compact routine on the page opposite.

This routine consists of two steps. The first identifies the invader that has advanced furthest along the web. The second then decides in which direction we must move to get nearer to that invader.

The first step is a simple matter of finding the invader with the highest 'Z Position' (which is measured on the Y axis by *Tempest* and stored in the `INVAY` array for each invader). The second step depends on finding the shortest distance between the line the player's cursor is in and that of the selected invader. A subroutine called `POLDEL`

is used to calculate this, and the direction is indicated by whether the value is positive (to the left) or negative (to the right).

```
   .SBTTL  PLAY-AUTO MOVE OF CURSOR
 AUTOCU:
   LDA I,-1         ; STORE -1 IN THE ACCUMULATOR.
   STA TEMP0        ; USE IT TO INITIALISE TEMP0.
   STA TEMP1        ; USE IT TO INITIALISE TEMP1.

   ; GET THE INDEX OF THE INVADER THAT IS FURTHEST ADVANCED
   ; ALONG THE WEB.
   LDX WINVMX       ; GET THE NUMBER OF INVADERS.
   BEGIN            ; LOOP FOR ALL INVADERS
     LDA X,INVAY    ; GET CURRENT INVADER'S DEPTH POSITION.
     IFNE           ; IF IT HAS ONE THEN..
       CMP TEMP0    ; IS IT THE HIGHEST SO FAR?
       IFCC         ; IF IT IS THEN..
       STA TEMP0    ; STORE THE NEW HIGHEST POSITION.
       STX TEMP1    ; STORE THE INDEX OF HIGHEST INVADER.
       ENDIF
     ENDIF
     DEX            ; GO THE NEXT INVADER.
   MIEND            ; LOOP UNTIL WE'VE CHECKED THEM ALL.

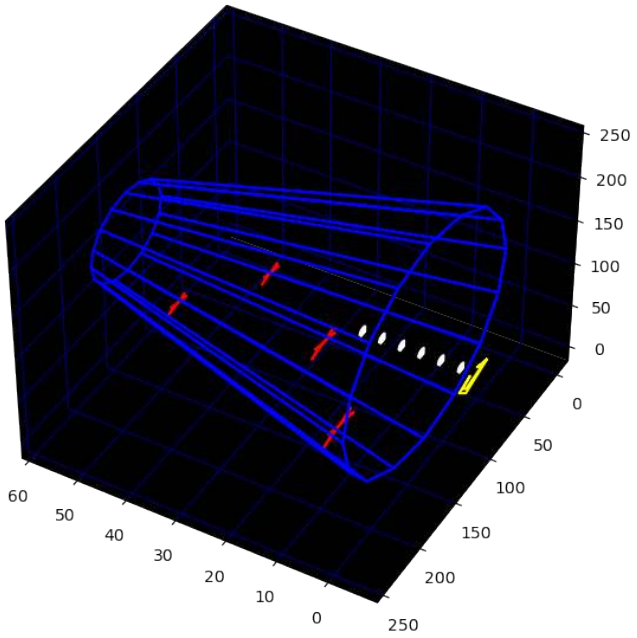
   ; FIGURE OUT IF MOVING LEFT OR RIGHT GETS US NEARER TO THE
   ; FURTHEST ADVANCED INVADER.
   LDX TEMP1       ; STORE INDEX TO FURTHEST ADVANCED INVADER IN X.
   IFPL            ; IF WE FOUND ONE THEN..
     LDA X,INVAL1  ; GET THE LINE THE INVADER IS ON.
     LDY CURSL1    ; GET THE LINE THE PLAYER IS ON.
     JSR POLDEL    ; CALCULATE HOW FAR AWAY ONE IS FROM THE OTHER.
     TAY           ; STORE THE RESULT IN Y REGISTER.
     IFNE          ; IF PLAYER IS NOT ALREADY ON SAME LINE AS ENEMY..
       IFPL        ; THEN IF RESULT FROM POLDEL IS POSITIVE..
       LDA I,-9    ; MOVE LEFT..
       ELSE        ; OTHERWISE..
       LDA I,9     ; MOVE RIGHT.
       ENDIF
     ENDIF
   ENDIF
   RTS
```

This isn't a perfect strategy, as we've already indicated. It means we potentially move away from invaders that are nearer to our line of fire. But it does at least ensure that we are always attempting to kill any enemy that is in danger of reaching the top of the web and overrunning us.

shooting admin

As long as we're trying to get nearer to the enemy that poses the most imminent danger, we should also have a strategy for what we do when we get there. The obvious one is: shoot. And that is indeed what we do, but with some latitude on top that improves our chances of killing any invader that happens to be in our field of fire. Our strategy for shooting, therefore, will be to look at every enemy, and every shot fired by the enemy, and fire a shot in their direction if the line they are on is within two moves from our present position.

This means we will fire a shot along lines even when there isn't an enemy currently there (in the hope that they might move there soon). It also means we no pay attention to how many shots we have already fired or any consideration to whether it might be better to wait for a better opportunity.



Shooting when no one is there: an example of firing player charges when enemies are adjacent.

We implement this simple control logic in the same routine responsible for shooting charges when the player presses the fire button: `FIREPC`.

```
   .SBTTL PLAY - FIRE PLAYER CHARGE
FIREPC:
  LDA CURSL2                ; GET THE PLAYER'S LINE POSITION.
  IFPL                      ; NON-NEGATIVE MEANS THEY'RE ALIVE SO..
  LDA QSTATUS               ; CHECK WHETHER IN ATTRACT MODE..
  IFPL                      ; IF WE ARE THEN..
    LDA CURMOD              ; UNLESS WE'RE DROPPING, CURMOD WILL BE 0.
    STA TEMP0               ; STORE AS OUR PROVISIONAL FIRE DECISION.

    ; LOOP THROUGH ALL INVADERS AND ALL SHOTS FIRED BY INVADERS TO LOOK
    ; FOR ANY ON A LINE NEAR TO THE PLAYER.
    LDX I,NICHARG+NINVAD-1  ; THE NUMBER OF INVADERS PLUS INVADER SHOTS.
    BEGIN                   ; LOOP THROUGH ALL INVADERS + INVADER SHOTS.
      LDA X,CHARY+NPCHAR    ; GET DEPTH OF CURRENT INVADER.
      IFNE                  ; IF NON-ZERO, IT IS IN PLAY SO..
        LDA X,CHARL1+NPCHAR ; GET THE LINE IT'S ON.
        SEC                 ; SET THE CARRY BIT BEFORE SUBTRACTING.
        SBC CURSL1          ; SUBTRACT PLAYER LINE FROM INVADER LINE.
        IFMI                ; IF NEGATIVE MAKE IT AN ABSOLUTE NUMBER..
        EOR I,0FF           ; BY X-OR'ING THE RESULT.
        CLC                 ; AND CLEAR THE CARRY BIT BEFORE..
        ADC I,1             ; ADDING 1 TO THE DIFFERENCE.
        ENDIF
        CMP I,2             ; COMPARE THE RESULT WITH 2..
        IFCC                ; IF THEY ARE LESS THAN 2 LINES APART THEN..
        INC TEMP0           ; SET TEMP0 SO THAT WE WILL FIRE A SHOT.
        ENDIF
      ENDIF
      DEX                   ; MOVE TO THE NEXT INVADER.
    MIEND                   ; LOOP UNTIL ALL INVADERS & SHOTS ARE DONE.
    LDA TEMP0               ; STORE OUR FIRE DECISION IN THE ACCUMULATOR.
  ELSE                      ; IF WE"RE NOT IN ATTRACT MODE THEN..
    LDA SWSTAT              ; READ INPUT FROM THE FIRE BUTTONS..
    AND I,MFIRE             ; IF THE FIRE BITS ARE SET, STORE THE DECISION.
  ENDIF
```

Hopefully the indentation above helps apprehend the structure of the above. In the first branch of the if statement we are calculating the fire decision for attract mode by looping through all the invaders and invaders shots. The second branch of the IF statement is much shorter by comparison: it reads the input from a human player and determines if the fire button has been pressed or not. The end result in both cases is stored in the accumulator register (A). If the value is non-zero it means we should go ahead and fire. Here's the remainder of the FIREPC routine. It looks for a vacant slot in the 8 active shots available to the player, if one is free then we can go ahead and fire. Otherwise the player (or in our case the attract mode) will have to try again the next time around.

```
  IFNE                    ; IF THE FIRE DECISION IS NON-ZERO THEN..
    LDX I,NPCHARG-1       ; GET THE NUMBER OF SLOTS FOR BULLETS (8)..
    BEGIN                 ; LOOP THROUGH EACH SLOT LOOKING FOR A SPARE..
      LDA X,CHARY         ; DOES THIS BULLET HAVE A DEPTH POSITION?
      IFEQ                ; IF NOT, THEN IT'S FREE AND WE CAN USE IT.
        INC CHACOU        ; INCREMENT THE BULLETS-USED COUNT.
        LDA CURSY         ; GET THE PLAYER'S DEPTH POSITION.
        STA X,CHARY       ; STORE IT AS THE NEW BULLET'S DEPTH POSITION.
        LDA CURSL1        ; GET THE PLAYER'S CURRENT LINE.
        STA X,CHARL1      ; STORE IT AS THE BULLET'S LINE.
        LDA CURSL2        ; GET THE PLAYER'S CURRENT LINE.
        STA X,CHARL2      ; STORE IT AS THE BULLET'S LINE.
        LDA I,0           ; INITIALIZE TO ZERO..
        STA X,CHARCO      ; THE BULLET'S COLLISION COUNTER.
        JSR SLAUNC        ; PLAY THE BULLET-FIRED SOUND.
        LDA CURSY         ; GET THE PLAYER'S DEPTH.
        JSR COLCHK        ; CHECK FOR COLLISION WITH ENEMY.
        LDX I,0           ; EXIT LOOP BY SETTING X TO 0.
      ENDIF
      DEX                 ; DECREMENT X TO MOVE TO NEXT BULLET.
    MIEND                 ; KEEP LOOPING UNTIL X GETS TO -1.
  ENDIF
```

We have specified just two mechanics: moving and shooting; and we've kept their operation very simple. Yet just a few seconds watching *Tempest* play by itself in *attract mode* testifies to their unreasonable effectiveness at emulating an accomplished player.

auto




```
tst auto      ; Check if in demo-mode.
beq namsg     ; If we're not, skip.
; Draw the demo text.
lea bfont,a1  ; Load the large font
lea autom1,a0 ; "Demo" string
move #50,d0   ; Set y position
jsr centext   ; Display text in center
```

As in the original *Tempest*, managing the attract mode behaviour of the claw comes down to deciding where to move and when to shoot. What we find in *Tempest 2000* is that shooting is simplified down to 'always shoot'. Unlike *Tempest*, no attention is paid to whether there may be an enemy present or not: the rule is that we will stop and shoot if there is an enemy on the same lane, and we'll also fire off a bullet whenever we move left or right. This keeps things simple, but does mean that we will sit on a lane firing bullets at an enemy until it is destroyed rather than moving on immediately.

We find this approach compactly implemented at the head of the `claw_con1` routine. It outsources the decision on whether left or right movement is the best choice to a routine called `lor` but looks afer everything else itself.

```
claw_con1:
        move #1,whichclaw     ; Flag that this is player 1.
        tst auto              ; Are we in demo mode?
        beq sclawr            ; If we're not, use player input.

        ; Otherwise we're in demo mode, so decide what to do.
        move.l pad_now,-(a7) ; Stash any player input on the stack.
        move droid_data,d0    ; droid_data has the lane of enemy nearest top.
```

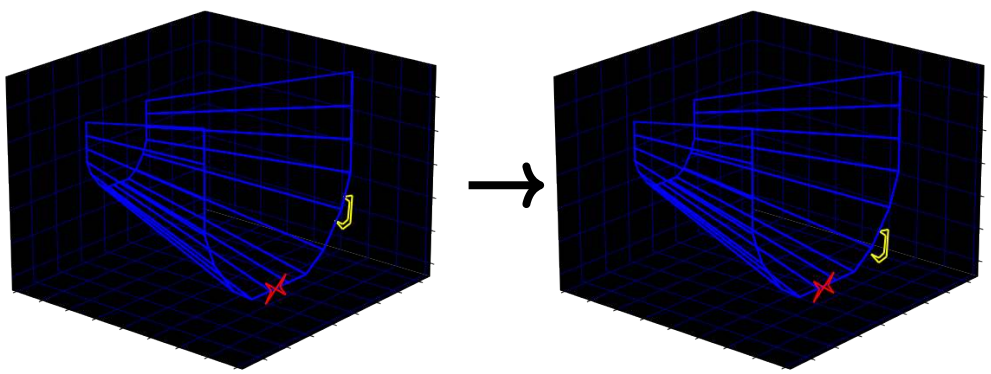
```
        bsr lor              ; Decide whether to move left or right.

        bpl moveme           ; If movement selected, perform the movement.
        clr.l 20(a6)         ; Otherwise, stop the claw here.
        move.l fire_1,pad_now ; Press fire.
gscl:   bsr sclawr           ; Shoot a bullet at the enemy.
        move.l (a7)+,pad_now ; Restore stashed player input from the stack.
        rts                  ; We're done - so return.

        ; Look after the movement.
moveme: bne clawright        ; If '0' from 'lor' then move right.
        ; Otherwise We need to move left.
        move.l fire_1,d0     ; Might as well fire a bullet too.
        or.l #$00400000,d0   ; Move left.
        move.l d0,pad_now    ; Add bullet and movement to input.
        bra gscl             ; Move, fire bullet, and return.

clawright:                   ; We need to move right.
        move.l fire_1,d0     ; Might as well fire a bullet too.
        or.l #$00800000,d0   ; Move right.
        move.l d0,pad_now    ; Add bullet and movement to input.
        bra gscl             ; Move, fire bullet, and return.
```

The `lor` subroutine is responsible for figuring out whether the best movement is left or right. On a plane that is open, i.e. isn't closed like a circle, it's a simple question of whether the claw is to the right or left of the target and should move in the opposite direction to get nearer to it.



But in the case of a closed plane, this might be suboptimal. So what we do instead is split the closed plane into two halves. If the target is in a lower-numbered lane than

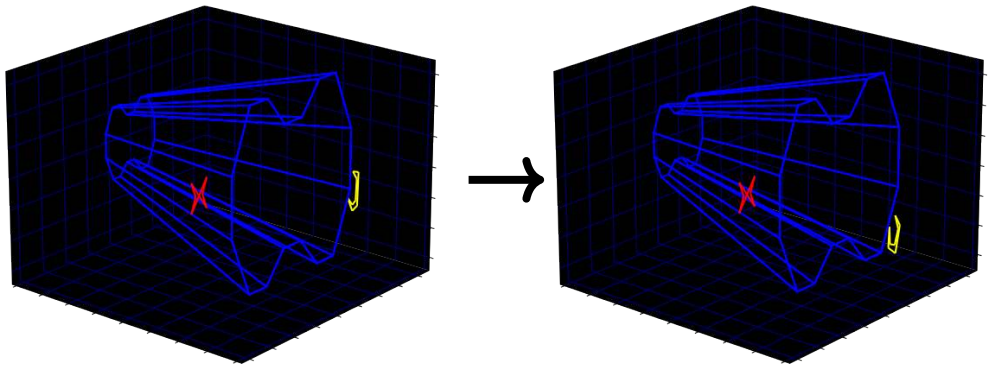
the claw, then the claw will move left if is in the ’lower’ half of the web:

```
; ********************************************************************
; lor: select whether to move left or right
; enter with d0=web column # of target.
; For any web, returns d1=0 for go left, 1 for go right,
; based on a6 object's position.
; ********************************************************************
lor:    move 16(a6),d1 ; Get the claw's current position.
        cmp d0,d1      ; Compare it to the topmost enemy's.
        bne nntrv      ; If positions not the same, go to nntrv.
        tst auto       ; Are we in demo mode?
        beq nntrv      ; If not, skip to nntrv.
        move #-1,d1    ; Tell demo mode to stop the claw here.
        rts            ; Return.

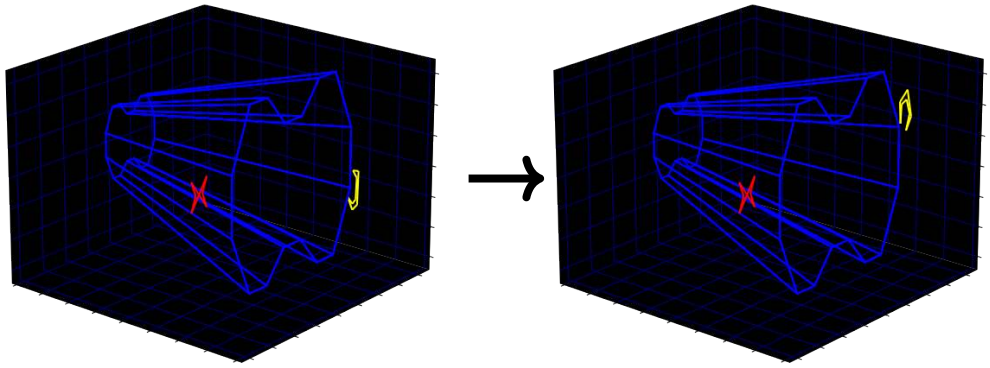
        ; Where position of claw and target are not the same, figure
        ; out whether to go left or right.
nntrv:  tst connect    ; Is it a closed/wrapper web?
        bne _lor1      ; If so, skip to _lor1 to handle it..
        ; (1) Othwerise the web is a plane that doesn't wrap so subtracting
        ; positions will tell us which way to move.
        sub d0,d1      ; Subtract the position of target from the claw.
        bgt lor0       ; If greater than 0, move left..
        ble lor1       ; If less than 0, move right..

        ; (2) Otherwise the web is a 'closed plane' that wraps, so need to
        ; figure the shortest path to the enemy.
_lor1:  move web_max,d2; Get number of lanes in web and put it in d2.
        asr #1,d2      ; Divide by 2 to get half total lanes in web.
        sub d0,d1      ; Subtract the position of target from the claw.
        bgt _lll       ; If target in a lower lane than claw, go to _lll.
        ble _rrr       ; If target in a higher lane than claw, go to _rrr.
        ; With target in a lower lane than the claw, if the claw is in
        ; the 'lower' half of the web then it should move left, otherwise
        ; it should move right.
_lll:   sub d2,d1      ; Subtract halfway-point in web from claw position.
        blt lor0       ; If it's less than zero, move left..
        bra lor1       ; Otherwise move right.

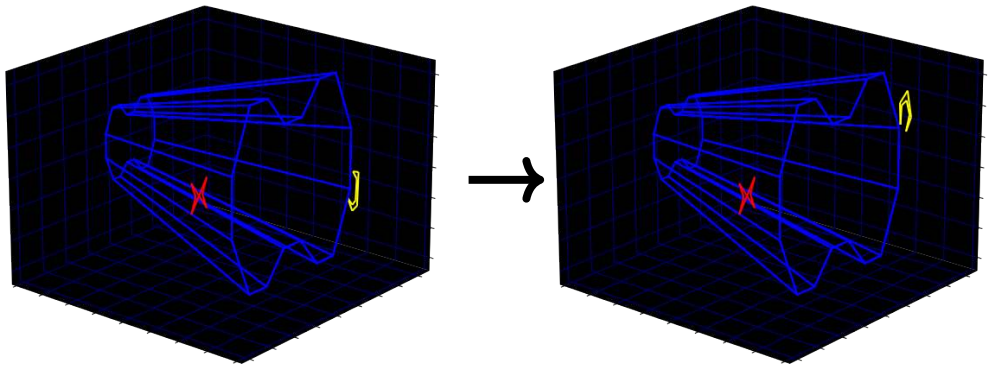
        ; With target in a higher lane than the claw, if the claw is in
        ; the 'lower' half of the web then it should move right, otherwise
        ; it should move left.
_rrr:   neg d1
        sub d2,d1      ; Subtract halfway-point in web from claw position.
        blt lor1       ; If it's less than zero, move right..
                       ; .. otherwise move left.
lor0:   clr d1         ; Decided to move left, so set d1 to 0.
        rts            ; Return
lor1:   move #1,d1     ; Decided to move right, so set d1 to 1.
        rts            ; Return
```



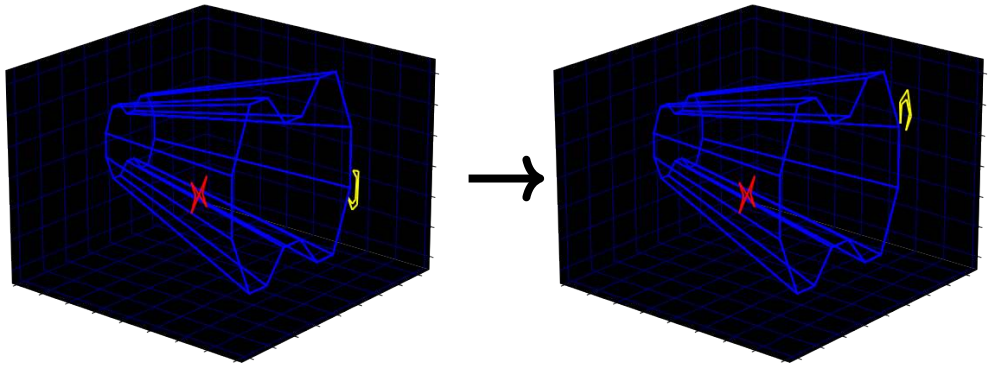
Otherwise it will move right:



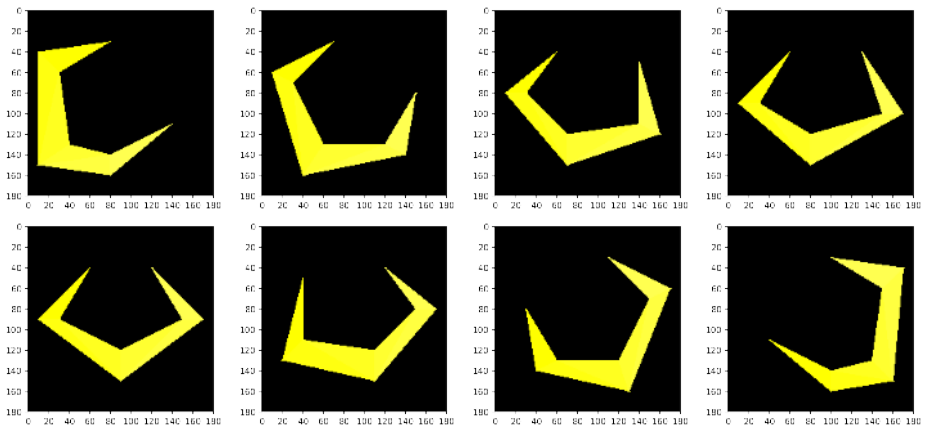
Likewise if the target is in a higher-numbered lane than the claw, then the claw will move right if it is in the 'lower' half of the web:



Otherwise it will move left:



more claws



The claws for Tempest 2000.

The Tempest 2000 claws are built using the same form of data structure we saw for 'flippers'. So although we don't need to rehearse the details in full again here. It is still worth seeing how a claw is built up from its component polygons.

The data structure for an upright claw in Tempest 2000 mode is as follows:

```
sclaw6:
  dc.l 6               ; 4 faces in this object, a shaded solid Flipper

 dc.w $ff             ; Face colour - Yellow.
  dc.w 0,$ff00         ; vertex index, intensity.
  dc.w 1,$ff00         ; vertex index, intensity.
  dc.w 2,$c000         ; vertex index, intensity.
  dc.w 0

  dc.w $fe             ; Face colour - Yellow.
  dc.w 1,$ff00         ; vertex index, intensity.
  dc.w 2,$c000         ; vertex index, intensity.
  dc.w 4,$ff00         ; vertex index, intensity.
  dc.w 0

  dc.w $fd             ; Face colour - Yellow.
  dc.w 2,$c000         ; vertex index, intensity.
  dc.w 3,$8000         ; vertex index, intensity.
  dc.w 4,$ff00         ; vertex index, intensity.
  dc.w 0

 dc.w $fd             ; Face colour - Yellow.
  dc.w 3,$8000         ; vertex index, intensity.
  dc.w 5,$c000         ; vertex index, intensity.
  dc.w 4,$ff00         ; vertex index, intensity.
  dc.w 0

  dc.w $fe             ; Face colour - Yellow.
  dc.w 4,$ff00         ; vertex index, intensity.
  dc.w 5,$c000         ; vertex index, intensity.
  dc.w 6,$ff00         ; vertex index, intensity.
  dc.w 0

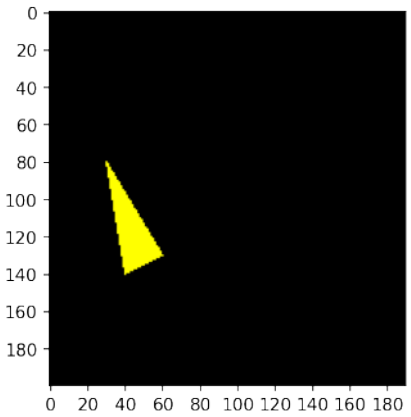
  dc.w $ff             ; Face colour - Yellow.
  dc.w 5,$c000         ; vertex index, intensity.
  dc.w 6,$ff00         ; vertex index, intensity.
  dc.w 7,$ff00         ; vertex index, intensity.
  dc.w 0

verts:
 dc.w 3,8            ; Index 0
  dc.w 4,14           ; Index 1
  dc.w 6,13           ; Index 2
  dc.w 12,13          ; Index 3
  dc.w 13,16          ; Index 4
  dc.w 15,7           ; Index 5
  dc.w 17,6           ; Index 6
  dc.w 11,3           ; Index 7
```

As before we are constructing our claw with a series of triangles. Each claw is made

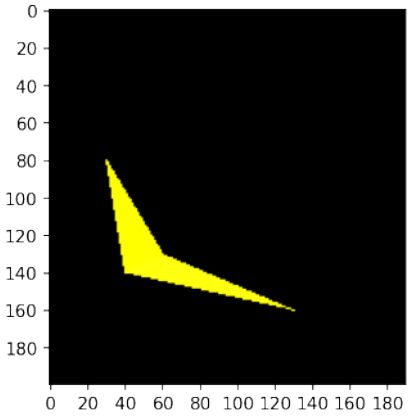
up 6 triangles, and for each triangle we specify 3 vertices.

With these, and our specified colour of yellow (`$ff`), we can make a triangle:



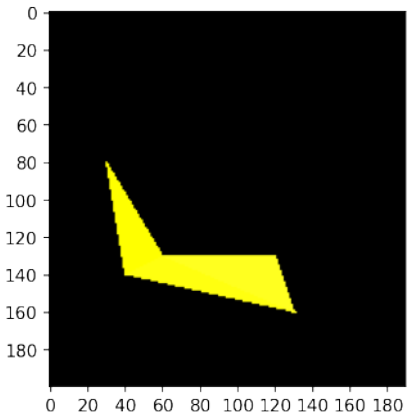

```
dc.w $ff     ; Face colour - Yellow.
dc.w 0,$ff00 ; vertex index, intensity.
dc.w 1,$ff00 ; vertex index, intensity.
dc.w 2,$c000 ; vertex index, intensity.
dc.w 0
```

```
dc.w 3,8     ; Index 0
dc.w 4,14    ; Index 1
dc.w 6,13    ; Index 2
```



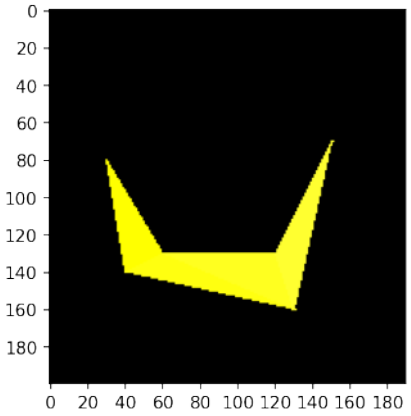

```
dc.w $fe      ; Face colour - Yellow.
dc.w 1,$ff00  ; vertex index, intensity.
dc.w 2,$c000  ; vertex index, intensity.
dc.w 4,$ff00  ; vertex index, intensity.
dc.w 0
```

```
dc.w 4,14    ; Index 1
dc.w 6,13    ; Index 2
dc.w 13,16   ; Index 4
```



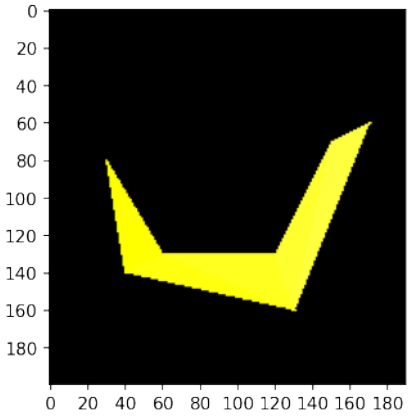

```
dc.w $fd       ; Face colour - Yellow.
dc.w 2,$c000   ; vertex index, intensity.
dc.w 3,$8000   ; vertex index, intensity.
dc.w 4,$ff00   ; vertex index, intensity.
dc.w 0
```

```
dc.w 6,13   ; Index 2
dc.w 12,13  ; Index 3
dc.w 13,16  ; Index 4
```



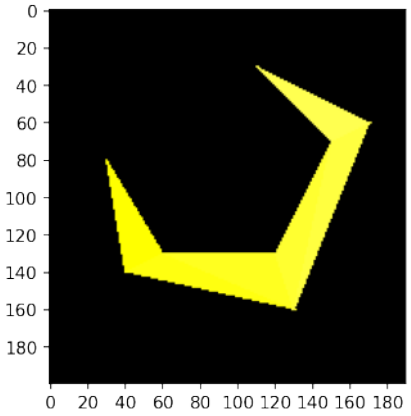

```
dc.w $fd       ; Face colour - Yellow.
dc.w 3,$8000   ; vertex index, intensity.
dc.w 5,$c000   ; vertex index, intensity.
dc.w 4,$ff00   ; vertex index, intensity.
dc.w 0
```

```
dc.w 12,13   ; Index 3
dc.w 15,7    ; Index 5
dc.w 13,16   ; Index 4
```




```
dc.w $fe      ; Face colour - Yellow.
dc.w 4,$ff00  ; vertex index, intensity.
dc.w 5,$c000  ; vertex index, intensity.
dc.w 6,$ff00  ; vertex index, intensity.
dc.w 0
```

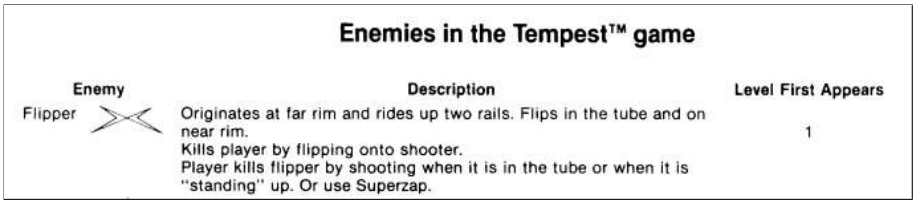
```
dc.w 13,16 ; Index 4
dc.w 15,7  ; Index 5
dc.w 17,6  ; Index 6
```




```
dc.w $ff      ; Face colour - Yellow.
dc.w 5,$c000  ; vertex index, intensity.
dc.w 6,$ff00  ; vertex index, intensity.
dc.w 7,$ff00  ; vertex index, intensity.
dc.w 0
```

```
dc.w 15,7 ; Index 5
dc.w 17,6 ; Index 6
dc.w 11,3 ; Index 7
```

flipper



Enemies in the Tempest™ game

Enemy	Description	Level First Appears
Flipper	Originates at far rim and rides up two rails. Flips in the tube and on near rim. Kills player by flipping onto shooter. Player kills flipper by shooting when it is in the tube or when it is "standing" up. Or use Superzap.	1

Figure .1: The description of the flipper in the Tempest manual.

The graphics in *Tempest* are generated using Atari's 'Quadrascan' vector technology. The images are all defined using a series of vectors. A vector in this case is a (X,Y) value pair that moves the beam in an X,Y direction to a new point on the screen. A series of vectors moving from point to point around the screen will eventually form a complete image. When we take a look at the data structure defining the player's principle enemy, the *Flipper*, we find this. I'm going to speculate that `INVA1S` is short for 'Invader 1 Start' and `INVA1E` for 'Invader 1 End':

```
INVA1S:
        VEC 4,1,1
        VEC 4,-1,1
        VEC -2,1
        VEC 1,1
        VEC -3,-1
        VEC -3,1
        VEC 1,-1
        VEC -2,-1
INVA1E:
```

The number pairs in the above listing are the vectors we're talking about, and we're going to see how we can use them to build up an image of the ship assuming an origin point of zero. Our first step, therefore, is to draw a line from (0,0) to (3,-2) as follows:

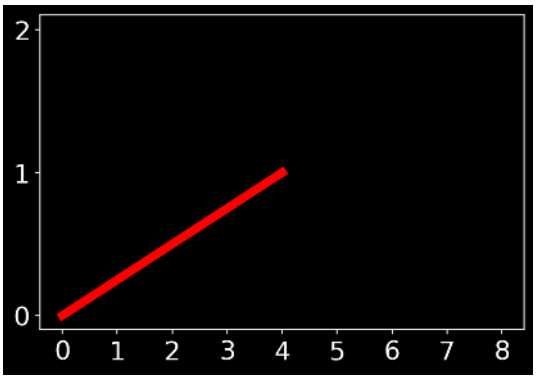


Figure .2: Draw a line from `(0,0)` to `(4,1)`.

Our next step reveals a bit more about the actual nature of the operation we are performing. To draw a line from our new position using a vector of `4,-1` we add it to our current position of `(4,1)` to draw a line to `8,0`:

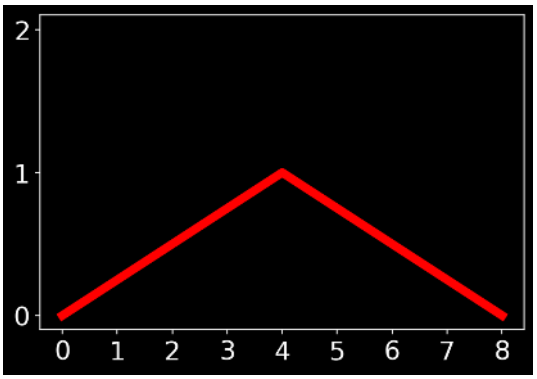
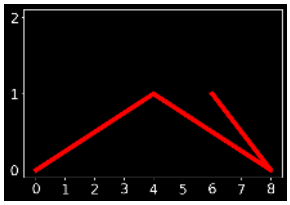
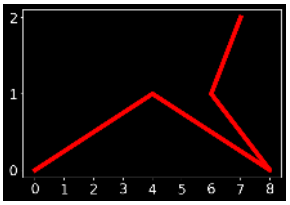


Figure .3: `(4,1)` + `(4,-1)`. ⟶ `(8,0)`

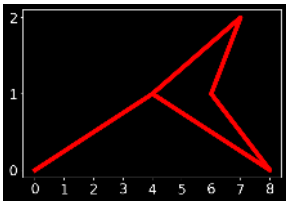
With this as our method, we can start building up our complete image, adding each vector in our array to the previous result to define a new line to draw.



(a) `(8,0) + (-2,1).` ⟶ `(6,1)`



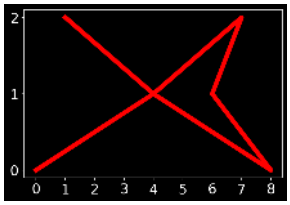
(b) `(6,1) + (1,1).` ⟶ `(7,2)`



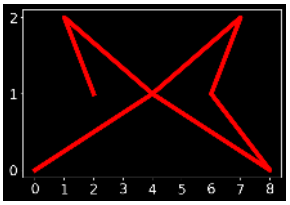
(c) `(7,2) + (-3,-1).` ⟶ `(4,1)`

Figure .4: Starting to take shape.

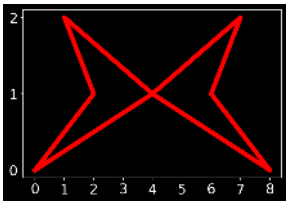
Adding our last three vectors completes the picture, and we have a claw.



(a) `(4,1) + (-3,1).` ⟶ `(1,2)`



(b) `(1,2) + (1,-1).` ⟶ `(2,1)`



(c) `(2,1) + (-2,-1).` ⟶ `(0,0)`

Figure .5: Putting the final pieces in place.

flipper 2000

The data structure as given in the source code is:

```
s_flipper:
  dc.l 4        ; 4 faces in this object, a shaded solid Flipper

 dc.w $f0       ; Face colour - Red.
 dc.w 3,$8000   ; vertex index, intensity.
 dc.w 1,$c000   ; vertex index, intensity.
 dc.w 0,$ff00   ; vertex index, intensity.
 dc.w 0

 dc.w $f4       ; Face colour - Orange.
 dc.w 4,$4000   ; vertex index, intensity.
 dc.w 2,$8000   ; vertex index, intensity.
 dc.w 0,$c000   ; vertex index, intensity.
 dc.w 0

 dc.w $f4       ; Face colour - Red.
 dc.w 5,$8000   ; vertex index, intensity.
 dc.w 1,$c000   ; vertex index, intensity.
 dc.w 0,$ff00   ; vertex index, intensity.
 dc.w 0

 dc.w $f0       ; Face colour - Orange.
 dc.w 6,$4000   ; vertex index, intensity.
 dc.w 2,$8000   ; vertex index, intensity.
 dc.w 0,$c000   ; vertex index, intensity.
 dc.w 0

fverts: dc.w 9,9,5,9,13,9,1,0,17,0,1,18,17,18
```

A little more explanation, taking the first entry in the structure:

```
  dc.w $f0    ; Face colour - Red.
  dc.w 3,$8000 ; Index into fverts, followed by intensity value.
  dc.w 1,$c000 ; Index into fverts, followed by intensity value.
  dc.w 0,$ff00 ; Index into fverts, followed by intensity value.
  dc.w 0        ; Padding that rounds up the entry to 16 bytes
```

We can see that the middle three entries start with an index into the array `fverts` we listed above. The index is pair-wise, so for example an index of 3 must be multiplied by 2, and then counting into `fverts` starting from 0 we end up at the 7th element and the pair: `1,0`. We can make this more explicit by breaking out the presentation of `fverts` to match the way it's used:

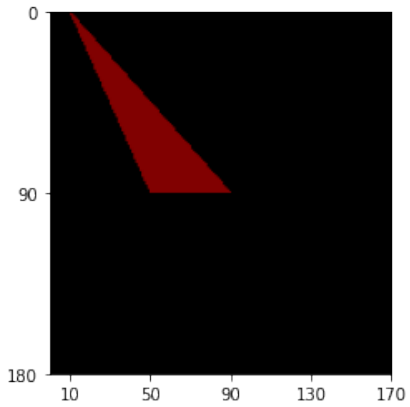
```
fverts: dc.w 9,9   ; Index 0
        dc.w 5,9   ; Index 1
        dc.w 13,9  ; Index 2
        dc.w 1,0   ; Index 3
        dc.w 17,0  ; Index 4
        dc.w 1,18  ; Index 5
        dc.w 17,18 ; Index 6
```

What are these pairs of numbers? They are (x,y) co-ordinates of course. It turns out `vertex` is just a fancy term for a point in two-dimensional space. So we can now begin to discern what this data structure is: it contains 4 sets of 3 vertices each. If you have 3 points in space, you have a triangle. So what we have here is a description of 4 triangles and each triangle is beign referred to as a 'face'.

Let's use the indices in the first entry to see what the first face gives us. We already saw how to get from an index of 3 to a vertex of `1,0` above. When we do the other two the three pairs of co-ordinates turn out as: to be as follows:

Index	X	Y	Description
3	1	0	First vertex.
1	5	9	Second Vertex.
0	9	9	Third Vertex.

With these, and our specified colour of red (`$f0`), we can make a triangle:




```
        dc.w $f0        ; Colour - Red.
        dc.w 3,$8000    ; index, intensity.
        dc.w 1,$c000    ; index, intensity.
        dc.w 0,$ff00    ; index, intensity.
        dc.w 0
```

```
        dc.w 9,9    ; Index 0
        dc.w 5,9    ; Index 1
        dc.w 1,0    ; Index 3
```

Figure .1: Our three vertices joined together in a triangle. Notice that we've scaled up our co-ordinates by 10. So `(9,9)`, for example, is given as `(90,90)`.

One triangle is all very nice, but we have a flipper to build. Should we try a second triangle? Let's see how that goes.

The instructions for our new triangle are in the second paragraph of the `s_flipper` data structure:

```
  dc.w $f4        ; Face colour - Orange.
  dc.w 4,$4000    ; vertex index, intensity.
  dc.w 2,$8000    ; vertex index, intensity.
  dc.w 0,$c000    ; vertex index, intensity.
  dc.w 0
```

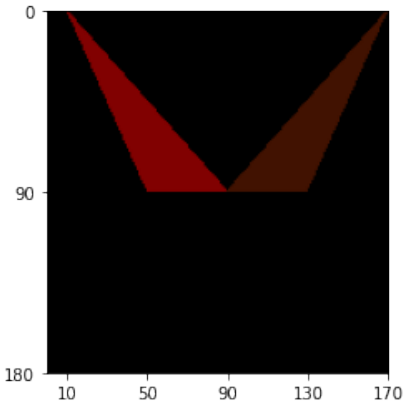
When we translate this to co-ordinates from `fverts` we get:

Index	X	Y	Description
4	17	0	First vertex.
2	13	9	Second Vertex.
0	9	9	Third Vertex.

Not wildly different than our previous effort. Putting our vertices for the two faces together in a single table we get:

Face	Vertex 1	Vertex 2	Vertex 3
1	1,0	5,9	9,9
2	17,0	13,9	9,9

Let's see what we get when we add it in:




```
dc.w $f4        ; Colour - Orange.
dc.w 4,$4000    ; index, intensity.
dc.w 2,$8000    ; index, intensity.
dc.w 0,$c000    ; index, intensity.
dc.w 0
```

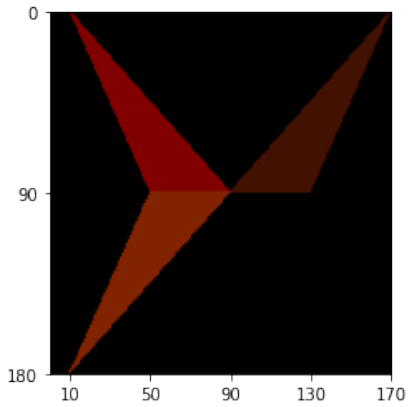
```
dc.w 9,9   ; Index 0
dc.w 13,9  ; Index 2
dc.w 17,0  ; Index 4
```

Figure .2: Adding in our second triangle

OK, things are starting to take shape. Here are the vertices for all four faces together:

Face	Vertex 1	Vertex 2	Vertex 3
1	1,0	5,9	9,9
2	17,0	13,9	9,9
3	1,18	5,9	9,9
4	17,18	13,9	9,9

Let's see what it looks like when we add in the third triangle:

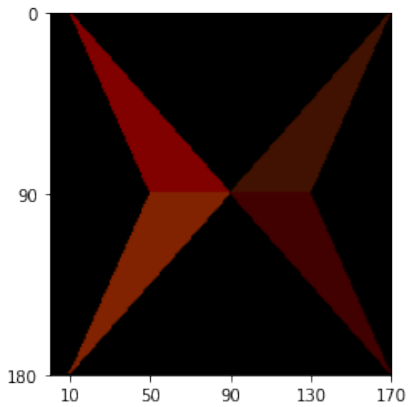



```
dc.w $f4        ; Colour - Red.
dc.w 5,$8000    ; index, intensity.
dc.w 1,$c000    ; index, intensity.
dc.w 0,$ff00    ; index, intensity.
dc.w 0
```

```
dc.w 9,9    ; Index 0
dc.w 5,9    ; Index 1
dc.w 1,18   ; Index 5
```

Figure .3: Adding in our third triangle

And the fourth and final triangle:




```
dc.w $f0        ; Colour - Orange.
dc.w 6,$4000    ; index, intensity.
dc.w 2,$8000    ; index, intensity.
dc.w 0,$c000    ; index, intensity.
dc.w 0
```

```
dc.w 9,9    ; Index 0
dc.w 13,9   ; Index 2
dc.w 17,18  ; Index 6
```

Figure .4: Adding in our final triangle

```
draw_sflipper:
  lea s_flipper,a1
  bra drawsolidxy
```

```
drawsolidxy:
  lea in_buf,a0
```

```
    move.l a1,(a0)+ ; pointer to our s_flipper data structure
    move.l d2,(a0)+ ; x position to draw flipper
    move.l d3,(a0)+ ; y position to draw flipper
    move.l d1,(a0)+ ; z position to draw flipper
    move.l d4,(a0)+ ; x position of flipper's origin
    move.l d5,(a0)+ ; y position of flipper's origin
    move.l d0,(a0)  ; angle

    move.l #0,gpu_mode
    lea equine,a0
    jsr gpurun      ;do clear screen
    jsr gpuwait
    rts
```

explosions one to four

It is almost impossible to notice when playing the game, but the explosion of enemies in *Tempest* is an animated sequence. It happens so quickly, in just a few frames, that the effect is almost subliminal: you can somehow detect that the explosion is animated if you look straight at it, but you cannot actually see it.

Since we have the luxury of slowing things down to a page-turning crawl let's look at the four frames of the explosion sequence one by one. We will also get to notice the particular economy with which it is achieved.

We'll start with an explosion in its infancy. On the left we show the code that defines how the explosion is drawn. The first three lines contain parameters used by the main drawing routine `SPOK16`. With a scaling factor of just `1` given by `CM`, the explosion starts small:



When we move on to the second frame of the explosion we start to see some progress, but may be surprised that the code for `EXPL2` is identical to `EXPL1`. We have to look

closely to observe the only actual difference, the scaling factor of CM is now set to 2. This has the apparently magical effect of scaling up the explosion by a second order of magnitude.



To understand this magic we first have to take a look at the SPOK16 routine being used to do the actual drawing.

```
.MACRO SPOK16           ; INPUT: CM=SCALE MULTIPLIER
                        ;        CB=INTENSITY
                        ;        CD=SCALE DIVIDER
    ICVEC               ; SET STARTING POINT TO CENTRE
    SCVEC 7,3           ; DRAW A LINE FROM 0,0 TO 7,3.
    SCVEC -7,-3,CB      ; DRAW A LINE FROM 7,3 to -7,-3.
    SCVEC -6,-6         ; DRAW A LINE FROM -7,-3 TO -6,-6.
    SCVEC 6,6,CB        ; DRAW A LINE FROM
    SCVEC 3,7           ; DRAW A LINE FROM
    SCVEC -3,-7,CB      ; DRAW A LINE FROM
    SCVEC 0,-8          ; DRAW A LINE FROM
    SCVEC 0,8,CB        ; DRAW A LINE FROM
    SCVEC -3,7          ; DRAW A LINE FROM
    SCVEC 3,-7,CB       ; DRAW A LINE FROM
    SCVEC 6,-6          ; DRAW A LINE FROM
    SCVEC -6,6,CB       ; DRAW A LINE FROM
    SCVEC -7,3          ; DRAW A LINE FROM
    SCVEC 7,-3,CB       ; DRAW A LINE FROM
    SCVEC 8,0           ; DRAW A LINE FROM
    SCVEC -8,0,CB       ; DRAW A LINE FROM
    SCVEC 0,0           ; DRAW A LINE FROM
.ENDM
```

At first the relevant magic is not apparent at all. SPOK16 draws our 16-spoked star-shaped explosion but uses the same co-ordinates each time. Where does the scaling happen? The answer to this lies under the hood in SCVEC, which is yet another *macro* that takes the hardcoded co-ordinates in icodeSPOK16 as parameters:

```
.MACRO SCVEC ...A,...B          ; A = X CO-ORD, B = Y CO-ORD
    CVEC ...A*CM/CD,...B*CM/CD ; SCALE UP CO-ORDS USING CM AND CD.
.ENDM
```

As you can see above it is in SCVEC that the scaling actually happens. The mysterious constant CM we saw in our first two diagrams finally comes in to play and is used to multiply the X and Y co-ordinates to achieve a scaling effect. For the indefatigably curious, I have given the contents of the CVEC macro below. This is where the hard yards are run to turn the co-ordinates into a line. Notice that it has to keep track of the position of the previous line in OLX and OLZ:

```
.MACRO CVEC NEWX,NEWZ,BRIT
  ...XN = NEWX-...OLX
  ...ZN = NEWZ-...OLZ
  ...BR=BRIT
  CS=1
  VCTR ...XN*CS,...ZN*CS,...BR
  ...OLX = NEWX
  ...OLZ = NEWZ
.ENDM
```

With the mystery of how our scaling works pretty much solved we can take a look what happens in the next frame, with our scale factor set to 4.



And in the final frame of the explosion, EXPL4, where the factor is set to 8, the enemy's destruction reaches its fullest extent.



meltovision

'melt-o-vision' was the name given to a neat interstitial feature in Tempest 2000 where the screen blurs, rotates and fades before transitioning to the next screen.



Figure .1: Melt-o-vision in action.

```
; ********************************************************************
; fade ; Do a meltovision transition.
; This is meltovision.
; https://www.trademarkia.com/meltovision-74534584
; Go into FADE after merging screen3 to current screen and
; turning off BEASTIES+64
; ********************************************************************
fade:   tst beasties+76             ; Are we ready for meltovision?
        bmi ofade                   ; If so, do a meltovision transition.
        ; If not do a merge first.
        move.l #screen3,a0          ; Point a0 at screen3.
        move.l gpu_screen,a1        ; Point a1 at gpu_screen.
        moveq #0,d0                 ; X position in source screen
        moveq #0,d1                 ; Y position in source screen
        move #384,d2                ; Width
        move #240,d3                ; Height
        add palfix1,d3              ; Adjust for PAL
        moveq #0,d4                 ; X position in destination screen.
        moveq #0,d5                 ; Y position in destination screen.
        tst mfudj                   ; Fudging height required?
        beq pmf2                    ; If not, skip.
        sub #8,d3                   ; Otherwise fudge source height by 8.

pmfade: tst mfudj                   ; Fudging height required?
        beq pmf2                    ; If not, skip.
        add #8,d5                   ; Otherwise fudge dest height by 8.
        clr mfudj                   ; Clear the fudge factor.

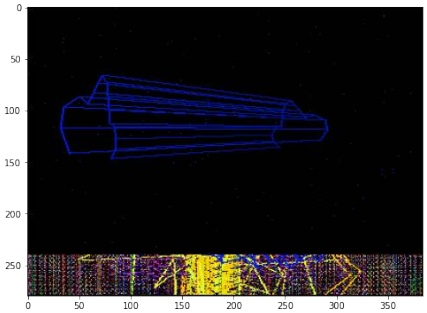
        ; Now we can merge the two screens together.
pmf2:   jsr MergeBlock              ; Merge screen3 and gpu_screen
        move #-1,beasties+76        ; Signal we can do a fade now.

        ; Now we can set up melt-o-vision (tm).
ofade:  clr _pauen                  ; Can't pause in fade
        move.l #ffade,demo_routine  ; Set meltovision as the demo_routine
        move.l #failcount,routine   ; Set the counter routine.
        move #150,pongx             ; Set the x counter to 150
        ; Get a random value for pongz.
        jsr rannum                  ; Random number between 0 and 255 in d0.
        and #$7,d0                  ; Just the last 3 bits please.
        sub #$03,d0                 ; Subtract 3.
        and #$ff,d0                 ; Keep the result between 0 and 255
        move d0,pongz               ; Set as pongz (our z counter).
        move.l #$f80000,delta_i     ; Set our delta intensity.
        move z,-(a7)                ; Stash z in the stack.
        clr z                       ; Clear hard reset signal.
        bsr gogame                  ; Run the meltovision transition.
        move.l #screen3,a0          ; Point a0 at screen3.
        jsr clrscreen               ; Clear the screen in a0.
        move (a7)+,z                ; Restore z from the stack.
        move #1,sync                ; Signal a sync is required.
        rts
```

This innovation was so important Atari decide to trademark it. They later decided it wasn't important at all and abandoned the trademark. That's how business works I guess.

In order to scale, rotate and zoom the current screen we need to merge the two layers its composed of. This means taking `gpu_screen`, which is the 'background', and `screen3`, which is our 'foreground', and compositing them so we have one region of data we can manipulate.

Raw Data	Parsed Data	Image at `gpu_screen` (`0x100000`)

`10,00,00,1D,6E,45,C1,60`	`TYPE: 0 LINK: 0xeb70`
	`YPOS: 44 DATA: 0x100000`
	`HEIGHT: 279`
		

`00,00,80,06,01,80,CF,F8`	`XPOS: -7 REFLECT: 0`
	`DEPTH: 4 RMW: 0`
	`PITCH: 1 TRANS: 1`
	`DWIDTH: 96 RELEASE: 0`
	`IWIDTH: 96 FIRSTPIX: 0`
	`INDEX: 0`	

Raw Data	**Parsed Data**	**Image at `screen3` (`0x50000`)**

`05,00,00,1D,72,45,C1,60`	`TYPE: 0 LINK: 0xeb90`
	`YPOS: 44 DATA: 0x50000`
	`HEIGHT: 279`
		

`00,00,80,06,01,80,CF,F8`	`XPOS: -7 REFLECT: 0`
	`DEPTH: 4 RMW: 0`
	`PITCH: 1 TRANS: 1`
	`DWIDTH: 96 RELEASE: 0`
	`IWIDTH: 96 FIRSTPIX: 0`
	`INDEX: 0`	

We do all of this in the `fade` routine on the opposite page. Once we have our source and destination co-ordinates ready we can call `MergeBlock` to do the compositing:

```
        ; Now we can merge the two screens together.
pmf2:   jsr MergeBlock              ; Merge screen3 and gpu_screen
```

`MergeBlock`, which we reproduce on the next page, consists of a call to the Jaguar's 'Blitter' with a simple command to draw the contents of `screen3` onto `gpu_screen`.

```
MergeBlock:
        move.l #0,B_PATD   ; Set transparency colour.
        move.l #0,B_PATD+4 ; Set transparency colour

        ; Set up the blitter command.
        move.l #PITCH1|PIXEL16|WID384|XADDINC,d7
        move.l d7,A1_FLAGS ; Set the Source screen flags.

        move.l #PITCH1|PIXEL16|WID384|XADDPIX|YADD1,d7
        move.l d7,A2_FLAGS ; Set the Dest screenf lags.

        ; d3 is the height, d2 is the width.
        move d3,d7         ; Put d3 (height) in lower word of d7.
        swap d7            ; Swap position of first 2 bytes with last 2.
        move d2,d7         ; Put d2 (width) in lower word of d7.
        move.l d7,B_COUNT  ; Store as inner and outer loop counts

        ; Set up the origin of source screen (x:d0, y:d1).
        move d1,d7         ; Put d1 (Y pos) in lower word of d7.
        swap d7            ; Swap position of first 2 bytes with last 2.
        move d0,d7         ; Put d0 (X pos) in lower word of d7.
        move.l d7,A1_PIXEL ; Store as X and Y origin of source screen.

        ; Set up the origin of dest screen (x:d4, y:d5).
        move d5,d7         ; Put d5 (Y pos) in lower word of d7.
        swap d7            ; Swap position of first 2 bytes with last 2.
        move d4,d7         ; Put d4 (X pos) in lower word of d7.
        move.l d7,A2_PIXEL ; Store as X and Y origin of dest screen.

        move.l #0,A1_FPIXEL
        move.l #$0001,A1_INC
        move.l #$0,A1_FINC

        ; Set our width (d2) as the number of loop steps.
        move #1,d7         ; Store 1 in d7
        swap d7            ; Swap position of first 2 bytes with last 2.
        move d2,d7         ; Put d2 (width) in lower word of d7.
        neg d7             ; Make it negative.
        move.l d7,A1_STEP  ; Store as the number of loops for source.
        move.l d7,A2_STEP  ; Store as the number of loops for dest.

        ; Copy the screen data for source and dest.
        move.l a0,d7       ; Copy the source screen (a0) to d7.
        move.l d7,A1_BASE  ; Set it as our source screen.
        move.l a1,d7       ; Copy the destination (a1) to d7.
        move.l d7,A2_BASE  ; Set it as our destination screen.

        ; Run the blitter command to draw one block onto the other.
        move.l #SRCEN|UPDA1|UPDA2|DSTA2|LFU_A|LFU_AN|DCOMPEN,d7
        move.l d7,B_CMD    ; Run the blitter command.
        bra WaitBlit       ; Wait for the blitter to finish.
```

The result is we go from having an object list with two screens, to just one, containing the `gpu_screen`. Here is what the object list contains with the two screens merged and the melt-o-vision transition from the high score screen just getting started.

Raw Data	Parsed Data	Image at `gpu_screen` (`0x100000`)

`10,00,00,1D,6E,45,C1,60`	`TYPE: 0 LINK: 0xeb70`
	`YPOS: 44 DATA: 0x100000`
	`HEIGHT: 279`
		

`00,00,80,06,01,80,CF,F8`	`XPOS: 4088 REFLECT: 0`
	`DEPTH: 4 RMW: 0`
	`PITCH: 1 TRANS: 1`
	`DWIDTH: 96 RELEASE: 0`
	`IWIDTH: 96 FIRSTPIX: 0`
	`INDEX: 0`	

Going back to the `fade` routine we can see that the next step after merging the two screens is to set up the meltovision transition itself. This consists of setting our GPU manager routine to `ffade` and a co-routine `failcount` that will track our progress along the X axis:

```
    move.l #ffade,demo_routine ; Set meltovision as the demo_routine
    move.l #failcount,routine  ; Co-routine to bail when pongx reaches 0.
```

And determining how much movement along the X axis (`pongx`) and how much rotation around the Z axis (`pongz`) the transition will consist of:

```
    move #150,pongx            ; Set the x counter to 150
    ; Get a random value for pongz.
    jsr rannum                 ; Random number between 0 and 255 in d0.
    and #$7,d0                 ; Just the last 3 bits please.
    sub #$03,d0                ; Subtract 3.
    and #$ff,d0                ; Keep the result between 0 and 255
    move d0,pongz              ; Set as pongz (our z counter).
```

Notice that we keep our rotation step (which is really an angle value) between 0 and 7, i.e. just the last 3 bits. With this done we can start running the meltovision transition itself:

```
    bsr gogame                 ; Run the meltovision transition.
```

`gogame`, which we won't print here, simply runs in a loop and invokes `ffade` until `failcount` has reduced `pongx` to zero.

```
ffade:
        ; This section ensures we only rotate a little, i.e. 7
        ; individual rotations of our value in pongz.
        add #1,pongy            ; Increment pongy.
        move pongy,d0           ; Move it to d0.
        and #$03,d0             ; Select the lowest 3 bits only.
        bne failfade            ; If non-zero, do a step in meltovision.
        tst.b pongz+1           ; Check pongz (out angle of rotation)..
        beq failfade            ; If it's zero, do a meltovision step.
        bmi ffinc               ; If it's negative, increment it.
        sub.b #2,pongz+1        ; Otherwise decrement it by 2.
ffinc:  add.b #1,pongz+1        ; Increment pongz.

        ; Run a step in the meltovision transition.
failfade:
        move.l #(PITCH1|PIXEL16|WID384),d0 ; Set d0 with our flags.
        move.l d0,source_flags ; Set as the source flags.
        move.l d0,dest_flags  ; And set as the dest flags.
        lea in_buf,a0           ; Point our GPU RAM input buffer at a0.
        move pongz,d0           ; Put angle of rotation in d0.
        and.l #$ff,d0           ; Keep it between 0 and 255
        move.l cscreen,(a0)     ; Put source screen in GPU buffer.
        move.l #384,4(a0)       ; Store full-screen width in GPU buffer.
        move.l #240,d1          ; Store full-screen height in d1.
        add palfix1,d1          ; Adjust for PAL screens if required.
        move.l d1,8(a0)         ; Store height in buffer.
        move.l #$1f4,12(a0)     ; Put X scale in GPU buffer.
        move.l #$1f4,16(a0)     ; Put Y scale in GPU buffer.
        move.l d0,20(a0)        ; Put initial angle in brads in GPU buffer.
        move.l #$c00000,24(a0); source x centre in 16:16
        move.l #$780000,d0      ; Set inital y centre position.
        add.l palfix3,d0        ; Adjust for PAL screens if required.
        move.l d0,28(a0)        ; Put y centre in GPU buffer.
        move.l #$0,32(a0)       ; offset of dest rectangle
        move.l delta_i,36(a0) ; change of i per increment
        ; Select 'scar' in antelope.gas.
        ; Op 2 of this module is Scale and Rotate
        move.l #2,gpu_mode      ; Select 'scar' in antelope.gas
        move.l #demons,a0       ; Load the demons module (antelope.gas).
        jsr gpurun              ; Run it.
        jmp gpuwait             ; Wait until GPU finished.
```

```
failcount:
        sub #1,pongx          ; Subtract 1 from pongx.
        bpl rrrts             ; If still positive we're not done yet.
        move #-1,x_end        ; Signal we're done
        move #1,screen_ready ; Signal screen ready for display.
        rts
```

We can play with a few parameters to alter the effect, and this will help us understand how it works.

For example, if we change our angle of rotation for each step to -16, by changing the following line in `fade` from:

```
        move d0,pongz           ; Store d0 in pongz.
```

to:

```
        move #$f0,pongz         ; Store -16 in pongz.
```

And if we increase the number of allowed rotations from 7 (3 bits) in `ffade` (opposite) from:

```
        and #$03,d0             ; Select the lowest 3 bits only.
```

to 256 (8 bits):

```
        and #$ff,d0             ; Select the whole byte instead.
```

We get an entertainingly rapid rotation of the screen!

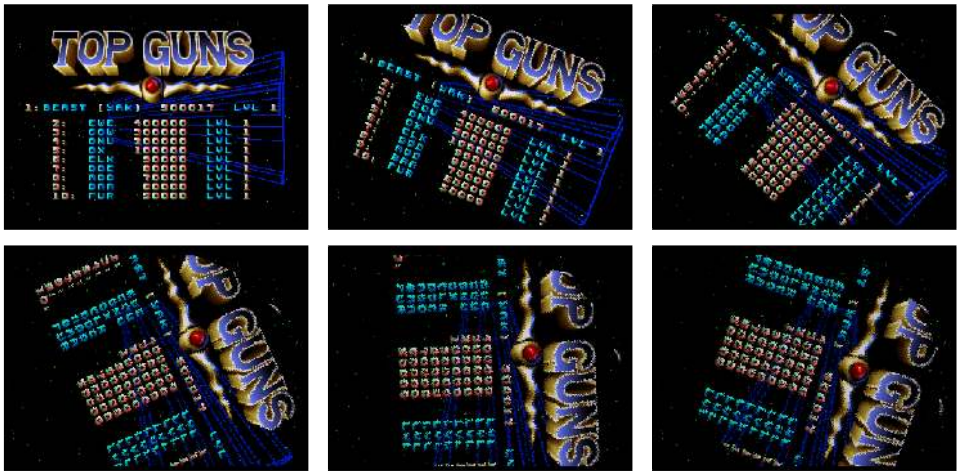


Figure .2: Melt-o-vision with a faster rotation.

We can also get rid of the rotation completely and alter the scaling factor to see what happens. Let's do that:

```
move pongz,d0          ; Initial angle.
..
move.l #$1f4,12(a0)    ; X scale.
move.l #$1f4,16(a0)    ; Y scale.
```

→

```
move #$00,d0           ; No rotation.
nop
..
move.l #$208,12(a0)    ; X scale.
move.l #$208,16(a0)    ; Y scale.
```

We've increased the scaling factor from `$1f4` to `$208`. The results are slightly strange.



Figure .3: Melt-o-vision with a higher scale factor and no rotation.

We get no obvious scaling of the image anymore, except across the central X/Y axis.

If we play with the centre Y position, by increasing it, we observe a different effect.

```
move pongz,d0          ; Initial angle.
..
move.l #$780000,d0     ; Initial Y centre
     .
```

→

```
move #$00,d0           ; No rotation.
nop
..
move.l #$7F0000,d0     ; Initial Y centre
     .
```

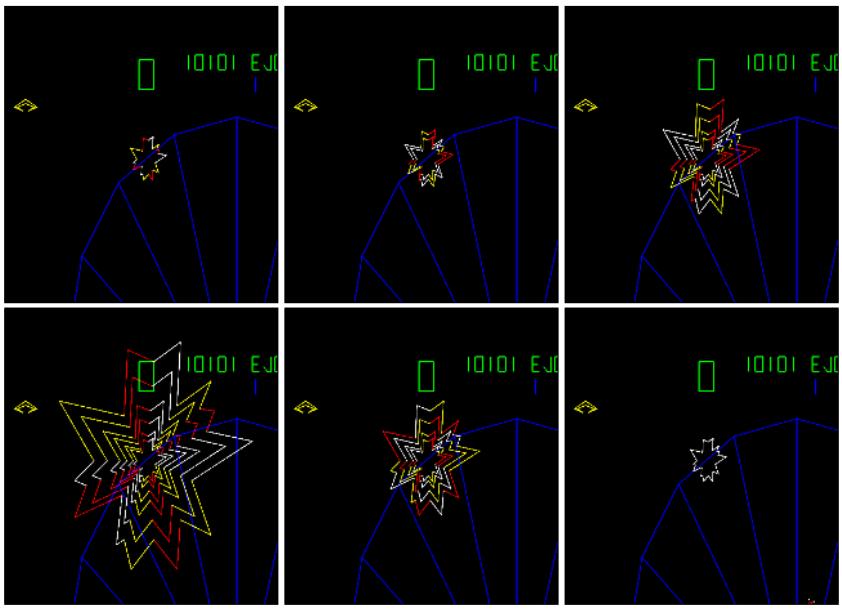
With a 'center' north of the centre of the screen the scaling ascends out of the top of the screen.



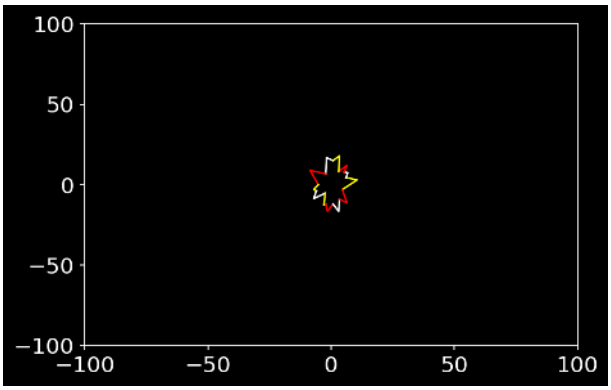
Figure .4: Melt-o-vision with a higher center factor and no rotation.

splat control

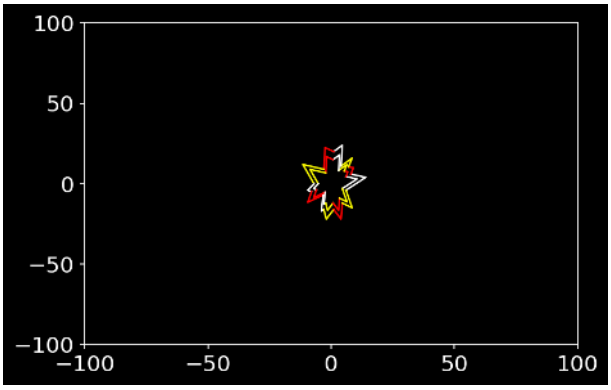
When the player is hit, he must die. And that means a good splat is required to ram the point home. *Tempest* achieves this with a simple but colorful effect that smears the remains of the player's cursor across the screen. We give an abbreviated extract of the full splat cycle below, ending in a white shell.



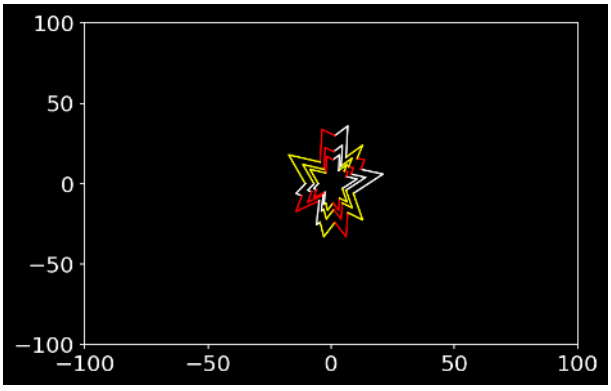
```
SPLAT6: SCAL 2,40
        JMPL SPLAT
```




```
SPLAT5: SCAL 2,0
        JSRL SPLAT
SPLAT6: SCAL 2,40
        JMPL SPLAT
```




```
SPLAT4: SCAL 1,40
        JSRL SPLAT
SPLAT5: SCAL 2,0
        JSRL SPLAT
SPLAT6: SCAL 2,40
        JMPL SPLAT
```



The full course of the player's explosion is described by what the *Tempest* source code rather quaintly describes as an EXPLOSION DATABASE. This is a list of nine entries that the game will invoke, one after another, to render each phase of the explosion. To help visualize it I've given each the 9 phases on the opposite page here and in the pages following.

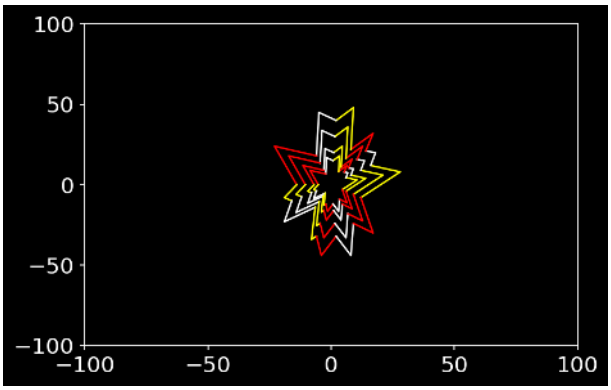
```
;EXPLOSION DATABASE FOR PLAYER CHARGE COLLISION
;
    JSRL SPLAT6   ; SMALLEST SPLAT
    JSRL SPLAT5   ; SECOND SMALLEST SPLAT
    JSRL SPLAT4   ; THIRD SMALLEST SPLAT
    JSRL SPLAT3   ; THIRD BIGGEST SPLAT
    JSRL SPLAT2   ; SECOND BIGGEST SPLAT
    JSRL SPLAT1   ; BIGGEST SPLAT OF ALL
    JSRL SPLAT3   ; THIRD BIGGEST SPLAT
    JSRL SPLAT5   ; SECOND SMALLEST SPLAT
    JSRL SPLAT6   ; SMALLEST SPLAT
```

The ingenuity of this little table lies in how it takes advantage of the way each entry has been ordered to draw the simple splat polygon in ever decreasing concentric scales. For example you'll notice that calling SPLAT6 results in a single splat shape being drawn. But when we call SPLAT5 for the second phase of the eplosion we will end up drawing two splat shapes: a small one, plus an even smaller one inside it. You can see how this is achieved in the SPLAT CONTROL listing below. The larger one, SPLAT5, is drawn first and then the smaller one, SPLAT6, is drawn after it.

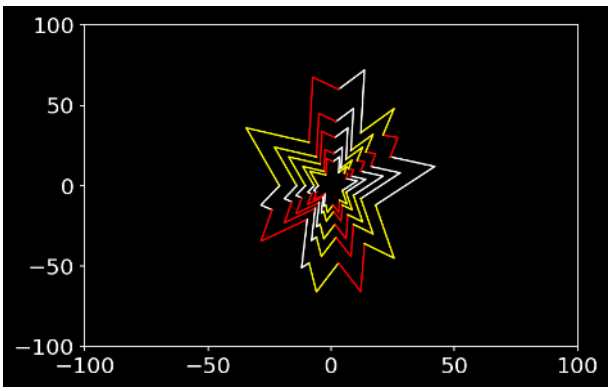
```
        .SBTTL  SPLAT CONTROL
SPLAT1: SCAL 0,0    ; SET SCALING TO 0.
        JSRL SPLAT  ; DRAW THE SPLAT.
SPLAT2: SCAL 0,40   ; DECREASE THE SCALING.
        JSRL SPLAT  ; DRAW THE SPLAT.
SPLAT3: SCAL 1,0    ; DECREASE THE SCALING.
        JSRL SPLAT  ; DRAW THE SPLAT.
SPLAT4: SCAL 1,40   ; DECREASE THE SCALING.
        JSRL SPLAT  ; DRAW THE SPLAT.
SPLAT5: SCAL 2,0    ; DECREASE THE SCALING.
        JSRL SPLAT  ; DRAW THE SPLAT.
SPLAT6: SCAL 2,40   ; DECREASE THE SCALING.
        JMPL SPLAT  ; DRAW THE SPLAT.
```

Still on the opposite page we can see the third phase in the explosion is rendered by calling SPLAT4 and falling through to execute SPLAT5 and SPLAT6 giving us three concentric, scaled splat shapes - each one smaller than the last. We see this pattern continue on the following page where we continue growing the explosion by entering the SPLAT CONTROL table earlier and earlier until we start at SPLAT1.

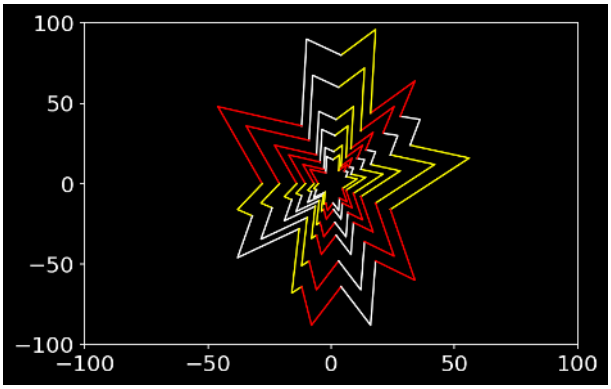
```
SPLAT3: SCAL 1,0
        JSRL SPLAT
SPLAT4: SCAL 1,40
        JSRL SPLAT
SPLAT5: SCAL 2,0
        JSRL SPLAT
SPLAT6: SCAL 2,40
        JMPL SPLAT
```




```
SPLAT2: SCAL 0,40
        JSRL SPLAT
SPLAT3: SCAL 1,0
        JSRL SPLAT
SPLAT4: SCAL 1,40
        JSRL SPLAT
SPLAT5: SCAL 2,0
        JSRL SPLAT
SPLAT6: SCAL 2,40
        JMPL SPLAT
```




```
SPLAT1: SCAL 0,0
        JSRL SPLAT
SPLAT2: SCAL 0,40
        JSRL SPLAT
SPLAT3: SCAL 1,0
        JSRL SPLAT
SPLAT4: SCAL 1,40
        JSRL SPLAT
SPLAT5: SCAL 2,0
        JSRL SPLAT
SPLAT6: SCAL 2,40
        JMPL SPLAT
```

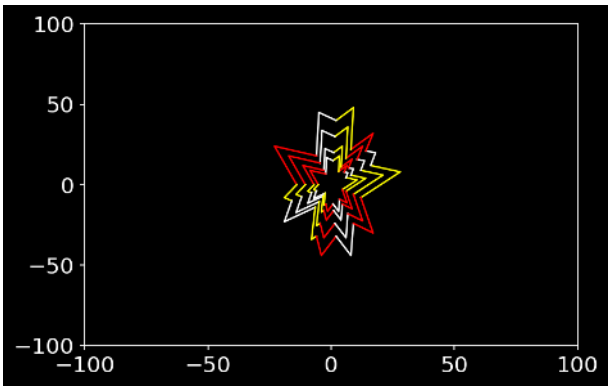


Our explanation so far has glossed over the actual drawing of the splat shape - the heavy lifting achieved by JSRL SPLAT. This is a common-or-garden set of vector instructions, similar to those we've seen elsewhere when discussing the rendering of claws and flippers.

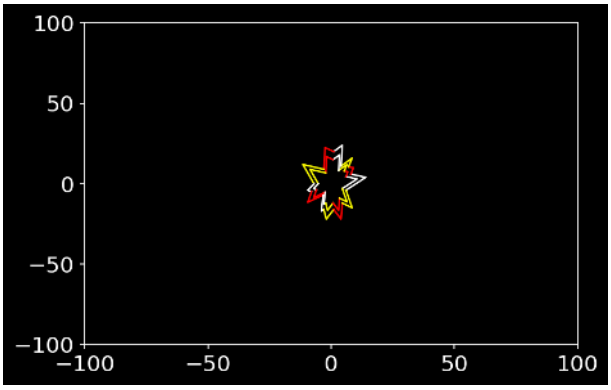
```
        .SBTTL  SPLAT PIC
SPLAT:  ICVEC           ; SET VECTOR POSITION TO CENTRE.
        CSTAT PDIWHI    ; SET LINE COLOR TO WHITE.
        SCVEC 18,-8,0   ; DRAW A BLANK LINE (NO COLOR).
        SCVEC 38,8,7    ; DRAW A WHITE LINE.
        SCVEC 20,0C,7   ; DRAW A WHITE LINE.
        CSTAT PDIRED    ; SET LINE COLOR TO RED.
        SCVEC 24,14,7   ; DRAW A RED LINE.
        SCVEC 1C,15,7   ; DRAW A RED LINE.
        CSTAT PDIYEL    ; SET LINE COLOR TO YELLOW.
        SCVEC 22,20,7   ; DRAW A YELLOW LINE.
        SCVEC 10,16,7   ; DRAW A YELLOW LINE.
        CSTAT PDIWHI    ; SET LINE COLOR TO WHITE.
        SCVEC 12,30,7   ; DRAW A WHITE LINE.
        SCVEC 4,28,7    ; DRAW A WHITE LINE.
        CSTAT PDIRED    ; SET LINE COLOR TO RED.
        SCVEC -0A,2D,7  ; DRAW A RED LINE.
        SCVEC -0C,12,7  ; DRAW A RED LINE.
        CSTAT PDIYEL    ; SET LINE COLOR TO YELLOW.
        SCVEC -2E,18,7  ; DRAW A YELLOW LINE.
        SCVEC -1C,0,7   ; DRAW A YELLOW LINE.
        CSTAT PDIWHI    ; SET LINE COLOR TO WHITE.
        SCVEC -26,-8,7  ; DRAW A WHITE LINE.
        SCVEC -20,-0A,7 ; DRAW A WHITE LINE.
        CSTAT PDIRED    ; SET LINE COLOR TO RED.
        SCVEC -26,-17,7 ; DRAW A RED LINE.
        SCVEC -0D,-0E,7 ; DRAW A RED LINE.
        CSTAT PDIWHI    ; SET LINE COLOR TO WHITE.
        SCVEC -10,-22,7 ; DRAW A WHITE LINE.
        SCVEC -0C,-20,7 ; DRAW A WHITE LINE.
        CSTAT PDIYEL    ; SET LINE COLOR TO YELLOW.
        SCVEC -8,-2C,7  ; DRAW A YELLOW LINE.
        SCVEC 4,-20,7   ; DRAW A YELLOW LINE.
        CSTAT PDIRED    ; SET LINE COLOR TO RED.
        SCVEC 10,-2C,7  ; DRAW A RED LINE.
        SCVEC 12,-18,7  ; DRAW A RED LINE.
        CSTAT PDIYEL    ; SET LINE COLOR TO YELLOW.
        SCVEC 22,-1E,7  ; DRAW A YELLOW LINE.
        SCVEC 18,-8,7   ; DRAW A YELLOW LINE.
        SCVEC 0,0,0     ; DRAW A BLANK LINE.
        RTSL            ; RETURN
```

The key is that this SPLAT routine doesn't need to worry about the scale or size of the vectors its drawing, just the dimensions. The actual size of the splat figure, which will determine where in the nested sequence of splats it sits, is controlled by setting the scale used by the vector hardware.

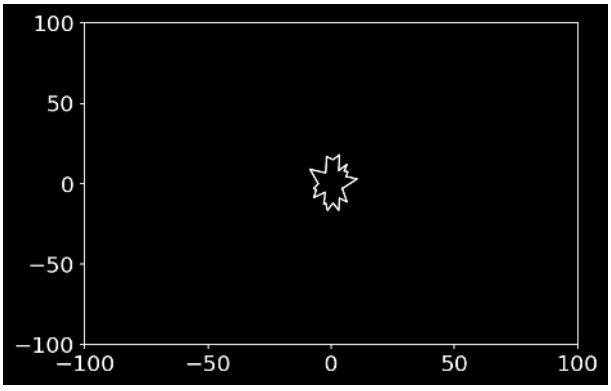
```
SPLAT3: SCAL 1,0
        JSRL SPLAT
SPLAT4: SCAL 1,40
        JSRL SPLAT
SPLAT5: SCAL 2,0
        JSRL SPLAT
SPLAT6: SCAL 2,40
        JMPL SPLAT
```




```
SPLAT5: SCAL 2,0
        JSRL SPLAT
SPLAT6: SCAL 2,40
        JMPL SPLAT
```




```
SPLAT6: SCAL 2,40
        JMPL SPLAT
```



This scaling is set by the preliminary calls to the SCAL macro. This generates a scaling vector instruction that tells the hardware how large or small to draw the final product. This instruction is a byte pair of the form 7000. So for example, SCAL 0,40 will translate into a vector bytcode instruction of7040, while SCAL 2,40 will translate into 7240. The two parameters act as power and linear scales respectively. The power scale acts as a broad brush, reducing the size by a half, or three quarters for example. The linear scale allows for fine tuning the result so that we're not limited to large step changes in size.

```
;  SCAL - SET VECTOR GENERATOR SCALE
;  THE FIRST PARAMETER IS THE POWER OF 2 SCALE.  A VALUE OF 0 MEANS
;  FULL SIZE, 1=1/2 SIZE, 2=1/4, ..., 7=1/128 SIZE.  THE SECOND
;  PARAMETER IS THE LINEAR SCALE (OPTIONAL).  A VALUE OF 0 MEANS
;  FULL SIZE, 80=1/2 SIZE, FF=1/256 SIZE.
  .MACRO SCAL S,LS
  ...1=0                        ; LINEAR SCALE DEFAULTS TO 0.
  .IIF NB,LS,...1=LS            ; UNLESS A VALUE IS PASSED IN 'LS'.
  .WORD ^H7000+...1+<S*^H100> ; BUILD THE VECTOR INSTRUCTION FROM S AND LS.
  .ENDM
```

One thing you might be wondering is how the colors of the explosion are managed. If you've looked closely at our diagrams you will have noticed that the line colors cycle rather than remain static. This rapid color rotation creates a glowing, pulsing effect to the explosion when run at full speed. But from what we've seen in the definition of SPLAT each segment of the splat seems to have its color determined statically by the variables PDIWHI (white), PDIRED (red), and PDIYEL (yellow). So how is the color rotation achieved?

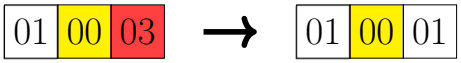
The answer lies in sneakily shifting the underlying values in the color ROM (COLPOR) that each of our color variables references. This color ROM is the table used by the vector instruction to determine the color of the line it is about to draw. When we initialize the player explosion we set up this ROM to contain our red, yellow, and white colors:

```
    ; SET UP THE SPLAT COLORS
    ; COLRAM: A HELPER BUFFER STORING THE CURRENT COLORS.
    ; COLPOR: THE COLOR ROM USED BY VECTOR DRAWING.
ALTCOL:
    LDA I,ZRED          ; GET THE RED COLOR.
    STA COLPOR+PDIRED   ; STORE IN THE 'RED' POSITION IN COLOR ROM.
    STA COLRAM+PDIRED   ; STORE IN THE 'RED' POSITION IN COLOR RAM.
    LDA I,ZYELLOW       ; GET THE YELLOW COLOR.
    STA COLPOR+PDIYEL   ; STORE IN THE 'YELLOW' POSITION IN COLOR ROM.
    STA COLRAM+PDIYEL   ; STORE IN THE 'YELLOW' POSITION IN COLOR RAM.
    LDA I,ZWHITE        ; GET THE WHITE COLOR.
    STA COLRAM+PDIWHI   ; STORE IN THE 'WHITE' POSITION IN COLOR RAM.
    STA COLPOR+PDIWHI   ; STORE IN THE 'WHITE' POSITION IN COLOR ROM.
    RTS
```

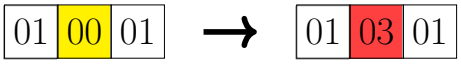
Later, when it comes time to draw the explosion we will occasionally update the color RAM by moving the color values along one position: moving white to the red slot, red to the yellow slot, and yellow to the white slot, for example.

```
    ; ROTATE COLORS FOR PLAYER EXPLOSION
ROTCOL:
  LDY COLRAM+PDIWHI   ; STORE 1ST COLOR IN HELPER BUFFER IN Y.
  LDX I,2             ; WE'RE GOING TO LOOP THROUGH THREE COLORS.
  BEGIN               ; LOOP FROM 2 to 0, USING X AS OUR COUNTER.
  LDA X,COLRAM+PDIWHI ; GET THE COLOR VALUE AT CURRENT POS IN COLOR RAM.
  PHA                 ; STORE IT ON THE STACK.
  STY X,COLRAM+PDIWHI ; STORE THE COLOR IN Y AT CURRENT POS IN COLOR RAM.
  TYA                 ; MOVE THE COLOR IN Y TO A.
  STA X,COLPOR+PDIWHI ; STORE THE COLOR IN THE CURRENT POS IN COLOR ROM.
  PLA                 ; GET THE OLD COLOR AT THE CURRENT POS FROM THE STACK.
  TAY                 ; STORE IN Y SO WE CAN USE IT IN NEXT ITERATION.
  DEX                 ; LOOP UNTIL X IS 0.
  MIEND               ; END LOOP
  RTS
```

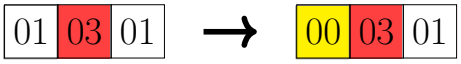
If we imagine we start with the colors in their initial position in Color ROM, after the first iteration in the loop in `ROTCOL`, the color ROM `COLPOR` will look like this. White has been moved to the end:



After the second iteration we have moved red to the middle position in `COLPOR`:



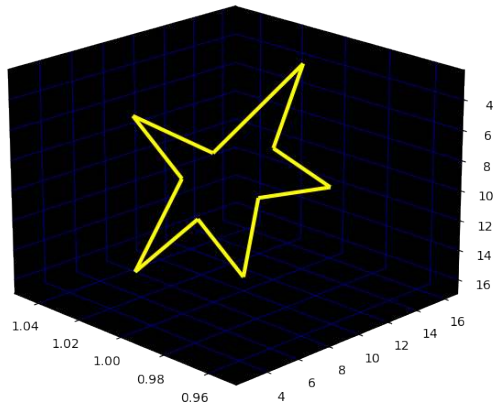
And after the third and final iteration we have moved yellow to the first position in `COLPOR`. The end result being that we have shifted the underlying colors left by one position.



This is what allows the colors to rotate each time we draw the splat.

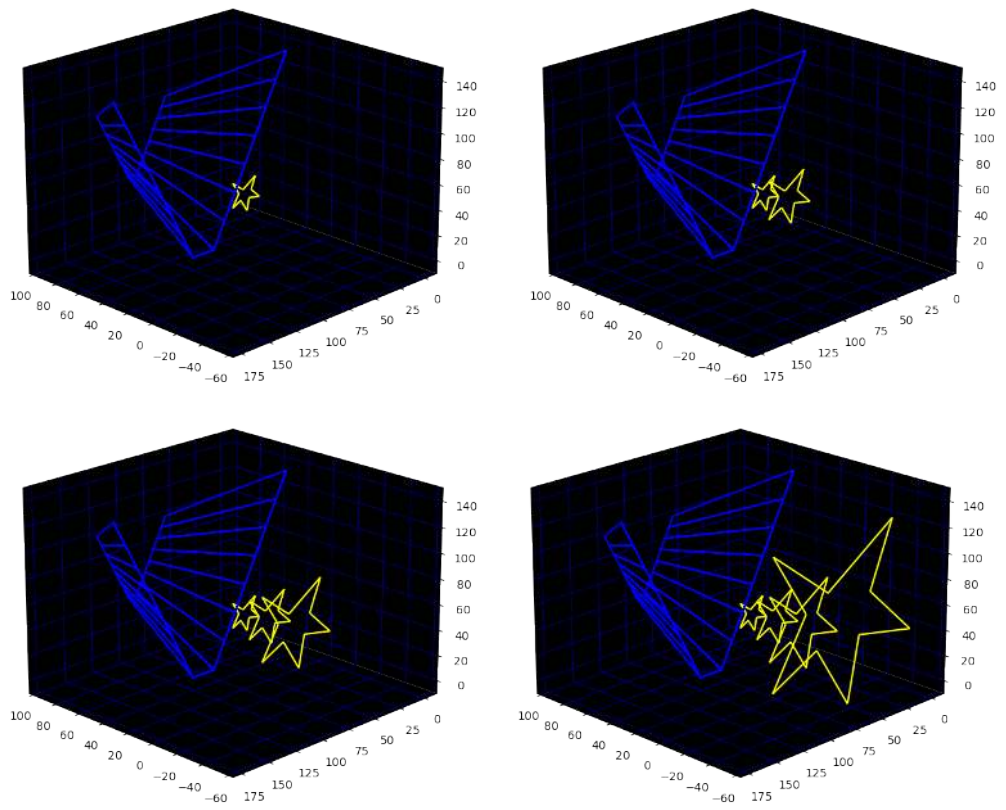
ouch

The player explosion in *Tempest 2000* is vector-based: that is, it is not a solid polygon like the claw or its enemies, but a line-based object using the same machinery as the objects in *Tempest Classic* mode. The data structure that defines the `zap` used to render the explosion is a series of x,y,z co-ordinate triplets followed by an index to the co-ordinate it should be connected to.




```
zap:
   dc.b 3,3,1    ; Co-ordinate 1
   dc.b 2,10,0   ; Connect co-ords 2 & 10
   dc.b 8,7,1    ; Co-ordinate 2
   dc.b 3,0      ; Connect to co-ord 3
   dc.b 14,3,1   ; Co-ordinate 3
   dc.b 4,0      ; Connect to co-ord 4
   dc.b 12,8,1   ; Co-ordinate 4
   dc.b 5,0      ; Connect to co-ord 5
   dc.b 16,12,1  ; Co-ordinate 5
   dc.b 6,0      ; Connect to co-ord 6
   dc.b 11,11,1  ; Co-ordinate 6
   dc.b 7,0      ; Connect to co-ord 7
   dc.b 10,16,1  ; Co-ordinate 7
   dc.b 8,0      ; Connect to co-ord 8
   dc.b 7,11,1   ; Co-ordinate 8
   dc.b 9,0      ; Connect to co-ord 9
   dc.b 3,13,1   ; Co-ordinate 9
   dc.b 10,0     ; Connect to co-ord 10
   dc.b 6,8,1    ; Co-ordinate 10
   dc.b 0,0      ; End of Data
```

The explosion effect will consist of four of these star-shaped `zaps` spinning clockwise in unison and racing towards the player. We can visualize the general character of the explosion as follows:



The first step towards setting this up is to override the *update* and *draw* routines associated with the claw object, replacing them with a routine that will control the progress of the explosion (`zap_player`) and another that will handle the drawing of the four star-shaped zaps (`draw_z`). We also need to clear some of the state related to the behaviour of the claw. And while we're at it we might as well play the explosion sound effect. All of this we cram into `ouch`:

```
; ********************************************************************
; ouch
; ********************************************************************
ouch:
  clr l_soltarg          ; Clear the solid background on the web.
  clr.l 20(a0)           ; Stop the claw moving.
  move.l #zap_player,routine ; Set update routine to 'zap_player'.
  move.l _zap,(a0)       ; Set address of the vector object to draw: _zap.
  clr 36(a0)             ; Set X centre to default.
  bsr clzapa             ; Clear more state.
  clr laser_type         ; Clear laser type.
  clr jenable            ; Disable jumping.
  clr bonum              ; Clear power-ups.
  move.l bshotspeed,shotspeed ; Reset shot speed.
  move #-17,38(a0)       ; Set color of object.
  move #3,44(a0)         ; Configure behaviour on rail.
  move #2,34(a0)         ; Draw routine in draw_vex is 'draw_z'.
  bra zapson             ; Set sound effect and return.
```

Now that we have updated the *update* and `draw` routines associated with the claw's object, the next time the main loop runs `zap_player` will take over the control of what was once the claw. In this routine we find the core of the explosion's behaviour is defined: specifically the pace at which it rolls clockwise and the rate at which it gets nearer to the player. Note that this is also where we point the claw object at the vector structure (`_zap`) which will be used to depict the explosion. Finally it is also in here that we detect when we have had quite enough exploding for one life, and switch off the process so that we can get on with the rest of the game.

```
; ********************************************************************
; zap_player
; ********************************************************************
zap_player:
  bsr rightit            ; Remove any 'roll' from the player's viewpoint.
  move.l _claw,a0        ; Point a0 at the claw object..
  move.l _zap,(a0)       ; Set address of the vector object to draw: _zap.
  move.l 4(a0),vp_xtarg ; Set player's X viewpoint from current X pos.
  move.l 8(a0),vp_ytarg ; Set player's Y viewpoint from current X pos.
  add #12,28(a0)         ; Roll the object by 12 degrees.
  sub #$80,36(a0)        ; Increase the speed of the explosion moving nearer.
  sub.l #$11000,vp_z     ; Move the player viewpoint away from the web.
  sub.l #$4000,12(a0)    ; Move the explosion nearer the player.
  bpl vp_xform           ; If not reached player yet, update player viewpoint
    .
  ; Otherwise the explosion has reached the player viewpoint,
  ; so we can switch it off.
  clr 34(a0)             ; Turn off the object's drawroutine.
  bsr zapson             ; Play sound effect.
  bra setsnatch          ; End the player's life.
  ; Returns
```

So far we've managed to specify the overall movement of the `zap` object in our *update* routine. However we haven't done anything about drawing 4 distinct `zap` objects that get increasingly closer. Our *draw* routine (`draw_z`) is where we take care of this detail. Note that the first `zap` is drawn by the call to `draw` at the start of the routine. We then use a loop enclosed within `dr_z` to iterate another three times, drawing three separate instances of the `zap` object. Each instance is drawn progressively closer to the player thanks to the parameter passed in `36(a0)`.

```
; ********************************************************************
; draw_z
; Draw  with a number of incrementally coloured layers.
; Used for vector objects in the activeobjects list.
; Called during the draw_objects sequence as a member of the draw_vex list.
; ********************************************************************
draw_z:
  bsr draw              ; Draw the original object
  move 40(a6),-(a7)     ; Save the original color of the object.
  move #2,d0            ; Set up our counter for the dr_z loop.
  move 36(a6),d1        ; How much to add to the Z pos at each iteration.
  ext.l d1              ; Extend
  asl.l #8,d1           ; Convert to 16:16
  move 38(a6),d2        ; Delta for colour
  move.l 12(a6),d3      ; Z position

dr_z:
  add d2,40(a6)         ; Add the delta to the colour
  add.l d1,d3           ; Add the z delta to the z position.
  movem.l d0-d3,-(a7)   ; Stash d0 to d3 inclusive on the stack.

  move.l (a6),a1        ; Get object header
  lea in_buf+4,a0       ; Get GPU buffer
  move.l 4(a6),d0       ; Get the X pos
  sub.l vp_x,d0         ; Subtract the viewpoint
  move.l d0,(a0)+       ; Add to GPU buffer
  move.l 8(a6),d0       ; Get the Y pos
  sub.l vp_y,d0         ; Subtract the viewpoint
  move.l d0,(a0)+       ; Add to GPU buffer
  move.l d3,d0          ; Get the z position
  bsr dra               ; Run the GPU routine to draw the object.
  movem.l (a7)+,d0-d3   ; Restore d0 to d3 inclusive from the stack.
  dbra d0,dr_z          ; Loop until we're done for all differences.

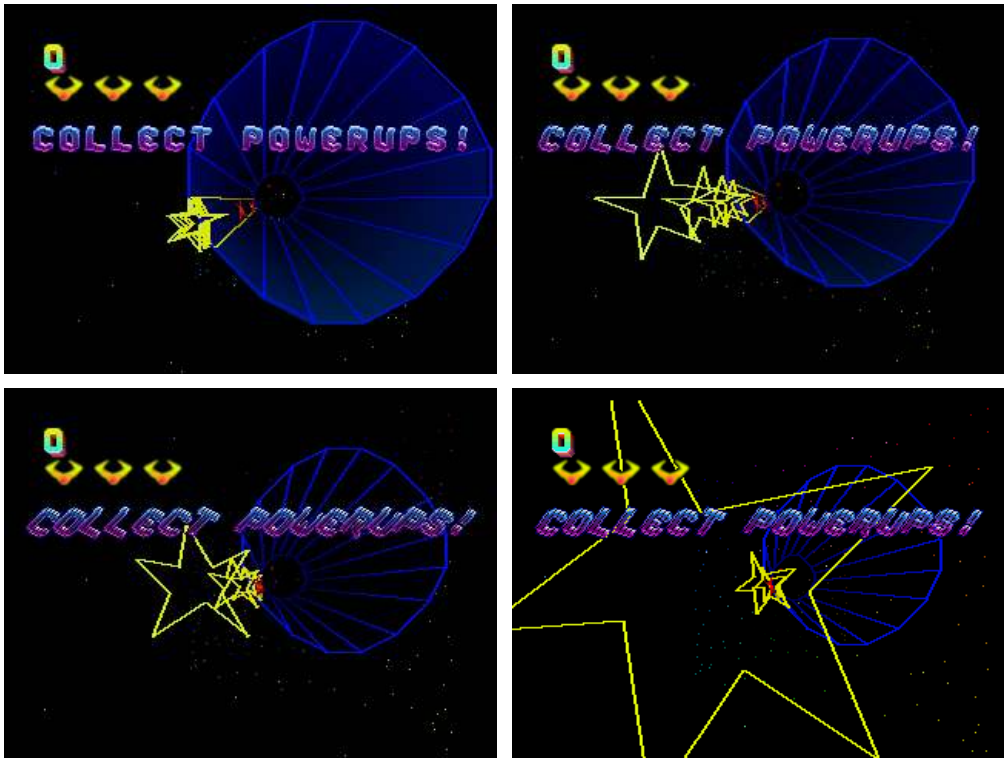
  move (a7)+,40(a6)     ; Get old colour back
  rts
```

A careful reading of the routine above shows that it has an additional capability we don't use here: adjusting the colour of each of the four instances we draw. The factor used to adjust the colour is specified in position `38(a0)` of the object's data structure. Now while `ouch` did write a value of -17 to this slot, the value isn't large enough to have any effect on the object's colour.

```
; ********************************************************************
; vector
; Draw a vector object
; These objects are constructed in advance by make_bits.
; ********************************************************************
vector:
        move.l d0,a1          ; Get object header, e.g. _flipper.
        lea in_buf+4,a0
        move.l 4(a6),d0       ; Get the source X position.
        sub.l vp_x,d0         ; Subtract the player viewpoint X position.
        move.l d0,(a0)+       ; Co-ordinates as X, Y, Z 16:16 frax
        move.l 8(a6),d0       ; Combine with camera viewpoint
        sub.l vp_y,d0         ; Subtract the player viewpoint Y position.
        move.l d0,(a0)+       ; Add it to our GPU RAM input buffer.
        move.l 12(a6),d0      ; Get the object's Z position.
; ********************************************************************
; Draw a vector without the above header information.
; ********************************************************************
dra:    sub.l vp_z,d0         ; Subtract the camera viewpoint.
        move.l d0,(a0)+       ; Add to the GPU buffer.
dragg:  lea fastvector,a2     ; Load the GPU module in 'llama.gas'.
draaa:  move 28(a6),d0        ; Get the XZ orientation
druuu:  and.l #$ff,d0         ; Only first byte.
        move.l d0,24(a1)      ; Copy to XY orientation
        move 30(a6),d0        ; Get Y rotation of object.
        and.l #$ff,d0         ; Only first byte.
        move.l d0,28(a1)      ; Copy to XZ orientation
        move 32(a6),d0        ; Get Z rotation
        and.l #$ff,d0         ; Only first byte.
        move.l d0,32(a1)      ; Copy to YZ orientation

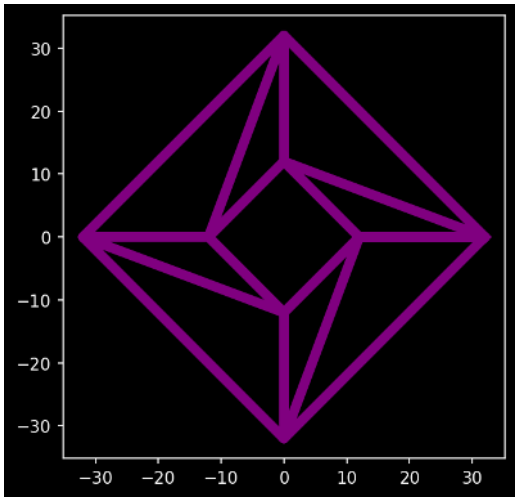
        ; Copy the first 48 bytes of object to GPU buffer.
        move #11,d0           ; Copy 48 bytes (12 * 4).
xhead:  move.l (a1)+,(a0)+    ; Copy 4 bytes to GPU input ram
        dbra d0,xhead         ; Keep looping until 48 bytes copied.

        move 40(a6),d0        ; Get the colour
        and.l #$ff,d0         ; Only first byte
        move.l d0,(a0)+       ; Add to the GPU buffer.
        move 42(a6),d0        ; Get the scale factor.
        ext.l d0              ; Make it a long.
        move.l d0,scaler      ; Move to scaler.
        move.l a1,oopss+4     ; Stash the updated object.
        move.l (a6),oopss+8   ; Stash the original object.
godraa: move.l #2,gpu_mode    ; Op 2 is vect3d in llama.gas.
        move.l a2,a0          ; Load the GPU shader in llama.gas.
        jsr gpurun            ; Run the selected gpu routine: vect3d.
        jsr gpuwait           ; Wait until finished.
        rts
```


COLLECT POWERUPS!
COLLECT POWERUPS!
COLLECT POWERUPS!
COLLECT POWERUPS!

tanker detail

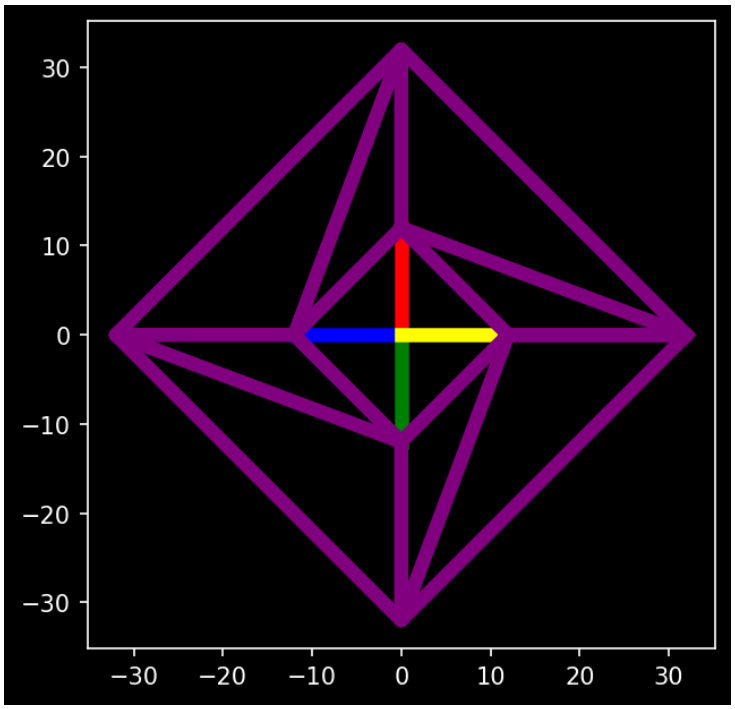
A standard tanker in *Tempest* releases flippers when it is struck.




```
TANKR:
        ICVEC
        SCVEC 20,0,0
GENTNK: CSTAT PURPLE
        SCVEC 0,20,CB
        SCVEC 0,0C,CB
        SCVEC 20,0,CB
        SCVEC 0C,0,CB
        SCVEC 0,0C,CB
        SCVEC -0C,0,CB
        SCVEC 0,20,CB
        SCVEC -20,0,CB
        SCVEC -0C,0,CB
        SCVEC 0,-0C,CB
        SCVEC -20,0,CB
        SCVEC 0,-20,CB
        SCVEC 0,-0C,CB
        SCVEC 0C,0,CB
        SCVEC 0,-20,CB
        SCVEC 20,0,CB
        SCVEC 0C,0,CB
        RTSL
```

A detail that is almost hidden, or at least very easy to miss, is that the tankers containing pulsars and fuses contain a rubric indicating their contents.

The tanker containing fuses contains a very basic representation of the fuse enemy.

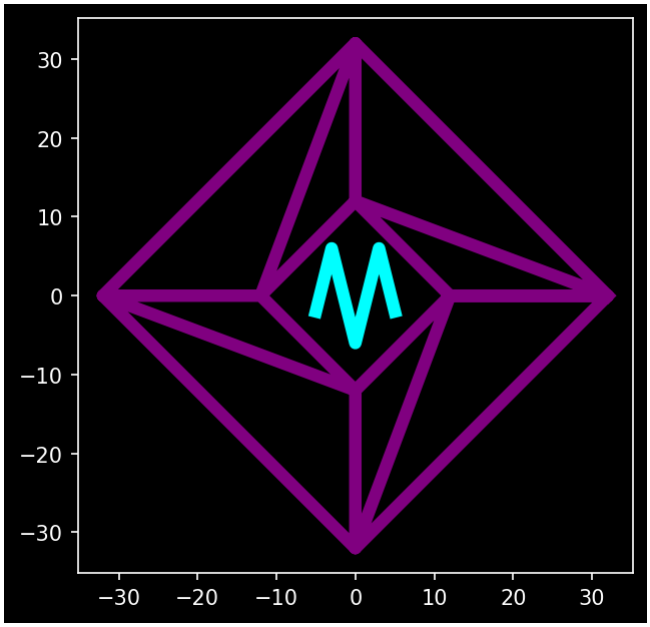



```
TANKF: ICVEC
       CSTAT BLUE
       SCVEC -0C,0,CB
       SCVEC 0,0C,0
       CSTAT RED
       SCVEC 0,0,CB
       CSTAT GREEN
       SCVEC 0,-0C,CB
       SCVEC 0,0,0
       CSTAT YELLOW
       SCVEC 0C,0,CB
       SCVEC 20,0,0
GENTNK:
       CSTAT PURPLE
       SCVEC 0,20,CB
       SCVEC 0,0C,CB
       SCVEC 20,0,CB
       SCVEC 0C,0,CB
       SCVEC 0,0C,CB
       SCVEC -0C,0,CB
       SCVEC 0,20,CB
       SCVEC -20,0,CB
       SCVEC -0C,0,CB
       SCVEC 0,-0C,CB
       SCVEC -20,0,CB
       SCVEC 0,-20,CB
       SCVEC 0,-0C,CB
       SCVEC 0C,0,CB
       SCVEC 0,-20,CB
       SCVEC 20,0,CB
       SCVEC 0C,0,CB
       RTSL
```

The tanker for *pulses*, given opposite, contains a more successful effort. It is much more immediately legible during play, compared to the cross on a *fuse* tanker which gets slightly lost in the detail.

Note the relative economy of coding used to draw each tanker. The rubrics are drawn by the first few lines, i.e. `TANKF` and `TANKP`. In each case the remaining section (`GENTNK`) is common to all tanker types. I've given `GENTNK` in full in each of the diagrams for the purposes of clarity - but in practice the duplication of code was avoided by replacing each with a `JMP` call to the `GENTNK` routine, which only had to be written once. For example, in the case of the pulsar tank:

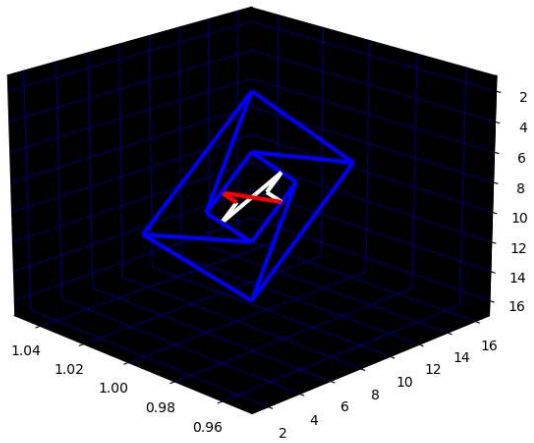
```
TANKP:ICVEC
      CSTAT TURQOI
      SCVEC -5,-2,0
      SCVEC -3,6,CB
      SCVEC 0,-6,CB
      SCVEC 3,6,CB
      SCVEC 5,-2,CB
      SCVEC 20,0,0
      JMP GENTNK
```




```
TANKP:ICVEC
      CSTAT TURQOI
      SCVEC -5,-2,0
      SCVEC -3,6,CB
      SCVEC 0,-6,CB
      SCVEC 3,6,CB
      SCVEC 5,-2,CB
      SCVEC 20,0,0
GENTNK:
      CSTAT PURPLE
      SCVEC 0,20,CB
      SCVEC 0,0C,CB
      SCVEC 20,0,CB
      SCVEC 0C,0,CB
      SCVEC 0,0C,CB
      SCVEC -0C,0,CB
      SCVEC 0,20,CB
      SCVEC -20,0,CB
      SCVEC -0C,0,CB
      SCVEC 0,-0C,CB
      SCVEC -20,0,CB
      SCVEC 0,-20,CB
      SCVEC 0,-0C,CB
      SCVEC 0C,0,CB
      SCVEC 0,-20,CB
      SCVEC 20,0,CB
      SCVEC 0C,0,CB
      RTSL
```

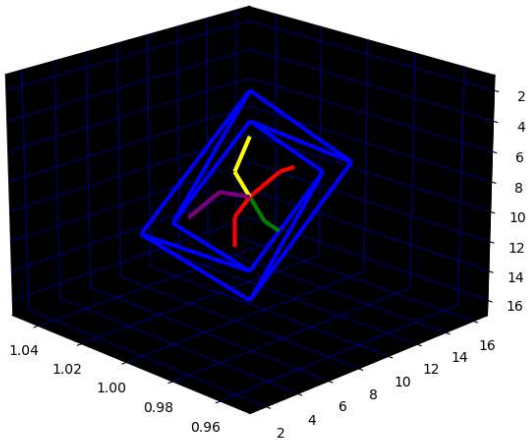
even more tanker detail

Tempest 2000 manages a little extra with the tankers in both *classic* and *2000* mode. The flipper tanker gets a badge of its own, in contrast to its anonymous appearance in the original.



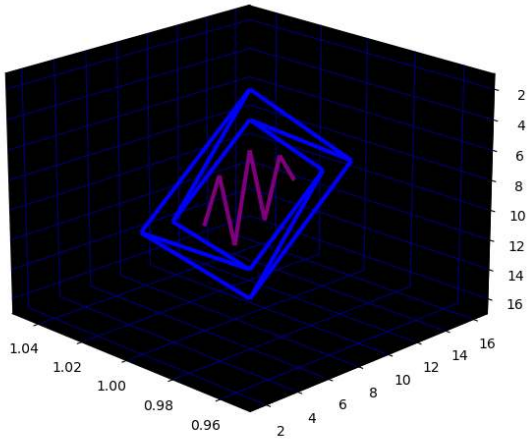

```
fliptank:
  dc.b 2,9,1,2,7,4,0
  dc.b 9,16,1,8,3,0
  dc.b 16,9,1,4,5,0
  dc.b 9,2,1,6,0
  dc.b 9,6,1,6,8,0
  dc.b 6,9,1,7,0
  dc.b 9,12,1,8,-1,240,0
  dc.b 12,9,1,0
  dc.b 7,8,1,10,14,-1,0,0
  dc.b 11,10,1,11,0
  dc.b 10,9,1,12,0
  dc.b 11,8,1,13,0
  dc.b 7,10,1,14,0
  dc.b 8,9,1,0,0
```

The fuse tanker meanwhile has a more spirited rendering of the fuse enemy as its rubric.




```
fusetank:
  dc.b 2,9,1,2,7,4,0
  dc.b 9,16,1,8,3,0
  dc.b 16,9,1,4,5,0
  dc.b 9,2,1,6,0
  dc.b 9,4,1,6,8,0
  dc.b 4,9,1,7,0
  dc.b 9,14,1,8,-1,240,0
  dc.b 14,9,1,0
  dc.b 9,9,1,10,0
  dc.b 8,10,1,11,0
  dc.b 8,12,1,0
  dc.b 10,11,1,-1,143,9,13,0
  dc.b 11,12,1,0
  dc.b 11,8,1,-1,240,9,15,0
  dc.b 12,8,1,0
  dc.b 8,7,1,-1,15,9,17,0
  dc.b 9,5,1,0
  dc.b 7,8,1,-1,255,9,19,0
  dc.b 5,9,1,0,0
```

The pulse tanker on the other hand is a relatively straightforward copy of that found in the original.




```
pulstank:
  dc.b 2,9,1,2,7,4,0
  dc.b 9,16,1,8,3,0
  dc.b 16,9,1,4,5,0
  dc.b 9,2,1,6,0
  dc.b 9,4,1,6,8,0
  dc.b 4,9,1,7,0
  dc.b 9,14,1,8,-1,255,0
  dc.b 14,9,1,0
  dc.b 6,10,1,10,0
  dc.b 7,7,1,11,0
  dc.b 8,12,1,12,0
  dc.b 9,6,1,13,0
  dc.b 10,11,1,14,0
  dc.b 11,7,1,15,0
  dc.b 12,9,1,0,0
```

bullet

The player's bullet in Tempest 2000 is something of an outlier compared to all the other graphics in the game. For some reason, it is not a polygon constructed from a data structure like the flipper or the player's claw. Instead it is pinched directly from a spritesheet, 'beasty3.cry'. Take a look at the sheet and see if you can spot the bullet:



Figure .1: `beasty3.cry`, with a bullet in there somewhere - can you find it?

```
; ****************************************************************
; draw_pixex
; Draws a pixel based explosion.
; A member of the draw_vex and solids list.
; Called during the draw_objects sequence as a member of the draw_vex list.
; Called during the draw_objects sequence as a member of the solids list.
; ****************************************************************
draw_pixex:
    move.l #4,gpu_mode    ; Use the 'rex' routine in camel.gas.
    lea in_buf,a0         ; Point our GPU RAM input buffer at a0.
    move.l #pic,(a0)+     ; Use 'pic' (beasty3.cry) as our sprite sheet.
    move.l 36(a6),(a0)+   ; Y/X position within the 'pic' sprite sheet.
    move.l 46(a6),(a0)+   ; Width and height to pluck from the sheet.
    move.l 42(a6),d0      ; Put the scale in d0.
    asr.l #2,d0           ; Divide by 4.
    move.l d0,(a0)+       ; Add the x-scale to our GPU RAM input buffer.
    move.l d0,(a0)+       ; Add the y-scale to our GPU RAM input buffer.
    move.l #0,(a0)+       ; Add no shear to the input buffer.
    move.l #0,(a0)+
    move.l #1,(a0)+       ; Add Mode 1 = Centered to input buffer.
    move.l 4(a6),d0       ; Get the source X position.
    sub.l vp_x,d0         ; Subtract the player/camera viewpoint X position.
    move.l d0,(a0)+       ; Add it to our GPU RAM input buffer.
    move.l 8(a6),d0       ; Get the object's Y position.
    sub.l vp_y,d0         ; Subtract the player/camera viewpoint Y position.
    move.l d0,(a0)+       ; Add it to our GPU RAM input buffer.
    move.l 12(a6),d0      ; Get the object's Z position.
    sub.l vp_z,d0         ; Subtract the player/camera viewpoint Z position.
    move.l d0,(a0)+       ; Add it to our GPU input buffer.
    lea parrot,a0         ; Load the GPU module in camel.gas.
    jsr gpurun            ; do clear screen
    jmp gpuwait           ; Wait for the GPU to finish.
```

claw says



Above, the routine `draw_pixex` is responsible for drawing the current state of a bullet. Despite it's name, it's not always drawing an 'explosion'. Instead it is drawing a segment selected from the sprite sheet 'pic' (beasty3.cry). The routine specifies the co-ordinates and dimensions of the segment to pluck from 'pic' and also details the expected scaling and shearing to apply. As you adjust this scale and shear over time you can achieve all sorts of interesting effects.

Look closely below, in the bottom left hand of the sheet and you can see it in a red box:

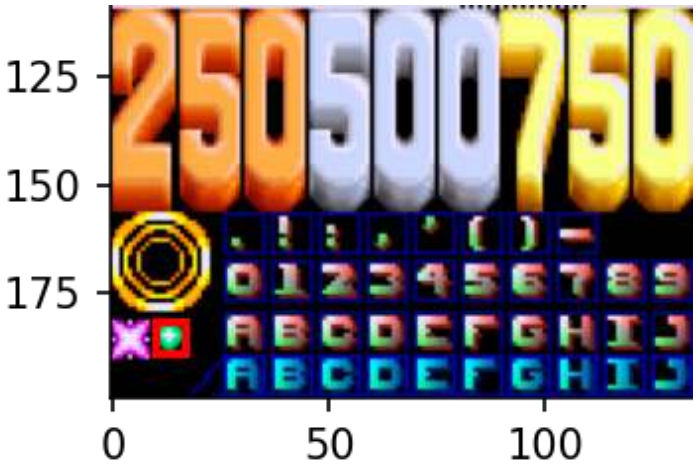


Figure .2: The 'bullet' glyph located in the lower left in a red box.

The delightfully named routine '`frab`' is responsible for `firing a` bullet. In here we can see the co-ordinates for plucking our bullet from the spritesheet being set. We define the X and Y position within the sheet and the width and height too.

```
frab:
    ; The bullet is taken from the sprite sheet in 'beasty3.cry'.
    ; This is an exception to the rule for nearly all other objects,
    ;  which are usually constructed polygons.
    ...
    move.l #$00b6000a,d0 ; Y (00b6) and X (000a) in sprite sheet 'pic'.
    move.l #$00070007,d1 ; Width (07) and height (07) for the bullet.
    ...
    move.l d0,36(a0)     ; Set the X/Y position in the sprite sheet.
    move.l d1,46(a0)     ; Set the width/height in the sprite sheet.
```

Notice how the Y and X values are encoded in a single 4-byte value:

```
    move.l #$00b6000a,d0 ; Y (00b6) and X (000a) in sprite sheet 'pic'.
```



Figure .3: Y of 182 (b6) and X of 10 (0a) encoded in a 4 byte long.

Likewise the width and height:

```
    move.l #$00070007,d1 ; Width (07) and height (07) for the bullet.
```

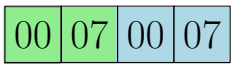


Figure .4: Width of 7 (07) and height of 07 (07) encoded in a 4 byte long.

With this setup complete, the routine `draw_pixex` on the opposite page will write these values to the buffer used by the GPU routine 'rex'. With everything we have configured the result will be to simply draw our selected segment from 'beasty3.cry' (i.e. our bullet) to the desired position on the screen.

I cheated earlier by shortening the `frab` listing above. It actually contains an additional detail where we can select a different type of bullet:

```
frab:
    ; The bullet is taken from the sprite sheet in 'beasty3.cry'.
    ; This is an exception to the rule for nearly all other objects,
    ;  which are usually constructed polygons.
    ...
    move.l #$00b6000a,d0 ; Y (00b6) and X (000a) in sprite sheet 'pic'.
    move.l #$00070007,d1 ; Width (07) and height (07) for the bullet.
    tst 32(a0)           ; What type of bullet are we using?
    beq konk             ; If default, skip to konk.
    ; We're using the other bullet!
    move.l #$b60000,d0   ; Y (00b6) and X (0000) in sprite sheet 'pic'.
    move.l #$090009,d1   ; Width (09) and height (09) for the bullet.
konk:
    move.l d0,36(a0)     ; Set the X/Y position in the sprite sheet.
    move.l d1,46(a0)     ; Set the width/height in the sprite sheet.
```

For this alternate bullet, we select the sprite to the left of our previous bullet. Notice how the co-ordinates and width/height are subtly altered for this version of the bullet.

```
    move.l #$b60000,d0   ; Y (00b6) and X (0000) in sprite sheet 'pic'.
```

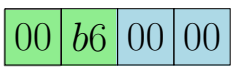


Figure .5: Y of 182 (b6) and X of 00 (00) encoded in a 4 byte long.

```
    move.l #$090009,d1   ; Width (09) and height (09) for the bullet.
```



Figure .6: Width of 9 (09) and height of 09 (09) encoded in a 4 byte long.

The result is that we instead select the bullet sprite to the left of our original bullet:

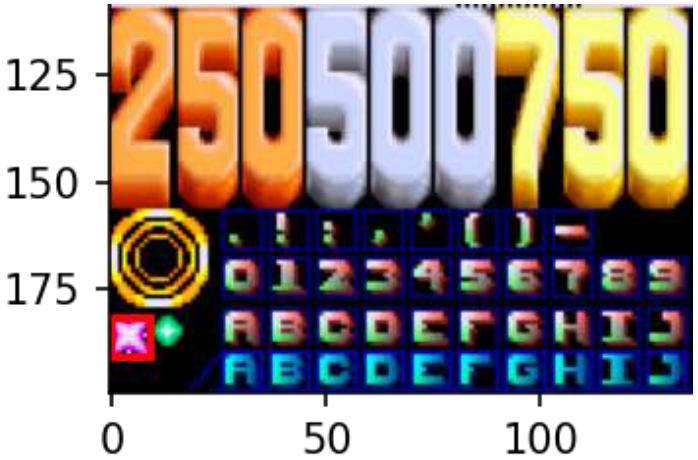
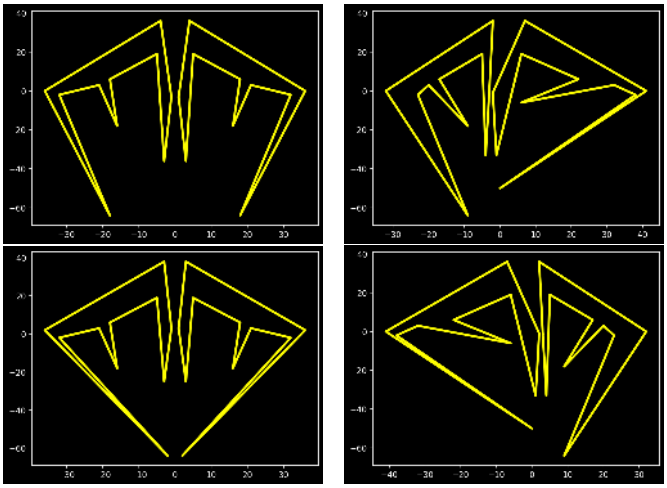


Figure .7: The alternate 'bullet' glyph located in the lower left in a red box.

things hidden

These appear to be enemy attack ships of a previously unknown configuration defined in the source file `ALVROM.MAC`. An assembly flag excluded these objects from the final release of the game but their full specification is available to us in a series of vector commands. This has enabled the us to painstakingly reconstruct the artefacts using modern equipment. We present them here to the reading public for the first time.



ENER21 to ENER24 in ALVROM.MAC.

The best of our finds is a clear predecessor to the iconic 'claw' ship. Each is defined using an array of X/Y co-ordinates that a macro by the name of `CALVEC` encodes into

a list of vector commands. For example, the first image in Figure 2 above is given as follows in lines 1483-1512 in ALVROM.MAC: [1]

```
ENER21:
  ICALVE               ; X:0   Y:0
  CALVEC -1,-3         ; X:-1  Y:-3
  .BRITE=VARBRT        ; Set brightness to 1
  CALVEC -4,24.        ; X:-4  Y:36
  CALVEC -24.,0        ; X:-36 Y:0
  CALVEC -12.,-40.     ; X:-18 Y:-64
  CALVEC -20.,-2       ; X:-32 Y:-2
  CALVEC -15.,3        ; X:-21 Y:3
  CALVEC -10.,-12.     ; X:-16 Y:-18
  CALVEC -12.,6        ; X:-18 Y:6
  CALVEC -5,13.        ; X:-5  Y:19
  CALVEC -3,-24.       ; X:-3  Y:-36
  CALVEC -1,-3         ; X:-1  Y:-3
  .BRITE=0             ; Set brightness to 0
  CALVEC 1,-3          ; X:1   Y:-3
  .BRITE=VARBRT        ; Set brightness to 1
  CALVEC 3,-24.        ; X:3   Y:-36
  CALVEC 5,13.         ; X:5   Y:19
  CALVEC 12.,6         ; X:18  Y:6
  CALVEC 10.,-12.      ; X:16  Y:-18
  CALVEC 15.,3         ; X:21  Y:3
  CALVEC 20.,-2        ; X:32  Y:-2
  CALVEC 12.,-40.      ; X:18  Y:-64
  CALVEC 24.,0         ; X:36  Y:0
  CALVEC 4,24.         ; X:4   Y:36
  CALVEC 1,-3          ; X:1   Y:-3
  .BRITE=0             ; Set brightness to 0
  CALVEC NXE,0         ; X: 0   Y:0
  RTSL
```

The listing gives X and Y co-ordinates in hex, which we can readily plot as vertices on a graph. During assembly these values were converted to 'relative draw'[2] vector commands for use by the Atari Analogue Vector Generator (AVG). These encode X and Y vectors, along with an intensity value I as follows:

			Vector Command Bits							
X	Y	I	000Y	YYYY	YYYY	YYYY	IIIX	XXXX	XXXX	XXXX
FF	FD	00	0001	1111	1111	1100	0001	1111	1111	1111

The values chosen above are not arbitrary: X is -1 (FF) and Y is -3 (FD). Together with an assumed intensity value of 0 these form the first entry in ENER21: CALVEC -1,-3,

[1] https://github.com/historicalsource/tempest/blob/ 6c783bee488ed736fc3fdc3a81fdc412c3bec386/ALVROM.MAC#L1483

[2] https://arcarc.xmission.com/Tech/neilw_xy.txt

which gets encoded in one's complement [3] for the thirteen bits of each value: 1FFD 1FFF.

Let's take that again, step by step. Our command is:

```
   CALVEC -1,-3         ; X:-1  Y:-3
```

This calls the macro CALVEC with the values -1 (FE) as X and -3 (FD) as Y. The end result we expect from this operation is a vector command that will draw a line from the current point on the screen to the new co-ordinates. To do this CALVEC manipulates the X and Y values we pass relative to the previous position of X and Y, and offloads the rest of the work to another macro, VCTR:

```
        .MACRO CALVEC NEWX,NEWZ
.XN     =NEWX-OLDX      ; Subtract the previous value of X from the current.
.ZN     =NEWZ-OLDZ      ; Subtract the previous value of Y from the current.
VCTR    .XN,.ZN,.BRITE  ; Call the VCTR macro with our new X and Y.
OLDX    =NEWX           ; Store the current X as our previous value.
OLDZ    =NEWZ           ; Store the current Y as our previous value.
        .ENDM
```

It is VCTR that will transform our humble parameters -1 and 3 into a fully formed 4 byte vector command. This will be 1FFD1FFF, let's see how that translation happens:

```
;       VCTR - DRAW VECTOR INSTRUCTION
;       THIS INSTRUCTION DRAWS A VECTOR ON THE DISPLAY AREA
;       RELATIVE TO THE PREVIOUS BEAM POSITION BEFORE THE
;       INSTRUCTION IS EXECUTED.  DX IS THE CHANGE IN BEAM X
;       POSITION; DY IS THE CHANGE IN BEAM Y POSITION; ZZ
;       SPECIFIES THE BEAM INTENSITY (0 THROUGH 7., 0 IS
;       NO INTENSITY, 7. IS BRIGHTEST INTENSITY).
;
        .SBTTL  VCTR
        .MACRO  VCTR DX,DY,ZZ
        ...1=DX             ; STORE DX AS '1'
        ...2=DY             ; STORE DY AS '2'
        .IF      LT,...1    ; IF NEGATIVE X
        ...1=-DX            ; MAKE IT POSITIVE
        .ENDC               ; END IF
        .IF      LT,...2    ; IF NEGATIVE Y
        ...2=-DY            ; MAKE IT POSITIVE
        .ENDC               ; END IF
        .WORD   DY&^H1FFF,<ZZ*^H2000>+<DX&^H1FFF> ; Turn into a vector
    command
        .ENDM
```

The crux is the penultimate line:

[3] https://en.wikipedia.org/wiki/Ones%27_complement

```
        .WORD   DY&^H1FFF,<ZZ*^H2000>+<DX&^H1FFF> ; Turn into a vector
    command
```

The macro assembler computes this into a single line of assembly:

```
        .WORD  1FFD,1FFF
```

Let's see the steps it follows to do this. First we convert each of -1 (FF) and -3 (FD) into 'positive' values:

Command	Left Operand	Operation	Result
...1=-DX	FF	-	00
...1=-DY	FD	-	02

Now we can run the bit operations that will composite everything into a sequence of 4 bytes:

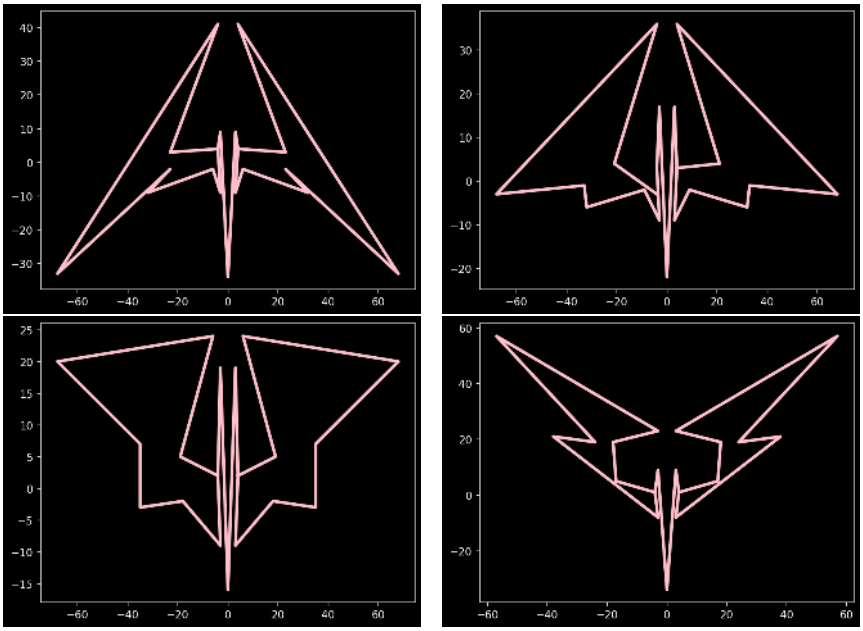
Command	Left Operand	Operation	Right Operand	Result	Cumulative Result
DY&^H1FFF	02	XOR	1FFF	1FFD	1FFD0000
ZZ*^H2000	00	AND	2000	0000	1FFD0000
DX&^H1FFF	00	XOR	1FFF	0000	1FFD1FFF

The successive operations give us our desired result, a vector command that draws a beam line from the current position to a new position along the vector -1,-3:

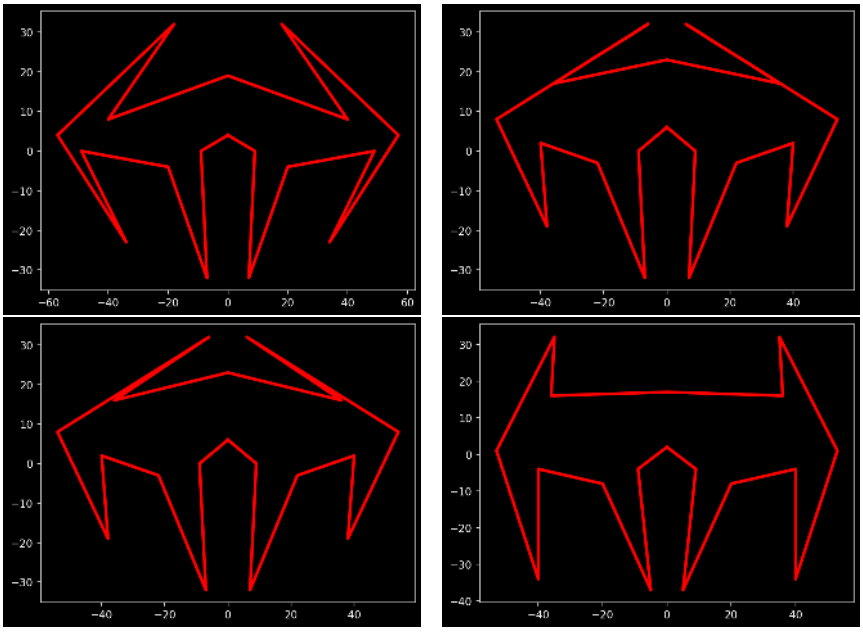
Y Value				Intensity/X Value			
1	F	F	D	1	F	F	F
000Y	YYYY	YYYY	YYYY	IIIX	XXXX	XXXX	XXXX
0001	1111	1111	1100	0001	1111	1111	1111

There are twelve other finds of interest given below. Unlike the set in Figure 2 above, none of these resemble early iterations of the player's 'claw'. All our finds appear in an area of the source code described as ENEMY PICTURES and are more likely to be just that: a set of alien enemies for a very early iteration of Tempest that according to programmer David Theurer was a 'First Person Space Invaders'. [4]

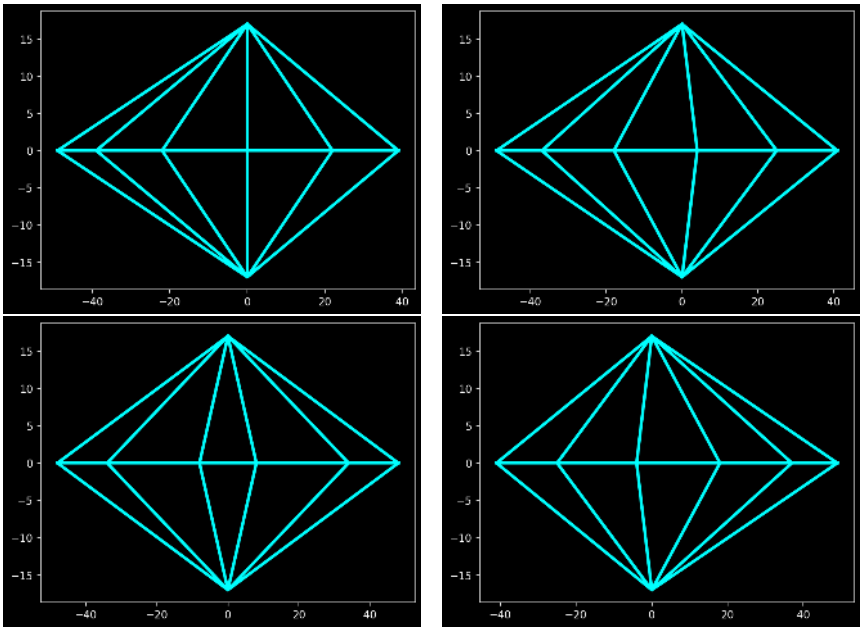
[4] https://arcadeblogger.com/2018/01/19/atari-tempest-dave-theurers-masterpiece/



ENER11 to ENER14 in ALVROM.MAC.



ENER41 to ENER44 in ALVROM.MAC.



SAU to SA4 in ALVROM.MAC.

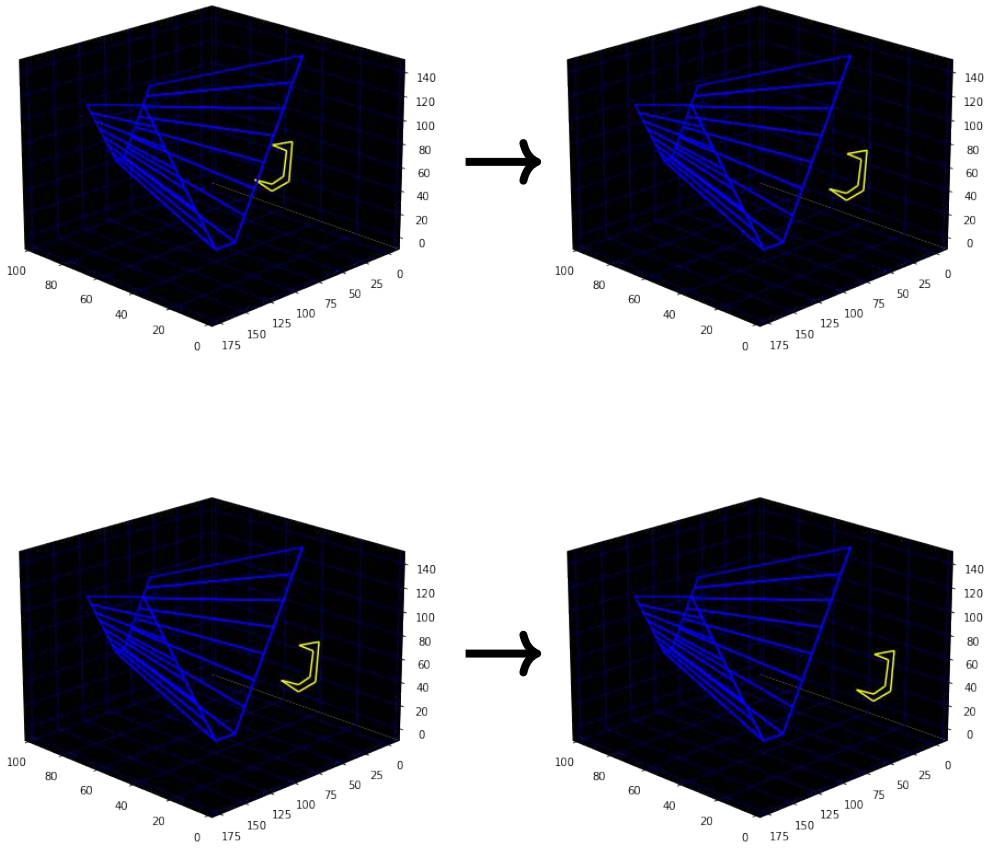
jump

Jumping off the web was a new game feature introduced by Tempest 2000. It allows the player to pop off the web and avoid colliding with enemies. This behaviour is managed by the routine responsible for updating the player's claw, `claw_con1`. We encountered this routine briefly in 'mainloop'. Here is the part concerned with detecting if the player has pressed the 'Fire 2' button, which intiates a jump.

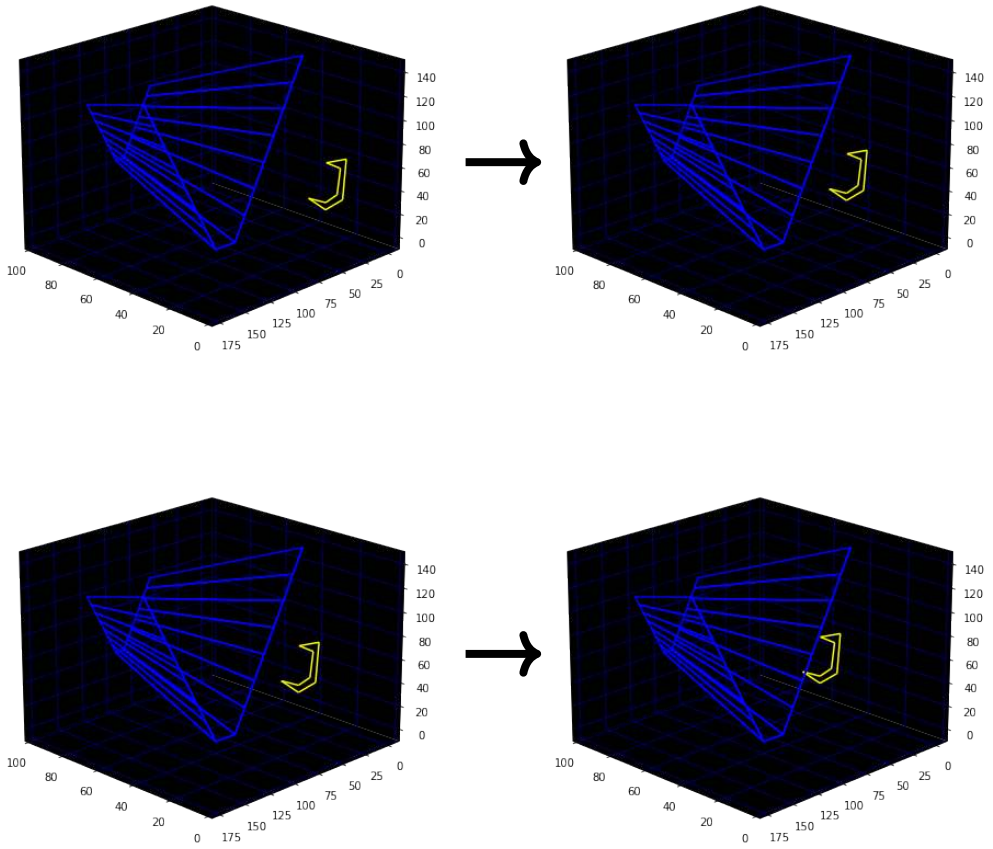
```
chek4jump:
        cmp #-2,wave_tim          ; Is the enemy wave over?
        beq cce                   ; If so, don't jump.
        tst jenable               ; Is jumping allowed?
        beq cce                   ; If not, don't jump.
        move.l fire_2,d0          ; Get the 'fire 2' button.
        and.l pad_now,d0          ; Check if it's been pressed.
        beq cce                   ; If not, skip to cce, otherwise..
        move.l #-$20000,cjump     ; .. start a jump
        move #6,sfx               ; Select the 'Player Jump' sound effect.
        jsr fox                   ; Make a v. silly noise
cce:    move.l pad_now,d0         ; Get currently pressed buttons.
        bra clawcon               ; Do the rest of the claw update routine.
```

From looking at the above, maybe you can divine that processing the jump is going to be managed by a single variable called `cjump`. All we do here is initialize it to the slightly eye-watering value of `-$20000`, which is -131072 in decimal. As we shall see, there is a trick to this number that allows the jump to be implemented nice and compactly.

Before we see how the magic was achieved, let's briefly consider what is actually required to achieve a jump-like effect. We visualize this in the simplified sequence of diagrams below: the claw moves gradually off the top of the web along its Z axis. That's it.



So to achieve a jump, we just have to adjust the Z position of the claw every time we are updating its position in `claw_con1`. There is one added complication of course, the claw has to come back down to earth again.



This is where the slightly magic value of `-$20000` starts to come in handy. It has the property of being negative. In assembler arithmetic, negative values have a very useful property if you keep adding positive values to them. At first they keep getting smaller, but when they reach zero any future additions will mean they start getting bigger again. In other words, they cycle around like a clock. So in our case if we keep adding some value to `-$20000` it will eventually reach `$00000` and then start climbing to `$20000` again.

This means that all we have to do to implement our jump is to choose a value to add to `cjump` and set that as the claw's Z position every time we're updating the claw object in `claw_con1`. At first the claw will get nearer to us, but once `cjump` becomes positive again it will start receding from us once more. In this case, the value we choose to add

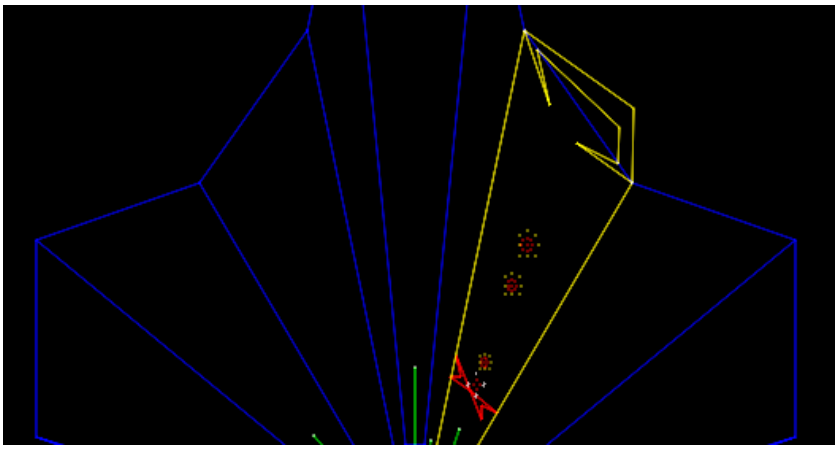
each time is $c11 (3089 in decimal). This results in approximately 42 steps on the way up and 42 on the way down.

```
        ; Process a claw jump if necessary.
cc2:    tst t2k                     ; Are we playing T2K?
        beq cce                     ; If not, skip to clawcon.
        ; Process a claw jump.
        move.l cjump,d0             ; Store the jump var in d0.
        beq chek4jump               ; Skip if not currently jumping..
        add.l d0,12(a6)             ; Otherwise set the claw's new Z position
    .
        move.l 12(a6),d1            ; Store the claw's Z position in d1.
        move #webz-80,d2            ; Store the web's Z position in d2.
        swap d2                     ; Swap first 2 bytes with last 2 bytes.
        clr d2                      ; Clear the last 2 bytes of d2.
        sub.l d2,d1                 ; Subtract the web's Z from the claw's Z.
        add.l vp_zbase,d1           ; Add the base viewpoint to the claw's Z.
        move.l d1,vp_z              ; Store as the new player's Z viewpoint.
        move.l d1,vp_ztarg          ; Store as the new player's Z viewpoint.
        add.l #$c11,cjump           ; Add 3089 to cjump.
```

Of course we also have to test for when the jump is over. We've 'hit the ground' when the claw finally reaches the same Z position as the web again, or event slightly past it. When this happens we clamp the claw to the web and reset everything so that the claw and the player's viewpoint are back to normal again. That was easy.

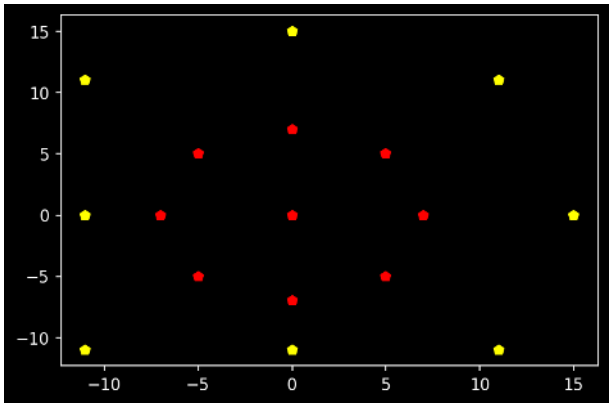
```
        tst.l d0                    ; Test prev val of cjump.
        bmi cce                     ; If less than 0, jump is still active.
        cmp #webz-80,12(a6)         ; If web Z less than claw Z..
        blt cce                     ; .. then jump is still active.
        move #webz-80,12(a6)        ; Otherwise jump over, attach claw to web
    .
        clr 14(a6)                  ; Clear the lower 2 bytes of claw's Z.
        clr.l cjump                 ; Clear the jump variable.
        and.l #$fffc0000,vp_z       ; Restore the player's viewpoint.
        and.l #$fffc0000,vp_ztarg   ; Restore the player's viewpoint.
```

player charges



Cursor Firing Charges, or as we like to call them: Bullets.

For some reason, the player's bullets in *TEMPEST*, are referred to as 'charges'. When fired they run along the rail the player occupies, into the well in the hope of striking an unwitting alien. Let's take a look at how we get these charges on to the screen.



The bullet as drawn by DIARA2 opposite.

```
DIARA2: ICVEC
        CSTAT PSHCTR
        SCDOT 0,0
        SCDOT 7,0
        SCDOT 5,5
        SCDOT 0,7
        SCDOT -5,5
        SCDOT -7,0
        SCDOT -5,-5
        SCDOT 0,-7
        SCDOT 5,-5
        CSTAT YELLOW
        SCDOT 0F,0
        SCDOT 0B,0B
        SCDOT 0,0F
        SCDOT -0B,0B
        SCDOT -0B,0
        SCDOT -0B,-0B
        SCDOT 0,-0B
        SCDOT 0B,-0B
        RTSL
```

We draw the charges at a given point in time by keeping a list of their depth in the well in CHARY (short for CHARGES Y POSITION). This means that drawing them is as simple as looping through this list and drawing any entry that has a non-zero depth value.

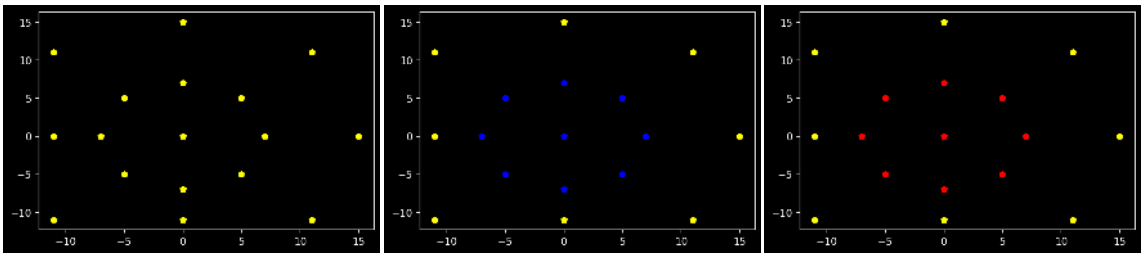
```
        .SBTTL  DISPLAY-CHARGES
        ; THIS ROUTINE WILL LOOP THROUGH ALL THE CHARGES
        ; AND DRAW ANY OF THEM THAT ARE ACTIVE.
DSPCHG:
        LDX I,NCHARG-1  ; GET THE NUMBER OF CHARGES (IT'S 8).
        STX INDEX1      ; STORE IT AS OUR INDEX.

        ; LOOP THROUGH ALL 8 CHARGES (BULLETS) AND DRAW
        ; ANY THAT ARE ACTIVE.
        BEGIN           ; START OF LOOP
        LDX INDEX1      ; GET CURRENT INDEX AND USE IT TO..
        LDA X,CHARY     ; ..GET THE VALUE IN THE CURRENT SLOT.
        IFNE            ; IS IT ACTIVE?
        STA PYL         ; IF YES, SCAPIC EXPECTS THE DEPTH IN PYL.
        LDY X,CHARL1    ; GET THE LINE THE BULLET IS ON.
        LDA I,PTCURS    ; GET THE PICTURE IN DIARA2.
        JSR SCAPIC      ; DRAW THE BULLET USING THE VALUES ABOVE.
        ENDIF
        DEC INDEX1      ; DECREMENT OUR INDEX..
        MIEND           ; AND LOOP AGAIN IF STILL NOT ZERO.

        ; SET THE COLOR OF THE CENTRE OF THE BULLET TO INDICATE
        ; HOW MANY SHOTS ARE AVAILABLE TO THE PLAYER.
        LDY I,ZYELLO    ; BY DEFAULT, YELLOW INDICATES PLENTY.
        LDA CHACOU      ; GET THE NO. OF BULLETS CURRENTLY IN PLAY.
        CMP I,NPCHARG-2 ; IF BETWEEN 6 AND 8..
        IFCS            ; THEN IT'S 'LOW' SO..
        LDY I,ZBLUE     ; SET COLOR TO BLUE ('LOW').
```

```
        CMP I,NPCHARG   ; IF IT'S 8...
        IFCS            ; THEN THERE ARE NO MORE LEFT..
        LDY I,ZRED      ; SET COLOR TO RED ('OUT').
        ENDIF
        ENDIF
        ; SET THE COLOR WE CHOSE FOR CENTER OF BULLET.
        STY COLPOR+PSHCTR     ; NOTICE PSHCTR IS REF'D BY DIARA2 ABOVE.
        RTS
```

There is a little-noticed subtlety in how we draw the bullets in the code above. Since there are eight bullets available to the player at any one time, we colour-code the bullets according to how many are currently in play. If all eight bullets are currently in play, we will colour them red. If there are between 5 and 7, we colour them blue. Otherwise we colour them all yellow. Personally I'd never realized this was happening until I read the code!



(a) YELLOW: PLENTY (b) BLUE: LOW (c) RED: OUT

Figure .1: Different colours of the bullets depending on how many shots are left.

The actual drawing to the screen of each bullet is worth pausing over. First of all, what do we want to do? We want to draw each bullet a little further along the line it was originally fired on. This means that we have to pick a position that is a combination of horizontal (X), vertical (Z), and depth (Y) points and draw the bullet centred at that point.

You may have noticed that we were using the list of 'depth' points (CHARY) to decide whether or not to draw a bullet in the first place, so we already have the depth value we want to draw at stored in PYL. This leaves the vertical and horizontal positions. To retrieve this for a particular bullet we used a list called CHARL1 that stores the line that each bullet belongs to. Armed with this 'line number' in the Y register the first part of our drawing routine (SCAPIC), we can use it to get the appropriate horizontal and vertical position for that line from a pair of special tables. Now that we have all three points we can draw the bullet in its position.

```
;FUNCTION: DISPLAY A PICTURE CENTERED BETWEEN 2 POINTS AND SCALED
;          DOWN ACCORDING TO ITS DEPTH
;INPUT: X = INDEX INTO LINEX,Z OF 1ST PT'S X & Z WC WORDS
;       Y = INDEX INTO LINEX,Z OF 2ND PT'S X & Z WC WORDS
;       COLOR=COLOR OF OBJECT
;       PYL = Y WC COORD FOR BOTH PTS.
;       ACC = CODE FOR PICTURE TO DISPLAY (INDEX INTO PICLO)
SCAPIC:
        STA OBJIND        ; STORE THE PICTURE INDEX IN OBJIND
        LDA Y,LINEXM      ; GET THE X COORD. OF MIDWAY PT.
        STA PXL           ; STORE IN PXL
        LDA Y,LINEZM      ; GET THE Z COORD OF MIDWAYPT.
        STA PZL           ; STORE IN PZL

        ; USE PXL,PYL, AND PZL TO CALCULATE THE CENTRE POINT FOR
        ; THE BULLET ON THE SCREEN. THE RESULT IS STORED IN
        ; SXL AND SZL. THEN MOVE THE BEAM TO THE DESIRED POSITION
        ; BY ADDING SOME OPCODE INSTRUCTION TO THE VECTOR GENERATOR
        ; LIST.
        JSR WORSCR        ; CALCULATE SXL AND SZL.
        LDX I,SXL         ; GET THE CALCULATED CENTRE PT.
        JSR VGYAB1        ; DRAW A BLANK VECTOR TO IT SO WE START THERE.

        ; CALCULATE THE SCALE AND ADD IT AS AN OPCODE TO THE VGLIST.
        LDA I,0           ; VGY IS OUR INDEX INTO THE VGLIST..
        STA VGY           ; ..SO SET IT TO 0 SO WE'RE AT THE START.
        JSR CASCAL        ; CALCULATE SCALE

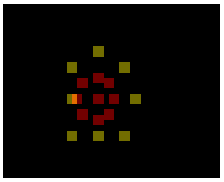
        ; CALCULATE THE BRIGHTNESS AND ADD IT AS AN OPCODE TO THE VGLIST.
        LDA BFACTR        ; GET THE BRIGHTNESS FACTOR.
        EOR I,7
        ASL
        CMP I,0A
        IFCC
        LDA I,0A
        ENDIF
        ASL
        ASL
        ASL
        ASL
        STA NY,VGLIST     ; STORE THE BRIGHTNESS BYTE
        INY               ; MOVE TO NEXT BYTE OF OPCODE
        LDA I,60          ; '60' IS THE ID OF THE BRIGHTNESS OPCODE.
        STA NY,VGLIST     ; STORE THE ID IN THE OPCODE LIST.
        INY               ; INCREMENT THE INDEX.
        STY VGY           ; STORE IT.

        ; ADD THE OPCODE TO THE VGLIST FOR DRAWING THE BULLET.
        LDY OBJIND        ; GET THE INDEX OF THE BULLET PIC.
        LDX Y,PICHI       ; GET THE HI BYTE OF THE BULLET PIC.
        LDA Y,PICLO       ; GET THE LO BYTE OF THE BULLET PIC.
        LDY VGY           ; GET THE CURRENT INDEX IN THE OPCODE LIST.
        JMP VGADD3        ; DRAW PIC AT PT.
```

So the result of a typical visit to the SCAPIC routine is to generate a series of Op-Codes that are added to the Vector Generator list. The first set is generated by calling VGYAB1 and is responsible for moving the beam to the desired position on screen.

Vector Commands	Command Meaning	Result on Screen
7100	Set Scale: 00	
8040	Move to Center	
1F2AE4AB	Draw Relative Vector:	

We follow that by setting the scale again, then the brightness, and finally drawing the bullet itself.

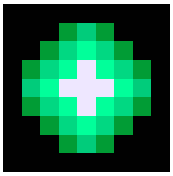
Vector Commands	Command Meaning	Result on Screen
7308	Set Scale: 308	
680A	New Color: Not Set Intensity: 0A	
AFA7	Jump to Routine: DIARA2	

activeobjects

`activeobjects` is a master list of every in-game object on the screen at a given time, with the exception of the player's claw. It makes sense to keep everything in a list like this and have each entry tell us what the game should do with the object, where it should draw it, what type of object it is, and so on. The best way of understanding the use of the `activobjects` list is to see it in action. To do that, we're going to take a look at a snapshot from a game and examine its full `activeobjects` list.

Before we do that, let's take a quick look at what a typical entry in the `activeobjects` list looks like. Each entry in the `activeobjects` list is 64 bytes long. Depending on how many bytes we want to use for each property in an object we could define anything from 16 to 64 properties. In practice, Tempest 2000 has a total of 21 properties, some are 4 bytes (a `long`), others are 2 bytes (a `word`).

Below we show a typical entry for a player's bullet.

Bullet	Object Data	
	Draw Routine: draw_pixex X: -22 Y: -12 Z: 47 Web Lane: 2 Velocity: 1 Acceleration: 0000ca7e Roll: 40 Pitch: 0 Yaw: 0 draw_vex routine: draw_pixex	YX in Sprite Sheet: 00b6000a Colour: 0078 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Width/Height in Sprite Sheet: 00070007 Marked for deletion: 0000 Enemy or Not: not an enemy run_vex routine: player_shot Address of Previous Object: ffffffff Address of Next Object: 0000cc38 Current Address: 0xcc78

As you can imagine there are usually a few of these. If you take a look at the components included in the 64 bytes of data representing the object you will see there are a few things whose utility is obvious: we give the X, Y, and Z co-ordinates of the object; which lane it is on, its velocity, and its colour. Some others might seem a bit more recherche: roll, pitch, and yaw, for example, define the angle and shear we apply to the object when drawing it in 3D space. Another way of describing this is the degree of tilt we apply in the X,Y, and Z directions when drawing the object. The most obscure-looking of all are the various 'routines' associated with the object. In the example given above we can see that the routines `draw_pixex` and `player_shot` are defined as properties. These are routines that the game will run when either drawing or updating the state of the object. Lastly we can see that the object contains addresses for the next and previou objects in the list. This is a useful feature when reading the `activeobjects` list - they tell us that when we're finishing processing this particular `activeobject`, which object we should jump to next.

So let's take a look at a random game screen and enumerate the `activeobject` list associated with it. We've chosen a rather busy screen:

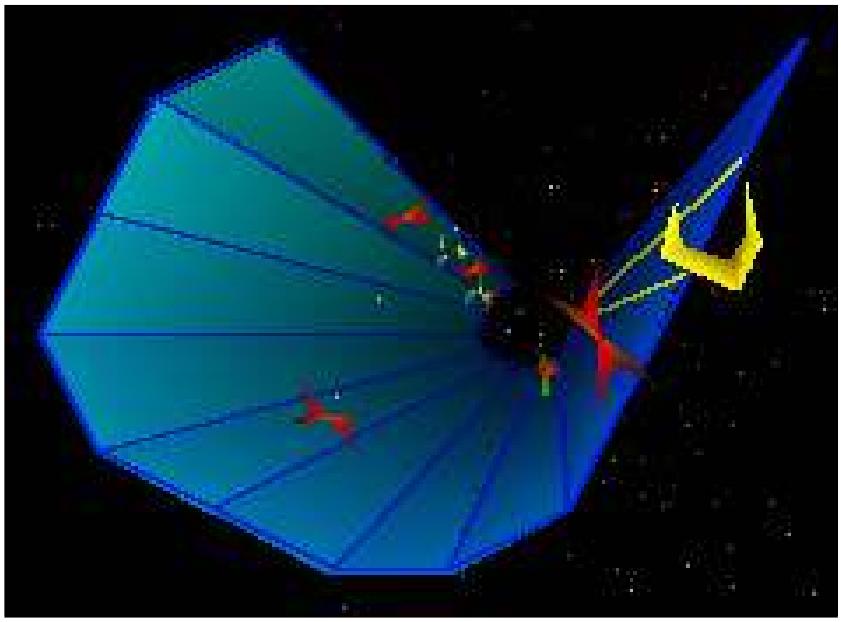
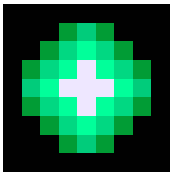
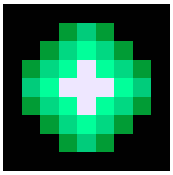


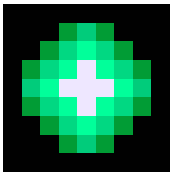
Figure .1: Our random screen.

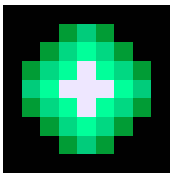
Let's enumerate the entire `activeobjects` list for this screen to give us an idea of how much is being managed at once. We give the objects in the order in which they appear in the list.

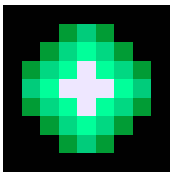
Enemy Bullet	Object Data	
	Draw Routine: s_shot X: -12 Y: -26 Z: 100 Web Lane: 0 Velocity: 0 Acceleration: 0000ca82 Roll: 124 Pitch: 0 Yaw: 0 draw_vex routine: draw	Y/X in Sprite Sheet: 00b6000a Colour: 0018 Scale factor: 0000 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Width/Height in Sprite Sheet: 00070007 Marked for deletion: 0000 Enemy or Not: enemy run_vex routine: run_ashot Address of Previous Object: ffffffff Address of Next Object: 0000d038 Current Address: 0xcbf8

Player Bullet	Object Data	
	Draw Routine: draw_pixex X: 22 Y: -16 Z: 194 Web Lane: 10 Velocity: 1 Acceleration: 0000ca6e Roll: 376 Pitch: 0 Yaw: 0 draw_vex routine: draw_pixex	Y/X in Sprite Sheet: 00b6000a Colour: 0058 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Width/Height in Sprite Sheet: 00070007 Marked for deletion: ffff Enemy or Not: not an enemy run_vex routine: player_shot Address of Previous Object: 0000cdf8 Address of Next Object: 0000cd38 Current Address: 0xcc38

Player Bullet	Object Data	
	Draw Routine: draw_pixex X: 26 Y: -24 Z: 191 Web Lane: 11 Velocity: 1 Acceleration: 0000ca72 Roll: 368 Pitch: 0 Yaw: 0 draw_vex routine: draw_pixex	Y/X in Sprite Sheet: 00b6000a Colour: 0028 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Width/Height in Sprite Sheet: 00070007 Marked for deletion: ffff Enemy or Not: not an enemy run_vex routine: player_shot Address of Previous Object: 0000ccf8 Address of Next Object: 0000d0b8 Current Address: 0xcc78

Player Bullet	Object Data	
	Draw Routine: draw_pixex	Y/X in Sprite Sheet: 00b6000a
	X: 26	Colour: 0028
	Y: -24	Scale factor: 0001
	Z: 191	0 = climb rail, 1 = cross rail, 2 = blowaway: 0000
	Web Lane: 11	Width/Height in Sprite Sheet: 00070007
	Velocity: 1	Marked for deletion: ffff
	Acceleration: 0000ca76	Enemy or Not: not an enemy
	Roll: 368	run_vex routine: player_shot
	Pitch: 0	Address of Previous Object: 0000d078
	Yaw: 0	Address of Next Object: 0000ccf8
	draw_vex routine: draw_pixex	Current Address: 0xccb8

Player Bullet	Object Data	
	Draw Routine: draw_pixex	Y/X in Sprite Sheet: 00b6000a
	X: 26	Colour: 0028
	Y: -24	Scale factor: 0001
	Z: 191	0 = climb rail, 1 = cross rail, 2 = blowaway: 0000
	Web Lane: 11	Width/Height in Sprite Sheet: 00070007
	Velocity: 1	Marked for deletion: ffff
	Acceleration: 0000ca7a	Enemy or Not: not an enemy
	Roll: 368	run_vex routine: player_shot
	Pitch: 0	Address of Previous Object: 0000ccb8
	Yaw: 0	Address of Next Object: 0000cc78
	draw_vex routine: draw_pixex	Current Address: 0xccf8

Player Bullet	Object Data	
	Draw Routine: draw_pixex	Y/X in Sprite Sheet: 00b6000a
	X: 22	Colour: 0058
	Y: -16	Scale factor: 0001
	Z: 194	0 = climb rail, 1 = cross rail, 2 = blowaway: 0000
	Web Lane: 10	Width/Height in Sprite Sheet: 00070007
	Velocity: 1	Marked for deletion: ffff
	Acceleration: 0000ca7e	Enemy or Not: not an enemy
	Roll: 376	run_vex routine: player_shot
	Pitch: 0	Address of Previous Object: 0000cc38
	Yaw: 0	Address of Next Object: 0000cb78
	draw_vex routine: draw_pixex	Current Address: 0xcd38

Flipper	Object Data	
	Draw Routine: cdraw_sflipper	Start address of pixel data.: fff70006
	X: 4	Colour: 00f0
	Y: 8	Scale factor: 0001
	Z: 30	0 = climb rail, 1 = cross rail, 2 = blowaway: 0000
	Position/lane on web.: 7	Size of Pixel Data: c9ce0007
	Velocity: 4	Marked for deletion: 0000
	Acceleration/Flipper Mode: 00010808	Enemy or Not: enemy
	XZ Orientation (Roll): 96	run_vex routine: run_flipper
	Y Rotation (Pitch): 0	Address of Previous Object: 0000cdb8
	Z Rotation (Yaw): 0	Address of Next Object: 0000cb38
	draw_vex routine: draw_vxc	Current Address: 0xcd78

Flipper	Object Data	
	Draw Routine: cdraw_sflipper X: 16 Y: -4 Z: 30 Position/lane on web.: 9 Velocity: 4 Acceleration/Flipper Mode: 00010808 XZ Orientation (Roll): 36 Y Rotation (Pitch): 0 Z Rotation (Yaw): 0 draw_vex routine: draw_vxc	Start address of pixel data.: fff70008 Colour: 00f0 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Size of Pixel Data: c9ca0101 Marked for deletion: 0000 Enemy or Not: enemy run_vex routine: run_flipper Address of Previous Object: 0000ce38 Address of Next Object: 0000cd78 Current Address: 0xcdb8

Enemy Bullet	Object Data	
	Draw Routine: s_shot X: -16 Y: 2 Z: 30 Web Lane: 4 Velocity: 0 Acceleration: 0000ca76 Roll: 204 Pitch: 13 Yaw: 15 draw_vex routine: draw	Y/X in Sprite Sheet: 009e0000 Colour: 00d8 Scale factor: 0000 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Width/Height in Sprite Sheet: 00a00001 Marked for deletion: ffff Enemy or Not: enemy run_vex routine: run_ashot Address of Previous Object: 0000cf78 Address of Next Object: 0000cc38 Current Address: 0xcdf8

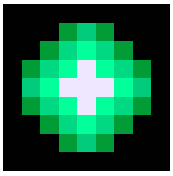
Flipper	Object Data	
	Draw Routine: cdraw_sflipper X: -12 Y: -26 Z: 71 Position/lane on web.: 0 Velocity: 0 Acceleration/Flipper Mode: 00000808 XZ Orientation (Roll): 112 Y Rotation (Pitch): 0 Z Rotation (Yaw): 0 draw_vex routine: draw_vxc	Start address of pixel data.: 00000000 Colour: 00f0 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Size of Pixel Data: c9d20007 Marked for deletion: 0000 Enemy or Not: enemy run_vex routine: run_flipper Address of Previous Object: 0000ce78 Address of Next Object: 0000cdb8 Current Address: 0xce38

Flipper	Object Data	
	Draw Routine: cdraw_sflipper X: -12 Y: -26 Z: 125 Position/lane on web.: 0 Velocity: 1 Acceleration/Flipper Mode: 00000808 XZ Orientation (Roll): 112 Y Rotation (Pitch): 0 Z Rotation (Yaw): 0 draw_vex routine: draw_vxc	Start address of pixel data.: 00000000 Colour: 00f0 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Size of Pixel Data: c9ce0000 Marked for deletion: 0000 Enemy or Not: enemy run_vex routine: run_flipper Address of Previous Object: 0000cef8 Address of Next Object: 0000ce38 Current Address: 0xce78

Spiker	Object Data	
	Draw Routine: 0000b6c4 X: 8 Y: 6 Z: 187 Position/lane on web.: 7 Velocity: 1 Acceleration/Flipper Mode: 0000cff8 XZ Orientation (Roll): 592 Y Rotation (Pitch): 0 Z Rotation (Yaw): 0 draw_vex routine: draw	Start address of pixel data.: ffff000a Colour: 008f Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0002 Size of Pixel Data: 00070007 Marked for deletion: 0000 Enemy or Not: enemy run_vex routine: run_spiker Address of Previous Object: 0000cff8 Address of Next Object: 0000cfb8 Current Address: 0xceb8

Fuseball	Object Data	
	Draw Routine: draw_sfuseball X: -16 Y: -25 Z: 163 Position/lane on web.: 0 Velocity: -1 Acceleration/Flipper Mode: 00004000 XZ Orientation (Roll): 526 Y Rotation (Pitch): 0 Z Rotation (Yaw): 0 draw_vex routine: draw	Start address of pixel data.: 00000001 Colour: 000f Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0001 Size of Pixel Data: 30300101 Marked for deletion: 0000 Enemy or Not: enemy run_vex routine: run_fuseball Address of Previous Object: 0000cf38 Address of Next Object: 0000ce78 Current Address: 0xcef8

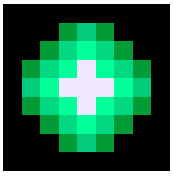
Power-Up	Object Data	
	Draw Routine: draw_pup1 X: 14 Y: 0 Z: 29 Position/lane on web.: 8 Velocity: 0 Acceleration/Flipper Mode: 00010808 XZ Orientation (Roll): -48 Y Rotation (Pitch): 0 Z Rotation (Yaw): 0 draw_vex routine: draw_prex	Start address of pixel data.: fff70007 Colour: 0058 Scale factor: 0000 0 = climb rail, 1 = cross rail, 2 = blowaway: 0023 Size of Pixel Data: 0010fffc Marked for deletion: 0000 Enemy or Not: not an enemy run_vex routine: run_prex Address of Previous Object: 0000cbb8 Address of Next Object: 0000cef8 Current Address: 0xcf38

Player Bullet	Object Data	
	Draw Routine: draw_pixex X: 22 Y: -16 Z: 194 Web Lane: 10 Velocity: 1 Acceleration: 0000ca72 Roll: 376 Pitch: 0 Yaw: 0 draw_vex routine: draw_pixex	Y/X in Sprite Sheet: 00b6000a Colour: 0058 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Width/Height in Sprite Sheet: 00070007 Marked for deletion: ffff Enemy or Not: not an enemy run_vex routine: player_shot Address of Previous Object: ffffffff Address of Next Object: 0000cdf8 Current Address: 0xcf78

Flipper	Object Data	
	Draw Routine: cdraw_sflipper X: -16 Y: 2 Z: 55 Position/lane on web.: 4 Velocity: -4 Acceleration/Flipper Mode: 00000808 XZ Orientation (Roll): 16 Y Rotation (Pitch): 0 Z Rotation (Yaw): 0 draw_vex routine: draw_vxc	Start address of pixel data.: 0000ffff Colour: 00f0 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Size of Pixel Data: 00070007 Marked for deletion: 0000 Enemy or Not: enemy run_vex routine: run_flipper Address of Previous Object: 0000ceb8 Address of Next Object: 0000cbb8 Current Address: 0xcfb8

Spiker	Object Data	
	Draw Routine: 0000b694 X: 8 Y: 6 Z: 190 Position/lane on web.: 7 Velocity: 0 Acceleration/Flipper Mode: 00000000 XZ Orientation (Roll): -16 Y Rotation (Pitch): 0 Z Rotation (Yaw): 0 draw_vex routine: draw_spike	Start address of pixel data.: 00576000 Colour: 008f Scale factor: ffff 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Size of Pixel Data: c9ca0000 Marked for deletion: 0000 Enemy or Not: not an enemy run_vex routine: run_spike Address of Previous Object: 0000d038 Address of Next Object: 0000ceb8 Current Address: 0xcff8

Flipper	Object Data	
	Draw Routine: cdraw_sflipper X: 0 Y: 8 Z: 330 Position/lane on web.: 6 Velocity: 0 Acceleration/Flipper Mode: 00000808 XZ Orientation (Roll): 0 Y Rotation (Pitch): 0 Z Rotation (Yaw): 0 draw_vex routine: fffd	Start address of pixel data.: 00000000 Colour: 00f0 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Size of Pixel Data: 00100000 Marked for deletion: 0000 Enemy or Not: enemy run_vex routine: run_flipper Address of Previous Object: 0000cbf8 Address of Next Object: 0000cff8 Current Address: 0xd038

Player Bullet	Object Data	
	Draw Routine: draw_pixex X: 26 Y: -24 Z: 191 Web Lane: 11 Velocity: 1 Acceleration: 0000ca86 Roll: 368 Pitch: 0 Yaw: 0 draw_vex routine: draw_pixex	Y/X in Sprite Sheet: 00b6000a Colour: 0028 Scale factor: 0001 0 = climb rail, 1 = cross rail, 2 = blowaway: 0000 Width/Height in Sprite Sheet: 00070007 Marked for deletion: ffff Enemy or Not: not an enemy run_vex routine: player_shot Address of Previous Object: 0000cb78 Address of Next Object: 0000ccb8 Current Address: 0xd078

superzap

Casting our minds back to our discussion of the mainline routine, you may recall that one of the things we do during each 'tick' of the game's clock is update the progress of any 'super zap' that is in progress:

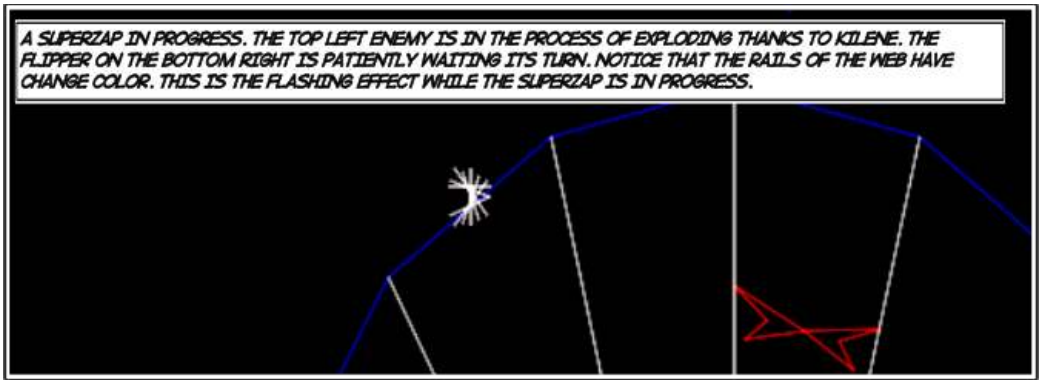
```
PLAY:
        JSR MOVCUR              ;MOVE CURSOR AROUND
        JSR FIREPC              ;FIRE PLAYER CHARGE
        JSR PROSUZ              ;PROCESS SUPER ZAP
        ...
```

In case you're not familiar with what a SUPER ZAP is when it's at home, this is when the player presses the 'kill everything' button on the *Tempest* console and the game logic dutifully annihilates your enemies for you. It can only be used twice on each level, so must be applied sparingly. The first press will kill all enemies and enemy bullets, with the exception of tankers. While the second press will kill just one additional enemy.

```
CSUMAX=2                        ; MAX USES OF SUPER ZAP IS 2.
CSUINT=1                        ; KILL ENEMIES AT EVERY 2ND TICK OF TIMER.
CSUSTA=3                        ; CONTROLS NUMBER OF ENEMIES TO KILL.
        .SBTTL INITIALIZE SUPER ZAP
INISUZ:
        LDA I,0                 ; STORE 0 IN THE 'A' REGISTER.
        STA SUZCNT              ; SET SUPER ZAP USE COUNTER TO 0.
        STA SUZTIM              ; SET SUPER ZAP TIMER TO 0.
        RTS
```

```
   .SBTTL PROCESS SUPER ZAPPER
PROSUZ:
  LDA QSTATUS                     ; GET CURRENT GAME STATE..
  IFMI                            ; IF WE'RE NOT IN ATTRACT MODE..
    LDA SUZTIM                    ; CHECK THE SUPERZAP TIMER..
    IFEQ                          ; IF ITS ZERO..
      ; ..THEN MAYBE WE NEED TO TURN SUPERZAPPING ON.
      ; FRST WE CHECK IF PLAYER IS ALIVE AND
      ; THE SUPERZAP BUTTON HAS BEEN PRESSED.
      LDA CURSL2                  ; GET PLAYER STATUS.
      IFPL                        ; IF THE PLAYER IS ALIVE..
      LDA SWFINA                  ; CHECK BUTTON PRESSES FOR..
      AND I,MSUZA                 ; THE SUPERZAPPER BUTTON.
      IFNE                        ; IF ZAP WAS PRESSED..
      LDA SUZCNT                  ; GET NO. OF ZAPS USED SO FAR.
      CMP I,CSUMAX                ; IS IT EQUAL TO THE MAX AVAIL (2)?
      IFCC                        ; IF NOT..
        INC SUZCNT                ; THEN INCREMENT THE ZAPS USED AND..
        LDA I,1                   ; .. PUT 1 IN THE ..
        STA SUZTIM                ; NEW ZAP TIMER
      ENDIF
      LDA SWFINA                  ; GET THE BUTTON PRESSES
      AND I,^C<MSUZA!MFAKE>       ; TURN OFF THE ZAP PRESS..
      STA SWFINA                  ; SO WE DON'T USE IT AGAIN NEXT TIME.
      ENDIF
      ENDIF
    ELSE  ; THE SUPERZAPPER TIMER IS NON-ZERO SO..
      ; ..WE'RE ALREADY ZAPPING. UPDATE OUR TIMERS AND KILL THINGS.
      INC SUZTIM                  ; INCREMENT ZAP TIMER BY ONE.
      LDX SUZCNT                  ; PUT NO. OF ZAPS USED IN X.
      LDA SUZTIM                  ; GET CURRENT VALUE OF ZAP TIMER.
      ; TIMAX IS AN ARRAY: .BYTE 00,13,05
      ; SUZCNT IS USED AS AN INDEX INTO TIMAX TO GET THE NUMBER OF TICKS
      ; FOR THIS ZAP.THIS MEANS THE FIRST ZAP GETS 13 FRAMES, WHILE THE
      ; SECOND ZAP GETS 5.
      CMP X,TIMAX                 ; TIMER REACHED MAX TIME YET?
      IFCS                        ; IF YES..
      LDA I,0                     ; LOAD 0 TO A REGISTER..
      STA SUZTIM                  ; .. AND USE IT TO DEACTIVATE ZAP
      ENDIF
      ; KILL THINGS!
      JSR KILENE                  ; WIPE OUT INVADERS & CHARGES
    ENDIF
  ENDIF

  LDA SWFINA                      ; GET THE BUTTON PRESSES
  AND I,^CMFAKE                   ; TURN OFF THE ZAP PRESS..
  STA SWFINA                      ; SO WE DON'T USE IT AGAIN NEXT TIME.
  RTS
```



PROSUZ, the routine opposite responsible for processing the super zapping decision making, does one of two things: it either turns superzapping on or off, or it determines that superzapping is already on and kills an enemy.

The main control unit in PROSUZ is SUZTIM, which tells us if we are in the middle of killing enemies with a superzap or not. If we are not then the first branch in PROSUZ figures out if the superzap button has been pressed and sets things up for a superzap session if it hasn't been used twice already.

The second branch handles a 'tick' in an ongoing superzap. This 'tick' is SUZTIM, which we increment every time we visit the branch: and every time we visit this branch we kill an enemy KILENE.

An important feature of this branch is figuring out if the zap has killed its maximum number of enemies yet. The way we do this is by figuring out how many times the superzapper button has been pressed so far, something we keep track of in SUZCNT. When used as an index into the TIMAX array it gives us what we're looking for. The TIMAX array contains 13 for the first button press and 5 for the second. Through some indirect logic this means that we will kill 7 enemies at the first press and 1 for the second. This slightly oblique result is achieved by setting a lower cut off of 3 (CSUSTA) before anything is killed and from then on only killing an enemy when our kill counter SUZTIM is an odd value. We eventually stop killing enemies when we exceed the value for this button press in the TIMAX array.

If we take a look at the KILENE routine - the one actually responsible for zapping our enemies - we find the rate limiting using the minimum cutoff on our timer given by CSUSTA and the logic that only kills when SUZTIM is an odd number. All the other zap-management overhead is done by PROSUZ. Notice that killing an enemy just means looping through the enemy list and killing the first one that has a positive Y position (a proxy for it being active and sitting on the web).

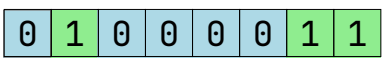
```
        .SBTTL SUPER ZAP-WIPE OUT ENEMY
        ; FIND ENEMIES THAT ARE ON THE PLAYFIELD (I.E. HAVE A
        ; NON-ZERO Y POSITION) AND KILL THEM.
KILENE: LDA SUZTIM              ; CHECK THE SUPERZAP TIMER..
        CMP I,CSUSTA            ; HAS IT REACHED 3 YET?
        IFCS                    ; IF YES THEN..
        AND I,CSUINT            ; IS SUZTIM AN ODD VALUE (LAST BIT IS A 1)?
        IFEQ                    ; IF YES THEN..
          LDY WINVMX            ; PUT NO OF INVADERS IN Y.
          BEGIN                 ; LOOP THRU INVADERS
          LDA Y,INVAY           ; GET Y POSITION OF INVADER.
          BNE EXIKIL            ; IF IT HAS A Y POS, KILL IT, AND EXIT.
          DEY                   ; OTHERWISE IT'S NOT ACTIVE SO TRY NEXT ONE.
          MIEND                 ; KEEP LOOPING TILL WE KILL ONE.
          ; IF WE REACH HERE, ALL ARE DEAD. DEACTIVATE ZAP.
          LDA I,0               ; STORE 0 IN THE A REGISTER.
          STA SUZTIM            ; AND USE IT TO CLEAR SUPERZAP TIMER.
        ENDIF
        ENDIF
        RTS

        ; IF IT'S A CARRIER/TANKER CONVERT IT TO A NORMAL ENEMY FIRST.
EXIKIL: LDA Y,INVAC2            ; STORE INVADER STATUS BYTE IN 'A'.
        AND I,^C<INVCAR>        ; 'AND' IT WITH 1's COMPLEMENT OF INVCAR.
        STA Y,INVAC2            ; STORE RESULT IN INVADER STATUS BYTE.
        JMP INCISQ              ;START EXPLOSION
```

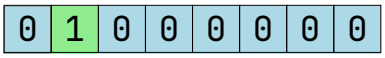
Let's take a quick look at what we are doing in the last few lines above with `INVAC2`, a byte array that governs the type, firepower, and movement direction of each active invader. With Y as an index into the array, we pick out the `INVAC2` byte for the invader we're about to kill. The table below gives you a sense of how each invader's byte in `INVAC2` is used to determine its behaviour:

Byte	Movement	Firepower	Carrier?
00000000	Up	No Fire	No
10000001	Down	No Fire	Flippers
11000010	Down	Fire	Pulsars
01000011	Up	Fire	Fuses

Here is the last entry in the table above, but prettier. As you can see above it represents a fuse carrier, that moves up and also fires.



Our aim is to turn this invader into a normal enemy before we destroy it, so that killing it doesn't unleash its fuse contents. To do this we need a handy way of clearing the last two bits that specify it as a 'carrier' without affecting the other bits. Like this:



One way we can achieve this is with an AND operation on our INVAC2 value and a byte with the carrier bits unset and everything else set (i.e. 11111100):

Byte	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
INVAC2	0	1	0	0	0	0	1	1
AND	1	1	1	1	1	1	0	0
New INVAC2:	0	1	0	0	0	0	0	0

AND'ing INVAC2 with a value to clear the 'carrier' bits.

So how do we come up with a value of 11111100 to use in this AND operation? One way of doing it is to take a value we already have, INVCAR, and flip it.

INVCAR has a value of 00000011. So if we flip every bit in it we get its polar opposite 11111100, which is the desired value for use in our AND operation above. This is what ^C<INVCAR> does: it flips 00000011 to 11111100, which we call the *one's complement*.

This means that our full instruction AND I,^C<INVCAR> will perform a logical AND operation between the INVAC2 value we've stored in the A register and this flipped value to give us a new value with just the 'carrier' bits reset to zero, as desired.

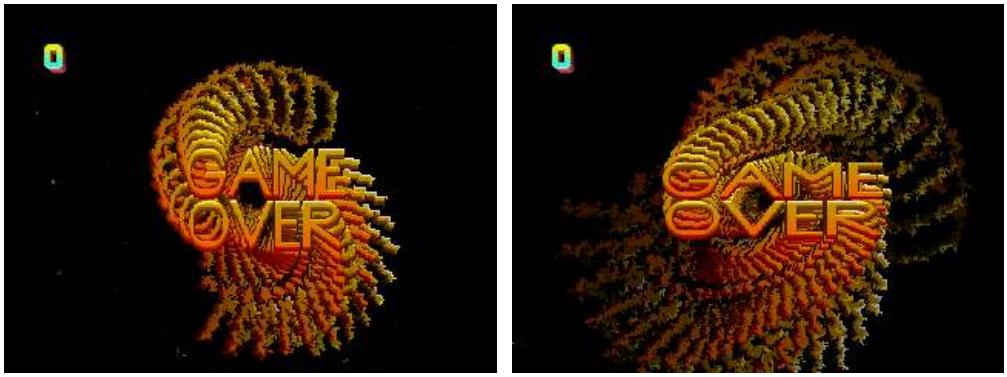
One final note, the web flashes while we're superzapping. How is that achieved? It turns out very simply: we take the frame counter in QFRAME and use that to give us a value between 0 and 7. We ignore 7, because that's the color value for black, and use white instead. This means that every time we draw the web while a superzap is active we effectively get a 'random' value for the color of the web, resulting in the flashing effect.

```
        .SBTTL DISPLAY-WELL
DSPWEL:
        ..
        LDY SUZTIM   ; GET THE SUPERZAP TIMER.
        IFNE         ; IS IT NON-ZERO?
          IFPL       ; IF SO IS IT POSITIVE?
          LDA QFRAME ; IF YES, USE QFRAME TO GIVE A COLOR..
          AND I,7    ; TAKE JUST THE LAST 3 BITS FOR A COLOR VALUE.
          CMP I,7    ; IS IT 7 (I.E. BLACK)?
            IFEQ     ; IF YES, THEN..
            LDA I,1  ; WE'LL SET IT TO WHITE.
            ENDIF
          ENDIF
```

```
        ENDIF
        STA TEMP0   ; STORE OUR CHOSEN COLOR.
```

game over

The 'game over' display in *Tempest 2000* is a pulsating, coruscating display achieved in a relatively simple fashion. It relies on two separate shader operations. The first takes the entire screen and scales and rotates it. This creates the effect of the 'Game Over' swirling and atomizing into the background. The second manipulates the height and width of the 'Game Over' message to create a pulsating effect. This also has the effect of creating an irregular profile to the background as it is scaled and rotated.



The 'Game Over' graphic itself is sourced from the `beasty4.cry` spritesheet. We find it located in the top right of the sheet, extracting it is a straightforward matter of providing the offset into the sheet and the number of pixels to extract:



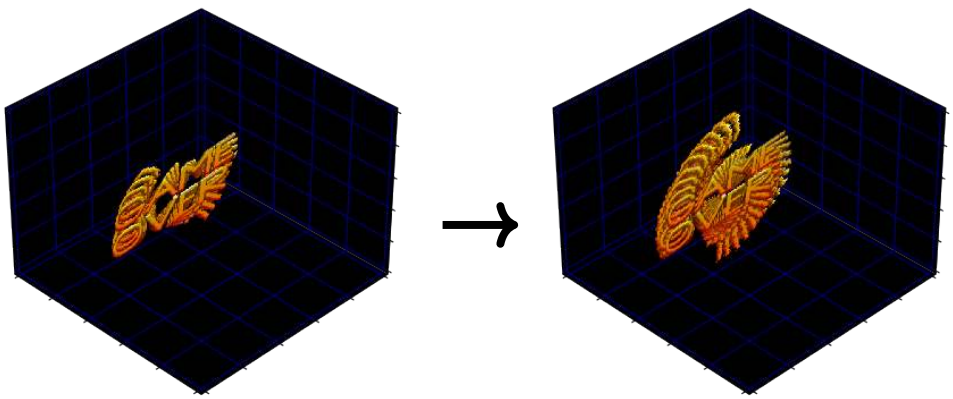

```
; Point GPU RAM at a0.
lea in_buf,a0
; Srce screen is pic2 aka beasty4.cry.
move.l #pic2,(a0)+
; Start pixel address for 'Game Over'
move.l #$150094,(a0)+
; Size for 'Game Over'
move.l #$440091,(a0)+
```

In the `gofeed` routine we find the two elements of the 'Game Over' effect implemented. First up is the scale and rotation of the currently displayed screen (`cscreen`). This is a call to the `scar` module in `antelope.gas`. The main task prior to calling `scar` is to calculate the angle that we will rotate the image by. This result is stored in register `d0` and passed to the GPU input buffer. The scaling factor on the other hand is a constant value `$1f4`.

```
; ******************************************************************
; gofeed
; A 'demo_routine' routine
; Implements the game over display effect.
; ******************************************************************
gofeed:
        ; Draw the rotating galaxy effect on the game over screen. This
        ; draws the spiralling arms of the main game over graphic. It does
        ; this by taking the currently displayed screen and stretching and
        ; rotating it.
        move.l #(PITCH1|PIXEL16|WID384),d0   ; Set the GPU flags in d0.
        move.l d0,source_flags      ; Pass them to the GPU's source flags.
        move.l d0,dest_flags        ; And the GPU's destination flags.
        lea in_buf,a0               ; Point our GPU RAM input buffer at a0.
        add.l #$2000,pongz          ; Add 2000 to pongz.
        move pongz,d0               ; Store pongz in d0.
        and.l #$0f,d0               ; Keep d0 between 0 and 15
        sub.l #$07,d0               ; Make sure d0 is between 0 and 8.
        move.l cscreen,(a0)         ; Image to rotate is the current screen.
        move.l #384,4(a0)           ; Full screen width.
        move.l #240,d1              ; Full screen height.
        add palfix1,d1              ; Adjust for PAL
        move.l d1,8(a0)             ; Store height in GPU input buffer.
        move.l #$1f4,12(a0)         ; Store X scale in GPU input buffer.
        move.l #$1f4,16(a0)         ; Store Y scale in GPU input buffer.
        move.l d0,20(a0)            ; Store initial angle in brads (d0).
```

```
        move.l #$c00000,24(a0)      ; source x centre in 16:16
        move.l #$780000,d0          ; Source Y centre
        add.l palfix3,d0            ; Adjust for PAL screens if required.
        move.l d0,28(a0)            ; Set as Y centre.
        move.l #$0,32(a0)           ; Offset of dest rectangle
        move.l delta_i,36(a0)       ; Set the color intensity delta.
        ; Op 2 of this module is Scale and Rotate ('scar').
        move.l #2,gpu_mode          ; Select 'scar' in antelope.gas.
        move.l #demons,a0           ; Load the GPU module in antelope.gas.
        jsr gpurun                  ; Run the selected GPU module.
        jsr gpuwait                 ; Wait for the GPU to finish.
```

If we think of this as a layer being drawn to the screen we can visualize it like so. We take the current screen and rotate and expand it clockwise. Our diagram shows the result of several iterations of scaling and rotating so that the effect is obvious. In reality, each visit to the `gofeed` routine will scale and rotate only a little, but when you're visiting the routine 20 or 30 times a second, those operations quickly add up.



The second and final part of the effect is to draw a new 'layer' of the 'Game Over' graphic onto the screen. We distort its width and height to create a pulsating effect. The new width and height are calculated in the section below and stored in the d1 and d2 registers respectively. Notice that we use a *sine* table as a source of values for the degree to which we will expand or contract the graphic.

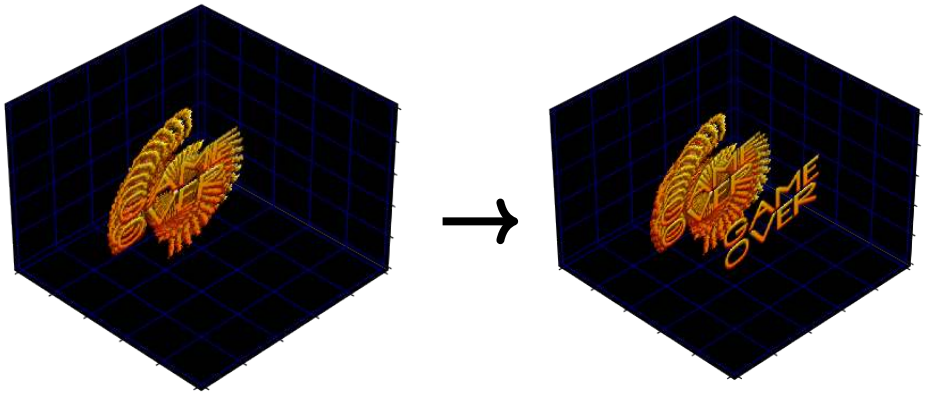
```
        ; Calculate the new X and Y scale for the 'Game Over' graphic.
        ; We use a table of positive sine values to pick an X and Y
        ; scale based on previous X and Y positions .
        move.l #4,gpu_mode          ; Select 'rex' in camel.gas.
        lea p_sines,a0              ; Load the positive sine table to a0.
        move pongx,d0               ; Store pongx in d0.
        move pongy,d3               ; Store pongy in d3.
        add #15,pongx               ; Increment pongx for next time.
```

```
        add #17,pongy               ; Increment pongy for next time.
        and #$ff,d0                 ; Keep d0 between 0 and 255
        and #$ff,d3                 ; Keep d3 between 0 and 255.
        ; Use d0 to get val in sine table and..
        move.b 0(a0,d0.w),d1        ; .. store result in d1.
        ; Use d3 to get val in sine table and..
        move.b 0(a0,d3.w),d2        ; .. store result in d2.
        and.l #$ff,d1               ; Keep d1 between 0 and 255
        and.l #$ff,d2               ; Keep d2 between 0 and 255
        lsl.l #6,d2                 ; Multiply d2 by 64.
        lsl.l #6,d1                 ; Multiply d1 by 64.
        add.l #$8000,d1             ; Make d1 negative?
        add.l #$8000,d2             ; Make d2 negative?
```

With the new scale along the X and Y axes calculated, we are ready to draw the graphic itself. For this we use the `rex` shader in `camel.gas`. This shader is capable of shearing as well as scaling the image. For our purposes here all we need to do is pass the scaling parameters we calculated above to the `shearx` and `sheary` fields in the GPU RAM input buffer `in_buf`.

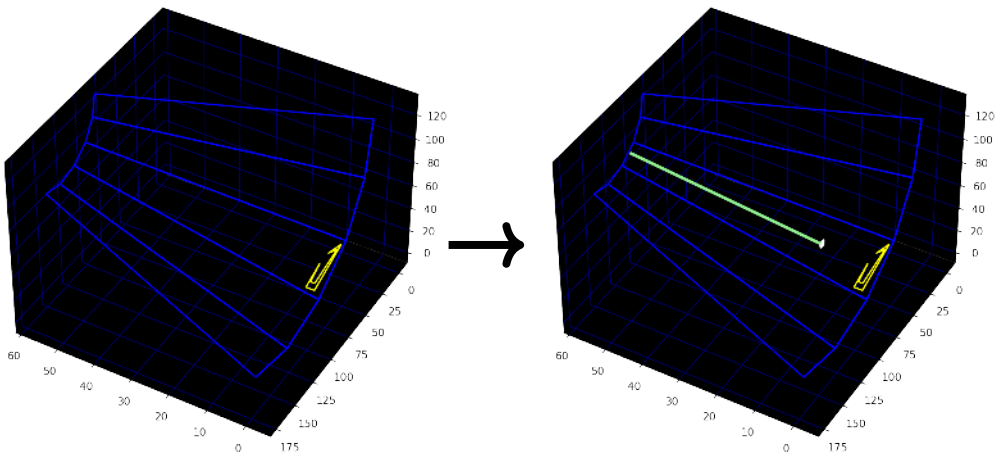
```
        ; Draw the Game Over graphic. This is the 'game over' message that
        ; expands and  contracts in the centre of the screen.
        lea in_buf,a0               ; Point our GPU RAM input buffer at a0.
        move.l #pic2,(a0)+          ; crex->_bass: srce screen is pic2.
        move.l #$150094,(a0)+       ; crex-> spixel: srce start pixel address
    .
        move.l #$440091,(a0)+       ; crex-> ssize: srce size for 'Game Over'
        move.l d1,(a0)+             ; crex-> scalex: x-scale calculated above
        move.l d2,(a0)+             ; crex-> scaley: y-scale calculated above
        move.l #0,(a0)+             ; crex-> shearx: shearx
        move.l #0,(a0)+             ; crex-> sheary: sheary
        move.l #1,(a0)+             ; crex-> mode: Mode 1 = Centered
        move.l #0,d0                ; Store 0 in d0 for X and Y dest pos..
        move.l d0,(a0)+             ; crex-> dstx: Destination X position.
        move.l d0,(a0)+             ; crex-> dsty: Destination Y position.
        move.l #$600000,d0          ; Store Z in d0.
        move.l d0,(a0)+             ; crex-> dstz:
        lea parrot,a0               ; Load the GPU module in camel.gas.
        jsr gpurun                  ; do clear screen
        jsr gpuwait                 ; Wait for the GPU to finish.
```

We can think of the final result as follows - two layers consisting of the rotated background we draw in the first part, followed by the second layer of the scaled graphic drawn on top of it.


GAME
OVER

spikes

Green spikes grow from the bottom of the well. They advance and retreat in a regular fashion until they are struck by a player bullet. When this happen a spike with a white dot is formed, presenting a peril to the player when they descend the well at the end of the level. Each time we draw a spike, the below is an approximation of the effect we want to achieve - a green line advancing from the far point of the well with a white dot at the tip.



We draw the spikes, like we draw everything else in *Tempest*, by adding appropriate vector generator opcodes to the display list (the VGLIST). We covered the basic details of this in 'tracing the beam.'

When we look at the routine responsible for drawing spikes, DSPENL, we find that we start off with a very simple opcode:

```
        .SBTTL  DISPLAY - ENEMY LINES
DSPENL: LDA I,1         ; STORE 1 IN A.
        JSR VGSCA1      ; SET NORMAL SCALE.
        LDA VGLIST      ; GET 1ST BYTE OF SCALE OPCODE.
        PHA             ; STORE IT TO STACK.
        LDA VGLIST+1    ; GET 2ND BYTE OF SCALE OPCODE.
        PHA             ; STORE IT TO STACK.
```

This adds a pair of bytes to the display list, which together form the opcode for setting the scale to 'full size'.

Vector Description	Op Code	Bitmap
Set Scale	0x7100	10000001 00000000

Our next step is to iterate through all of the active spikes and draw each one. The number of active spikes is stored in NLINES. We are going to use this number as a counter and draw a spike until we decrement our counter to 0. We will also use this number as an index into arrays that provide information about the state of each spike.

```
        LDA I,0             ; STORE 0 IN A.
        STA INDEX2          ; AND SET AS LINE LOOP INDEX.
        STA VGY             ; AND SET AS INITAL INDEX IN VGLIST.
        LDX I,NLINES-1      ; GET NUMBER OF SPIKES.
        STX INDEX1          ; AND SET AS OUR INDEX.
        BEGIN               ; LOOP FOR EACH LINE FROM 0 TO TOP
          LDX I,3           ; SET X TO 3.
          LDY VGY           ; STORE CUR POS IN VGLIST IN Y.
          BEGIN             ; LOOP THROUGH ENLFIX AND STORE IN VGLIST
            LDA X,ENLFIX    ; GET CURRENT BYTE FROM ENLFIX.
            STA NY,VGLIST   ; STORE IT IN DISPLAY LIST.
            INY             ; INCREMENT VGLIST POSITION.
            DEX             ; MOVE TO NEXT BYTE IN ENLFIX.
          MIEND             ; LOOP UNTIL END.
          STY VGY           ; STORE CUR POS IN VGLIST IN VGY.
```

As you may notice above, for each line we start by adding some 'static' opcodes that are common to all spikes. These are found in the ENLFIX array.

```
ENLFIX: .BYTE 80,40,68,05
```

The contents of ENLFIX provides two new opcodes to the VGLIST, centering the draw-

ing position and setting the color of our 'pen' to green:

Vector Description	Op Code	Bitmap
Set Scale	0x7100	10000001 00000000
Set Color to Green	0x6805	01101000 00001001
Center	0x8040	10000000 01000000

Our next step is to draw the actual line. To do this we just need to retrieve and update the 'far point' for each line. For each line we keep an X and Y co-ordinate in two separate sets of arrays. Since the co-ordinate values can be greater than 255 we will need two bytes to express each. For this reason we have two arrays for the X co-ordinate, one that stores the 'low byte' for each line (`LIFSXL`), and another that stores the 'high byte' (`LIFSXH`). We do the same for the Y co-ordinate (`LIFSZL` and `LIFSZH`). In order to calculate the point to which we should draw the line we get whatever the current value is and increment with whatever the value is in the line adjacent to it.

```
FIXSTU: ;CALCULATE SCREEN LOCATION OF FAR POINT
        LDA INDEX2    ; GET THE CURRENT LINE NUMBER.
        TAX           ; STORE IT IN X.
        CLC           ; CLEAR THE CARRY BIT.
        ADC I,1       ; ADD 1 TO GET INDEX TO ADJACENT LINE.
        AND I,0F      ; CLAMP IT TO BETWEN 0 AND 15.
        TAY           ; STORE THE ADJACENT LINE INDEX IN Y.

        LDA X,LIFSXL  ; GET THE LOW X-COORDINATE BYTE FOR CUR LINE.
        SEC           ; ADD ANY CARRIED VALUE.
        ADC Y,LIFSXL  ; ADD VALUE FROM ADJACENT LINE.
        STA SXL       ; STORE AS NEW X POS LOW BYTE.

        LDA X,LIFSXH  ; GET THE HIGH X-COORDINATE BYTE FOR CUR LINE.
        ADC Y,LIFSXH  ; ADD VALUE FROM ADJACENT LINE.
        STA SXH       ; STORE AS NEW X POS HIGH BYTE.

        ASL
        ROR SXH       ; DIVIDE BY 2 TO GET MIDWAY POINT.
        ROR SXL       ; DIVIDE BY 2 TO GET MIDWAY POINT.

        LDA X,LIFSZL  ; GET THE LOW Y-COORDINATE BYTE FOR CUR LINE.
        SEC           ; ADD ANY CARRIED VALUE.
        ADC Y,LIFSZL  ; ADD VALUE FROM ADJACENT LINE.
        STA SZL       ; STORE AS NEW Y POS LOW BYTE.

        LDA X,LIFSZH  ; GET THE HIGH Y-COORDINATE BYTE FOR CUR LINE.
        ADC Y,LIFSZH  ; ADD VALUE FROM ADJACENT LINE.
        STA SZH       ; STORE AS NEW Y POS HIGH BYTE.

        ASL
        ROR SZH       ; DIVIDE BY 2 TO GET MIDWAY POINT.
        ROR SZL       ; DIVIDE BY 2 TO GET MIDWAY POINT.
```

Now we add the values we have calculated above to the *Display List* (VGLIST) as a 4 byte opcode that encodes a *Draw Relative Vector* instruction. The special thing about this line is that it is invisible. It is invisible because what we are actually doing is moving the 'pen' from the center of the screen to the 'far point' at which the green spike line will begin. The instruction for this blank line consists of the X and Y co-ordinates to which we should draw the invisible line from its origin at the center of the screen.

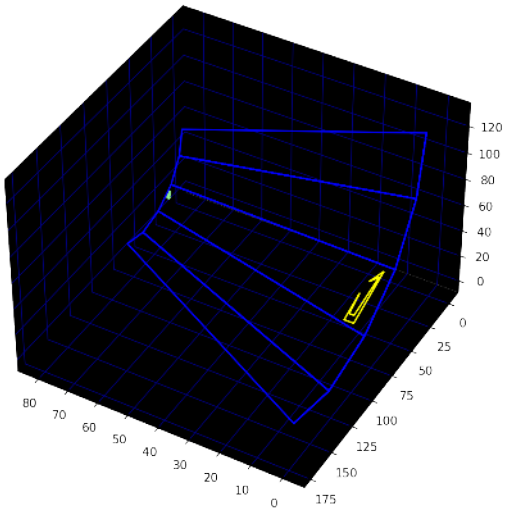
As we mentioned above there are four bytes in total. Two bytes for the X co-ordinate and two for the Y co-ordinate. In each case the high byte has a special property that makes the line we draw work the way we want it to. For the X co-ordinate the top three bits are zero so that the opcode is correctly identified as a *Draw Relative Vector* instruction. For the Y co-ordinate the top three bits are zero so that the intensity (i.e. the color) of the line is invisible. Any value at all in these three bits (between 0 and 7) will make the line progressively more visible to the player.

```
YVGVCT: LDY VGY       ; PUT CUR POS IN VGLIST IN Y
        LDA SZL       ; PUT Y-COORDINATE LOW BYTE IN A.
        STA NY,VGLIST ; STORE Y-COORDINATE LOW BYTE IN VGLIST.
        INY           ; INCREMENT CUR POS IN VG LIST.
        STA CURNTY    ; STASH LOW BYTE IN CURNTY.
        LDA SZH       ; PUT Y-COORDINATE HIGH BYTE IN A.
        STA CURNTY+1  ; STASH HIGH BYTE IN CURNTY.
        ; MAKE SURE TOP 3 BITS ARE ZERO SO THAT THIS IS A
        ; 'DRAW RELATIVE VECTOR' INSTRUCTION.
        AND I,1F      ; CLAMP HIGH BYTE TO LAST 5 BITS.
        STA NY,VGLIST ; STORE Y-COORDINATE HIGH BYTE IN VGLIST.
        INY           ; INCREMENT CUR POS IN VG LIST.
        LDA SXL       ; PUT X-COORDINATE LOW BYTE IN A.
        STA NY,VGLIST ; STORE X-COORDINATE LOW BYTE IN VGLIST.
        INY           ; INCREMENT CUR POS IN VG LIST.
        STA CURNTX    ; STASH LOW BYTE IN CURNTX.
        LDA SXH       ; PUT X-COORDINATE HIGH BYTE IN A.
        STA CURNTX+1  ; STASH HIGH BYTE IN CURNTX.
        ; MAKE SURE TOP 3 BITS ARE ZERO SO THAT THE LINE
        ; HAS 'ZERO INTENSITY', I.E. NO COLOR.
        AND I,1F      ; CLAMP HIGH BYTE TO LAST 5 BITS.
        STA NY,VGLIST ; STORE X-COORDINATE HIGH BYTE IN VGLIST.
        INY           ; INCREMENT CUR POS IN VG LIST.
        STY VGY       ; STORE CUR POS IN VG LIST.
        RTS           ; WE'RE DONE - RETURN.
```

With this, our current list of opcodes in VGLIST now looks like this.

Vector Description	Op Code	Bitmap
Set Scale	0x7100	01110001 00000000
Set Color to Green	0x6805	01101000 00001001
Center	0x8040	10000000 01000000
Draw Relative Vector (Blank)	0x0021 0x0034	00000000 00010001 00000000 00111000

Just to reiterate: we still haven't drawn any part of our line yet. We have simply moved our pen to the origin of the spike line we are about to draw, indicated by the green dot below.



So our next step is to draw a line from this origin. The routine TIPACT, which is responsible for drawing both the line and the item at its tip, starts out by figuring out the midway point between the web lines. With this it will be able to project a line from the furthest point of the screen to the nearest.

```
        .SBTTL  DISPLAY - ENEMY LINES (TIP STUFF)
TIPACT: LDX INDEX2    ; PUT INDEX TO CURRENT LINE IN X.
        LDA X,LINEY   ; GET Y POS OF CURRENT LINE.
        STA PYL       ; STORE IN PYL.
        ; CALCULATE NEAR PT.
        LDA X,LINEXM  ; GET X COORD OF MIDWAY PT BETWEEN LINES.
        STA PXL       ; STORE IN PXL.
        LDA X,LINEZM  ; GET Y COORD OF MIDWAY PT BETWEEN LINES..
        STA PZL       ; STORE IN PZL.
        JSR WORSCR    ; CALCULATE SXL, SYL, SZL: NEAR POINT OF LINE.
```

This is an involved business performed by the WORSCR routine in the last line above. It is so long and complicated that I have not attempted to annotate it here. For our purposes, we just need to know that it uses the values in PXL, PYL, and PZL to perform this calculation (this is origin) and the end result is captured in the SXL/SXH and SZL/SZH high/low byte pairs (this is the endpoint of the line).

```
WORSCR: LDA PYL
        SEC
        SBC EYL
        STA MXPL
        LDA I,0
        SBC EYH
        STA MXPH
        IFMI
        LDA I,0
        STA MXPH
        LDA I,1
        STA MXPL
        ENDIF
        LDA PZL
        CMP EZL
        IFCS
        SBC EZL
        LDX I,0
        ELSE
        LDA EZL
        SEC
        SBC PZL
        LDX I,-1
        ENDIF
        STA MZLH
        STA MSZXD
        STX MTEMP+2
        LDA PXL
        CMP EXL
        IFCS
        SBC EXL
        LDX I,0
        ELSE
        LDA EXL
        SEC
        SBC PXL
        LDX I,-1
        ENDIF
```

```
        STA MTEMP+1
        STX MTEMP+3
        BEGIN
        BIT MSTAT
        PLEND
        LDA MYLOW
        STA SZL
        LDA MYHIGH
        STA SZH
        LDA MTEMP+1
        STA MZLH
        STA MSZXD
        LDA MTEMP+2
        IFPL
        LDA SZL
        CLC
        ADC ZADJL
        STA SZL
        LDA SZL+1
        ADC ZADJL+1
        IFVS
        LDA I,0FF
        STA SZL
        LDA I,7F
        ENDIF
        STA SZL+1
        ELSE
        LDA ZADJL
        SEC
        SBC SZL
        STA SZL
        LDA ZADJL+1
        SBC SZL+1
        IFVS
        LDA I,0
        STA SZL
        LDA I,80
        ENDIF
```

```
        STA SZL+1
        ENDIF
        BEGIN
        BIT MSTAT
        PLEND
        LDA MYLOW
        STA SXL
        LDA MYHIGH
        STA SXH
        LDX MTEMP+3
        IFPL
        LDA SXL
        CLC
        ADC XADJL
        STA SXL
        LDA SXL+1
        ADC XADJL+1
        IFVS
        LDA I,0FF
        STA SXL
        LDA I,7F
        ENDIF
        STA SXL+1
        RTS
        ENDIF
        LDA XADJL
        SEC
        SBC SXL
        STA SXL
        LDA XADJL+1
        SBC SXL+1
        IFVS
        LDA I,0
        STA SXL
        LDA I,80
        ENDIF
        STA SXL+1
        RTS
```

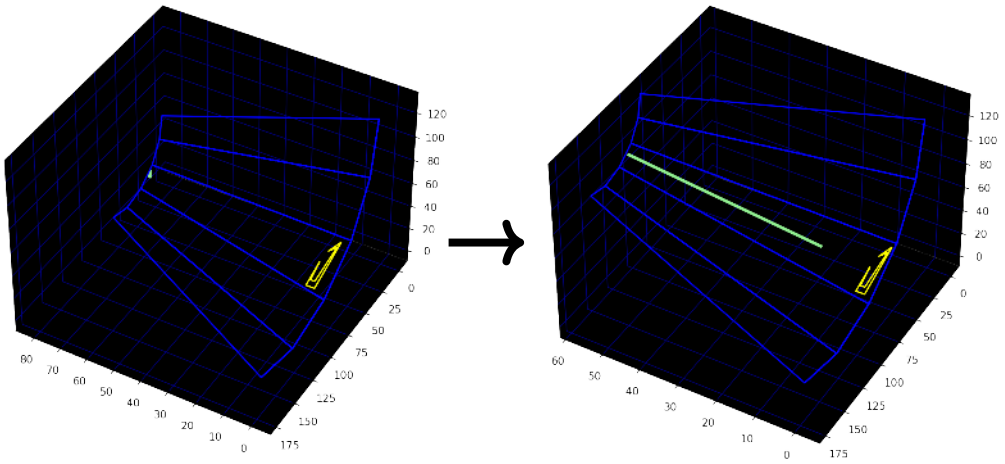
With these in hand, i.e. the co-ordinates of our near point, we can now draw our line vector. As before, this means calculating the values for our *Draw Relative Vector* instruction.

```
FCONNEC:LDY VGY         ; STORE THE CURRENT POS IN VGLIST IN Y.
        LDA SZL         ; GET THE LOW BYTE OF THE CALCULATED Y POSITION.
        SEC             ; SET CARRY.
        SBC CURNTY      ; SUBTRACT THE LOW BYTE OF THE ORIGIN.
        STA NY,VGLIST   ; STORE IN THE VGLIST.
        INY             ; INCREMENT POSITION IN THE VGLIST.
        LDA SZH         ; GET THE HIGH BYTE OF THE CALCULATED Y POSITION.
        SBC CURNTY+1    ; SUBTRACT THE HIGH BYTE OF THE ORIGIN.
        AND I,1F        ; CLAMP TO THE LOWER 5 BITS (SO INTENSITY NOT SET).
        STA NY,VGLIST   ; STORE IN THE VGLIST.
        INY             ; INCREMENT POSITION IN THE VGLIST.
        LDA SXL         ; GET THE LOW BYTE OF THE CALCULATED X POSITION.
        SEC             ; SET CARRY.
        SBC CURNTX      ; SUBTRACT THE LOW BYTE OF THE ORIGIN.
        STA NY,VGLIST   ; STORE IN THE VGLIST.
        INY             ; INCREMENT POSITION IN THE VGLIST.
        LDA SXH         ; GET THE HIGH BYTE OF THE CALCULATED Y POSITION.
        SBC CURNTX+1    ; SUBTRACT THE HIGH BYTE OF THE ORIGIN.
        AND I,1F        ; CLAMP TO THE LOWER 5 BITS (SO INTENSITY NOT SET).
        ORA I,0A0       ; MAKE SURE THE LINE IS VISIBLE BY SETTING INTENSITY
    TO A0..
        STA NY,VGLIST   ; STORE IN THE VGLIST.
        INY             ; INCREMENT THE POSITION IN THE VGLIST.
        STY VGY         ; STORE THE NEW CURRENT POSITION IN THE VGLIST.
        RTS
```

And now our updated list of opcode instructions looks as follows. Note that we make the line visible by setting the topmost three bits in the second-last byte.

Vector Description	Op Code	Bitmap
Set Scale	0x7100	01110001 00000000
Set Color to Green	0x6805	01101000 00001001
Center	0x8040	10000000 01000000
Draw Relative Vector (Blank)	0x0021 0x0034	00000000 00010001 00000000 00111000
Draw Relative Vector (Green)	0x0005 0xA007	00000000 00001001 10100000 00000111

And this is what the instructions will generate on the screen:



Our next step is to add a 'white dot' to end of the line.

```
WHITIP: LDY VGY         ; PUT CUR POS IN VGLIST IN Y
        LDA I,WHITE     ; STORE 'WHITE VALUE' IN A.
        STA NY,VGLIST   ; STORE WHITE COLOR VALUE IN VGLIST.
        INY             ; INCREMENT CUR POS IN VGLIST.
        LDA I,68        ; STORE 'SET COLOR' OPCODE IN A.
        STA NY,VGLIST   ; STORE 'SET COLOR' OPCODE IN VGLIST.
        INY             ; INCREMENT CUR POS IN VGLIST.
        LDA JSRDOT      ; GET LOW BYTE OF 'DOT' ROUTINE.
        STA NY,VGLIST   ; STORE IN VGLIST.
        INY             ; INCREMENT CUR POS IN VGLIST.
        LDA JSRDT1      ; GET HIGH BYTE OF 'DOT' ROUTINE.
        STA NY,VGLIST   ; STORE IN VGLIST.
        INY             ; INCREMENT CUR POS IN VGLIST.
        STY VGY         ; UPDATE CUR POS IN VGLIST.
```

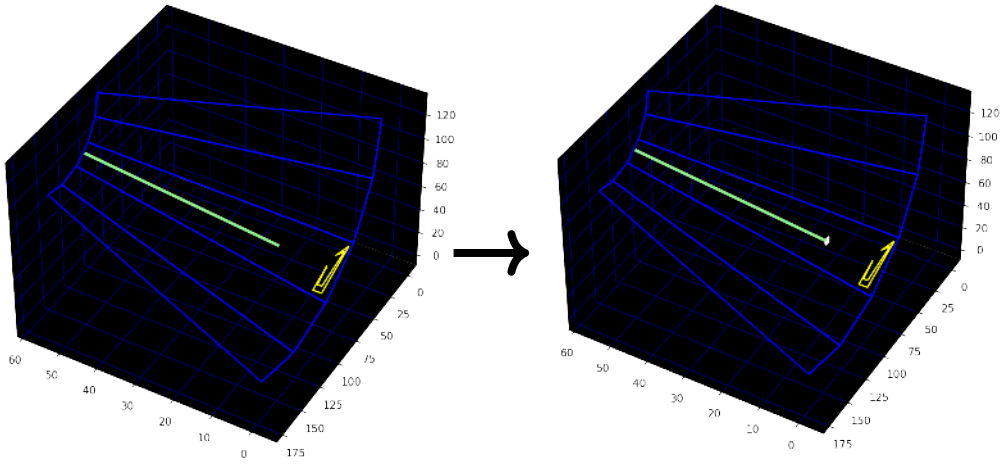
This adds two new opcodes to the `VGLIST`. One to set the pen color to white, and another that jumps to a subroutine that will draw our white dot. This 'white dot' routine is as simple as the following:

```
JADOT: VCTR 0,0,CB
       RTSL
```

Our final list of opcodes in the `VGLIST` reads as follows:

Vector Description	Op Code	Bitmap
Set Scale	0x7100	01110001 00000000
Set Color to Green	0x6805	01101000 00001001
Center	0x8040	10000000 01000000
Draw Relative Vector (Blank)	0x0021 0x0034	00000000 00010001 00000000 00111000
Draw Relative Vector (Green)	0x0005 0xA007	00000000 00001001 10100000 00000111
Set Color to White	0x6800	01101000 00000000
Jump to Subroutine (JSRL JADOT)	0xAD7F	10101101 01111111

And this is how it looks on screen:



rotary club

Sprinkled throughout `yak.s`, the principal source code file for *Tempest 2000*, we find references to a never-produced 'rotary controller'. This was a device that would emulate the controller present in the coin-operated original of Tempest in 1981.

Towards the end of the `Frame` interrupt routine we find the main logic used to detect and, if a rotary controller is present, detect its input:

```
        ; Check for joypad input. The logic for rotary controllers
        ; was added to support the potential release of a rotary
        ; controller for the Jaguar, but this never happened.
        ; So all rotary controller logic in Tempest 2000 is unused.
        tst roconon             ; Is Rotary Controller enabled?
        bne roco                ; Yes, do the rotary controller.
        jsr dopad               ; No, check for normal joypad input.
        bra exxit               ; We're done - so exit interrupt.

        ; Logic for reading input from the never-built and never-
        ; released rotary controller. We only look for button input
        ; every 8 interrupts.
roco:   move pitcount,d1        ; Get the rotary input interval counter.
        and #$07,d1             ; Do a modulus 7 on the counter.
        bne rotonly             ; If non-zero, read the rotary control only.
        jsr dopad               ; Otherwise, look for button presses too.
rotonly:
        add #1,pitcount         ; Increment the interval counter.
        bsr readrotary          ; Read input from the rotary controller.
```

You might wonder how it was possible to test a rotary controller that doesn't exist. The answer to this is that you make one of your own. In this case, Yak adapted a an Atari

drive controller marketed for the antiquated Atari 2600 game console and used it as a substitute for the rotary input promised by the Jaguar hardware team.



The humble 2600 Atari drive controller.

Once we've decided we're reading from this hacked-up rotary controller our first task is to hide it behind some secret button presses on the options screen.

```
selector:
        cmp.l #option2,the_option ; Are we on the 'Options' screen?
        bne selector2             ; If not, skip.
        btst.b #6,sysflags        ; Are controllers enabled?
        bne sopt2                 ; If not, skip.
        btst.b #4,pad_now         ; Pause pressed by Player 1?
        beq selector2             ; If not, skip.
        btst.b #4,pad_now+4       ; Pause pressed by Player 2?
        beq selector2             ; If not, skip.
        bset.b #6,sysflags        ; Enable controllers.
        jsr sayex                 ; Say 'Excellent!'.
```

If we have 2 controllers and both press the pause button it enables us to select the controller type and enable the rotary controller:



Selecting and enabling the rotary controller.

Next we have to figure out what it's telling us. For this, and for the ordinary joypad controller too, we inspect a 4-byte phrase (JOYIN) that tells us what buttons or dials have been pressed most recently. These 4 bytes are a bitmap: if a bit is set it tells us a specific button has been pressed.

Button	Hex	Bitmap
abutton	20000000	00100000000000000000000000000000
pausebutton	10000000	00010000000000000000000000000000
bbutton	02000000	00000010000000000000000000000000
right	00800000	00000000100000000000000000000000
left	00400000	00000000010000000000000000000000
down	00200000	00000000001000000000000000000000
up	00100000	00000000000100000000000000000000
seven	00080000	00000000000010000000000000000000
four	00040000	00000000000001000000000000000000
one	00020000	00000000000000100000000000000000
asterisk	00010000	00000000000000010000000000000000
cbutton	00002000	00000000000000000010000000000000
optionbutton	00000200	00000000000000000000001000000000
two	00000080	00000000000000000000000010000000
five	00000040	00000000000000000000000001000000
eight	00000020	00000000000000000000000000100000
zerobutton	00000010	00000000000000000000000000010000
three	00000008	00000000000000000000000000001000
six	00000004	00000000000000000000000000000100
nine	00000002	00000000000000000000000000000010
hash	00000001	00000000000000000000000000000001

All controller buttons and the bit they set in the JOYIN bitmap.

readrotary does this job. It is only interested in a subset of the buttons. These are:

Button	Hex	Bitmap
right	00800000	00000000100000000000000000000000
left	00400000	00000000010000000000000000000000

But the rotary controller has everything in a slightly different position, so when we read JOYIN, we'll find the right and left movement bits 4 bits to the left like so:

Button	Hex	Bitmap
right	08000000	00001000000000000000000000000000
left	04000000	00000100000000000000000000000000

Thus readrotary's first task is to extract the most recently pressed buttons and get them into the register d0. It must then move everything 4 bits to the right. It does this as follows.

```
readrotary:
    btst.b #3,sysflags      ; Rotary controller enabled for Player 1?
    beq op2                 ; If not, skip to checking Player 2.
    move.l   #$f0fffffc,d1 ; d1 = Joypad data mask   (Player 1)
    moveq.l  #$ffffffff,d2 ; d2 = Cumulative joypad reading
    move.w   #$81fe,JOYOUT ; Ask for the most recent button presses.
    move.l   JOYIN,d0       ; Read joypad, pause button, A button
    or.l     d1,d0          ; Mask off unused bits
    ; Rotate d0 4 bits to the right so that the bits are where we
    ; expect them to be.
    ror.l    #4,d0          ; Shift everything 4 bits to the right.
    and.l    d0,d2          ; d2 = xxAPxxxx RLDUxxxx xxxxxxxx xxxxxxxx
    swap     d2             ; Swap position of the first 2 with last 2 bytes.
    move     d2,d0          ; Store result in d0.

    lea lstcon,a1           ; Point a1 at lstcon (last read from controller).
    lea roconsens,a2        ; Point a2 at roconsens.
    bsr rroco               ; Calculate the input movement and store in d0.
    add.l d0,rot_cum        ; Add result to the cumulative movement.
op2:
    ; We omit the duplicated logic for player 2.
    rts
```

Once we have a value to work with, the question is how we decide if it's telling us to move the claw right or left. What we will do is compare the current input with the last input reported by the controller and depending on whether it is before or after the input value's position in the `conseq` array move it right or left, respectively.

```
conseq: dc.b 0,1,3,2
```

We can capture some of these potential outcomes as follows.

Previous Value	Current Value	Result
1	0	Move Right
1	3	Move Left
3	1	Move Right
3	2	Move Left
2	3	Move Right
2	0	Move Left
0	2	Move Right
0	1	Move Left

It's not exactly intuitive why this is the case.

But anyway, here is the code that does the job.

```
rroco:
```

```
        rol.b #2,d0         ; Phase Bits to bottom of word
        and #$03,d0         ; Get juicy bits
        move (a1),d4        ; Put last value read (lstcon) in d4.
        cmp d4,d0           ; Compare to most recent read.
        beq decsens         ; Did not move, so exit early.

        move d0,(a1)        ; Store current read in lstcon.
        lea conseq,a0       ; Point to sequence values
        clr d5              ; Set d5 to 0.

        ; Search sequence values (conseq) for a match to the last value
        ; we read (lstcon).
slocate:
        cmp.b 0(a0,d5.w),d4 ; Get val using d5 as index into conseq.
        beq slocated        ; If it equals last value read (d4), exit loop.
        addq #1,d5          ; Otherwise, increment d5..
        bra slocate         ; .. and loop.

        ; d5 now points to the position in conseq matching the previous
        ; value read from the rotary controller. To determine whether
        ; to move left or right we now see if the currently read value
        ; is before (right) or after (left) the most recent value read
        ; from the controller.
slocated:
        subq #1,d5          ; Subtract 1 from d5.
        and #3,d5           ; Clamp to 0 - 7.
        cmp.b 0(a0,d5.w),d0 ; Use it as in index into conseq.
        beq rclaw_right     ; If val is equal to d0(curr read), move right.
        addq #2,d5          ; Add two to d5
        and #3,d5           ; Clamp to 0 - 7.
        cmp.b 0(a0,d5.w),d0 ; Use it as in index into conseq.
        beq rclaw_left      ; If val is equal to d0 (curr read), move left.
        clr.l d0            ; Clear d0.
        rts                 ; Return
```

space game reenactment society

It was called *ALIEN SPACE GAME*. It didn't test well. It was meant to be a kind of three-dimensional space invaders knock-off, if we can imagine what that might look like. And we definitely shouldn't.

According to Theuerer: "The initial gameplay was to be simply a 'First Person Space Invaders'. I got it up and running, but it wasn't much fun. Too much like Space Invaders (surprise surprise). So after the review they said do you want to kill it? Or have you got any other ideas?"

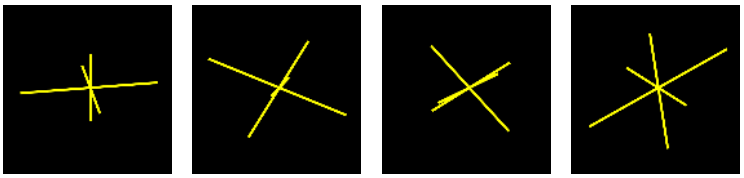
He did have another idea, it was *Tempest*, so he made that instead. But as we saw in 'things hidden' there is just enough wreckage of that initial concept in the source code for us to imagine what that first attempt at a game might have looked like.

Scattered here and there we find traces of the crime.

```
SPACG=0   ;SUPPRESS SPACE GAME CODE
```

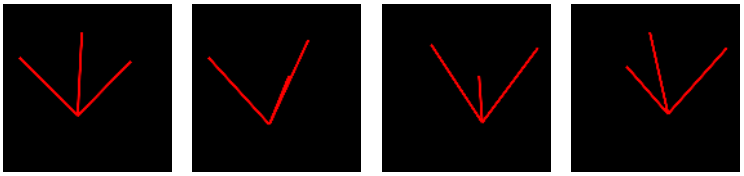
Well I'm sorry but the `SPACE GAME` refuses to be fully suppressed. We have already seen some of the intriguing enemies defined in `ALVROM.MAC`.

But there are other remnants to be found in there. We find the player's bullets:



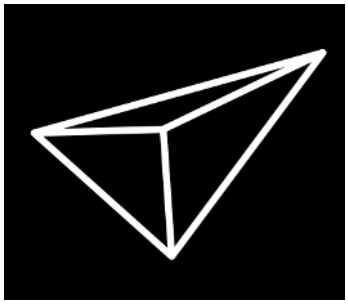
DSHTBL, DS2TBL, DS3TBL and DS4TBL in ALVROM.MAC.

And the 'spears' fired by the player's enemies:



ESHTBL, ES2TBL, ES3TBL and ES4TBL in ALVROM.MAC.

There is also an 'asteroid', which appears to have been some kind of bomb to be deployed when necessary.



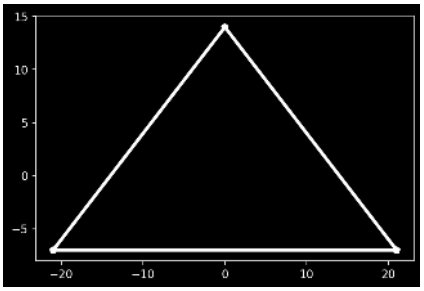
```
ASTTBL: .BYTE ENDAST-ASTTBL
        .BYTE -6,0,0
        .BYTE 3,5,4
        .BYTE 3,5,-4
        .BYTE 0,-5,0
```

ASTTBL in ALVROM.MAC.

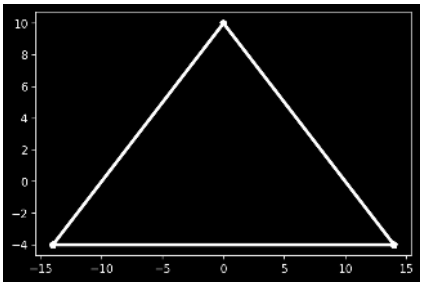
We find the definition of a primitive-looking 'fort', a simple pyramid shape that perhaps served as an obstacle that the player could hide behind to avoid enemy 'spears' and which was perhaps eventually degraded by enemy attack over time. To draw the forts a macro called FORTMAC was used to draw the same pyramidal shape with the initial dimension specified as a parameter FS.

```
.MACRO FORTMAC .FS
  VCTR .FS,-<.FS/3>,0
  VCTR -.FS,.FS,6
  VCTR -.FS,-.FS,6
  VCTR 2*.FS,0,6
  VCTR -.FS,<.FS/3>,0
  RTSL
.ENDM
```

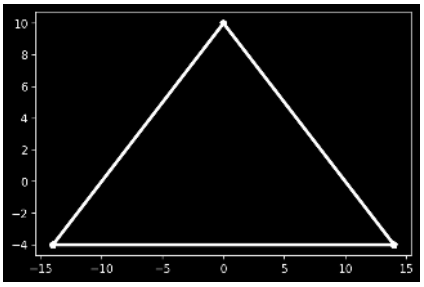
It appears this degradation was implemented by gradually reducing the size of the fort - as you can see by the values on the axis on the images below.



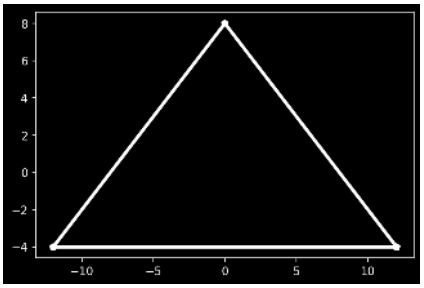
`FORTMAC 15` in `ALVROM.MAC`.



`FORTMAC 10` in `ALVROM.MAC`.

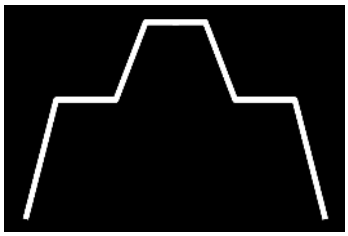


`FORTMAC 0E` in `ALVROM.MAC`.



`FORTMAC 0C` in `ALVROM.MAC`.

Finally we find the remains of the player's ship. Unfortunately it is something of a disappointment - so clearly little more than a placeholder.



GUNPIC in ALVROM.MAC.

```
GUNPIC: VCTR -8,0,6
        VCTR -8,-10,6
        VCTR -10,0,6
        VCTR -8,-18,6
        VCTR 50,0,0
        VCTR -8,18,6
        VCTR -10,0,6
        VCTR -8,10,6
        VCTR -8,0,6
        RTSL
```

There are other clues. We find the remnants of what must have been the game's playefield, a grid-like structure. which despite its name contain 10 vertical lines rather than just 5.

```
        .SBTTL DRAW TABLES:GRID LINES
VRT4DRW:
VRT5DRW:                        ;DRAW 5 VERT LINES
        TLABS 0
        SBRITE 040
        TVCTR 1
        SBRITE 0
        TVCTR 3
        SBRITE 040
        TVCTR 2
        SBRITE 0
        TVCTR 4
        SBRITE 040
        TVCTR 5
        SBRITE 0
        TVCTR 7
        SBRITE 040
        TVCTR 6
        SBRITE 0
        TVCTR 8
        SBRITE 040
        TVCTR 9
        OBJEND
```

If we put all these elements together we can make a wild and unfounded speculation as to how *ALIEN SPACE GAME* might have looked.



I probably shouldn't have even tried.

This is my deeply wonky attempt at visualising such a thing. Note that I've assumed that 'first person' space invaders must involve some kind of version of the classic space invaders playfield but with a three-dimensional depth of field: the baddies come at us slowly from the back of the screen, getting larger as they approach. Our player sits at the near edge of the field shooting along the grid that the enemies advance across. The 'forts' sit in front of the player and presumably can shield him from enemy attack. Meanwhile the asteroid is hanging out there in space in search of a purpose. Aren't we all, darling.

sexy yes

The sound effects in Tempest 2000 are stpred in the game as Pulse Code Modulation (PCM) data. This is the most common format for storing raw audio and flavors of it are used in everything from CDs to telephony. PCM data is as simple as a stream of numbers. Each of the numbers is between a lower and upper limit:, for example between -128 and 128. If you plot enough of these numbers along a graph you get what looks like a wave. This is pretty much the sound wave, the analog version of our digital values. If you put these values through the right piece of hardware, such as a Digital Analog Converter (DAC) you can get a sound out the other side.

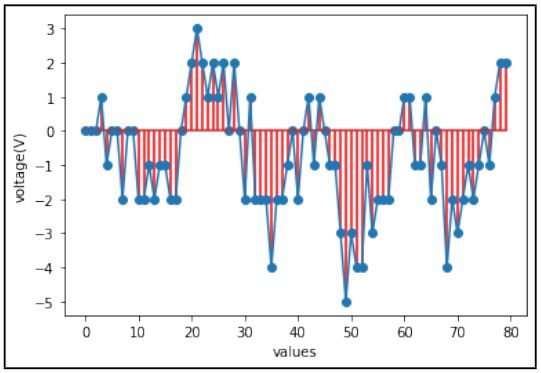


Figure .1: A sample of 80 bytes plotted as a wave graph.

The PCM data in Tempest 2000's sound samples is nothing more than this series of bytes, each one representing a value between -128 and 128, which can be converted

into sound. A single series of bytes will give us one 'channel' of sound, i.e. a mono sound sample. What we actually have are stereo sound samples so each sample contains two channels. The simplest way of incorporating two separate channels in the sample (one for the left speaker and one for the right) is to interleave them, in alternating bytes. We can see that this is the way the Tempest samples are encoded if we split out an arbitrary sample into separate streams of alternate bytes and graph them.

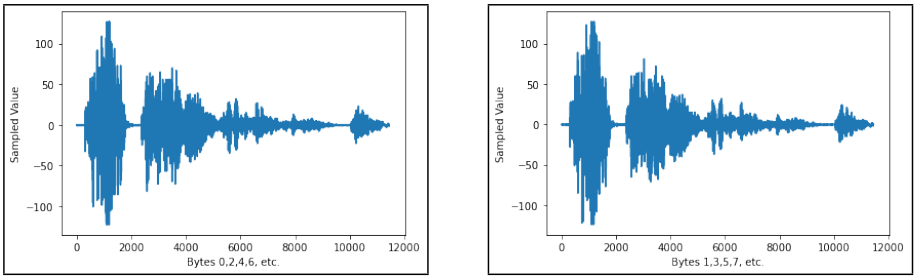


Figure .2: Plotted sound waves for the left and right channels respectively.

Since the sounds we want to play are just blobs of data, we need a way of knowing where these blobs are, and perhaps some information on what we have to do with them. This is provided by `samtab`, a block of data at address `$9ac800` that tells us everything we need to know:

```
   samtab       EQU $9ac800
```

Just byt itself, the `samtab` table reveals some interesting potential for ways that we can use the raw PCM bytes. Here it is in its raw form. With the bytes in `samtab` exactly as they are.

```
                      Prio-                                    Repeat     Repeat
 Name                 rity   Period Start      Length        Start      Length
---------------------- -----  ------ ---------  -------  --   ---------  -------  ---
| 'Engine Noise 1      ', $0001, $01ac, $009acd00, $0011a0, $00, $009acd02, $00119e, $00 |
| 'Player Shot Normal 2', $0002, $01ac, $009adea4, $0008e8, $00, $009adea4, $000000, $00 |
| 'Engine Noise        ', $0003, $01ac, $009ae290, $003378, $00, $009aee9e, $002594, $00 |
| 'Player Death        ', $0004, $00d6, $00000000, $00549a, $00, $00000000, $000000, $00 |
| 'Player Death 2      ', $0005, $01ac, $00000000, $002458, $00, $00000000, $000000, $00 |
| 'Player Shot Normal  ', $0006, $01ac, $009b160c, $0007a4, $00, $009b160c, $000000, $00 |
| 'Player Jump         ', $0007, $01ac, $009b1db4, $0018de, $00, $009b1db4, $000000, $00 |
| 'Crackle             ', $0008, $00d6, $009b3696, $004594, $00, $009b3696, $000000, $00 |
| 'Cleared Level       ', $0009, $01ac, $009b7c2e, $0037a2, $00, $009b7c2e, $000000, $00 |
| 'Warp                ', $000a, $0238, $009bb3d4, $006ec8, $00, $009bb3d4, $000000, $00 |
| 'Large Explosion     ', $000b, $01ac, $009c22a0, $0050c2, $00, $009c22a0, $000000, $00 |
| 'Powered Up Shot     ', $000c, $01ac, $009c7366, $001976, $00, $009c7366, $000000, $00 |
| 'Get Power Up        ', $000d, $01ac, $009c8ce0, $001aea, $00, $009c8ce0, $000000, $00 |
| 'Tink For Spike      ', $000e, $00fe, $009ca7ce, $00040e, $00, $009ca7ce, $000000, $00 |
| 'NME At Top Of Web   ', $000f, $01ac, $009cabe0, $00001e, $00, $009cabe0, $000000, $00 |
| 'Pulse For Pulsar    ', $0010, $0358, $009cac02, $0019fe, $00, $009cac02, $000000, $00 |
```

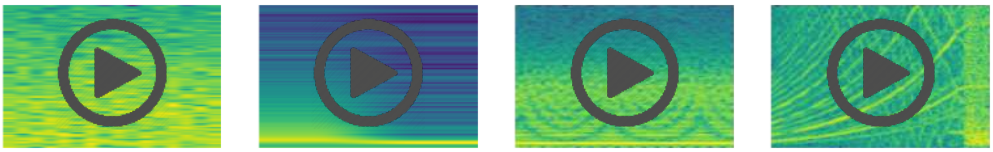
```
| 'Normal Explosion    ', $0011, $00d6, $009cc604, $002ab6, $00, $009cc604, $000000, $00  |
| 'Extra Explosion     ', $0012, $0358, $009cf0be, $0018ca, $00, $009cf0be, $000000, $00  |
| 'Static or Pulsar    ', $0013, $011c, $009d098c, $003fe4, $00, $009d098c, $000000, $00  |
| 'Pulsar Pulse        ', $0014, $0358, $009d4974, $000f0c, $00, $009d4974, $000000, $00  |
| 'Off Shielded NME    ', $0015, $00aa, $009d5884, $0027ca, $00, $009d5884, $000000, $00  |
| 'Excellent           ', $0016, $0200, $009d8052, $005976, $00, $009d8052, $000000, $00  |
| 'Superzapper Recharge', $0016, $0200, $009dd9cc, $00a958, $00, $009dd9cc, $000000, $00  |
| 'yes                 ', $0018, $0200, $009e8328, $005a6c, $00, $009e832a, $005a6a, $00  |
| 'oneup               ', $0019, $0200, $009edd98, $0043ae, $00, $009edd98, $000000, $00  |
| 'screeeam            ', $001a, $0200, $009f214a, $004568, $00, $009f214a, $000000, $00  |
| 'sexy yes 1          ', $001b, $0200, $009f66b6, $002c54, $00, $009f66b6, $000000, $00  |
| 'sexy yes 2          ', $001c, $0200, $009f9362, $003236, $00, $009f9362, $000000, $00  |
| 'tink                ', $001e, $0200, $009fc59c, $0005ce, $00, $009fc59c, $000000, $00  |
| 'zero                ', $001f, $0200, $009fcb6e, $000008, $00, $009fcb6e, $000000, $00  |
| 'dummy               ', $0020, $0200, $009fcb7a, $00a1d8, $00, $009fcb7a, $000000, $00  |
```

Listing .1: The contents of `samtab` at `$9AC800`.

And here is a 'normalized' version of the table, with appropriate values converted to decimal for convenience.

Name	Priority	Period	Start	Length	Repeat Start	Repeat Length
'Engine Noise 1'	1	428	$009acd00	4512	$009acd02	4510
'Player Shot Normal 2'	2	428	$009adea4	2280	$009adea4	0
'Engine Noise'	3	428	$009ae290	13176	$009aee9e	9620
'Player Death'	4	214	$00000000	21658	$00000000	0
'Player Death 2'	5	428	$00000000	9304	$00000000	0
'Player Shot Normal'	6	428	$009b160c	1956	$009b160c	0
'Player Jump'	7	428	$009b1db4	6366	$009b1db4	0
'Crackle'	8	214	$009b3696	17812	$009b3696	0
'Cleared Level'	9	428	$009b7c2e	14242	$009b7c2e	0
'Warp'	10	568	$009bb3d4	28360	$009bb3d4	0
'Large Explosion'	11	428	$009c22a0	20674	$009c22a0	0
'Powered Up Shot'	12	428	$009c7366	6518	$009c7366	0
'Get Power Up'	13	428	$009c8ce0	6890	$009c8ce0	0
'Tink For Spike'	14	254	$009ca7ce	1038	$009ca7ce	0
'NME At Top Of Web'	15	428	$009cabe0	30	$009cabe0	0
'Pulse For Pulsar'	16	856	$009cac02	6654	$009cac02	0
'Normal Explosion'	17	214	$009cc604	10934	$009cc604	0
'Extra Explosion'	18	856	$009cf0be	6346	$009cf0be	0
'Static or Pulsar'	19	284	$009d098c	16356	$009d098c	0
'Pulsar Pulse'	20	856	$009d4974	3852	$009d4974	0
'Off Shielded NME'	21	170	$009d5884	10186	$009d5884	0
'Excellent'	22	512	$009d8052	22902	$009d8052	0
'Superzapper Recharge'	22	512	$009dd9cc	43352	$009dd9cc	0
'yes'	24	512	$009e8328	23148	$009e832a	23146
'oneup'	25	512	$009edd98	17326	$009edd98	0
'screeeam'	26	512	$009f214a	17768	$009f214a	0
'sexy yes 1'	27	512	$009f66b6	11348	$009f66b6	0
'sexy yes 2'	28	512	$009f9362	12854	$009f9362	0
'tink'	30	512	$009fc59c	1486	$009fc59c	0
'zero'	31	512	$009fcb6e	8	$009fcb6e	0
'dummy'	32	512	$009fcb7a	41432	$009fcb7a	0

Turning the raw bytes of `samtab` into something we can read.

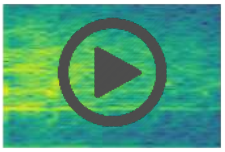


'Player Death'

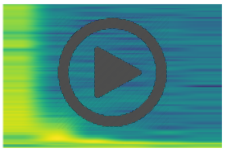
'Player Death 2'

'Engine Noise 1'

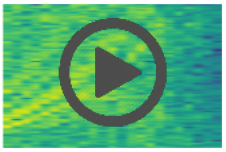
'Player Shot Normal 2'



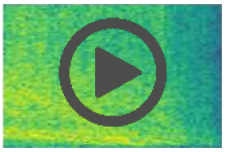
'Engine Noise'



'Player Shot Normal'



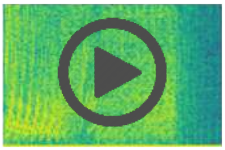
'Player Jump'



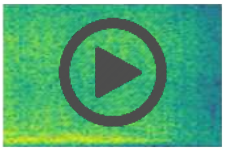
'Crackle'



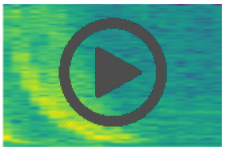
'Cleared Level'



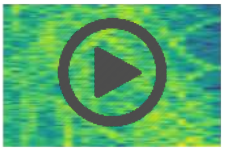
'Warp'



'Large Explosion'



'Powered Up Shot'



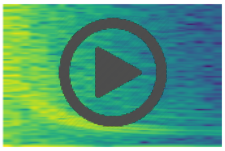
'Get Power Up'



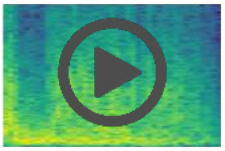
'Tink For Spike'



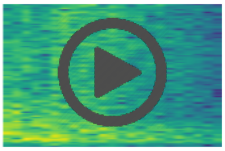
'NME At Top Of Web'



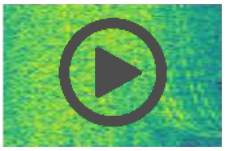
'Pulse For Pulsar'



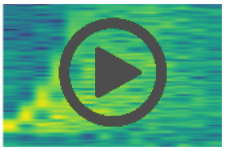
'Normal Explosion'



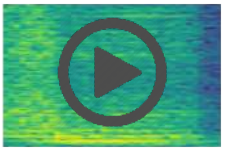
'Extra Explosion'



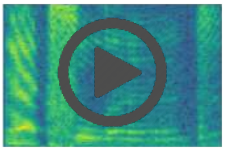
'Static or Pulsar'



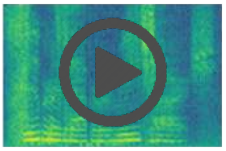
'Pulsar Pulse'



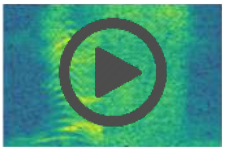
'Off Shielded NME'



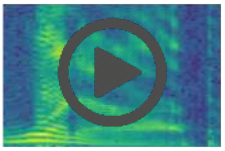
'Excellent'



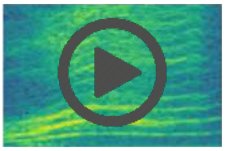
'Superzapper Recharge'



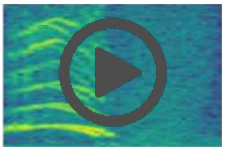
'yes'



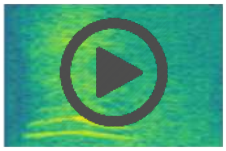
'oneup'



'screeeam'



'sexy yes 1'



'sexy yes 2'

All 28 sound samples and their spectrograms. If your reader supports it you can play the samples.

Of course the best way of seeing how this works is to follow one through in practice. Here is the little routine that shouts 'Excellent!' when you navigate the options menu. We can see that we do a little set up such as selecting the sample from the table above ('Excellent' is at position 21 ($16 in hex)), setting its pitch and then calling another routine called `fox` that inches nearer the actual work of playing the sample:

```
; ********************************************************************
; Say 'Excellent', e.g. when the user naviates to an option.
; ********************************************************************
sayex:
  move #21,sfx           ; Select effect $16 (21) from samtab, 'Excellent'.
  move #101,sfx_pri      ; Set the priority.
  move.l #$160,sfx_pitch ; Set the pitch.
  jsr fox                ; Play the effect (see below).
  move #101,sfx_pri      ; Do it again.
  move.l #$162,sfx_pitch ; This time with a different pitch.
  jmp fox                ; Play it.
```

`fox` (why waste an opportunity to pun on something in the animal kingdom) sets up our data for playing the sample. It's main preoccupation is turning our index into the `samtab` table to a pointer to the data there.

```
; ********************************************************************
; Play a selected sound sample.
; ********************************************************************
fox:
  movem.l d0-d3/a0,-(a7)  ; Save d0,d1,d2,d3,a0 to the a7 register.
                          ; Because we're going to clobber them here.
  move sfx,d0             ; Store sfx in d0, this is '$16' for 'Excellent'.

  ; Use '21' to point to the address of sample data, i.e. directly after
  ; the 'Excellent    ' in this entry in samtab:
  ; 'Excellent           ', $0016, $0200, $009d8052, $005976, $00,

  ; Since each entry in samtab is 40 bytes long, multiplying our index by
  ; 40 will bring us to the start of the 'Excellent' entry. The steps below
  ; are a fast way of doing this multiplication by 40.
  lsl #3,d0   ; Turn $16(21) into $a8 by left-shifting.
  move d0,d1  ; Store in d1.
  lsl #2,d0   ; Turn $a8 into $2a0 by left-shifting.
  add d1,d0   ; d0 is now $348 (840), i.e. 21 * 40.

  ; Now that we have an offset for our sample information in samtab, we'll
  ; add 20 to it (so that it's after the sample name) and store it in a0.
  lea samtab,a0           ; Store samtab address in a0.
  lea 20(a0,d0.w),a0      ; Add 840 + 20 to it.

  ; Now we can set up the last few items our sound synth needs.
  move sfx_pri,d1
  move sfx_vol,d2
  move.l sfx_pitch,d3
```

```
; Invoke the sound synth to play our sample.
jsr PLAYFX2

; Clean up.
move d0,handl
clr sfx_pri
clr sfx_vol
clr.l sfx_pitch

movem.l (a7)+,d0-d3/a0 ; Restore the old unclobbered values we saved off
                       ; at the start.
rts
```

To recap, at this point we have set up our registers as follows:

Register	Value	Description
a0	$9acb5c	Address of the 'Excellent' metadata in `samtab`.
d1	101	`sfx_pri`.
d2	$ff	`sfx_vol`: the volume to play at.
d3	$162	`sfx_pitch`: the pitch to play at.

These are the garnish for a choice piece of mystery meat, PLAYFX2. PLAYFX2 is a library routine provided to Atari by a crowd called 'Imagitec Designs'. It is not something we have the source code for, and nor did Minter when he was writing Tempest 2000. Instead, he was given a binary blob called `syn6.o` and a list of routine names to use when he wanted to play one of his samples. This list is given at the start of the main `yak.s` source file:

```
    INIT_SOUND     EQU  $4040  ;jump table for the SFX/Tunes module
    NT_VBL         EQU  $4046
    PT_MOD_INIT    EQU  $404c
    START_MOD      EQU  $4052
    STOP_MOD       EQU  $4058
    PLAYFX2        EQU  $405e
    CHANGE_VOLUME  EQU  $4064
    SET_VOLUME     EQU  $406a
    NOFADE         EQU  $4070
    FADEUP         EQU  $4076
    FADEDOWN       EQU  $407c
    ENABLE_FX      EQU  $4082
    DISABLE_FX     EQU  $4088
    CHANGEFX       EQU  $409a  ;new in syn6
    HALT_DSP       EQU  $408e
    RESUME_DSP     EQU  $4094
    intmask        EQU  $40a0
```

These are all entry points in the game binary for each routine. So for PLAYFX2 the entry point is $405e. In the source code this routine exists as raw machine code only. Along with all the other Imagitec sound routines it lives in a file called moomoo.dat:

```
    .include "moomoo.dat"
```

This is a slightly modified version of the syn6.o sound synthesizer binary provided by Imagitec Designs. With modern tooling, it is possible for us to recreate moomoo.dat by partially disassembling it, then reassembling and relinking it. This will more or less re-enact the build steps Minter would have followed to generate it.

The initial bytes of moomoo.dat can be disassembled as follows into a file we'll call moomoo_header.s.

```
; Header for moomoo.s
;.org $4040
.include "jaguar.inc"

init_sound    EQU $000041ba
nt_vbl        EQU $0000425e
pt_mod_init   EQU $000041d6
start_mod     EQU $000045e2
stop_mod      EQU $00004606
playfx2       EQU $00004370
change_volume EQU $000044ea
set_volume    EQU $00004546
nofade        EQU $00004598
fadeup        EQU $000045b2
fadedown      EQU $000045ca
enable_fx     EQU $0000464e
disable_fx    EQU $00004660
resume_dsp    EQU $00004470

INIT_SOUND:    jmp     (init_sound).l
NT_VBL:        jmp     (nt_vbl).l
PT_MOD_INIT:   jmp     (pt_mod_init).l
START_MOD:     jmp     (start_mod).l
STOP_MOD:      jmp     (stop_mod).l
PLAYFX2:       jmp     (playfx2).l
CHANGE_VOLUME: jmp     (change_volume).l
SET_VOLUME:    jmp     (set_volume).l
NOFADE:        jmp     (nofade).l
FADEUP:        jmp     (fadeup).l
FADEDOWN:      jmp     (fadedown).l
ENABLE_FX:     jmp     (enable_fx).l
DISABLE_FX:    jmp     (disable_fx).l
CHANGEFX:      jmp     (disable_fx).l
HALT_DSP:      jmp     (disable_fx).l
RESUME_DSP:    jmp     (resume_dsp).l
```

```
intmask:        dc.b 0
; Address 0x40A2
return_early:
                rts

; Address 0x40A4
update_interrupt:
                move.w  d0,-(sp)
                bset    #3,(intmask).l
                clr.w   d0
                move.b  (intmask).l,d0
                move.w  d0,(INT1).l ; RW CPU Interrupt Control Register
                move.w  (sp)+,d0
                rts
```

Listing .2: The contents of `moomoo_header.s`.

It turns out that the address we're using to invoke PLAYFX2 simply jumps to another adress in the binary, one that contains the actual `playfx2` routine:

```
PLAYFX2:        jmp     (playfx2).l
```

So to recreate the steps used to generate `moomoo.dat` we first assemble this file:

```
rmac -fa -i moomoo_header.s -o moomoo_header.o
```

Then we link `moomoo_header.o` with the `syn6.o` file provided by Imagitec:

```
rln -n -z -u -v -a 4040 xd xd -e moomoo_header.o syn6.o -o moomoo.dat
```

Voila, we have our `moomoo.dat`. If you'd like to follow these steps yourself, you can use this Python notebook.

story of a beep

Whenever the cursor is moved left or right, *Tempest* emits a beep. This is the story of that beep. To tell the story we will start at the end rather than the beginning.

To magic up this beep, we have to emit a series of commands to the Atari cabinet's sound chip, known as the POKEY. These commands are as follows:

Register	Description	Value	Bits	Meaning
AUDC1	Channel 1 Control	A2	10110010	Rectangular curve, volume 2
AUDF1	Channel 1 Frequency	0F	00001111	High pitched sound

Then after a short interval, we follow up with the commands to stop the sound:

Register	Description	Value	Bits	Meaning
AUDC1	Channel 1 Control	00	00000000	Stop
AUDF1	Channel 1 Frequency	00	00000000	No sound

It's as simple as that. But since we will want to generate more complex sounds later, we have to develop some machinery that's capable of emitting more than just a single beep. So it will be instructive, with this minimally simple example, to see how this machinery generates the above commands, which consist of little more than telling the sound chip to play a beep and then stop. We find our sound definitions and the

machinery underlying them in ALSOUN.MAC.

So below is the data driving our 'cursor crossing a line' beep. There are two sequences of bytes. The first defines the values and the timing for writing those values to the AUDF1 register. Writing something to AUDF1 is like playing a note. The value we write determines the note that will be played.

The second sequence defines the way that note is played. We can manipulate the performance of the note by writing to the AUDC1 register. Depending on the value we write we can alter the volume and the distortion applied to the note we selected.

```
LO5F:   .BYTE 0F,4,0,1   ; CURSOR CROSSED A LINE
        .BYTE 0,0
LO5A:   .BYTE 0A2,4,40,1
        .BYTE 0,0
```

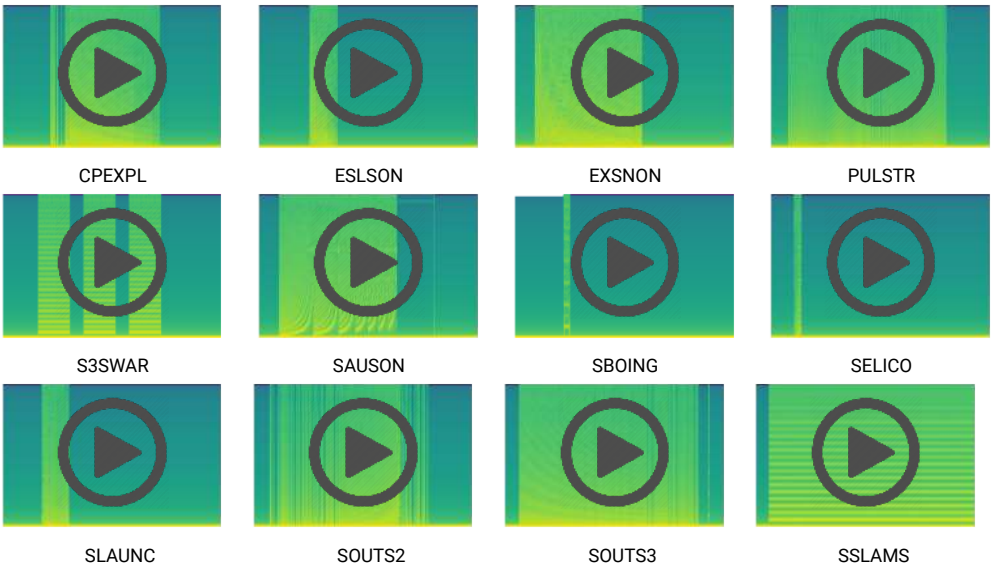
It will be easier to understand the above sequence definition if we comment the purpose of each byte in the sequence:

```
LO5F:   .BYTE 0F  ; VALUE TO WRITE TO AUDF1
        .BYTE 4   ; BEATS TO WAIT BEFORE NEXT CHANGE
        .BYTE 0   ; AMOUNT TO CHANGE BY
        .BYTE 1   ; NUMBER OF TIMES TO CHANGE (1 MEANS 0)
        .BYTE 0   ; RESTART POSITION (0 MEANS NO RESTART)
        .BYTE 0   ; STOP

LO5A:   .BYTE 0A2 ; VALUE TO WRITE TO AUDC1
        .BYTE 4   ; BEATS TO WAIT BEFORE NEXT CHANGE
        .BYTE 40  ; AMOUNT TO CHANGE BY
        .BYTE 1   ; NUMBER OF TIMES TO CHANGE (1 MEANS 0)
        .BYTE 0   ; RESTART POSITION (0 MEANS NO RESTART)
        .BYTE 0   ; STOP
```

Close inspection of the above example shows that the commands we set out on the previous page are the result of following the instructions of each of these byte sequences. For the LO5F sequence, which controls what we write to the AUDF1 register, we write the following commands to turn on the note, wait 4 frames, and then turn it off again:

Register	Description	Value	Bits	Meaning
AUDF1	Channel 1 Frequency	0F	00001111	High pitched sound
AUDF1	Channel 1 Frequency	00	00000000	No sound



All 28 sound samples and their spectrograms. If your reader supports it you can play the samples.

Simultaneously the `L05A` sequence instructs us to set the volume and the sound type by writing to the `AUDC1` register and then, after 4 frames, silence it again:

Register	Description	Value	Bits	Meaning
`AUDC1`	Channel 1 Control	`A2`	`101,10010`	Rectangular curve, volume 2
`AUDC1`	Channel 1 Control	`00`	`00000000`	Stop

We can look at a slightly more involved example to get an idea of the manipulation that this scheme affords us. Here are the two sequences that define an 'Enemy Shot' (`ESLSON`):

```
        ; ENEMY SHOT
ES8F:   .BYTE 0   ; VALUE TO WRITE TO AUDF1
        .BYTE 3   ; BEATS TO WAIT BEFORE NEXT CHANGE
        .BYTE 2   ; AMOUNT TO CHANGE BY
        .BYTE 9   ; NUMBER OF TIMES TO CHANGE (1 MEANS 0)
        .BYTE 0   ; RESTART POSITION (0 MEANS NO RESTART)
        .BYTE 0   ; STOP

ES8A:   .BYTE 8   ; VALUE TO WRITE TO AUDC1
        .BYTE 3   ; BEATS TO WAIT BEFORE NEXT CHANGE
        .BYTE 0FF ; AMOUNT TO CHANGE BY (0FF means -1)
```

```
        .BYTE 9   ; NUMBER OF TIMES TO CHANGE (1 MEANS 0)
        .BYTE 0   ; RESTART POSITION (0 MEANS NO RESTART)
        .BYTE 0   ; STOP
```

Notice that the commands are to be run simultaneously: every 3 frames. You can also notice that we are to run each command 8 times in total (9 - 1). We also note that `0FF` is another way of saying `-1`. So equipped with these provisions we can execute the commands as follows:

Register	Value	Meaning	Register	Value	Meaning
AUDC1	08	Volume 8	AUDF1	00	No Sound
AUDC1	07	Volume 7	AUDF1	02	Very Very Low Pitch
AUDC1	06	Volume 6	AUDF1	04	Very Low Pitch
AUDC1	05	Volume 5	AUDF1	06	Still Low Pitch
AUDC1	04	Volume 4	AUDF1	08	Low Pitch
AUDC1	03	Volume 3	AUDF1	0A	Getting Higher
AUDC1	02	Volume 2	AUDF1	0C	Higher Still
AUDC1	01	Volume 1	AUDF1	0F	Higher again

play us a tune pal

Music in *Tempest 2000* comes in the form of `mod` files. The mechanics of processing these files are hidden from us in a blob of bytes provide by Imagitec Ltd and included in the game binary. This blob of bytes is a sound synthesiser that transforms the contents of a `mod` file into a tune that you can hear with your ears. What we can do instead is dig into the contents and structure of a mod file to understand how it stores a tune. The obvious candidate for this research is the music played over the title screen.

I'm not sure if this music ever had a proper title, however the source code knows it fondly as `tune13.mod`. This file, along with all the other tunes included in the game, are stored in a reference table called `modbase`. This allows us to pick a tune for playing by referencing it using an index into the table.

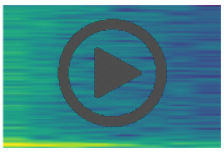
```
modbase      EQU $8d6800
modtable:
.dc.l modbase + $100   ; tune13.mod
.dc.l modbase + $18d2c ; tune7.mod
.dc.l modbase + $37a04 ; tune1.mod
.dc.l modbase + $5bd7c ; tune3.mod
.dc.l modbase + $80b22 ; rave4.mod
.dc.l modbase + $99d72 ; tune5.mod
.dc.l modbase + $b0a50 ; tune12.mod
.dc.l 0
```

In the little routine that plays a selected tune we can see this happening in practice. The index value for the selected tune is passed in the `d0` register and used as an index into `modbase` to retrieve the address in memory of the tune we've picked to play.

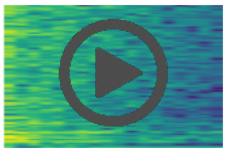
```
playtune:
        jsr STOP_MOD                ; Stop any current tune.
        lsl #2,d0                   ; Multiply index (d0) by 4
        lea modbase,a0              ; Point a0 at modbase.
        move.l 0(a0,d0.w),a0        ; Get tune base using index.
        jsr PT_MOD_INIT             ; Point the MOD player at a0.
        move.b vols,d0              ; Store the volume in d0.
        and.l #$ff,d0               ; Keep it between 0 and 255
        clr d1                      ; Clear d1
        jsr SET_VOLUME              ; Actually set the volume.
        move.l d0,vset              ; Store the selected volume.
        jsr NOFADE                  ; Turn off 'fading'.
        jmp START_MOD               ; Start playing the tune.
```

So to play the title screen tune, which is the first entry in `modtable`, we point the sound synthesiser at the region in memory containing `tune13.mod` by calling its black-box routine `PT_MOD_INIT` and then once we've set the volume, instruct it to start playing the darn thing (`jmp START_MOD`).

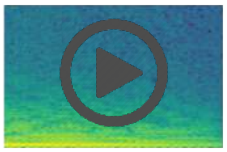
To understand the contents of `tune13.mod` and how it gives us a tune, it is best to start with the most basic building blocks it contains. These are 11 raw sound samples of Pulse Code Modulation (PCM) data like the ones we discussed in 'sexy yes'. They are stored at the very end of the file, one after another from byte 9276 to byte 101416. This means they constitute the bulk of the `tune13.mod` tune file - which is not surprising since they are the part of the file that carries all the sounds. If you want to hear what each sample sounds like, your reader might support clicking on each sample to play it.



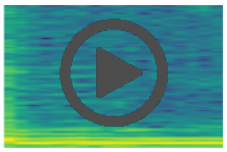
Sample 1



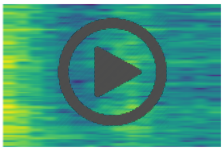
Sample 2



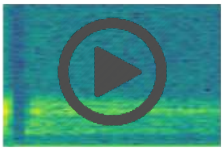
Sample 4



Sample 5



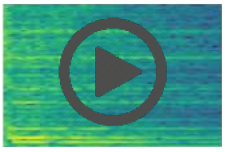
Sample 6



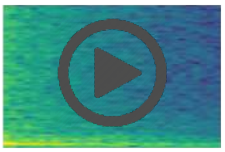
Sample 7



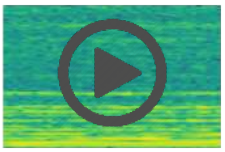
Sample 8



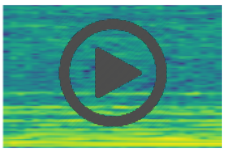
Sample 10



Sample 11



Sample 12



Sample 13

All 11 sound samples and their spectrograms. If your reader supports it you can play the samples.

If you try playing the files you'll notice that the majority of them are single notes on an instrument. This gives us a clue as to what the rest of the `tune13.mod` file might contain: instructions on how to play these notes in the correct order. More than this, the instructions might even tell us what pitch to play each note at, for example, whether a C sharp or a B flat.

We find these instructions at byte 1084 in the file and they occupy all the space in the file up to the beginning of the samples at byte 9276. The instructions are broken up into chunks called 'Patterns'. Each 'Pattern' describes a passage of music, using the samples to play notes across four separate audio channels at once. Another way of thinking of a 'Pattern' is like a piece of sheet music that describes the notes to play and the instrument to play them for four different players. In this case our instruments are the 'samples'. To give you an idea of what I mean here is the first note for all four players (which we call channels) given in the first 16 bytes of the data for the first of the seven patterns in `tune13.mod`.

	Bytes	Sample	Note	Effect
Channel 1	`007f1000`	1	A3	
Channel 2	`01944f09`	4	C#2	Set Speed 09
Channel 3	`011d5000`	5	G2	
Channel 4	`00000000`			

Decoding the first 16 bytes (4 per channel) in Pattern 000.

One the following page we see the complete decoded set of instructions contained in Pattern 000. As you look through it you can get a sense of the note being played on each channel and the instrument used, along with any special instruction (effect) specified.

Channel 1			Channel 2			Channel 3			Channel 4		
Sample	Note	Effect	Sample	Note	Effect	Sample	Note	Effect	Sample	Note	Effect
1	A3		4	C#2	Set Speed 09	5	G2				
8	A#3					5	G2	Set Volume 15			
6	A3					5	G2				
6	A3					5	G2	Set Volume 25			
8	A#3										
8	A#3					5	G2	Set Volume 15			
6	A3					5	G2				
1	A3					5	G2	Set Volume 25			
2	D3										
8	B3					5	G2	Set Volume 15		G2	
8	B3					5	G2		10	D3	
6	A3					5	G2	Set Volume 25	10	G2	
0									10	D3	
8	B3					5	G2	Set Volume 15	10	G2	
6	A3					5	G2		10	D#3	
1	A3					5	G2	Set Volume 25	10	D3	
1	A3										
8	B3					5	G2	Set Volume 15			
6	A3					5	G2				
6	A3					5	G2	Set Volume 25			
8	A#3										
8	A#3					5	G2	Set Volume 15			
6	A3					5	G2				
1	A3					5	G2	Set Volume 25			
2	D3										
0						5	G2	Set Volume 15			
6	A3					5	G2			F2	
6	A3					5	G2	Set Volume 25			
8	A#3										
8	A#3		4	D2		5	F2	Set Volume 15		F2	
6	A3					5	F2				
1	A3					5	F2	Set Volume 25			
1	A3		4	C#2						F2	
8	A#3					5	G2	Set Volume 15		G2	
6	A3					5	G2				
6	A3					5	G2	Set Volume 25			
8	A#3										
8	A#3					5	G2	Set Volume 15			
6	A3					5	G2				
1	A3					5	G2	Set Volume 25			
2	D3										
0						5	G2	Set Volume 15			
6	A3					5	G2			D3	
6	A3					5	G2	Set Volume 25		G2	
8	A#3									D3	
8	A#3					5	G2	Set Volume 15		G2	
6	A3					5	G2			D#3	
1	A3					5	G2	Set Volume 25		B2	
1	A3										
8	A#3					5	G2	Set Volume 15			
6	A3					5	G2				
6	A3					5	G2	Set Volume 25			
8	A#3										
8	A#3					5	G2	Set Volume 15			
6	A3					5	G2				
1	A3					5	G2	Set Volume 25			
2	D3										
0						5	G2	Set Volume 15			
6	A3					5	G2				
6	A3					5	G2	Set Volume 25			
8	A#3										
1	A3					5	G2	Set Volume 15			
6	A3					5	G2				
1	A3					5	G#2	Set Volume 25			

Decoding the full instructions for Pattern 000.

What if we attempted to turn the notes in Pattern 0 into some sheet music? It's just a question of representing each channel as a stave and transcribing the notes from the pattern. We can't display the sample that's being used to play the note, but we do get a sense of the music that's being played.



Pattern 0 rendered as sheet music.

With a single chunk of music like this we now have the makings of a tune, built up from using our samples in a series of machine-readable sheet music instructions in a pattern.

Our next step is to come up with as many of these patterns as we need to form a complete tune. In theory we could mix and match the patterns in any sequence we like and play them one after the other to form the finished article. In the case of `tune13.mod`, the structure is quite simple. At byte 952 in our `tune13.mod` file we list the order in which the patterns are to be played. This also lets us know how many patterns in total the file contains. The full list is: 0, 1, 2, 3, 4, 3, 6, 7, 5. With this information we can illustrate the title tune in its entirety by piecing together the transcriptions of each of the patterns.



Pattern 0



Pattern 1



Pattern 2



Pattern 3



Pattern 4



Pattern 3



Pattern 6



Pattern 7



Pattern 5

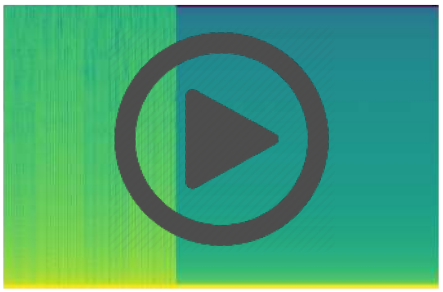
The patterns in the order that they're played, giving the complete tune.

unused explosions

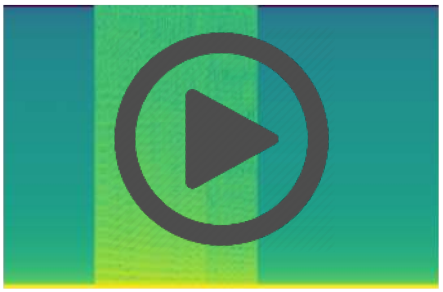
The *Tempest* source in `ALSOUN.MAC` contains a couple of unused sound definitions. The comment next to them suggests that they were originally intended as explosion effects.

```
;
;EXPLOSION SOUND
;
T51F:   .BYTE 0C0,8,4,10
        .BYTE 0,0
T51A:   .BYTE 0A6,20,0F8,4
        .BYTE 0,0
T52F:   .BYTE 40,8,4,10
        .BYTE 0,0
T52A:   .BYTE 0A6,20,0FE,4
        .BYTE 0,0
```

When you listen to them they do sound like that alright. Give them a try below.



T51F



T52F

The unused explosion sounds in `ALSOUN.MAC`.

ntsc pal

In the 1980s and 1990s games were played on cathode ray tube televisions. Like with everything, there were competing standards. And like with everything, Americans did things one way (NTSC) and Europe did things another (PAL). The rest of the world got to choose which horse to back.

There are many technical differences between NTSC and PAL, but only one that Jeff Minter really needed to worry about in *Tempest 2000*: how many lines of pixels each standard allowed. NTSC allowed a measly 241, while PAL televisions came with a princely 287.

Faced with releasing a video game that could be played on televisions in both America and Europe, the developer needed to figure out which of the two it was playing on and adjust the dimensions of the screen it was writing to accordingly.

When launched one of the earliest tasks is to do this figuring out. Fortunately there was a magic number available from the Jaguar hardware that would tell it what was what. This magic number lived at the hardware address register `$F14002`. Here we can see the `VideoIni` routine take care of this business, querying the register, setting some dimensions based on the reading it gets, and most importantly for the rest of the game, setting a global variable called `pal` that can be used to perform any ad-hoc fiddling during gameplay to ensure a display consistent with the player's ageing television hardware.

```
VideoIni:
        movem.l d0-d6,-(sp)         ; Store d0-d6 in the stack.
        clr pal                     ; Clear the global pal variable
```

```
        move.w  BASE+$14002,d0      ; Check the JOY2(!) port.
        and.w   #$10,d0             ; Check if the magic bit is set..
        beq     ispal               ; .. if not, it's PAL so skip to ispal.

        ; Otherwise it's NTSC.
        move.w  #ntsc_hmid,d2       ; Set the Horizontal mid-point
        move.w  #ntsc_width,d0      ; Set the width.
        move.w  #ntsc_vmid,d6       ; Set the vertical mid-point.
        move.w  #ntsc_height,d4     ; Set the height.

        bra     doit                ; Actually apply the above values.
ispal:
        move.w  #pal_hmid,d2        ; Set the Horizontal mid-point
        move.w  #pal_width,d0       ; Set the width.
        move.w  #pal_vmid,d6        ; Set the vertical mid-point.
        move.w  #pal_height,d4      ; Set the height.
        move #1,pal                 ; Set the global pal variable.
```

There are numerous places where this kind of fiddling will be necessary. Early on in the game's launch sequence we will query this `pal` setting to set various fix-up variables that can be used to centre text and images depending on the height of the screen:

```
; ********************************************************************
; Set up the NTSC/PAL options.
; ********************************************************************
spall:
        btst.b #6,sysflags          ; Are controllers enabled?
        beq slopt
        move.l #o2s3,option2+16
slopt:
        clr palside
        clr paltop
        bclr.b #5,sysflags          ; Flag PAL as not enabled.
        tst pal
        beq notpal1
        ; Set screen adjustments for PAL
        move #6,palside
        move #10,paltop
        move #40,palfix1            ; Y centre fix for rectangles in PAL
        move #20,palfix2            ; Y centre fix for text in PAL
        move.l #$140000,palfix3     ; Y centre fix for PAL
        bset.b #5,sysflags          ; Set PAL as enabled.
notpal1: rts
```

Here's an example where we're going to display some text on the screen ("Press Option for Beastly Mode!") and need to figure out whether we're on a PAL or NTSC screen to decide where to place the text so that it's vertically centred.

```
draw_oo:
        bsr draw_o                  ; Call the common draw_o rotuine first.
        btst.b #2,sysflags          ; Is Beastly Mode enabled?
```

```
        beq rrrts                   ; If so, return early.
        tst h2h                     ; Are we playing a head-to-head game?
        bne rrrts                   ; If so, return early.
        lea bstymsg,a0              ; "PRESS option FOR BEASTLY MODE!"
        lea cfont,a1                ; Load the small regular font to a1.
        move #180,d0                ; Set Y position of text.
        tst pal                     ; Are we on a PAL screen?
        beq gnopal2                 ; If not, skip.
        add palfix2,d0              ; If we are, adjust Y POS for PAL screens
    .
gnopal2:jmp centext                 ; Display horizontally centred text.
        ; Returns
```

The rest of the code is littered with such fix-ups. We even find an artefact of the PAL/NTSC schism when we look at the image data used in our 'Object Lists'. The image data assumes a screen that is 287 pixels high, i.e. one that will display a full screen on PAL. When playing on NTSC this area of the image is not written to and instead we find a bar of pleasing junk pixels.

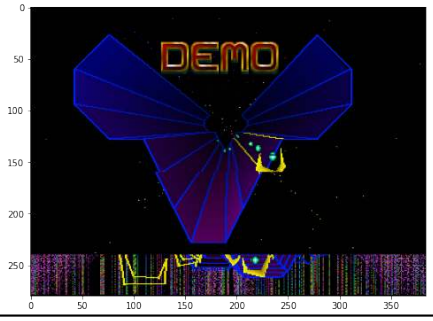
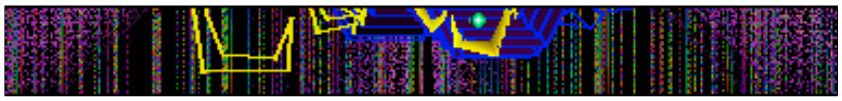
Raw Data	Parsed Data	Image at DATA: 0x134800
13,48,00,1D,6E,45,C1,60	TYPE: 0 LINK: 0xeb70 YPOS: 44 DATA: 0x134800 HEIGHT: 279	
00,00,80,06,01,80,CF,F8	XPOS: -7 REFLECT: 0 DEPTH: 4 RMW: 0 PITCH: 1 TRANS: 1 DWIDTH: 96 RELEASE: 0 IWIDTH: 96 FIRSTPIX: 0 INDEX: 0	

Figure .1: First Bit Mapped Object in the Object List

On PAL this field of colourful noise would be written to properly and the full game field would extend for the whole 287 pixels of screen height. The solution when playing on an NTSC television screen was simply to truncate the height and give the player just enough space to play the game while missing out on activity that extended below the direct field of play.





a random walk through the source code of two video games from the dawn of the computer age

